9.15 (⋆⋆) By making use of Jensen's inequality (2.102) for the special case of the convex function $f(x) = x^2$, show that the average expected sum-of-squares error E_{AV} of the members of a simple committee model, given by (9.46), and the expected error E_{COM} of the committee itself, given by (9.47), satisfy

$$E_{\mathrm{COM}} \leqslant E_{\mathrm{AV}}. \tag{9.64}$$

9.16 (⋆⋆) By making use of Jensen's in equality (2.102), show that the result (9.64) derived in the previous exercise holds for any error function $E(y)$, not just sum-of-squares, provided it is a convex function of y.

9.17 (⋆⋆) Consider a committee in which we allow unequal weighting of the constituent models, so that

$$y_{\mathrm{COM}}(\mathbf{x}) = \sum_{m=1}^{M} \alpha_m y_m(\mathbf{x}). \tag{9.65}$$

To ensure that the predictions $y_{\mathrm{COM}}(\mathbf{x})$ remain within sensible limits, suppose that we require that they be bounded at each value of $\mathbf{x}$ by the minimum and maximum values given by any of the members of the committee, so that

$$y_{\min}(\mathbf{x}) \leqslant y_{\mathrm{COM}}(\mathbf{x}) \leqslant y_{\max}(\mathbf{x}). \tag{9.66}$$

Show that a necessary and sufficient condition for this constraint is that the coefficients α_m satisfy

$$\alpha_m \geqslant 0, \qquad \sum_{m=1}^{M} \alpha_m = 1. \tag{9.67}$$

9.18 (⋆⋆⋆) Here we explore the effect of dropout regularization on a simple linear regression model trained using least squares. Consider a model of the form

$$y_k = \sum_{i=1}^{D} w_{ki} x_i \tag{9.68}$$

along with a sum-of-squares error function given by

$$E(\mathbf{W}) = \sum_{n=1}^{N} \sum_{k=1}^{K} \left\{ t_{nk} - \sum_{i=1}^{D} w_{ki} R_{ni} x_{ni} \right\}^2 \tag{9.69}$$

where the elements $R_{ni} \in \{0, 1\}$ of the dropout matrix are chosen randomly from a Bernoulli distribution with parameter ρ. We now take an expectation over the distribution of random dropout parameters. Show that

$$\mathbb{E}[R_{ni}] = \rho \tag{9.70}$$

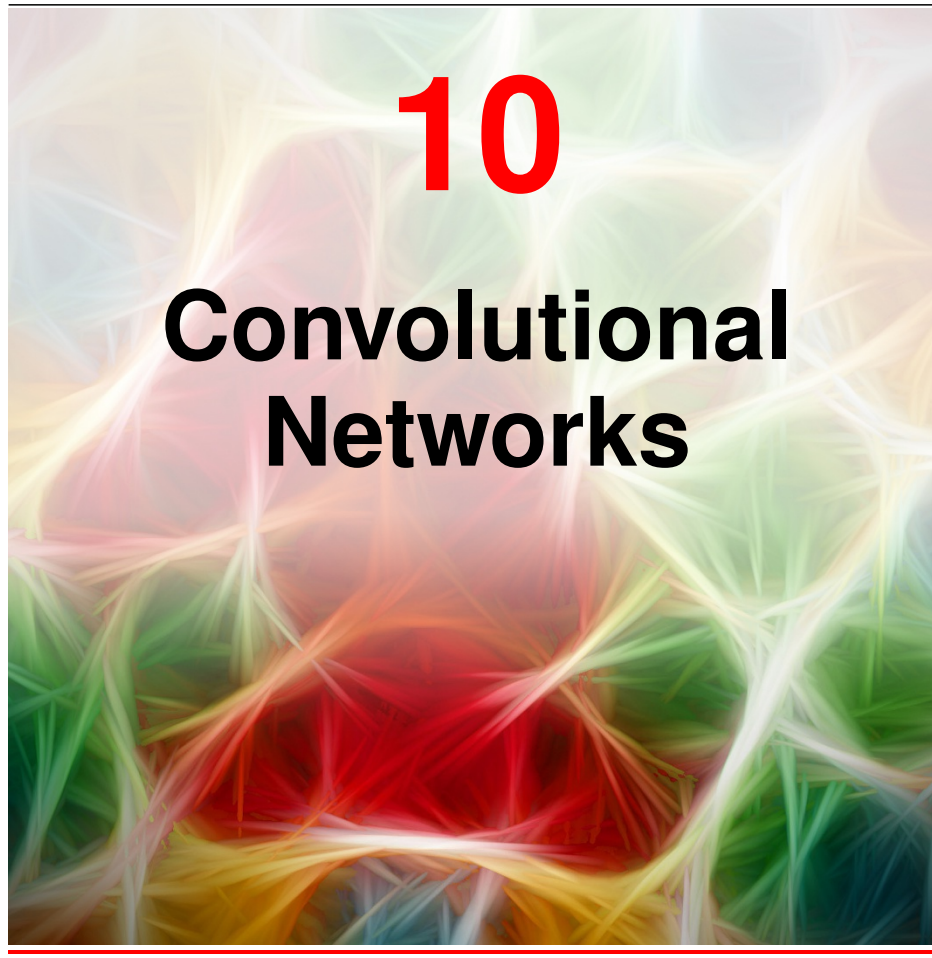
$$\mathbb{E}[R_{ni} R_{nj}] = \delta_{ij}\rho + (1 - \delta_{ij})\rho^2. \tag{9.71}$$

Hence, show that the expected error function for this dropout model is given by

$$\mathbb{E}\left[E(\mathbf{W})\right] = \sum_{n=1}^{N}\sum_{k=1}^{K}\left\{y_{nk} - \rho\sum_{i=1}^{D} w_{ki}x_{ni}\right\}^2 \tag{9.72}$$

$$+ \rho(1-\rho)\sum_{n=1}^{N}\sum_{k=1}^{K}\sum_{i=1}^{D} w_{ki}^2 x_{ni}^2. \tag{9.73}$$

Thus, we see that the expected error function corresponds to a sum-of-squares error with a quadratic regularizer in which the regularization coefficient is scaled separately for each input variable according to the data values seen by that input. Finally, write down a closed-form solution for the weight matrix that minimizes this regularized error function.



10 Convolutional Networks

The simplest machine learning models assume that the observed data values are unstructured, meaning that the elements of the data vectors $\mathbf{x} = (x_1, \ldots, x_D)$ are treated as if we do not know anything in advance about how the individual elements might relate to each other. If we were to make a random permutation of the ordering of these variables and apply this fixed permutation consistently on all training and test data, there would be no difference in the performance for the models considered so far.

Many applications of machine learning, however, involve *structured data* in which there are additional relationships between input variables. For example, the words in natural language form a *sequence*, and if we were to model language as a generative autoregressive process then we would expect each word to depend more strongly on the immediately preceding words and less so on words much earlier in the sequence. Likewise, the pixels of an image have a well-defined spatial relation-

Chapter 12

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024
C. M. Bishop, H. Bishop, *Deep Learning*, https://doi.org/10.1007/978-3-031-45468-4_10

ship to each other in which the input variables are arranged in a two-dimensional grid, and nearby pixels have highly correlated values.

We have already seen that our knowledge of the structure of specific data modalities can be utilized through the addition of a regularization term to the error function in the training objective, through data augmentation, or through modifications to the model architecture. These approaches can help guide the model to respect certain properties such as invariance and equivariance with respect to transformations of the input data. In this chapter we will take a look at an architectural approach called a *convolutional neural network* (CNN), which we will see can be viewed as a sparsely connected multilayer network with parameter sharing, and designed to encode invariances and equivariances specific to image data.

Section 9.1

Section 9.1.4

10.1. Computer Vision

The automatic analysis and interpretation of image data form the focus of the field of computer vision and represent a major application area for machine learning (Szeliski, 2022). Historically, computer vision was based largely on 3-dimensional projective geometry. Hand-crafted features were constructed and used as input to simple learning algorithms (Hartley and Zisserman, 2004). However, it was one of the first fields to be transformed by the deep learning revolution, predominantly thanks to the CNN architecture. Although the architecture was originally developed in the context of image analysis, it has also been applied in other domains such as the analysis of sequential data. Recently alternative architectures based on transformers have become competitive with convolutional networks in some applications.

Chapter 12

There are many applications for machine learning in computer vision, of which some of the most commonly encountered are the following:

Figure 1.1

1. **Classification** of images, for example classifying an image of a skin lesion as benign or malignant. This is sometimes called 'image recognition'.

Figure 10.19

2. **Detection** of objects in an image and determining their locations within the image, for example detecting pedestrians from camera data collected by an autonomous vehicle.

Figure 10.26

3. **Segmentation** of images, in which each pixel is classified individually thereby dividing the image into regions sharing a common label. For example, a natural scene might be segmented into sky, grass, trees, and buildings, whereas a medical scan image could be segmented into cancerous tissue and normal tissue.

Figure 12.27

4. **Caption generation** in which a textual description is generated automatically from an image.

Figure 1.3

5. **Synthesis** of new images, for example generating images of human faces. Images can also be synthesized based on a text input describing the desired image content.

Figure 20.9

6. **Inpainting** in which a region of an image is replaced with synthesized pixels that are consistent with the rest of the image. This is used, for example, to remove unwanted objects during image editing.

Figure 10.32

7. **Style transfer** in which an input image in one style, for example a photograph, is transformed into a corresponding image in a different style, for example an oil painting.

Figure 20.8

8. **Super-resolution** in which the resolution of an image is improved by increasing the number of pixels and synthesizing associated high-frequency information.

9. **Depth prediction** in which one or more views are used to predict the distance of the scene from the camera at each pixel in a target image.

10. **Scene reconstruction** in which one or more two-dimensional images of a scene are used to reconstruct a three-dimensional representation.

10.1.1 Image data

An image comprises a rectangular array of pixels, in which each pixel has either a grey-scale intensity or more commonly a triplet of red, green, and blue *channels* each with its own intensity value. These intensities are non-negative numbers that also have some maximum value corresponding to the limits of the camera or other hardware device used to capture the image. For the most part, we will view the intensities as continuous variables, but in practice they are represented with finite precision, for example as 8-bit numbers represented as integers in the range $0, \ldots, 255$. Some images, such as the magnetic resonance imaging (MRI) scans used in medical diagnosis, comprise three-dimensional grids of *voxels*. Similarly, videos comprise a sequence of two-dimensional images and therefore can also be viewed as three-dimensional structures in which successive frames are stacked through time.

Now consider the challenge of applying neural networks to image data to address some of the applications highlighted above. Images generally have a high dimensionality, with typical cameras capturing images comprising tens of megapixels. Treating the image data as unstructured may therefore require a model with a vast number of parameters that would be infeasible to train. More significantly, such an approach fails to take account of the highly structured nature of image data, in which the relative positions of different pixels play a crucial role. We can see this because if we take the pixels of an image and randomly permute them, then the result no longer looks like a natural image. Similarly, if we generate a synthetic image by drawing random values for the pixel intensities independently for each pixel, there

Figure 6.8

is essentially zero chance of generating something that looks like a natural image. Local correlations are important, and in a natural image there is a much higher probability that two nearby pixels will have similar colours and intensities compared to two pixels that are far apart. This represents powerful prior knowledge and can be used to encode strong inductive biases into a neural network, leading to models with far fewer parameters and with much better generalization accuracy.

10.2. Convolutional Filters

One motivation for the introduction of convolutional networks is that for image data, which is the modality for which CNNs were designed, a standard fully connected architecture would require vast numbers of parameters due to the high-dimensional nature of images. To see this, consider a colour image with $10^3 \times 10^3$ pixels, each with three values corresponding to red, green, and blue intensities. If the first hidden layer of the network has, say, 1,000 hidden units, then we already have 3×10^9 weights in the first layer. Furthermore, such a network would have to learn any invariances and equivariances by example, which would require huge data sets. By designing an architecture that incorporates our inductive bias about the structure of images, we can reduce the data set requirements dramatically and also improve generalization with respect to symmetries in the image space.

To exploit the two-dimensional structure of image data to create inductive biases, we can use four interrelated concepts: hierarchy, locality, equivariance, and invariance. Consider the task of detecting faces in images. There is a natural hierarchical structure because one image may contain several faces, and each face includes elements such as eyes, and each eye has structure such as an iris, which itself has structure such as edges. At the lowest level of the hierarchy, a node in a neural network could detect the presence of a feature such as an edge using information that is *local* to a small region of an image, and therefore it would only need to see a small subset of the image pixels. More complex structures further up the hierarchy can be detected by composing multiple features found at previous levels. A key point, however, is that although we want to build the general concept of hierarchy into the model, we want the details of the hierarchy, including the type of features extracted at each level, to be learned from data, not hand-coded. Hierarchical models fit naturally within the deep learning framework, which already allows very complex concepts to be extracted from raw data through a succession of, possibly very many, 'layers' of processing, in which the whole system is trained end-to-end.

10.2.1 Feature detectors

For simplicity we will initially restrict our attention to grey-scale images (i.e., ones having a single channel). Consider a single unit in the first layer of a neural network that takes as input just the pixel values from a small rectangular region, or patch, from the image, as illustrated in Figure 10.1(a). This patch is referred to as the *receptive field* of that unit, and it captures the notion of locality. We would like weight values associated with this unit to learn to detect some useful low-level feature. The output of this unit is given by the usual functional form comprising a weighted linear combination of the input values, which is subsequently transformed using a nonlinear activation function:

$$z = \mathrm{ReLU}(\mathbf{w}^{\mathrm{T}}\mathbf{x} + w_0) \tag{10.1}$$

where $\mathbf{x}$ is a vector of pixel values for the receptive field, and we have assumed a ReLU activation function. Because there is one weight associated with each input

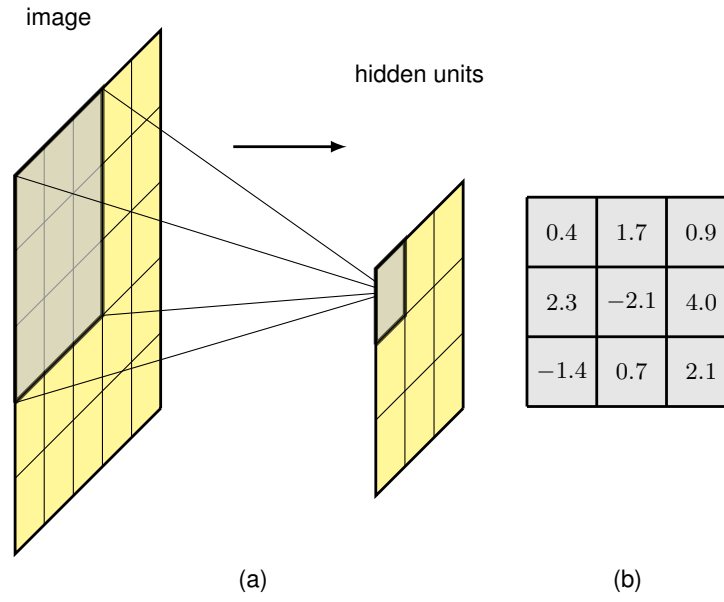


Figure 10.1 (a) Illustration of a receptive field, showing a unit in a hidden layer of a network that receives input from pixels in a 3×3 patch of the image. Pixels in this patch form the receptive field for this unit. (b) The weight values associated with this hidden unit can be visualized as a small 3×3 matrix, known as a kernel. There is also an additional bias parameter that is not shown here.

pixel, the weights themselves form a small two-dimensional grid known as a *filter*, sometimes also called a *kernel*, which itself can be visualized as an image. This is illustrated in Figure 10.1(b).

Suppose that $\mathbf{w}$ and w_0 in (10.1) are fixed and we ask for which value of the input image patch $\mathbf{x}$ will this hidden unit give the largest output response. To answer this we need to constrain $\mathbf{x}$ in some way, so let us suppose that its norm $\|\mathbf{x}\|^2$ is fixed. Then the solution for $\mathbf{x}$ that maximizes $\mathbf{w}^{\mathrm{T}}\mathbf{x}$, and hence maximizes the response of the hidden unit, is of the form $\mathbf{x} = \alpha\mathbf{w}$ for some coefficient α. This says that the maximum output response from this hidden unit occurs when it detects a patch of image that, up to an overall scaling, looks like the kernel image. Note that the ReLU generates a non-zero output only when $\mathbf{w}^{\mathrm{T}}\mathbf{x}$ exceeds a threshold of $-w_0$, and therefore the unit acts as a feature detector that signals when it finds a sufficiently good match to its kernel.

Exercise 10.1

10.2.2 Translation equivariance

Next note that if a small patch in a face image represents, for example, an eye at that location, then the same set of pixel values in a different part of the image must represent an eye at the new location. Our neural network needs to be able to generalize what it has learned in one location to all possible locations in the image, without needing to see examples in the training set of the corresponding feature at every possible location. To achieve this, we can simply replicate the *same* hidden-unit weight values at multiple locations across the image, as illustrated for a one-dimensional input space in Figure 10.2.

The units of the hidden layer form a *feature map* in which all the units share the same weights. Consequently if a local patch of an image produces a particular

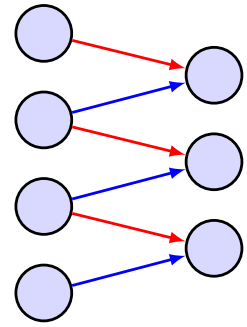


Figure 10.2 Illustration of convolution for a one-dimensional array of input values and a kernel of width 2. The connections are sparse and are shared by the hidden units, as shown by the red and blue arrows in which links with the same colour have the same weight values. This network therefore has six connections but only two independent learnable parameters.

response in the unit connected to that patch, then the same set of pixel values at a different location will produce the same response in the corresponding translated location in the feature map. This is an example of *equivariance*. We see that the connections in this network are *sparse* in that most connections are absent. Also, the values of the weights are *shared* by all the hidden units, as indicated by the colours of the connections. This transformation is an example of a *convolution*.

Section 9.1.3

We can extend the idea of convolution to two-dimensional images as follows (Dumoulin and Visin, 2016). For an image $\mathbf{I}$ with pixel intensities $I(j,k)$, and a filter $\mathbf{K}$ with pixel values $K(l,m)$, the feature map $\mathbf{C}$ has activation values given by

$$C(j,k) = \sum_l \sum_m I(j+l, k+m) K(l,m) \tag{10.2}$$

where we have omitted the nonlinear activation function for clarity. This again is an example of a *convolution* and is sometimes expressed as $\mathbf{C} = \mathbf{I} * \mathbf{K}$. Note that strictly speaking (10.2) is called a *cross-correlation*, which differs slightly from the conventional mathematical definition of convolution, but here we will follow common practice in the machine learning literature and refer to (10.2) as a convolution. The relationship (10.2) is illustrated in Figure 10.3 for a 3×3 image and a 2×2 filter. Importantly, when using batch normalization in a convolutional network, the same value of mean and variance must be used at every spatial location within a feature map when normalizing the states of the units to ensure that the statistics of the feature map are independent of location.

Exercise 10.4

Section 7.4.2

As an example of the application of convolution, we consider the problem of detecting edges in images using a fixed, hand-crafted convolutional filter. Intuitively, we can think of a vertical edge as occurring when there is a significant local change in the intensity between pixels as we move horizontally across the image. We can measure this by convolving the image with a 3×3 filter of the form

$$\begin{array}{|c|c|c|} \hline -1 & 0 & 1 \\ \hline -1 & 0 & 1 \\ \hline -1 & 0 & 1 \\ \hline \end{array} \tag{10.3}$$

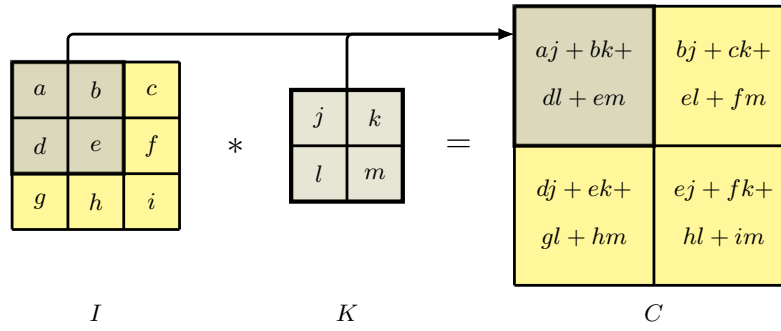


Figure 10.3 Example of a 3×3 image convolved with a 2×2 filter to give a resulting 2×2 feature map.

Similarly we can detect horizontal edges by convolving with the transpose of this filter:

$$\begin{array}{|c|c|c|}\hline -1 & -1 & -1 \\ \hline 0 & 0 & 0 \\ \hline 1 & 1 & 1 \\ \hline \end{array} \tag{10.4}$$

Figure 10.4 shows the results of applying these two convolutional filters to a sample image. Note that in Figure 10.4(b) if a vertical edge corresponds to an increase in pixel intensity, the corresponding point on the feature map is positive (indicated by a light colour), whereas if the vertical edge corresponds to a decrease in pixel intensity, the corresponding point on the feature map is negative (indicated by a dark colour), with analogous properties for Figure 10.4(c) for horizontal edges.

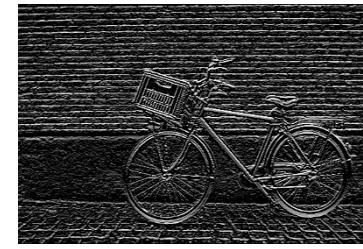
Comparing this convolutional structure with a standard fully connected network, we see several advantages: (i) the connections are sparse, leading to far fewer weights even with large images, (ii) the weight values are shared, greatly reducing the number of independent parameters and consequently reducing the required size



(a)



(b)



(c)

Figure 10.4 Illustration of edge detection using convolutional filters showing (a) the original image, (b) the result of convolving with the filter (10.3) that detects vertical edges, and (c) the result of convolving with the filter (10.4) that detects horizontal edges.

of the training set needed to learn those parameters, and (iii) the same network can be applied to images of different sizes without the need for retraining. We will return to this final point later, but for the moment, simply note that changing the size of the input image simply changes the size of the feature map but does not change the number of weights, or the number of independent learnable parameters, in the model. One final observation regarding convolutional networks is that, by exploiting the massive parallelism of graphics processing units (GPUs) to achieve high computational throughput, convolutions can be implemented very efficiently.

Section 10.4.3

10.2.3 Padding

We see from Figure 10.3 that the convolution map is smaller than the original image. If the image has dimensionality $J \times K$ pixels and we convolve with a kernel of dimensionality $M \times M$ (filters are typically chosen to be square) the resulting feature map has dimensionality $(J - M + 1) \times (K - M + 1)$. In some cases we want the feature map to have the same dimensions as the original image. This can be achieved by *padding* the original image with additional pixels around the outside, as illustrated in Figure 10.5. If we pad with P pixels then the output map has dimensionality $(J+2P-M+1)\times(K+2P-M+1)$. If there is no padding, so that $P = 0$, this is called a *valid* convolution. When the value of P is chosen such that the output array has the same size as the input, corresponding to $P = (M - 1)/2$, this is called a *same* convolution, because the image and the feature map have the same dimensions. In computer vision, filters generally use odd values of M, so that the padding can be symmetric on all sides of the image and that there is a well-defined central pixel associated with the location of the filter. Finally, we have to choose a suitable value for the intensities associated with the padding pixels. A typical choice is to set the padding values to zero, after first subtracting the mean from each image so that zero represents the average value of the pixel intensity. Padding can also be applied to feature maps for processing by subsequent convolutional layers.

Exercise 10.6

10.2.4 Strided convolutions

In typical image processing applications, the images can have very large numbers of pixels, and since the kernels are often relatively small, so that $M \ll J, K$, the convolutional feature map will be of a similar size to the original image and will be the same size if *same* padding is used. Sometimes we wish to use feature maps that are significantly smaller than the original image to provide flexibility in the design of convolutional network architectures. One way to achieve this is to use *strided convolutions* in which, instead of stepping the filter over the image one pixel at a time, it is moved in larger steps of size S, called the *stride*. If we use the same stride horizontally and vertically, then the number of elements in the feature map will be

Exercise 10.7

$$\left\lfloor \frac{J + 2P - M}{S} - 1 \right\rfloor \times \left\lfloor \frac{K + 2P - M}{S} - 1 \right\rfloor \tag{10.5}$$

where $\lfloor x \rfloor$ denotes the 'floor' of x, i.e., the largest integer that is less than or equal to x. For large images and small filter sizes, the image map will be roughly a factor of $1/S$ smaller than the original image.

Figure 10.5 Illustration of a 4×4 image that has been padded with additional pixels to create a 6×6 image.

0	0	0	0	0	0
0	X_{11}	X_{12}	X_{13}	X_{14}	0
0	X_{21}	X_{22}	X_{23}	X_{24}	0
0	X_{31}	X_{32}	X_{33}	X_{34}	0
0	X_{41}	X_{42}	X_{43}	X_{44}	0
0	0	0	0	0	0

10.2.5 Multi-dimensional convolutions

So far we have considered convolutions over a single grey-scale image. For a colour image there will be three channels corresponding to the red, green, and blue colours. We can easily extend convolutions to cover multiple channels by extending the dimensionality of the filter. An image with $J \times K$ pixels and C channels will be described by a *tensor* of dimensionality $J \times K \times C$. We can introduce a filter described by a tensor of dimensionality $M \times M \times C$ comprising a separate $M \times M$ filter for each of the C channels. Assuming no padding and a stride of 1, this again gives a feature map of size $(J-M+1) \times (K-M+1)$, as is illustrated in Figure 10.6.

Section 6.3.7

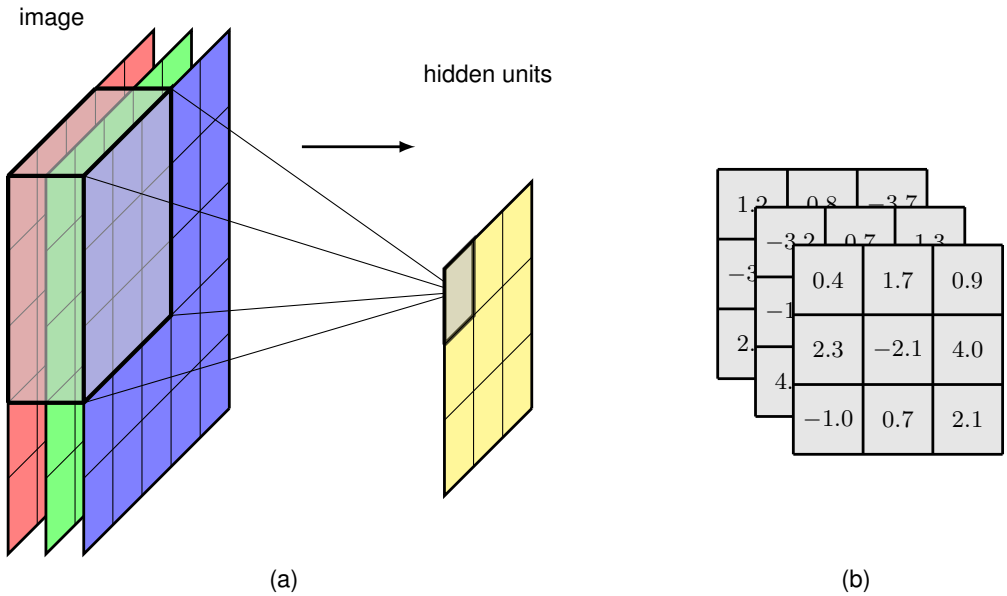


Figure 10.6 (a) Illustration of a multi-dimensional filter that takes input from across the R, G, and B channels. (b) The kernel here has 27 weights (plus a bias parameter not shown) and can be visualized as a $3 \times 3 \times 3$ tensor.

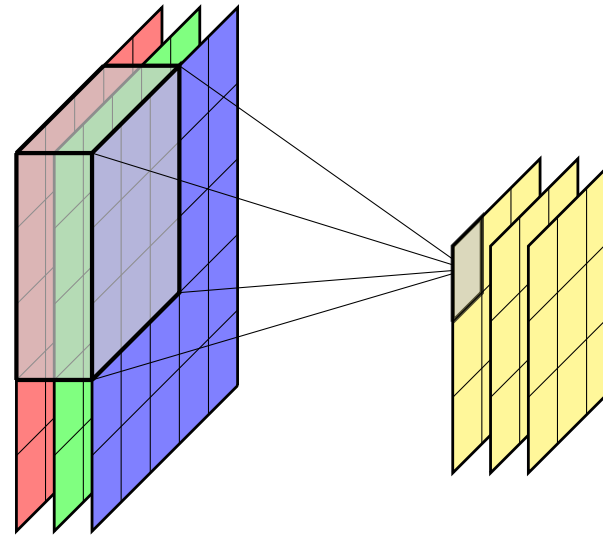


Figure 10.7 The multi-dimensional convolutional filter layer shown in Figure 10.6 can be extended to include multiple independent filter channels.

We now make a further important extension to convolutions. Up to now we have created a single feature map in which all the points in the feature map share the same set of learnable parameters. For a filter of dimensionality $M \times M \times C$, this will have M^2C weight parameters, irrespective of the size of the image. In addition there will be a bias parameter associated with this unit. Such a filter is analogous to a single hidden node in a fully connected network, and it can learn to detect only one kind of feature and is therefore very limited. To build more flexible models, we simply include multiple such filters, in which each filter has its own independent set of parameters giving rise to its own independent feature map, as illustrated in Figure 10.7. We will again refer to these separate feature maps as *channels*. The filter tensor now has dimensionality $M \times M \times C \times C_{\text{OUT}}$ where C is the number of input channels and C_{OUT} is the number of output channels. Each output channel will have its own associated bias parameter, so the total number of parameters will be $(M^2C + 1)C_{\text{OUT}}$.

A useful concept in designing convolutional networks is the 1×1 *convolution* (Lin, Chen, and Yan, 2013), which is simply a convolutional layer in which the filter size is a single pixel. The filters have C weights, one for each input channel, plus a bias. One application for 1×1 convolutions is simply to change the number of channels (typically to reduce the number of channels) without changing the size of the feature maps, by setting the number of output channels to be different to the number of input channels. It is therefore complementary to strided convolutions or pooling in that it reduces the number of channels rather than the dimensionality of the channels.

10.2.6 Pooling

A convolutional layer encodes translation *equivariance*, whereby if a small patch of pixels, representing the receptive field of a hidden unit, is moved to a different

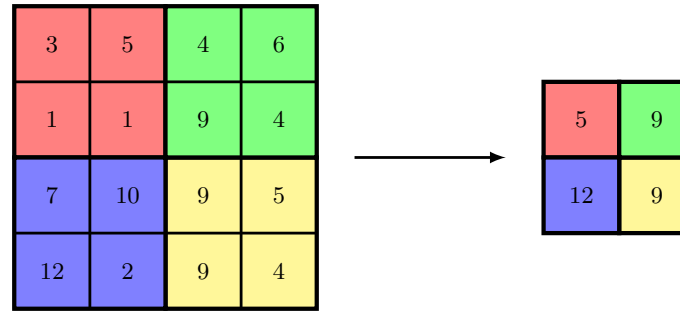


Figure 10.8 Illustration of max-pooling in which blocks of 2×2 pixels in a feature map are combined using the 'max' operator to generate a new feature map of smaller dimensionality.

location in the image, the associated outputs of the feature map will move to the corresponding location in the feature map. This is valuable for applications such as finding the location of an object within an image. For other applications, such as classifying an image, we want the output to be *invariant* to translations of the input. In all cases, however, we want the network to be able to learn hierarchical structure in which complex features at a particular level are built up from simpler features at the previous level. In many cases the spatial relationship between those simpler features will be important. For example, it is the relative positions of the eyes, nose, and mouth that help determine the presence of a face and not just the presence of these features in arbitrary locations within the image. However, small changes in the relative locations do not affect the classification, and we want to be invariant to such small translations of individual features. This can be achieved using *pooling* applied to the output of the convolutional layer.

Pooling has similarities to using a convolutional layer in that an array of units is arranged in a grid, with each unit taking input from a receptive field in the previous feature map layer. Again, there is a choice of filter size and of stride length. The difference, however, is that the output of a pooling unit is a simple, fixed function of its inputs, and so there are no learnable parameters in pooling. A common example of a pooling function is *max-pooling* (Zhou and Chellappa, 1988) in which each unit simply outputs the $\max$ function applied to the input values. This is illustrated with a simple example in Figure 10.8. Here the stride length is equal to the filter width, and so there is no overlap of the receptive fields.

As well as building in some local translation invariance, pooling can also be used to reduce the dimensionality of the representation by down-sampling the feature map. Note that using strides greater than 1 in a convolutional layer also has the effect of down-sampling the feature maps.

We can interpret the activation of a unit in a feature map as a measure of the strength of detection of a corresponding feature, so that the max-pooling preserves information on whether the feature is present and with what strength but discards some positional information. There are many other choices of pooling function, for example *average pooling* in which the pooling function computes the average of the values from the corresponding receptive field in the feature map. These all introduce some degree of local translation invariance.

Pooling is usually applied to each channel of a feature map independently. For

example, if we have a feature map with 8 channels, each of dimensionality 64×64, and we apply max-pooling with a receptive field of size 2×2 and a stride of 2, the output of the pooling operation will be a tensor of dimensionality $32 \times 32 \times 8$.

We can also apply pooling across multiple channels of a feature map, which gives the network the potential to *learn* other invariances beyond simple translation invariance. For example, if several channels in a convolutional layer learn to detect the same feature but at different orientations, then max-pooling across those feature maps will be approximately invariant to rotations.

Pooling also allows a convolutional network to process images of varying sizes. Ultimately, the output, and generally some of the intermediate layers, of a convolutional network must have a fixed size. Variable-sized inputs can be accommodated by varying the stride length of the pooling according to the size of the image such that the number of pooled outputs remains constant.

10.2.7 Multilayer convolutions

The convolutional network structure described so far is analogous to a single layer in a standard fully connected neural network. To allow the network to discover and represent hierarchical structure in the data, we now extend the architecture by considering multiple layers of the kind described above. Each convolutional layer is described by a filter tensor of dimensionality $M \times M \times C_{\text{IN}} \times C_{\text{OUT}}$ in which the number of independent weight and bias parameters is $(M^2 C_{\text{IN}} + 1)C_{\text{OUT}}$. Each such convolutional layer can optionally be followed by a pooling layer. We can now apply multiple such layers of convolution and pooling in succession, in which the C_{OUT} output channels of a particular layer, analogous to the RGB channels of the input image, form the input channels of the next layer. Note that the number of channels in a feature map is sometimes called the 'depth' of the feature map, but we prefer to reserve the term depth to mean the number of layers in a multilayer network.

A key property that we built into the convolutional framework is that of locality, in which a given unit in a feature map takes information only from a small patch, the *receptive field*, in the previous layer. When we construct a deep neural network in which each layer is convolutional then the effective receptive field of a unit in later layers in the network becomes much larger than those in earlier layers, as seen in Figure 10.9.

In many applications, the output units of the network need to make predictions about the image as a whole, for example in a classification task, and so they need to combine information from across the whole of the input image. This is typically achieved by introducing one or two standard fully connected layers as the final stages of the network, in which each unit is connected to every unit in the previous layer. The number of parameters in such an architecture can be manageable because the final convolutional layer generally has much lower dimensionality than the input layer due to the intermediate pooling layers. Nevertheless, the final fully connected layers may contain the majority of the independent degrees of freedom in the network even if the number of (shared) connections in the network is larger in the convolutional layers.

A complete CNN therefore comprises multiple layers of convolutions inter-

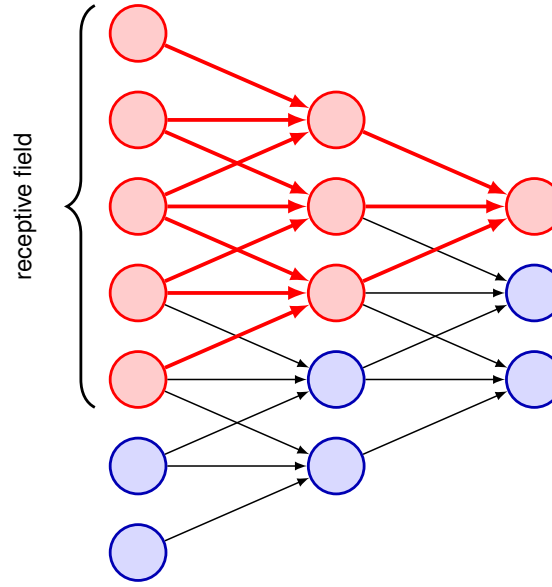


Figure 10.9 Illustration of how the effective receptive field grows with depth in a multilayer convolutional network. Here we see that the red unit at the top of the output layer takes inputs from a receptive field in the middle layer of units, each of which has a receptive field in the first layer of units. Thus, the activation of the red unit in the output layer depends on the outputs of 3 units in the middle layer and 5 units in the input layer.

spersed with pooling operations, and often with conventional fully connected layers in the final stages of the network. There are many choices to be made in designing such an architecture including the number of layers, the number of channels in each layer, the filter sizes, the stride widths, and multiple other such hyperparameters. A wide variety of different architectures have been explored, although in practice it is difficult to make a systematic comparison of hyperparameter values using hold-out data due to the high computational cost of training each candidate configuration.

10.2.8 Example network architectures

Convolutional networks were the first deep neural networks (i.e., ones with more than two learnable layers of parameters) to be successfully deployed in applications. An early example was *LeNet*, which was used to classify low-resolution monochrome images of handwritten digits (LeCun *et al.*, 1989; LeCun *et al.*, 1998). The development of more powerful convolutional networks was accelerated through the introduction of a large-scale benchmark data set called *ImageNet* (Deng *et al.*, 2009) comprising some 14 million natural images each of which has been hand labelled into one of nearly 22,000 categories. This was a much larger data set than had been used previously, and the advances in the field driven by ImageNet served to emphasize the importance of large-scale data, alongside well-designed models having appropriate inductive biases, in building successful deep learning solutions.

A subset of images comprising 1,000 non-overlapping categories formed the basis for the annual *ImageNet Large Scale Visual Recognition Challenge*. Again, this was a much larger number of categories than the typically few dozen classes previously considered. Having so many categories made the problem much more challenging because, if the classes were distributed uniformly, random guessing would

have an error rate of 99.9%. The data set has just over 1.28 million training images, 50,000 validation images, and 100,000 test images. The classifiers are designed to produce a ranked list of predicted output classes on test images, and results are reported in terms of top-1 and top-5 error rates, meaning an image is deemed to be correctly classified if the true class appears at the top of the list or if it is in one of the five highest-ranked class predictions. Early results with this data set achieved a top-5 error rate of around 25.5%. An important advance was made by the *AlexNet* convolutional network architecture (Krizhevsky, Sutskever, and Hinton, 2012), which won the 2012 competition and reduced the top-5 error rate to a new record of 15.3%. Key aspects of this model were the use of the ReLU activation function, the application of GPUs to train the network, and the use of dropout regularization. Subsequent years saw further advances, leading to error rates of around 3%, which is somewhat better than human-level performance for the same data, which is around 5% (Dodge and Karam, 2017). This can be attributed to the difficulty humans have in distinguishing subtly different classes (for example multiple varieties of mushrooms).

Section 9.6.1

As an example of a typical convolutional network architecture, we look in detail at the VGG-16 model (Simonyan and Zisserman, 2014), where VGG stands for the Visual Geometry Group, who developed the model, and 16 refers to the number of learnable layers in the model. VGG-16 has some simple design principles leading to a relatively uniform architecture, shown in Figure 10.10, that minimizes the number of hyperparameter choices that need to be made. It takes an input image having 224×224 pixels and three colour channels, followed by sets of convolutional layers interspersed with down-sampling. All convolutional layers have filters of size 3×3 with a stride of 1, same padding, and a ReLU activation function, whereas the max-pooling operations all use stride 2 and filter size 2×2 thereby down-sampling the number of units by a factor of 4. The first learnable layer is a convolutional layer in which each unit takes input from a $3 \times 3 \times 3$ 'cube' from the stack of input channels and so has 28 parameters including the bias. These parameters are shared across all units in the feature map for that channel. There are 64 such feature channels in the first layer, giving an output tensor of size $224 \times 224 \times 64$. The second layer is also convolutional and again has 64 channels. This is followed by max-pooling giving feature maps of size 112×112. Layers 3 and 4 are again convolutional, of dimensionality 112×112, and each was chosen to have 128 channels. This increase in the number of channels offsets to some extent the down-sampling in the max-pooling layer to ensure that the number of variables in the representation at each layer does not decrease too rapidly through the network. Again, this is followed by a max-pooling operation to give a feature map size of 56×56. Next come three more convolutional layers each with 256 channels, thereby again doubling the number of channels in association with the down-sampling. This is followed by another max-pooling to give feature maps of size 28×28 followed by three more convolutional layers each having 512 channels, followed by another max-pooling, which down-samples to feature maps of size 14×14. This is followed by three more convolutional layers, although the number of feature maps in these layers is kept at 512, followed by another max-pooling, which brings the size of the feature maps down to 7×7. Finally there are three more layers that are *fully connected* meaning that they are

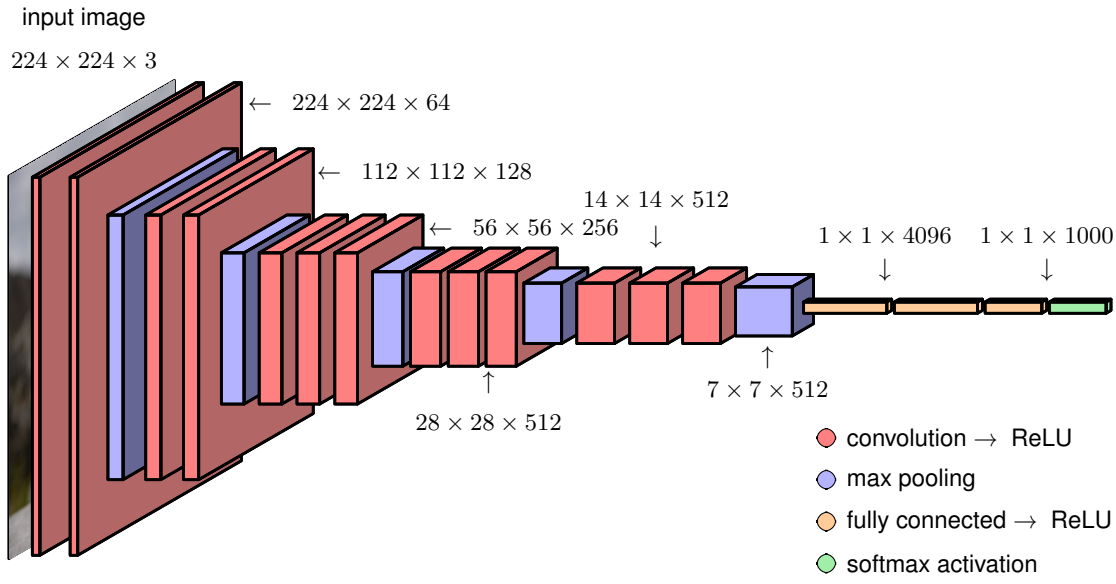


Figure 10.10 The architecture of a typical convolutional network, in this case a model called VGG-16.

standard neural network layers with full connectivity and no sharing of parameters. The final max-pooling layer has 512 channels each of size 7×7 giving 25,088 units in total. The first fully connected layer has 4,096 units, each of which is connected to each of the max-pooling units. This is followed by a second fully connected layer again with 4,096 units, and finally there is a third fully connected layer with 1,000 units so that the network can be applied to a classification problem involving 1,000 classes. All the learnable layers in the network have nonlinear ReLU activation functions, except for the output layer, which has a softmax activation function. In total there are roughly 138 million independently learnable parameters in VGG-16, the majority of which (nearly 103 million) are in the first fully connected layer, whereas most of the connections are in the first convolutional layer.

Exercise 10.8

Earlier CNNs typically had fewer convolutional layers, as they had larger receptive fields. For example, Alexnet (Krizhevsky, Sutskever, and Hinton, 2012) has 11×11 receptive fields with a stride of 4. We saw in Figure 10.9 that larger receptive fields can also be achieved implicitly by using multiple layers each having smaller receptive fields. The advantage of the latter approach is that it requires significantly fewer parameters, effectively imposing an inductive bias on the larger filters as they must be composed of convolutional sub-filters. Although this is a highly complex architecture, only the network function itself needs to be coded explicitly since the derivatives of the cost function can be evaluated using automatic differentiation and the cost function optimized using stochastic gradient descent.

Section 8.2

10.3. Visualizing Trained CNNs

We turn now to an exploration of the features learned by modern deep CNNs, and we will see some remarkable similarities to the properties of the mammalian visual cortex.

10.3.1 Visual cortex

Historically, much of the motivation for CNNs came from pioneering research in neuroscience, which gave insights into the nature of visual processing in mammals including humans. Electrical signals from the retina are transformed through a series of processing layers in the visual cortex, which is at the back of the brain, where the neurons are organized into two-dimensional sheets each of which forms a map of the two-dimensional visual field. In their pioneering work, Hubel and Wiesel (1959) measured the electrical responses of individual neurons in the visual cortex of cats while presenting visual stimuli to the cats' eyes. They discovered that some neurons, called 'simple cells', have a strong response to visual inputs with a simple edge oriented at a particular angle and located at a particular position within the visual field, whereas other stimuli generated relatively little response in those neurons. More detailed studies showed that the response of these simple cells can be modelled using *Gabor filters*, which are two-dimensional functions defined by

$$G(x, y) = A \exp\left(-\alpha\widetilde{x}^2 - \beta\widetilde{y}^2\right) \sin\left(\omega\widetilde{x} + \phi\right) \tag{10.6}$$

where

$$\widetilde{x} = (x - x_0)\cos(\theta) + (y - y_0)\sin(\theta) \tag{10.7}$$
$$\widetilde{y} = -(x - x_0)\sin(\theta) + (y - y_0)\cos(\theta). \tag{10.8}$$

Equations (10.7) and (10.8) represent a rotation of the coordinate system through an angle θ and therefore the $\sin(\cdot)$ term in (10.6) represents a sinusoidal spatial oscillation oriented in a direction defined by the polar angle θ, with frequency ω and phase angle ϕ . The exponential factor in (10.6) creates a decay envelope that localizes the filter in the neighbourhood of position (x_0, y_0) and with decay rates governed by α and β. Example Gabor filters are shown in Figure 10.11.

Hubel and Wiesel also discovered the presence of 'complex cells', which respond to more complex stimuli and which seem to be derived by combining and processing the output of simple cells. These responses exhibit some degree of invariance to small changes in the input such as shifts in location, analogous to the pooling units in a convolutional deep network. Deeper levels of the mammalian visual processing system have even more specific responses and even greater invariance to transformations of the visual input. Such cells have been termed 'grandmother cells' because such a cell could notionally respond if, and only if, the visual input corresponds to a person's grandmother, irrespective of location, scale, lighting, or other transformations of the scene. This work directly inspired an early form of deep neural network called the *neocognitron* (Fukushima, 1980), which was the forerunner of

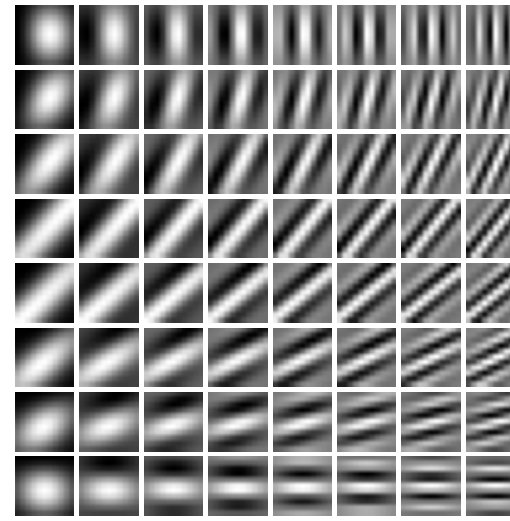


Figure 10.11 Examples of Gabor filters defined by (10.6). The orientation angle θ varies from 0 in the top row to $\pi/2$ in the bottom row, whereas the frequency varies from $\omega = 1$ in the left column to $\omega = 10$ in the right column.

convolutional neural networks. The neocognitron had multiple layers of processing comprising local receptive fields with shared weights followed by local averaging or max-pooling to confer positional invariance. However, it lacked an end-to-end training procedure since it predated the development of backpropagation, relying instead on greedy layer-wise learning through an unsupervised clustering algorithm.

10.3.2 Visualizing trained filters

Suppose we have a trained deep CNN and we wish to explore what the hidden units have learned to detect. For the filters in the first convolutional layer this is relatively straightforward, as they correspond to small patches in the original input image space, and so we can visualize the network weights associated with these filters directly as small images. The first convolutional layer computes inner products between the filters and the corresponding image patches, and so the unit will have a large activation when the inner product has a large magnitude.

Figure 10.12 shows some example filters from the first layer of a CNN trained on the ImageNet data set. We see a remarkable similarity between these filters and the Gabor filters of Figure 10.11. However, this does not imply that a convolutional neural network is a good model of how the brain works, because very similar results can be obtained from a wide variety of statistical methods (Hyvärinen, Hurri, and Hoyer, 2009). This is because these characteristic filters are a general property of the statistics of natural images and therefore prove useful for image understanding in both natural and artificial systems.

Although we can visualize the filters in the first layer directly, the subsequent layers in the network are harder to interpret because their inputs are not patches of images but groups of filter responses. One approach, analogous to that used by Hubel and Wiesel, is to present a large number of image patches to the network and

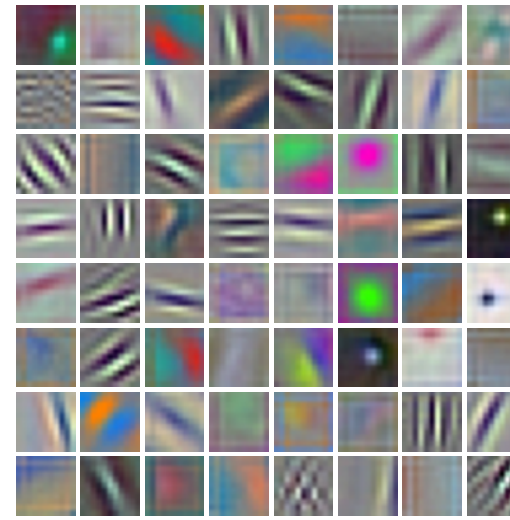


Figure 10.12 Examples of learned filters from the first layer of AlexNet. Note the remarkable similarity of many of the learned filters to the Gabor filters in Figure 10.11, which correspond to features detected by living neurons in the visual cortex of mammals.

see which produce the highest activation value in any particular hidden unit. Figure 10.13 shows examples obtained using a network with five convolutional layers, followed by two fully connected layers, trained on 1.3 million ImageNet data points spanning 1,000 classes. We see a natural hierarchical structure, with the first layer responding to edges, the second layer responding to textures and simple shapes, layer 3 showing components of objects (such as wheels), and layer 5 showing entire objects.

We can extend this technique to go beyond simply selecting image patches from the validation set and instead perform a numerical optimization over the input variables to maximize the activation of a particular unit (Zeiler and Fergus, 2013; Simonyan, Vedaldi, and Zisserman, 2013; Yosinski *et al.*, 2015). If we chose the unit to be one of the outputs then we can look for an image that is most representative of the corresponding class label. Because the output units generally have a softmax activation function, it is better to maximise the pre-activation value that feeds into the softmax rather than the class probability directly, as this ensures the optimization depends on only one class. For example, if we seek the image that produces the strongest response to the class 'dog', then if we optimize the softmax output it could drive the image to be, say, less like a cat because of the denominator in the softmax.

Chapter 17

This approach is related to adversarial training. Unconstrained optimization of the output-unit activation, however, leads to individual pixel values being driven to infinity and also creates high-frequency structure that is difficult to interpret, and so some form of regularization is required to find solutions that are closer to natural images. Yosinski *et al.* (2015) used a regularization function comprising the sum of squares of the pixel values along with a procedure that alternates gradient-based updates to the image pixel values with a blurring operation to remove high-frequency structure and a clipping operation that sets to zero those pixel values that make only small contributions to the class label. Example results are shown in Figure 10.14.

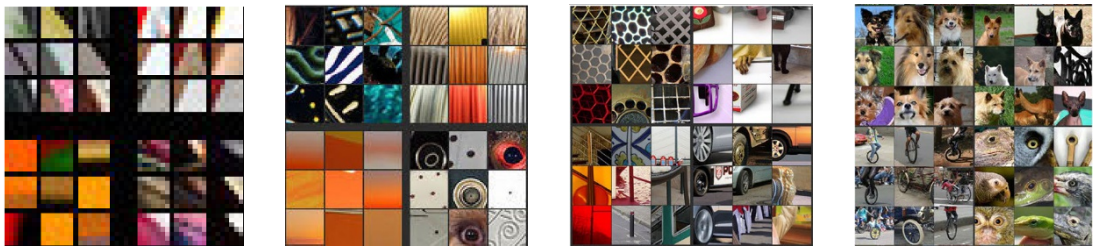


Figure 10.13 Examples of image patches (taken from a validation set) that produce the strongest activation in the hidden units in a network having five convolutional layers trained on ImageNet data. The top nine activations in each feature map are arranged as a 3×3 grid for four randomly chosen channels in each of the corresponding layers. We see a steady progression in complexity with depth, from simple edges in layer 1 to complete objects in layer 5. [From Zeiler and Fergus (2013) with permission.]

10.3.3 Saliency maps

Another way to gain insight into the features used by a convolutional network is through *saliency maps*, which aim to identify those regions of an image that are most significant in determining the class label. This is best done by investigating the final convolutional layer because this still retains spatial localization, which becomes lost in the subsequent fully connected layers, and yet it has the highest level of semantic representation. The Grad-CAM (gradient class activation mapping) method (Selvaraju *et al.*, 2016) first computes, for a given input image, the derivatives of the output-unit pre-activation $a^{(c)}$ for a given class c, before the softmax, with respect to the pre-activations $a_{ij}^{(k)}$ of all the units in the final convolutional layer for channel k. For each channel in that layer, the average of those derivatives is evaluated to give

$$\alpha_k = \frac{1}{M_k} \sum_i \sum_j \frac{\partial a^{(c)}}{\partial a_{ij}^{(k)}} \tag{10.9}$$

where i and j index the rows and columns of channel k, and M_k is the total number of units in that channel. These averages are then used to form a weighted sum of the form:

$$\mathbf{L} = \sum_k \alpha_k \mathbf{A}^{(k)} \tag{10.10}$$

in which $\mathbf{A}^{(k)}$ is a matrix with elements $a_{ij}^{(k)}$. The resulting array has the same dimensionality as the final convolutional layer, for example 14×14 for the VGG network shown in Figure 10.10, and can be superimposed on the original image in the form of a 'heat map' as seen in Figure 10.15.

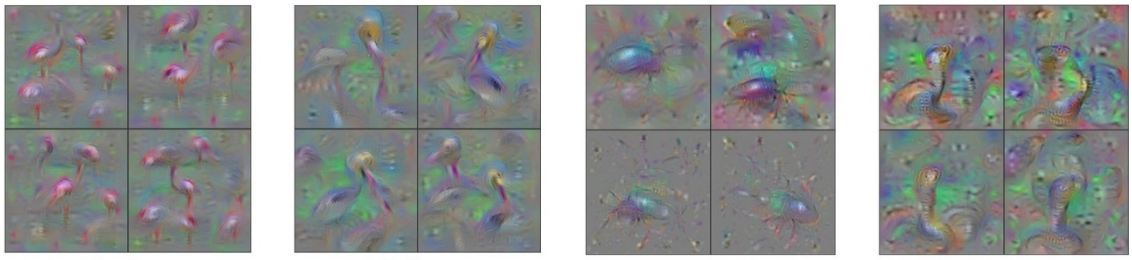


Figure 10.14 Examples of synthetic images generated by maximizing the class probability with respect to the image pixel channel values for a trained convolutional classifier. Four different solutions, obtained with different settings of the regularization parameters, are shown for each of four object classes. [From Yosinski *et al.* (2015) with permission.]

10.3.4 Adversarial attacks

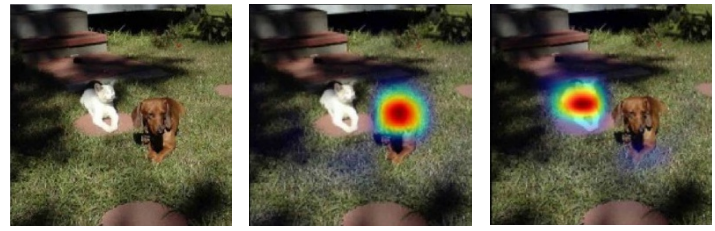
Gradients with respect to changes in the input image pixel values can also be used to create *adversarial attacks* against convolutional networks (Szegedy *et al.*, 2013). These attacks involve making very small modifications to an image, at a level that is imperceptible to a human, which cause the image to be misclassified by the neural network. One simple approach to creating adversarial images is called the *fast gradient sign* method (Goodfellow, Shlens, and Szegedy, 2014). This involves changing each pixel value in an image $\mathbf{x}$ by a fixed amount ϵ with a sign determined by the gradient of an error function $E(\mathbf{x}, t)$ with respect to the pixel values. This gives a modified image defined by

$$\mathbf{x}' = \mathbf{x} + \epsilon \, \text{sign}(\nabla_{\mathbf{x}} E(\mathbf{x}, t)). \tag{10.11}$$

Here t is the true label of $\mathbf{x}$, and the error $E(\mathbf{x}, t)$ could, for example, be the negative log likelihood of $\mathbf{x}$. The required gradient can be computed efficiently using backpropagation. During conventional training of a neural network, the network parameters are adjusted to minimize this error, whereas the modification defined by (10.11) alters the image (while keeping the trained network parameters fixed) so as

Chapter 8

Figure 10.15 Saliency maps for the VGG-16 network with respect to the 'dog' and 'cat' categories. [From Selvaraju *et al.* (2016) with permission.]



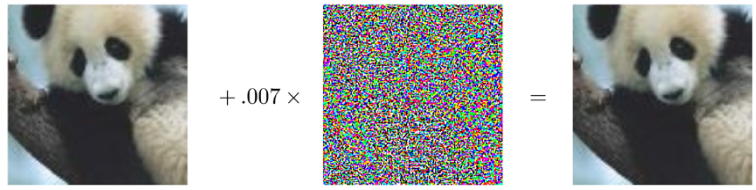


Figure 10.16 Example of an adversarial attack against a trained convolutional network. The image on the left is classified as a panda with confidence 57.7%. The addition of a small level of a random-looking perturbation (that itself is classified as a nematode with confidence 8.2%) results in the image on the right, which is classified as a gibbon with confidence 99.3%. [From Goodfellow, Shlens, and Szegedy (2014) with permission.]

to increase the error. By keeping ϵ small, we ensure that the changes to the image are undetectable to the human eye. Remarkably, this can give images that are misclassified by the network with high confidence, as seen in the example in Figure 10.16.

The ability to fool neural networks in this way raises potential security concerns as it creates opportunities for attacking trained classifiers. It might appear that this issue arises from over-fitting, in which a high-capacity model has adapted precisely to the specific image such that small changes in the input produce large changes in the predicted class probabilities. However, it turns out that an image that has been adapted to give a spurious output for a particular trained network can give similarly spurious outputs when fed to other networks (Goodfellow, Shlens, and Szegedy, 2014). Moreover, a similar adversarial result can be obtained with much less flexible linear models. It is even possible to create physical artefacts such that a regular, uncorrupted image of the artefact will give erroneous predictions when presented to a trained neural network, as seen in Figure 10.17. Although these basic kinds of adversarial attacks can be addressed by simple modifications to the network training process, more sophisticated approaches are harder to defeat. Understanding the implications of these results and mitigating their potential pitfalls remain open areas of research.



Figure 10.17 Two examples of physical stop signs that have been modified. Images of these objects are robustly classified as 45 mph speed-limit signs by CNNs. [From Eykholt *et al.* (2018) with permission.]

10.3.5 Synthetic images

As a final example of image modification that provides additional insights into the operation of a trained convolutional network, we consider a technique called *DeepDream* (Mordvintsev, Olah, and Tyka, 2015). The goal is to generate a synthetic image with exaggerated characteristics. We do this by determining which nodes in a particular hidden layer of the network respond strongly to a particular image and then modifying the image to amplify those responses. For example, if we present an image of some clouds to a network trained on object recognition and a particular node detects a cat-like pattern at a particular region of the image, then we modify the image to be more 'cat like' in that region. To do this, we apply an image to the input of the network and forward propagate through to some particular layer. We then set the backpropagation δ variables for that layer equal to the pre-activations of the nodes and run backpropagation to the input layer to get a gradient vector over the pixels of the image. Finally, we modify the image by taking a small step in the direction of the gradient vector. This procedure can be viewed as a gradient-based method for increasing the function

Section 8.1.2

Exercise 10.10

$$F(\mathbf{I}) = \sum_{i,j,k} a_{ijk}(\mathbf{I})^2 \tag{10.12}$$

where $a_{ijk}(\mathbf{I})$ is the pre-activation of the unit in row i and column j of channel k in the chosen layer when the input image is $\mathbf{I}$, and the sum is over all units and over all channels in that layer. To generate smooth-looking images, some regularization is applied in the form of spatial smoothing and pixel clipping. This process can then be repeated multiple times if stronger enhancements are desired. Examples of the resulting image are shown in Figure 10.18. It is interesting that even though convolutional networks are trained to discriminate between object classes, they seem able to capture at least some of the information needed to generate images from those classes.

This technique can be applied to a photograph, or we can start with inputs consisting of random noise to obtain an image generated entirely from the trained network. Although DeepDream provides some insights into the operation of the trained network, it has primarily been used to generate interesting looking images as a form of artwork.

10.4. Object Detection

We have motivated the design of CNNs primarily by the image classification problem, in which an entire image is assigned to a single class, for example 'cat' or 'bicycle'. This is reasonable for data sets such as ImageNet where, by design, each image is dominated by a single object. However, there are many other applications for CNNs that are able to exploit the inbuilt inductive biases. More generally, the convolutional layers of a CNN trained on a large image data base for a particular task can learn internal representations that have broad applicability, and therefore a

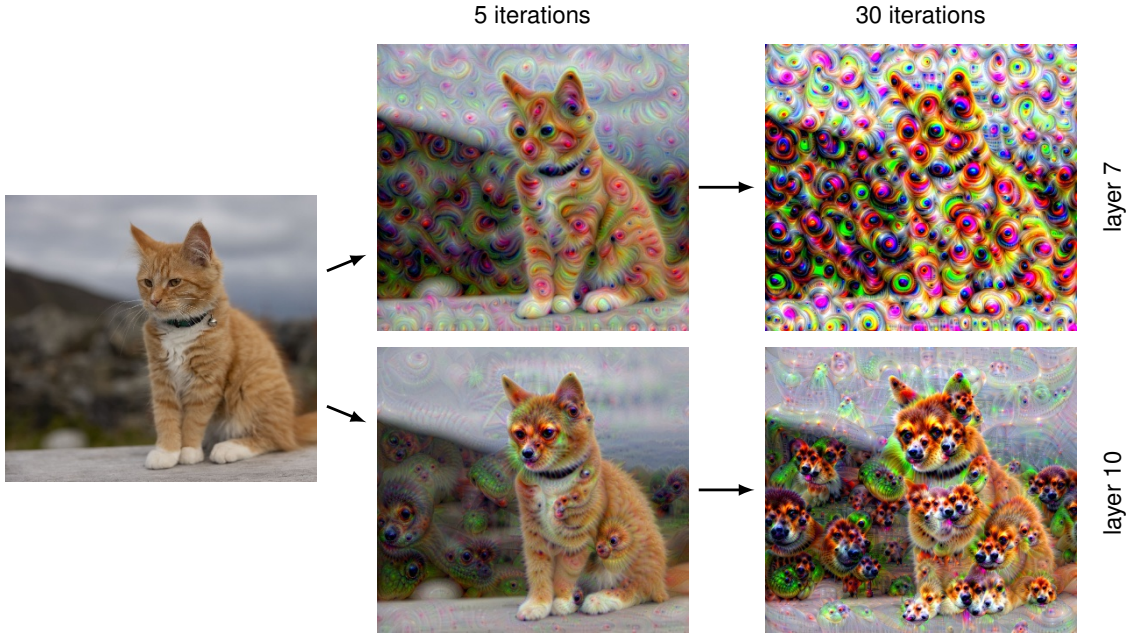


Figure 10.18 Examples of DeepDream applied to an image. The top row shows outputs when the algorithm is applied using the activations from the 7th convolutional layer of the VGG-16 network after five iterations and after 30 iterations. Similarly, the bottom row shows examples using the 10th layer, again after five iterations and after 30 iterations.

Section 6.3.4 CNN can be fine-tuned for a wide range of specific tasks. We have already seen an example of a convolutional network trained on ImageNet data, which through transfer learning was able to achieve human-level performance on skin lesion classi-
Section 1.1.1 fication.

10.4.1 Bounding boxes

Many images have multiple objects belonging to one or more classes, and we may wish to detect the presence and class of each object. Moreover, in many applications of computer vision we also need to determine the locations within the image of any objects that are detected. For example, an autonomous vehicle that uses RGB cameras may need to detect the presence and location of pedestrians and also identify road signs, other vehicles, etc.

Consider the problem of specifying the location of an object in an image. A widely used approach is to define a *bounding box*, which consists of a rectangle that fits closely to the boundary of the object, as illustrated in Figure 10.19. The bounding box can be defined by the coordinates of its centre along with its width and height in the form of a vector $\mathbf{b} = (b_x, b_y, b_{\mathrm{W}}, b_{\mathrm{H}})$. Here the elements of $\mathbf{b}$ can be specified in terms of pixels or as continuous numbers where, by convention, the top left of the image is given coordinates $(0, 0)$ and the bottom right is given coordinates $(1, 1)$.

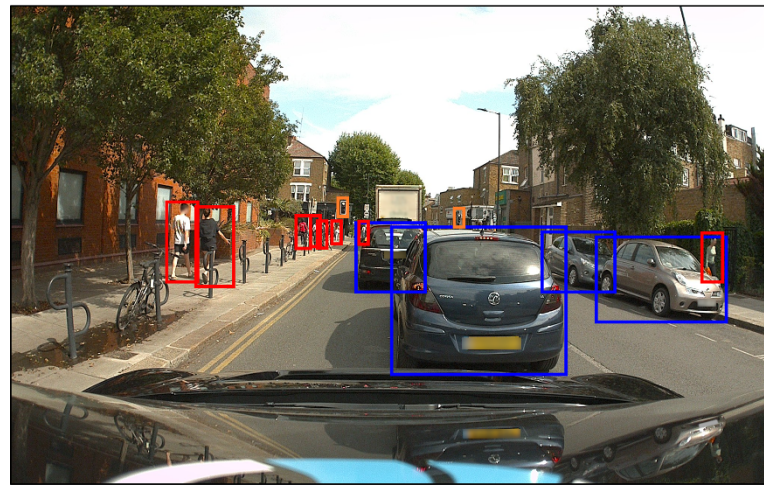


Figure 10.19 An image containing several objects from different classes in which the location of each object is labelled by a close-fitting rectangle known as a bounding box. Here blue boxes correspond to the class 'car', red boxes to the class 'pedestrian', and orange boxes to the class 'traffic light'. [Original image courtesy of Wayve Technologies Ltd.]

When images are assumed to contain one, and only one, object drawn from a predefined set of C classes, a CNN will generally have C output units whose activation functions are defined by the softmax function. An object can be localized by using an additional four outputs, with linear activation functions trained to predict the bounding box coordinates $(b_x, b_y, b_{\mathrm{W}}, b_{\mathrm{H}})$. Since these quantities are continuous, a sum-of-squares error function over the corresponding outputs may be appropriate. This is used for example by Redmon *et al.* (2015), who first divide the image into a 7×7 grid. For each grid cell, they use a convolutional network to output the class and bounding box coordinates of any object associated with that grid cell, based on features taken from the whole image.

Section 5.3

10.4.2 Intersection-over-union

We need a meaningful way to measure the performance of a trained network that can predict bounding boxes. In image classification the output of the network is a probability distribution over class labels, and we can measure performance by looking at the log likelihood for the true class labels on a test set. For object localization, however, we need some way to measure the accuracy of a bounding box relative to some ground truth, where the latter could, for example, be obtained by human labelling. The extent to which the predicted and target boxes overlap can be used as the basis for such a measure, but the area of the overlap will depend on the size of the object within the image. Also a predicted bounding box should be be penalized for the region of the prediction that lies outside the ground truth bounding box. A better metric that addresses both of these issues is called *intersection-over-union*, or IoU, and is simply the ratio of the area of the intersection of the two bounding boxes

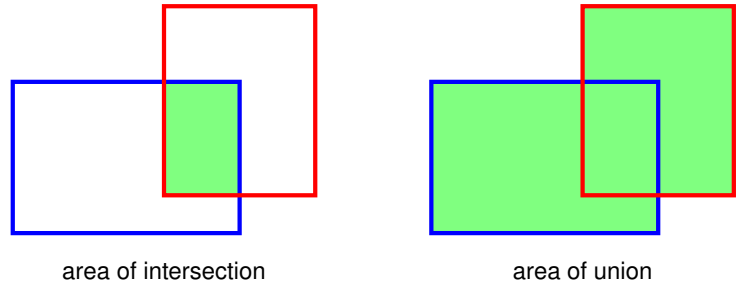


Figure 10.20 Illustration of the intersection-over-union metric for quantifying the accuracy of a bounding box prediction. If the predicted bounding box is shown by the blue rectangle and the ground truth by the red rectangle, then the intersection-over-union is defined as the ratio of the area of the intersection of the boxes, shown in green on the left, divided by the area of their union, shown in green on the right.

to that of their union, as illustrated in Figure 10.20. Note that the IoU measure lies in the range 0 to 1. Predictions can be labelled as correct if the IoU measure exceeds a threshold, which is typically set at 0.5. Note that IoU is not generally used directly as a loss function for training as it is hard to optimize by gradient descent, and so training is typically performed using centred objects, and the IoU score is mainly used an evaluation metric.

10.4.3 Sliding windows

One approach to object detection and object localization starts by creating a training set consisting of tightly cropped examples of the object to be detected, as well as examples of similarly cropped sections of images that do not contain any object (the 'background' class). This data set is used to train a classifier, such as a deep CNN, whose outputs represent the probability of there being an object of each particular class in the input window. The trained model is then used to detect objects in a new image by 'scanning' an input window across the image and, for each location, taking the resulting subset of the image as input to the classifier. This is called a *sliding window*. When an object is detected with high probability, the associated window location then defines the corresponding bounding box.

One obvious drawback of this approach is that it can be computationally very costly due to the large number of potential window positions in the image. Furthermore, the process may have to be repeated using windows of various scales to allow for different sizes of object within the image. A cost saving can be made by moving the input window in strides across the image, both horizontally and vertically, which are larger than one pixel. However, there is a trade-off between precision of location using a small stride and reducing the computational cost by using a larger stride. The computational cost of a sliding window approach may be reasonable for simple classifiers, but for deep neural networks potentially containing millions of parameters, the cost of a naive implementation can be prohibitive.

Fortunately, the convolutional structure of the neural network allows for a dramatic improvement in efficiency (Sermanet *et al.*, 2013). We note that a convolutional layer within such a network itself involves sliding a feature detector, with shared weights, across the input image in strides. Consequently, when a sliding window is used to generate multiple forward passes through a convolutional network

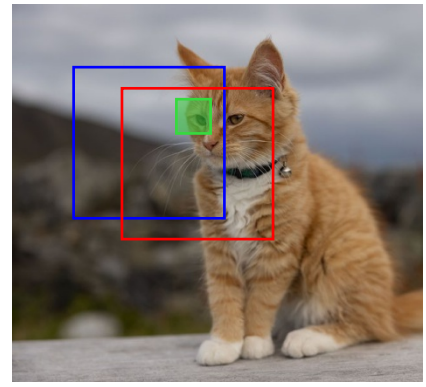


Figure 10.21 Illustration of replicated calculations when a CNN is used to process data from a sliding input window, in which the red and blue boxes show two overlapping locations for the input window. The green box represents one of the locations for the receptive field of a hidden unit in the first convolutional layer, and the evaluation of the corresponding hidden-unit activation is shared across the two window positions.

there is substantial redundancy in the computation, as illustrated in Figure 10.21.

Because the computational structure of sliding windows mirrors that of convolutions, it turns out to be remarkably simple to implement sliding windows efficiently in a convolutional network. Consider the simplified convolutional network in Figure 10.22, which consists of a convolutional layer followed by a max-pooling layer followed by a fully connected layer. For simplicity we have shown only a single channel in each layer, but the extension to multiple channels is straightforward. The input image to the network has size 6×6, the filters in the convolutional layer have size 3×3 with stride 1, and the max-pooling layer has non-overlapping receptive fields of size 2×2 with stride 1. This is followed by a fully connected layer with a single output unit. Note that we can also view this final layer as another convolutional layer with a filter size that is 2×2, so that there is only a single position for the filter and hence a single output.

Now suppose this network is trained on centred images of objects and then applied to a larger image of size 8×8, as shown in Figure 10.23 in which we simply enlarge the network by increasing the size of the convolutional and max-pooling layers. The convolution layer now has size 6×6 and the pooling layer has size

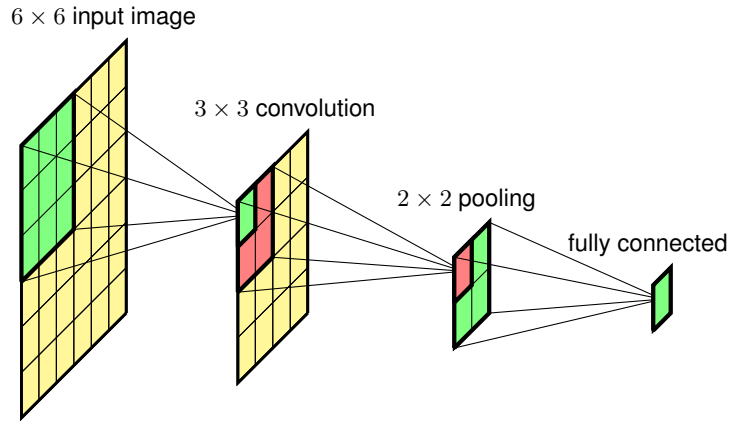


Figure 10.22 Example of a simple convolutional network having a single channel at each layer used to illustrate the concept of a sliding window for detecting objects in images.

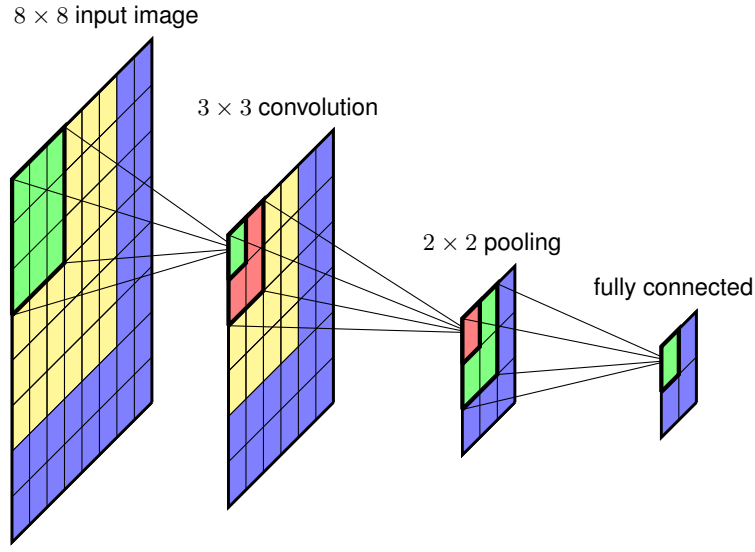


Figure 10.23 Application of the network shown in Figure 10.22 to a larger image in which the additional computation required corresponds to the blue regions.

3×3. There are now four output units each of which has its own softmax function. The weights into this unit are shared across the four units. We see that the calculations needed to process the input corresponding to a window position in the top left corner of the input image are the same as those used to process the original 6×6 inputs used in training. For the remaining window positions, only a small amount of additional computation is needed, as indicated by the blue squares, leading to a significant increase in efficiency compared to a naive repeated application of the full convolutional network. Note that the fully connected layers themselves now have a convolutional structure.

Exercise 10.12

10.4.4 Detection across scales

As well as looking for objects in different positions in the image, we also need to look for objects at different scales and at different aspect ratios. For example, a tight bounding box drawn around a cat will have a different aspect ratio when the cat is sitting upright compared to when it is lying down. Instead of using multiple detectors with different sizes and shapes of input window, it is simpler but equivalent to use a fixed input window and to make multiple copies of the input image each with a different pair of horizontal and vertical scaling factors. The input window is then scanned over each of the image copies to detect objects, and the associated scaling factors are then used to transform the bounding box coordinates back into the original image space, as illustrated in Figure 10.24.



Figure 10.24 Illustration of the detection and localization of objects at multiple scales and aspect ratios using a fixed input window. The original image (a) is replicated multiple times and each copy is scaled in the horizontal and/or vertical directions, as illustrated for a horizontal scaling in (b). A fixed-sized window is then scanned over the scaled images. When an object is detected with high probability, as illustrated by the red box in (b), the corresponding window coordinates can be projected back into the original image space to determine the corresponding bounding box as shown in (c).

10.4.5 Non-max suppression

By scanning a trained convolutional network over an image, it is possible to detect multiple instances of the same class of object within the image as well as instances of objects from other classes. However, this also tends to produce multiple detections of the same object at similar locations, as illustrated in Figure 10.25. This can be addressed using *non-max suppression*, which, for each object class in turn, works as follows. It first runs the sliding window over the whole image and evaluates the probability of an object of that class being present at each location. Next it eliminates all the associated bounding boxes whose probability is below some threshold, say 0.7, giving a result of the kind illustrated in Figure 10.25. The box with the highest probability is considered to be a successful detection, and the corresponding bounding box is recorded as a prediction. Next, any other boxes whose IoU with the successful detection box exceeds some threshold, say 0.5, is discarded. This is intended to eliminate multiple nearby detections of the same object. Then of the remaining boxes, the one with the highest probability is declared to be another successful detection, and the elimination step is repeated. The process continues until all bounding boxes have either been discarded or declared as successful detections.

10.4.6 Fast region CNNs

Another way to speed up object detection and localization is to note that a scanning window approach applies the full power of a deep convolutional network to all areas of the image, even though some areas may be unlikely to contain an object. Instead, we can apply some form of computationally cheaper technique, for example a segmentation algorithm, to identify parts of the image where there is a higher probability of finding an object, and then apply the full network only to these areas, leading to techniques such as *fast region proposals with CNN* or *fast R-CNN*

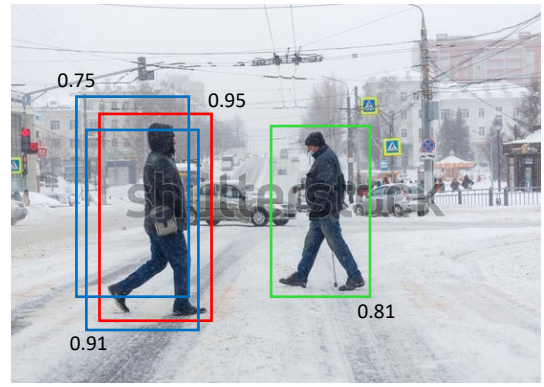


Figure 10.25 Schematic illustration of multiple detections of the same object at nearby locations, along with their associated probabilities. The red bounding box corresponds to the highest overall probability. Non-max suppression eliminates the other overlapping candidate bounding boxes shown in blue, while preserving the detection of another instance of the same object class shown by the bounding box in green.

(Girshick, 2015). It is also possible to use a *region proposal* convolutional network to identify the most promising regions, leading to *faster R-CNN* (Ren *et al.*, 2015), which allows end-to-end training of both the region proposal network and the detection and localization network.

A disadvantage of the sliding window approach is that if we want a very precise localization the objects then we need to consider large numbers of finely spaced window positions, which becomes computationally costly. A more efficient approach is to combine sliding windows with the direct bounding box predictions that we discussed at the start of this section (Sermanet *et al.*, 2013). In this case, the continuous outputs predict the position of the bounding box relative to the window position and therefore provide some fine-tuning to the predicted position.

10.5. Image Segmentation

Section 10.4

In an image classification problem, an entire image is assigned to a single class label. We have seen that more detailed information is provided if multiple objects are detected and their positions recorded using bounding boxes. An even more detailed analysis is obtained with *semantic segmentation* in which every pixel of an image is assigned to one of a predefined set of classes. This means that the output space will have the same dimensionality as the input image and can therefore be conveniently represented as an image with the same number of pixels. Although the input image will generally have three channels for R, G, and B, the output array will have C channels, if there are C classes, representing the probability for each class. If we associate a different (arbitrarily chosen) colour with each class, then the prediction of a segmentation network can be represented as an image in which each pixel is coloured according to the class having the highest probability, as illustrated in Figure 10.26.

10.5.1 Convolutional segmentation

A simple way to approach a semantic segmentation problem would be to construct a convolutional network that takes as input a rectangular section of the image



Figure 10.26 Example of an image and its corresponding semantic segmentation in which each pixel is coloured according to its class. For example, blue pixels correspond to the class 'car', red pixels to the class 'pedestrian', and orange pixels to the class 'traffic light'. [Courtesy of Wayve Technologies Ltd.]

centred on a pixel and that has a single softmax output that classifies that pixel. By applying such a network to each pixel in turn, the entire image can be segmented (this would require edge padding around the image depending on the size of the input window). However, this approach would be extremely inefficient due to redundant calculations caused by overlapping patches. As we have seen, we can remove this inefficiency by grouping together the forward-pass calculations for different input locations into a single network, which results in a model in which the final fully connected layers are also convolutional. We could therefore create a CNN in which each layer has the same dimensionality as the input image, by having stride 1 at each layer with same padding and no pooling. Each output unit has a softmax activation function with weights that are shared across all outputs. Although this could work, such a network would still need many layers, with multiple channels in each layer, to learn the complex internal representations needed to achieve high accuracy, and overall this would be prohibitively costly for images of reasonable resolution.

Figure 10.21

Figure 10.23

10.5.2 Up-sampling

As we have already seen, most convolutional networks use several levels of down-sampling so that as the number of channels increases, the size of the feature maps decreases, keeping the overall size and cost of the network tractable, while allowing the network to extract semantically meaningful high-order features from the image. We can use this concept to create a more efficient architecture for semantic segmentation by taking a standard deep convolutional network and adding additional learnable layers that take the low-dimensional internal representation and transform it back up to the original image resolution (Long, Shelhamer, and Darrell, 2014; Noh, Hong, and Han, 2015; Badrinarayanan, Kendall, and Cipolla, 2015), as illustrated in Figure 10.27.

To do this we need a way to reverse the down-sampling effects of strided convolutions and pooling operations. Consider first the up-sampling analogue of pooling, where the output layer has a larger number of units than the input layer, for example

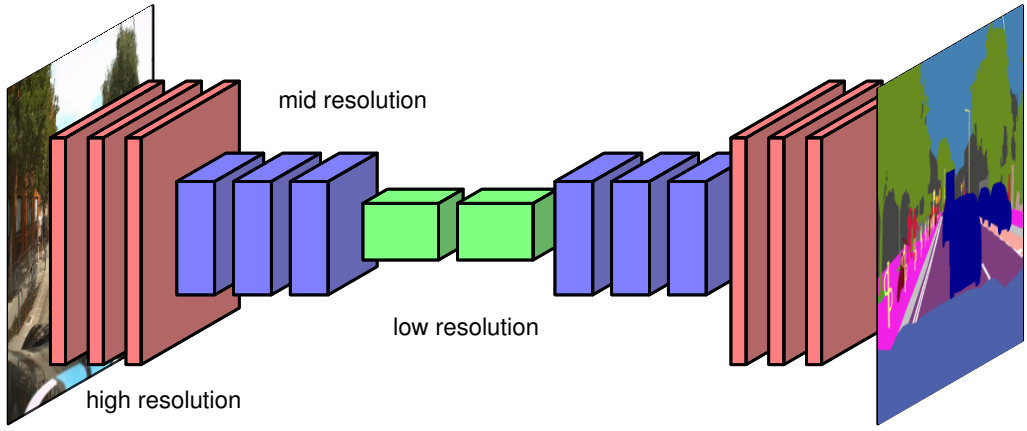


Figure 10.27 Illustration of a convolutional neural network used for semantic image segmentation, showing the reduction in the dimensionality of the feature maps through a series of strided convolutions and/or pooling operations, followed by a series of transpose convolutions and/or unpooling which increase the dimensionality back up to that of the original image.

with each input unit corresponding to a 2×2 block of output units. The question is then what values to use for the outputs. To find an up-sampling analogue of average pooling, we can simply copy over each input value into all the corresponding output units, as shown in Figure 10.28(a). We see that applying average pooling to the output of this operation regenerates the input.

For max-pooling, we can consider the operation shown in Figure 10.28(b) in which each input value is copied into the first unit of the corresponding output block, and the remaining values in each block are set to zero. Again we see that applying a max-pooling operation to the output layer regenerates the input layer. This is sometimes called *max-unpooling*. Assigning the non-zero value to the first element of the output block seems arbitrary, and so a modified approach can be used that also preserves more of the spatial information from the down-sampling layers (Badrinarayanan, Kendall, and Cipolla, 2015). This is done by choosing a network architecture in which each max-pooling down-sampling layer has a corresponding

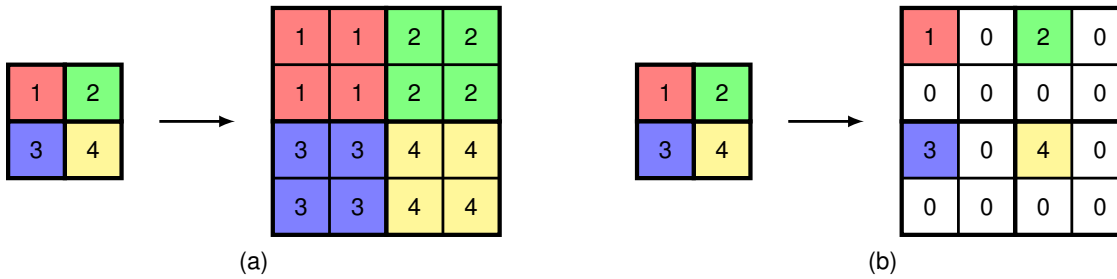


Figure 10.28 Illustration of unpooling operations showing (a) an analogue of average pooling and (b) an analogue of max-pooling.

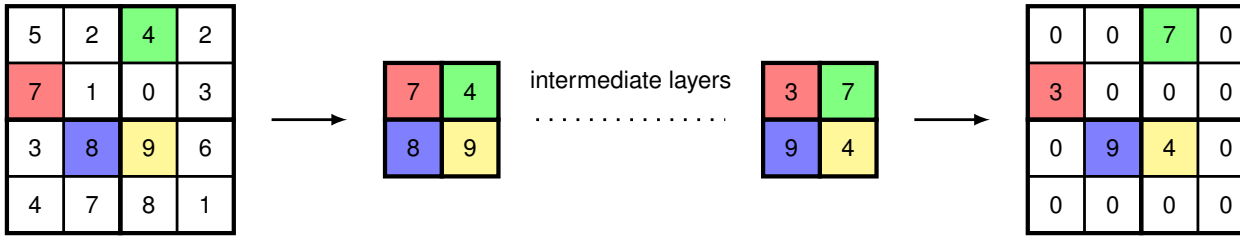


Figure 10.29 Some of the spatial information from a max-pooling layer, shown on the left, can be preserved by noting the location of the maximum value for each 2×2 block in the input array, and then in the corresponding up-sampling layer, placing the non-zero entry at the corresponding location in the output array.

up-sampling layer later in the network. Then during down-sampling, a record is kept of which element in each block had the maximum value, and then in the corresponding up-sampling layer, the non-zero element is chosen to have the same location, as illustrated for 2×2 max-pooling in Figure 10.29.

10.5.3 Fully convolutional networks

The up-sampling methods considered above are fixed functions, much like the average-pooling and max-pooling down-sampling operations. We can also use a *learned* up-sampling that is analogous to strided convolution for down-sampling. In strided convolution, each unit on the output map is connected via shared learnable weights to a small patch on the input map, and as we move one step through the output array, the filter is moved two or more steps across the input array, and hence the output array has lower dimensionality than the input array. For up-sampling, we use a filter that connects one pixel in the input array to a patch in the output array, and then chose the architecture so that as we move one step across the input array, we move two or more steps across the output array (Dumoulin and Visin, 2016). This is illustrated for 3×3 filters, and an output stride of 2, in Figure 10.30. Note that there are output cells for which multiple filter positions overlap, and the corresponding output values can be found either by summing or by averaging the contributions from the individual filter positions.

Exercise 10.13

This up-sampling is called *transpose convolution* because, if the down-sampling convolution is expressed in matrix form, the corresponding up-sampling is given by the transpose matrix. It is also called 'fractionally strided convolution' because the stride of a standard convolution is the ratio of the step size in the output layer to the step size in the input layer. In Figure 10.30, for example, this ratio is $1/2$. Note that this is sometimes also referred to as 'deconvolution', but it is better to avoid this term since deconvolution is widely used in mathematics to mean the inverse of the operation of convolution used in functional analysis, which is a different concept. If we have a network architecture with no pooling layers, so that the down-sampling and up-sampling are done purely using convolutions, then the architecture is known as a *fully convolutional network* (Long, Shelhamer, and Darrell, 2014). It can take an arbitrarily sized image and will output a segmentation map of the same size.

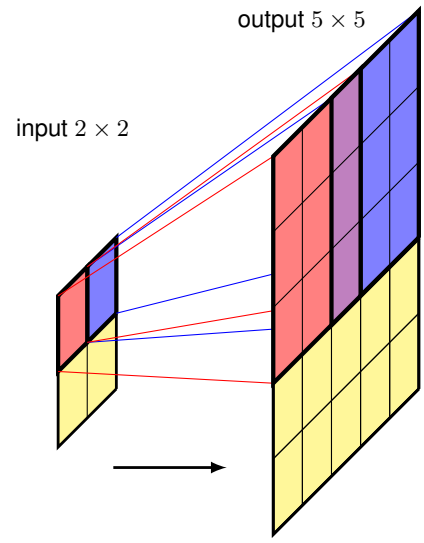


Figure 10.30 Illustration of transpose convolution for a 3×3 filter with an output stride of 2. This can be thought of as the inverse operation to a 3×3 convolution. The red output patch is given by multiplying the kernel by the activation of the red unit in the input layer, and similarly for the blue output patch. The activations of cells for which patches overlap are calculated by summing or averaging the contributions from the individual patches.

10.5.4 The U-net architecture

We have seen that the down-sampling associated with strided convolutions and pooling allows the number of channels to be increased without the size of the network becoming prohibitive. This also has the effect of reducing the spatial resolution and hence discarding positional information as the signals flow through the network. Although this is fine for image classification, the loss of spatial information is a problem for semantic segmentation as we want to classify each pixel. One approach for addressing this is the *U-net* architecture (Ronneberger, Fischer, and Brox, 2015) illustrated in Figure 10.31, where the name comes from the U-shape of the diagram.

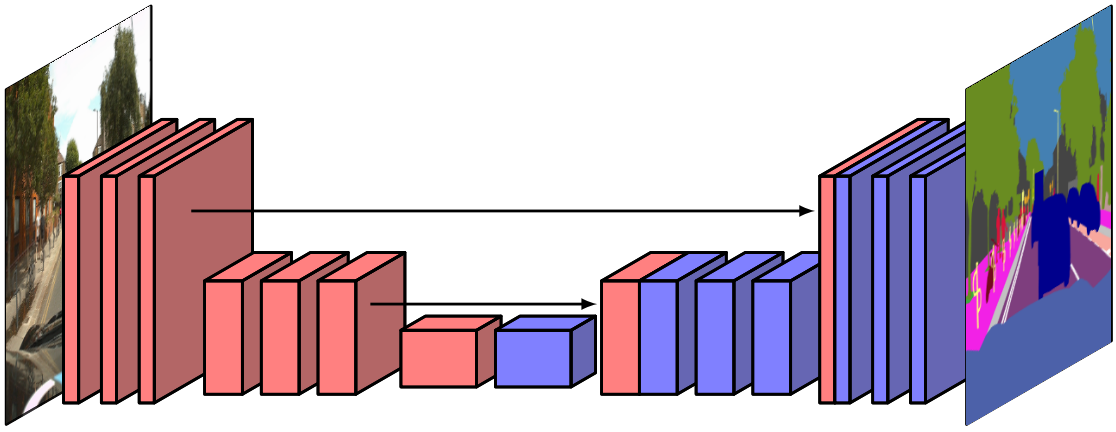


Figure 10.31 The U-net architecture has a symmetrical arrangement of down-sampling and up-sampling layers, and the output from each down-sampling layer is concatenated with the corresponding up-sampling layer.



Figure 10.32 An example of neural style transfer showing a photograph of a canal scene (left) that has been rendered in the style of *The Wreck of a Transport Ship* by J. M. W. Turner (centre) and in the style of *The Starry Night* by Vincent van Gogh (right). In each case the image used to provide the style is shown in the inset. [From Gatys, Ecker, and Bethge (2015) with permission.]

The core concept is that for each down-sampling layer there is a corresponding up-sampling layer, and the final set of channel activations at each down-sampling layer is concatenated with the corresponding first set of channels in the up-sampling layer, thereby giving those layers access to higher-resolution spatial information. Note that 1×1 convolutions may be used in the final layer of a U-net to reduce the number of channels down to the number of classes, which is then followed by a softmax activation function.

10.6. Style Transfer

Section 10.3

As we have seen, early layers in a deep convolutional network learn to detect simple features such as edges and textures whereas later layers learn to detect more complex entities such as objects. We can exploit this property to re-render an image in the style of a different image using a process called *neural style transfer* (Gatys, Ecker, and Bethge, 2015). This is illustrated in Figure 10.32.

Our goal is to generate a synthetic image $\mathbf{G}$ whose 'content' is defined by an image $\mathbf{C}$ and whose 'style' is taken from some other image $\mathbf{S}$. This is achieved by defining an error function $E(\mathbf{G})$ given by the sum of two terms, one of which encourages $\mathbf{G}$ to have a similar content to $\mathbf{C}$ whereas the other encourages $\mathbf{G}$ to have a similar style to $\mathbf{S}$:

$$E(\mathbf{G}) = E_{\text{content}}(\mathbf{G}, \mathbf{C}) + E_{\text{style}}(\mathbf{G}, \mathbf{S}). \tag{10.13}$$

The concepts of content and style are defined implicitly by the functional forms of these two terms. We can then find $\mathbf{G}$ by starting from a randomly initialized image and using gradient descent to minimize $E(\mathbf{G})$.

To define $E_{\text{content}}(\mathbf{G}, \mathbf{C})$, we can pick a particular convolutional layer in the network and measure the activations of the units in that layer when image $\mathbf{G}$ is used as input and also when image $\mathbf{C}$ is used as input. We can then encourage the

corresponding pre-activations to be similar by using a sum-of-squares error function of the form

$$E_{\text{content}}(\mathbf{G}, \mathbf{C}) = \sum_{i,j,k} \{a_{ijk}(\mathbf{G}) - a_{ijk}(\mathbf{C})\}^2 \tag{10.14}$$

where $a_{ijk}(\mathbf{G})$ denotes the pre-activation of the unit at position (i, j) in channel k of that layer when the input image is $\mathbf{G}$, and similarly for $a_{ijk}(\mathbf{C})$. The choice of which layer to use in defining the pre-activations will influence the final result, with earlier layers aiming to match low-level features like edges and later layers matching more complex structures or even entire objects.

In defining $E_{\text{style}}(\mathbf{G}, \mathbf{C})$, the intuition is that style is determined by the co-occurrence of features from different channels within a convolutional layer. For example, if the style image $\mathbf{S}$ is such that vertical edges are generally associated with orange blobs, then we would like the same to be true for the generated image $\mathbf{G}$. However, although $E_{\text{content}}(\mathbf{G}, \mathbf{C})$ tries to match features in $\mathbf{G}$ at the same locations as corresponding features in $\mathbf{C}$, for the style error $E_{\text{style}}(\mathbf{G}, \mathbf{S})$ we want $\mathbf{G}$ to have characteristics that match those of $\mathbf{S}$ but taken from any location, and so we take an average over locations in a feature map. Again, consider a particular convolutional layer. We can measure the extent to which a feature in channel k co-occurs with the corresponding feature in channel k' for input image $\mathbf{G}$ by forming the cross-correlation matrix

$$F_{kk'}(\mathbf{G}) = \sum_{i=1}^{I} \sum_{j=1}^{J} a_{ijk}(\mathbf{G}) a_{ijk'}(\mathbf{G}) \tag{10.15}$$

where I and J are the dimensions of the feature maps in this particular convolutional layer, and the product $a_{ijk}a_{ijk'}$ will be large if both features are activated. If there are K channels in this layer, then $F_{kk'}$ form the elements of a $K \times K$ matrix, called the *style matrix*. We can measure the extent to which the two images $\mathbf{G}$ and $\mathbf{S}$ have the same style by comparing their style matrices using

$$E_{\text{style}}(\mathbf{G}, \mathbf{S}) = \frac{1}{(2IJK)^2} \sum_{k=1}^{K} \sum_{k'=1}^{K} \{F_{kk'}(\mathbf{G}) - F_{kk'}(\mathbf{S})\}^2 . \tag{10.16}$$

Although we could again make use of a single layer, more pleasing results are obtained by using contributions from multiple layers in the form

$$E_{\text{style}}(\mathbf{G}, \mathbf{S}) = \sum_{l} \lambda_l E^{(l)}_{\text{style}}(\mathbf{G}, \mathbf{S}) \tag{10.17}$$

where l denotes the convolutional layer. The coefficients λ_l determine the relative weighting between the different layers and also the weighting relative to the content error term. These weighting coefficients are adjusted empirically using subjective judgement.

Exercises

10.1 ($\star$) Consider a fixed weight vector $\mathbf{w}$ and show that the input vector $\mathbf{x}$ that maximizes the scalar product $\mathbf{w}^{\mathrm{T}}\mathbf{x}$, subject to the constraint that $\|\mathbf{x}\|^2$ is constant, is given by $\mathbf{x} = \alpha\mathbf{w}$ for some scalar α. This can most easily be done using a Lagrange multiplier.

Appendix C

10.2 ($\star\star$) Consider a convolutional network layer with a one-dimensional input array and a one-dimensional feature map as shown in Figure 10.2, in which the input array has dimensionality 5 and the filters have width 3 with a stride of 1. Show that this can be expressed as a special case of a fully connected layer by writing down the weight matrix in which missing connections are replaced by zeros and where shared parameters are indicated by using replicated entries. Ignore any bias parameters.

10.3 ($\star$) Explicitly calculate the output of the following convolution of a 4×4 input matrix with a 2×2 filter:

$$\begin{array}{|c|c|c|c|} \hline 2 & 5 & -3 & 0 \\ \hline 0 & 6 & 0 & -4 \\ \hline -1 & -3 & 0 & 2 \\ \hline 5 & 0 & 0 & 3 \\ \hline \end{array} \quad * \quad \begin{array}{|c|c|} \hline -2 & 0 \\ \hline 4 & 6 \\ \hline \end{array} \quad = \quad \begin{array}{|c|c|c|} \hline ? & ? & ? \\ \hline ? & ? & ? \\ \hline ? & ? & ? \\ \hline \end{array} \tag{10.18}$$

10.4 ($\star\star$) If an image $\mathbf{I}$ has $J \times K$ pixels and a filter $\mathbf{K}$ has $L \times M$ elements, write down the limits for the two summations in (10.2). In the mathematics literature, the operation (10.2) would be called a *cross-correlation*, whereas a *convolution* would be defined by

$$C(j,k) = \sum_{l}\sum_{m} I(j-l, k-m)K(l,m). \tag{10.19}$$

Write down the limits for the summations in (10.19). Show that (10.19) can be written in the equivalent 'flipped' form

$$C(j,k) = \sum_{l}\sum_{m} I(j+l, k+m)K(l,m) \tag{10.20}$$

and again write down the limits for the summations.

10.5 ($\star$) In mathematics, a convolution for a continuous variable x is defined by

$$F(x) = \int_{-\infty}^{\infty} G(y)k(x-y)\,\mathrm{d}y \tag{10.21}$$

where $k(x-y)$ is the kernel function. By considering a discrete approximation to the integral, explain the relationship to a convolutional layer, defined by (10.19), in a CNN.

10.6 ($\star$) Consider an image of size $J \times K$ that is padded with an additional P pixels on all sides and which is then convolved using a kernel of size $M \times M$ where M is an odd number. Show that if we choose $P = (M-1)/2$, then the resulting feature map will have size $J \times K$ and hence will be the same size as the original image.

10.7 ($\star$) Show that if a kernel of size $M \times M$ is convolved with an image of size $J \times K$ with padding of depth P and strides of length S then the dimensionality of the resulting feature map is given by (10.5)

10.8 ($\star\star$) For each of the 16 layers in the VGG-16 CNN shown in Figure 10.10, evaluate (i) the number of weights (i.e., connections) including biases and (ii) the number of independently learnable parameters. Confirm that the total number of learnable parameters in the network is approximately 138 million.

10.9 ($\star\star$) Consider a convolution of the form (10.2) and suppose that the kernel is separable so that

$$K(l, m) = F(l)G(m) \tag{10.22}$$

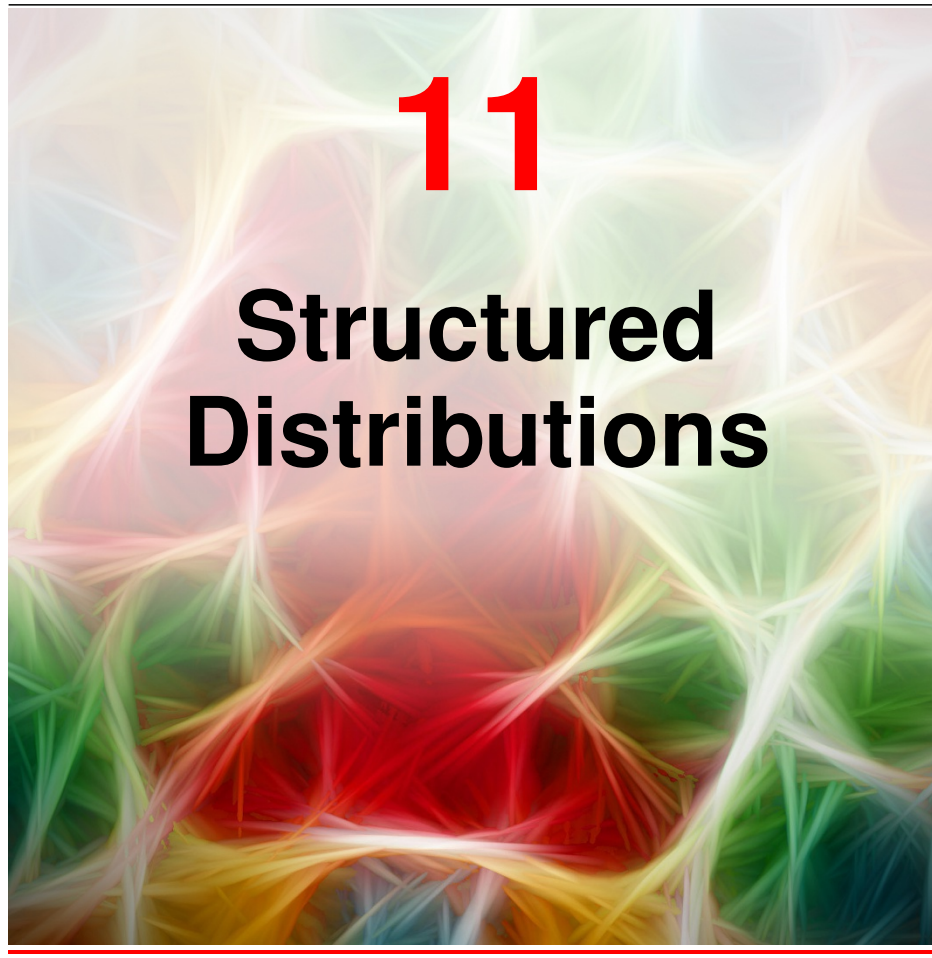
for some functions $F(\cdot)$ and $G(\cdot)$. Show that instead of performing a single two-dimensional convolution it is now possible to compute the same answer using two one-dimensional convolutions thereby resulting in a significant improvement in efficiency.

10.10 ($\star$) The DeepDream update procedure involves setting the δ variables for backpropagation equal to the pre-activations of the nodes in the chosen layer and then running backpropagation to the input layer to get a gradient vector over the pixels of the image. Show that this can be derived as a gradient optimization with respect to the pixels of an image $\mathbf{I}$ applied to the function (10.12).

10.11 ($\star\star$) When designing a neural network to detect objects from C different classes in an image, we can use a 1-of-$(C+1)$ class label with one variable for each object class and one additional variable representing a 'background' class, i.e., an input image region that does not contain an object belonging to any of the defined classes. The network will then output a vector of probabilities of length $(C + 1)$. Alternatively, we can use a single binary variable to denote the presence or absence of an object and then use a separate 1-of-C vector to denote the specific object class. In this case, the network outputs a single probability representing the presence of an object and a separate set of probabilities over the class label. Write down the relationship between these two sets of probabilities.

10.12 ($\star\star$) Calculate the number of computational steps required to make one forward pass through the convolutional network shown in Figure 10.22, ignoring biases and ignoring the evaluation of activation functions. Similarly, calculate the total number of computational steps for a single forward pass through the expanded network shown in Figure 10.23. Finally, evaluate ratio of nine repeated naive applications of the network in Figure 10.22 to an 8×8 image compared to a single application of the network in Figure 10.23. This ratio indicates the improvement in efficiency from using a convolutional implementation of the sliding window technique.

10.13 (⋆⋆) In this exercise we use one-dimensional vectors to demonstrate why a convolutional up-sampling is sometimes called a transpose convolution. Consider a one-dimensional strided convolutional layer with an input having four units with activations (x_1, x_2, x_3, x_4), which is padded with zeros to give $(0, x_1, x_2, x_3, x_4, 0)$, and a filter with parameters (w_1, w_2, w_3). Write down the one-dimensional activation vector of the output layer assuming a stride of 2. Express this output in the form of a matrix $\mathbf{A}$ multiplied by the vector $(0, x_1, x_2, x_3, x_4, 0)$. Now consider an up-sampling convolution in which the input layer has activations (z_1, z_2) with a filter having values (w_1, w_2, w_3) and an output stride of 2. Write down the six-dimensional output vector assuming that overlapping filter values are summed and that the activation function is just the identity. Show that this can be expressed as a matrix multiplication using the transpose matrix $\mathbf{A}^{\mathrm{T}}$.



11 Structured Distributions

We have seen that probability forms one of the most important foundational concepts for deep learning. For example, a neural network used for binary classification is described by a conditional probability distribution of the form

$$p(t|\mathbf{x}, \mathbf{w}) = y(\mathbf{x}, \mathbf{w})^t \{1 - y(\mathbf{x}, \mathbf{w})\}^{(1-t)} \tag{11.1}$$

where $y(\mathbf{x}, \mathbf{w})$ represents a neural network function that takes a vector $\mathbf{x}$ as input and is governed by a vector $\mathbf{w}$ of learnable parameters. The corresponding cross-entropy likelihood forms the basis for defining an error function used to train the neural network. Although the network function might be extremely complex, the conditional distribution in (11.1) has a simple form. However, there are many important deep learning models that have a much richer probabilistic structure, such as large language models, normalizing flows, variational autoencoders, diffusion models, and many others. To describe and exploit this structure, we introduce a powerful

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024
C. M. Bishop, H. Bishop, *Deep Learning*, https://doi.org/10.1007/978-3-031-45468-4_11

framework called *probabilistic graphical models*, or simply *graphical models*, which allows structured probability distributions to be expressed in graphical form. When combined with neural networks to define associated probability distributions, graphical models offer huge flexibility when creating sophisticated models that can be trained end to end using stochastic gradient descent in which gradients are evaluated efficiently using auto-differentiation. In this chapter, we will focus on the core concepts of graphical models needed for applications in deep learning, whereas a more comprehensive treatment of graphical models for machine learning can be found in Bishop (2006).

11.1. Graphical Models

Section 2.1

Probability theory can be expressed in terms of two simple equations known as the *sum rule* and the *product rule*. All of the probabilistic manipulations discussed in this book, no matter how complex, amount to repeated application of these two equations. In principle, we could therefore formulate and use complex probabilistic models purely by using algebraic manipulation. However, we will find it advantageous to augment the analysis using diagrammatic representations of probability distributions, as these offer several useful properties:

1. They provide a simple way to visualize the structure of a probabilistic model and can be used to design and motivate new models.
2. Insights into the properties of the model, including conditional independence properties, can be obtained by inspecting the graph.
3. The complex computations required to perform inference and learning in sophisticated models can be expressed in terms of graphical operations, such as message-passing, in which the underlying mathematical operations are carried out implicitly.

Although such graphical models have nodes and edges much like neural network diagrams, their interpretation is specifically probabilistic and carries a richer semantics. To help avoid confusion, in this book we denote neural network diagrams in blue and probabilistic graphical models in red.

11.1.1 Directed graphs

A graph comprises *nodes*, also called *vertices*, connected by *links*, also known as *edges*. In a probabilistic graphical model, each node represents a random variable, and the links express probabilistic relationships between these variables. The graph then captures the way in which the joint distribution over all the random variables can be decomposed into a product of factors each depending only on a subset of the variables. In this chapter we will focus on graphical models in which the links of the graphs have a particular direction indicated by arrows. These are known as *directed graphical models* and are also called *Bayesian networks* or *Bayes nets*.

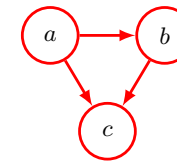


Figure 11.1 A directed graphical model representing the joint probability distribution over three variables a, b, and c, corresponding to the decomposition on the right-hand side of (11.3).

The other major class of graphical models are *Markov random fields*, also known as *undirected graphical models*, in which the links do not carry arrows and have no directional significance. Directed graphs are useful for expressing causal relationships between random variables, whereas undirected graphs are better suited to expressing soft constraints between random variables. Both directed and undirected graphs can be viewed as special cases of a representation called *factor graphs*. From now on we focus our attention on directed graphical models. Note, however, that undirected graphs, without the probabilistic interpretation, will also arise in our discussion of *graph neural networks* in which the nodes represent deterministic variables as in standard neural networks.

Chapter 13

11.1.2 Factorization

To motivate the use of directed graphs to describe probability distributions, consider first an arbitrary joint distribution $p(a, b, c)$ over three variables a, b, and c. Note that at this stage, we do not need to specify anything further about these variables, such as whether they are discrete or continuous. Indeed, one of the powerful aspects of graphical models is that a specific graph can make probabilistic statements for a broad class of distributions. By application of the product rule of probability (2.9), we can write the joint distribution in the form

$$p(a, b, c) = p(c|a, b)p(a, b). \tag{11.2}$$

A second application of the product rule, this time to the second term on the right-hand side of (11.2), gives

$$p(a, b, c) = p(c|a, b)p(b|a)p(a). \tag{11.3}$$

Note that this decomposition holds for any choice of the joint distribution. We now represent the right-hand side of (11.3) in terms of a simple graphical model as follows. First we introduce a node for each of the random variables a, b, and c and associate each node with the corresponding conditional distribution on the right-hand side of (11.3). Then, for each conditional distribution we add directed links (depicted as arrows) from the nodes corresponding to the variables on which the distribution is conditioned. Thus, for the factor $p(c|a, b)$, there will be links from nodes a and b to node c, whereas for the factor $p(a)$, there will be no incoming links. The result is the graph shown in Figure 11.1. If there is a link going from node a to node b, then we say that node a is the *parent* of node b, and we say that node b is the *child* of node a. Note that we will not make any formal distinction between a node and the variable to which it corresponds but will simply use the same symbol to refer to both.

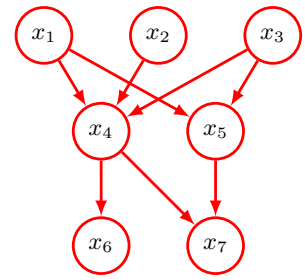


Figure 11.2 Example of a directed graph describing the joint distribution over variables $x_1, \ldots, x_7$. The corresponding decomposition of the joint distribution is given by (11.5).

An important point to note about (11.3) is that the left-hand side is symmetrical with respect to the three variables a, b, and c, whereas the right-hand side is not. In making the decomposition in (11.3), we have implicitly chosen a particular ordering, namely a, b, c, and had we chosen a different ordering we would have obtained a different decomposition and hence a different graphical representation.

For the moment let us extend the example of Figure 11.1 by considering the joint distribution over K variables given by $p(x_1, \ldots, x_K)$. By repeated application of the product rule of probability, this joint distribution can be written as a product of conditional distributions, one for each of the variables:

$$p(x_1, \ldots, x_K) = p(x_K|x_1, \ldots, x_{K-1}) \ldots p(x_2|x_1)p(x_1). \tag{11.4}$$

For a given choice of K, we can again represent this as a directed graph having K nodes, one for each conditional distribution on the right-hand side of (11.4), with each node having incoming links from all lower numbered nodes. We say that this graph is *fully connected* because there is a link between every pair of nodes.

So far, we have worked with completely general joint distributions, and so their factorization, and associated representation as fully connected graphs, will be applicable to any choice of distribution. As we will see shortly, it is the *absence* of links in the graph that conveys interesting information about the properties of the class of distributions that the graph represents. Consider the graph shown in Figure 11.2. Note that it is not a fully connected graph because, for instance, there is no link from x_1 to x_2 or from x_3 to x_7. We take this graph and extract the corresponding representation of the joint probability distribution written in terms of the product of a set of conditional distributions, one for each node in the graph. Each such conditional distribution will be conditioned only on the parents of the corresponding node in the graph. For instance, x_5 will be conditioned on x_1 and x_3. The joint distribution of all seven variables is therefore given by

$$p(x_1)p(x_2)p(x_3)p(x_4|x_1, x_2, x_3)p(x_5|x_1, x_3)p(x_6|x_4)p(x_7|x_4, x_5). \tag{11.5}$$

The reader should take a moment to study carefully the correspondence between (11.5) and Figure 11.2.

We can now state in general terms the relationship between a given directed graph and the corresponding distribution over the variables. The joint distribution defined by a graph is given by the product, over all of the nodes of the graph, of

a conditional distribution for each node conditioned on the variables corresponding to the parents of that node in the graph. Thus, for a graph with K nodes, the joint distribution is given by

$$p(x_1, \ldots, x_K) = \prod_{k=1}^{K} p(x_k|\mathrm{pa}(k)) \tag{11.6}$$

where $\mathrm{pa}(k)$ denotes the set of parents of x_k. This key equation expresses the *factorization* properties of the joint distribution for a directed graphical model. Although we have considered each node to correspond to a single variable, we can equally well associate sets of variables and vector-valued or tensor-valued variables with the nodes of a graph. It is easy to show that the representation on the right-hand side of (11.6) is always correctly normalized provided the individual conditional distributions are normalized.

Exercise 11.1

The directed graphs that we are considering are subject to an important restriction, namely that there must be no *directed cycles*. In other words, there are no closed paths within the graph such that we can move from node to node along links following the direction of the arrows and end up back at the starting node. Such graphs are also called *directed acyclic graphs*, or *DAGs*. This is equivalent to the statement that there exists an ordering of the nodes such that there are no links that go from any node to any lower-numbered node.

Exercise 11.2

11.1.3 Discrete variables

We have discussed the importance of probability distributions that are members of the exponential family, and we have seen that this family includes many well-known distributions as special cases. Although such distributions are relatively simple, they form useful building blocks for constructing more complex probability distributions, and the framework of graphical models is very useful in expressing the way in which these building blocks are linked together. There are two particular choices for the component distributions that are widely used, corresponding to discrete variables and to Gaussian variables. We begin by examining the discrete case.

Section 3.4

The probability distribution $p(\mathbf{x}|\boldsymbol{\mu})$ for a single discrete variable $\mathbf{x}$ having K possible states (using the 1-of-K representation) is given by

$$p(\mathbf{x}|\boldsymbol{\mu}) = \prod_{k=1}^{K} \mu_k^{x_k} \tag{11.7}$$

and is governed by the parameters $\boldsymbol{\mu} = (\mu_1, \ldots, \mu_K)^{\mathrm{T}}$. Due to the constraint $\sum_k \mu_k = 1$, only $K - 1$ values for μ_k need to be specified to define the distribution.

Now suppose that we have two discrete variables, $\mathbf{x}_1$ and $\mathbf{x}_2$, each of which has K states, and we wish to model their joint distribution. We denote the probability of observing both $x_{1k} = 1$ and $x_{2l} = 1$ by the parameter μ_{kl}, where x_{1k} denotes the

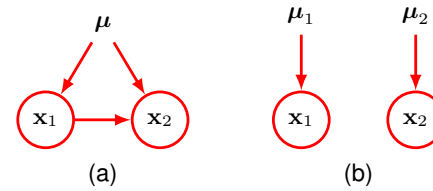


Figure 11.3 (a) This fully connected graph describes a general distribution over two K-state discrete variables having a total of $K^2 - 1$ parameters. (b) By dropping the link between the nodes, the number of parameters is reduced to $2(K - 1)$.

kth component of $\mathbf{x}_1$, and similarly for x_{2l}. The joint distribution can be written

$$p(\mathbf{x}_1, \mathbf{x}_2|\boldsymbol{\mu}) = \prod_{k=1}^{K}\prod_{l=1}^{K} \mu_{kl}^{x_{1k}x_{2l}}.$$

Because the parameters μ_{kl} are subject to the constraint $\sum_k \sum_l \mu_{kl} = 1$, this distribution is governed by $K^2 - 1$ parameters. It is easily seen that the total number of parameters that must be specified for an arbitrary joint distribution over M variables is $K^M - 1$ and therefore grows exponentially with the number M of variables.

Using the product rule, we can factor the joint distribution $p(\mathbf{x}_1, \mathbf{x}_2)$ in the form $p(\mathbf{x}_2|\mathbf{x}_1)p(\mathbf{x}_1)$, which corresponds to a two-node graph with a link going from the $\mathbf{x}_1$ node to the $\mathbf{x}_2$ node as shown in Figure 11.3(a). The marginal distribution $p(\mathbf{x}_1)$ is governed by $K - 1$ parameters, as before. Similarly, the conditional distribution $p(\mathbf{x}_2|\mathbf{x}_1)$ requires the specification of $K - 1$ parameters for each of the K possible values of $\mathbf{x}_1$. The total number of parameters that must be specified in the joint distribution is therefore $(K - 1) + K(K - 1) = K^2 - 1$ as before.

Now suppose that the variables $\mathbf{x}_1$ and $\mathbf{x}_2$ are independent, corresponding to the graphical model shown in Figure 11.3(b). Each variable is then described by a separate discrete distribution, and the total number of parameters would be $2(K-1)$. For a distribution over M independent discrete variables, each having K states, the total number of parameters would be $M(K - 1)$, which therefore grows linearly with the number of variables. From a graphical perspective, we have reduced the number of parameters by dropping links in the graph, at the expense of having a more restricted class of distributions.

More generally, if we have M discrete variables $\mathbf{x}_1, \ldots, \mathbf{x}_M$, we can model the joint distribution using a directed graph with one variable for each node. The conditional distribution at each node is given by a set of non-negative parameters subject to the usual normalization constraint. If the graph is fully connected, then we have a completely general distribution having $K^M - 1$ parameters, whereas if there are no links in the graph, the joint distribution factorizes into the product of the marginal distributions, and the total number of parameters is $M(K - 1)$. Graphs having intermediate levels of connectivity allow for more general distributions than the fully factorized one while requiring fewer parameters than the general joint distribution. As an illustration, consider the chain of nodes shown in Figure 11.4. The marginal distribution $p(\mathbf{x}_1)$ requires $K - 1$ parameters, whereas each of the $M - 1$ conditional distributions $p(\mathbf{x}_i|\mathbf{x}_{i-1})$, for $i = 2, \ldots, M$, requires $K(K - 1)$ parameters.

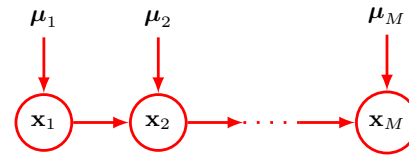


Figure 11.4 This chain of M discrete nodes, each having K states, requires the specification of $K-1+(M-1)K(K-1)$ parameters, which grows linearly with the length M of the chain. In contrast, a fully connected graph of M nodes would have K^M-1 parameters, which grows exponentially with M.

This gives a total parameter count of $K-1+(M-1)K(K-1)$, which is quadratic in K and which grows linearly (rather than exponentially) with the length M of the chain.

An alternative way to reduce the number of independent parameters in a model is by *sharing* parameters (also known as *tying* of parameters). For instance, in the chain example of Figure 11.4, we can arrange that all the conditional distributions $p(\mathbf{x}_i|\mathbf{x}_{i-1})$, for $i=2,\ldots,M$, are governed by the same set of $K(K-1)$ parameters, giving the model shown in Figure 11.5. Together with the $K-1$ parameters governing the distribution of $\mathbf{x}_1$, this gives a total of K^2-1 parameters that must be specified to define the joint distribution.

Another way of controlling the exponential growth of the number of parameters in models of discrete variables is to use parameterized representations for the conditional distributions instead of complete tables of conditional probability values. To illustrate this idea, consider the graph in Figure 11.6 in which all the nodes represent binary variables. Each of the parent variables x_i is governed by a single parameter μ_i representing the probability $p(x_i=1)$, giving M parameters in total for the parent nodes. The conditional distribution $p(y|x_1,\ldots,x_M)$, however, would require 2^M parameters representing the probability $p(y=1)$ for each of the 2^M possible settings of the parent variables. Thus, in general the number of parameters required to specify this conditional distribution will grow exponentially with M. We can obtain a more parsimonious form for the conditional distribution by using a logistic sigmoid function acting on a linear combination of the parent variables, giving

Section 3.4

$$p(y=1|x_1,\ldots,x_M)=\sigma\left(w_0+\sum_{i=1}^{M}w_ix_i\right)=\sigma(\mathbf{w}^{\mathrm{T}}\mathbf{x}) \tag{11.8}$$

where $\sigma(a)=(1+\exp(-a))^{-1}$ is the logistic sigmoid, $\mathbf{x}=(x_0,x_1,\ldots,x_M)^{\mathrm{T}}$ is an $(M+1)$-dimensional vector of parent states augmented with an additional variable x_0 whose value is clamped to 1, and $\mathbf{w}=(w_0,w_1,\ldots,w_M)^{\mathrm{T}}$ is a vector of $M+1$ parameters. This is a more restricted form of conditional distribution than the general case but is now governed by a number of parameters that grows linearly with M. In

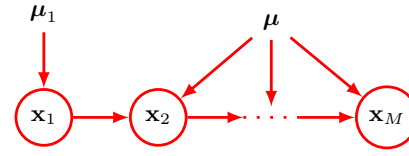


Figure 11.5 As in Figure 11.4 but with a single set of parameters $\boldsymbol{\mu}$ shared amongst all the conditional distributions $p(\mathbf{x}_i|\mathbf{x}_{i-1})$.

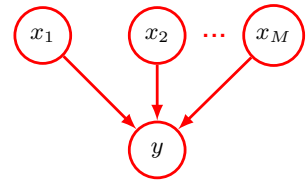


Figure 11.6 A graph comprising M parents $x_1, \ldots, x_M$ and a single child y, used to illustrate the idea of parameterized conditional distributions for discrete variables.

this sense, it is analogous to the choice of a restrictive form of covariance matrix (for example, a diagonal matrix) in a multivariate Gaussian distribution.

11.1.4 Gaussian variables

We now turn to graphical models in which the nodes represent continuous variables having Gaussian distributions. Each distribution is conditioned on the state of its parents in the graph. That dependence could take many forms, and here we focus on a specific choice in which the mean of each Gaussian is some linear function of the states of the Gaussian parent variables. This leads to a class of models called *linear-Gaussian models*, which include many cases of practical interest such as probabilistic principal component analysis, factor analysis, and linear dynamical systems (Roweis and Ghahramani, 1999).

Section 16.2

Consider an arbitrary directed acyclic graph over D variables in which node i represents a single continuous random variable x_i having a Gaussian distribution. The mean of this distribution is taken to be a linear combination of the states of its parent nodes $\mathrm{pa}(i)$ of node i:

$$p(x_i|\mathrm{pa}(i)) = \mathcal{N}\left(x_i \,\middle|\, \sum_{j\in\mathrm{pa}(i)} w_{ij}x_j + b_i, v_i\right) \tag{11.9}$$

where w_{ij} and b_i are parameters governing the mean and v_i is the variance of the conditional distribution for x_i. The log of the joint distribution is then the log of the product of these conditionals over all nodes in the graph and hence takes the form

$$\ln p(\mathbf{x}) = \sum_{i=1}^{D} \ln p(x_i|\mathrm{pa}(i)) \tag{11.10}$$

$$= -\sum_{i=1}^{D} \frac{1}{2v_i}\left(x_i - \sum_{j\in\mathrm{pa}(i)} w_{ij}x_j - b_i\right)^2 + \text{const} \tag{11.11}$$

where $\mathbf{x} = (x_1, \ldots, x_D)^{\mathrm{T}}$ and 'const' denotes terms independent of $\mathbf{x}$. We see that this is a quadratic function of the components of $\mathbf{x}$, and hence the joint distribution $p(\mathbf{x})$ is a multivariate Gaussian.

We can find the mean and covariance of this joint distribution as follows. The mean of each variable is given by the recursion relation

Exercise 11.6

$$\mathbb{E}[x_i] = \sum_{j\in\mathrm{pa}(i)} w_{ij}\mathbb{E}[x_j] + b_i. \tag{11.12}$$



Figure 11.7 A directed graph over three Gaussian variables with one missing link.

and so we can find the components of $\mathbb{E}[\mathbf{x}] = (\mathbb{E}[x_1], \ldots, \mathbb{E}[x_D])^{\mathrm{T}}$ by starting at the lowest numbered node and working recursively through the graph, where we assume that the nodes are numbered such that each node has a higher number than its parents. Similarly, the elements of the covariance matrix of the joint distribution satisfy a recursion relation of the form

Exercise 11.7

$$\mathrm{cov}[x_i, x_j] = \sum_{k \in \mathrm{pa}(j)} w_{jk} \mathrm{cov}[x_i, x_k] + I_{ij} v_j \tag{11.13}$$

and so the covariance can similarly be evaluated recursively starting from the lowest numbered node.

We now consider two extreme cases of possible graph structures. First, suppose that there are no links in the graph, which therefore comprises D isolated nodes. In this case, there are no parameters w_{ij} and so there are just D parameters b_i and D parameters v_i. From the recursion relations (11.12) and (11.13), we see that the mean of $p(\mathbf{x})$ is given by $(b_1, \ldots, b_D)^{\mathrm{T}}$ and the covariance matrix is diagonal of the form $\mathrm{diag}(v_1, \ldots, v_D)$. The joint distribution has a total of $2D$ parameters and represents a set of D independent univariate Gaussian distributions.

Now consider a fully connected graph in which each node has all lower numbered nodes as parents. In this case the total number of independent parameters $\{w_{ij}\}$ and $\{v_i\}$ in the covariance matrix is $D(D+1)/2$ corresponding to a general symmetric covariance.

Exercise 11.8

Graphs having some intermediate level of complexity correspond to joint Gaussian distributions with partially constrained covariance matrices. Consider for example the graph shown in Figure 11.7, which has a link missing between variables x_1 and x_3. Using the recursion relations (11.12) and (11.13), we see that the mean and covariance of the joint distribution are given by

Exercise 11.9

$$\boldsymbol{\mu} = (b_1, b_2 + w_{21}b_1, b_3 + w_{32}b_2 + w_{32}w_{21}b_1)^{\mathrm{T}} \tag{11.14}$$

$$\boldsymbol{\Sigma} = \begin{pmatrix} v_1 & w_{21}v_1 & w_{32}w_{21}v_1 \\ w_{21}v_1 & v_2 + w_{21}^2 v_1 & w_{32}(v_2 + w_{21}^2 v_1) \\ w_{32}w_{21}v_1 & w_{32}(v_2 + w_{21}^2 v_1) & v_3 + w_{32}^2(v_2 + w_{21}^2 v_1) \end{pmatrix}. \tag{11.15}$$

We can readily extend the linear-Gaussian graphical model to a situation in which the nodes of the graph represent multivariate Gaussian variables. In this case, we can write the conditional distribution for node i in the form

$$p(\mathbf{x}_i | \mathrm{pa}(i)) = \mathcal{N}\left(\mathbf{x}_i \,\middle|\, \sum_{j \in \mathrm{pa}(i)} \mathbf{W}_{ij}\mathbf{x}_j + \mathbf{b}_i, \boldsymbol{\Sigma}_i\right) \tag{11.16}$$

where now $\mathbf{W}_{ij}$ is a matrix (which is non-square if $\mathbf{x}_i$ and $\mathbf{x}_j$ have different dimensionality). Again it is easy to verify that the joint distribution over all variables is Gaussian.

Exercise 11.10

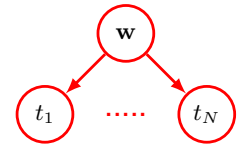


Figure 11.8 Directed graphical model representing the binary classifier model described by the joint distribution (11.17) showing only the stochastic variables $\{t_1, \ldots, t_N\}$ and $\mathbf{w}$.

11.1.5 Binary classifier

Section 2.6.2

We can illustrate the use of directed graphs to describe probability distributions using a two-class classifier model with Gaussian prior over the learnable parameters. We can write this in the form

$$p(\mathbf{t}, \mathbf{w}|\mathbf{X}, \lambda) = p(\mathbf{w}|\lambda) \prod_{n=1}^{N} p(t_n|\mathbf{w}, \mathbf{x}_n) \tag{11.17}$$

where $\mathbf{t} = (t_1, \ldots, t_N)^{\mathrm{T}}$ is the vector of target values, $\mathbf{X}$ is the data matrix with rows $\mathbf{x}_1^{\mathrm{T}}, \ldots \mathbf{x}_N^{\mathrm{T}}$, and the distribution $p(t|\mathbf{x}, \mathbf{w})$ is given by (11.1). We also assume a Gaussian prior over the parameter vector $\mathbf{w}$ given by

$$p(\mathbf{w}|\lambda) = \mathcal{N}(\mathbf{w}|\mathbf{0}, \lambda\mathbf{I}). \tag{11.18}$$

The stochastic variables in this model are $\{t_1, \ldots, t_N\}$ and $\mathbf{w}$. In addition, this model contains the noise variance σ^2 and the hyperparameter λ, both of which are parameters of the model rather than stochastic variables. If we consider for a moment only the stochastic variables, then the distribution given by (11.17) can be represented by the graphical model shown in Figure 11.8.

When we start to deal with more complex models, it becomes inconvenient to have to write out multiple nodes of the form $t_1, \ldots, t_N$ explicitly as in Figure 11.8. We therefore introduce a graphical notation that allows such multiple nodes to be expressed more compactly. We draw a single representative node t_n and then surround this with a box, called a *plate*, labelled with N to indicate that there are N nodes of this kind. Rewriting the graph of Figure 11.8 in this way, we obtain the graph shown in Figure 11.9.

11.1.6 Parameters and observations

We will sometimes find it helpful to make the parameters of a model, as well as its stochastic variables, explicit in the graphical representation. To do this, we will adopt the convention that random variables are denoted by open circles and deterministic parameters are denoted by floating variables. If we take the graph of

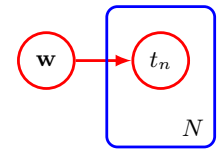


Figure 11.9 An alternative, more compact, representation of the graph shown in Figure 11.8 in which we have introduced a *plate* (the box labelled N) that represents N nodes of which only a single example t_n is shown explicitly.

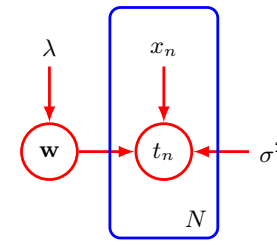


Figure 11.10 The same model as in Figure 11.9 but with the deterministic parameters shown explicitly by the floating variables.

Figure 11.9 and include the deterministic parameters, we obtain the graph shown in Figure 11.10.

When we apply a graphical model to a problem in machine learning, we will typically set some of the random variables to specific observed values. For example, the stochastic variables $\{t_n\}$ in the linear regression model will be set equal to the specific values given in the training set. In a graphical model, we denote such *observed variables* by shading the corresponding nodes. Thus, the graph corresponding to Figure 11.10 in which the variables $\{t_n\}$ are observed is shown in Figure 11.11.

Note that the value of $\mathbf{w}$ is not observed, and so $\mathbf{w}$ is an example of a *latent* variable, also known as a *hidden* variable. Such variables play a crucial role in many of the models discussed in this book. We therefore have three kinds of variables in a directed graphical model. First, there are unobserved (also called latent, or hidden) stochastic variables, which are denoted by open red circles. Second, when stochastic variables are observed, so that that they are set to specific values, they are denoted by red circles shaded with blue. Finally, non-stochastic parameters are denoted by floating variables, as seen in Figure 11.11.

Note that model parameters such as $\mathbf{w}$ are generally of little direct interest in themselves, because our ultimate goal is to make predictions for new input values. Suppose we are given a new input value $\widehat{x}$ and we wish to find the corresponding probability distribution for $\widehat{t}$ conditioned on the observed data. The joint distribution of all the random variables in this model, conditioned on the deterministic parameters, is given by

$$p(\widehat{t}, \mathbf{t}, \mathbf{w}|\widehat{x}, \mathbf{X}, \lambda) = p(\mathbf{w}|\lambda)p(\widehat{t}|\mathbf{w}, \widehat{x}) \prod_{n=1}^{N} p(t_n|\mathbf{w}, \mathbf{x}_n) \tag{11.19}$$

and the corresponding graphical model is shown in Figure 11.12.

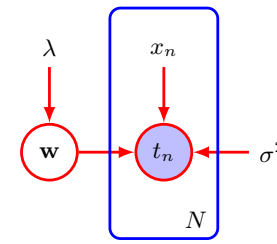


Figure 11.11 As in Figure 11.10 but with the nodes $\{t_n\}$ shaded to indicate that the corresponding random variables have been set to their observed values given by the training set.

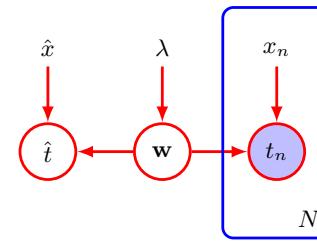


Figure 11.12 The classification model, corresponding to Figure 11.11, showing a new input value $\widehat{x}$ together with the corresponding model prediction $\widehat{t}$.

The required predictive distribution for $\widehat{t}$ is then obtained from the sum rule of probability by integrating out the model parameters $\mathbf{w}$. This integration over parameters represents a fully Bayesian treatment, which is rarely used in practice, especially with deep neural networks. Instead, we approximate this integral by first finding the most probable value $\mathbf{w}_{\text{MAP}}$ that maximizes the posterior distribution and then using just this single value to make predictions using $p(\widehat{t}|\mathbf{w}_{\text{MAP}}, \widehat{x})$.

11.1.7 Bayes' theorem

When stochastic variables in a probabilistic model are set equal to observed values, the distributions over other unobserved stochastic variables change accordingly. The process of calculating these updated distributions is known as *inference*. We can illustrate this by considering the graphical interpretation of Bayes' theorem. Suppose we decompose the joint distribution $p(x, y)$ over two variables x and y into a product of factors in the form $p(x, y) = p(x)p(y|x)$. This can be represented by the directed graph shown in Figure 11.13(a). Now suppose we observe the value of y, as indicated by the shaded node in Figure 11.13(b). We can view the marginal distribution $p(x)$ as a prior over the latent variable x, and our goal is to infer the corresponding posterior. Using the sum and product rules of probability we can evaluate

$$p(y) = \sum_{x'} p(y|x')p(x'), \tag{11.20}$$

which can then be used in Bayes' theorem to calculate

$$p(x|y) = \frac{p(y|x)p(x)}{p(y)}. \tag{11.21}$$

Thus, the joint distribution is now expressed in terms of $p(x|y)$ and $p(y)$. From a graphical perspective, the joint distribution $p(x, y)$ is represented by the graph shown in Figure 11.13(c), in which the direction of the arrow is reversed. This is the simplest example of an inference problem for a graphical model.

For complex graphical models that capture rich probabilistic structure, the process of calculating posterior distributions once some of the stochastic variables are observed can be complex and subtle. Conceptually, it simply involves the systematic application of the sum and product rules of probability, or equivalently Bayes' theorem. In practice, however, managing these calculations efficiently can benefit greatly from an exploitation of the graphical structure. These calculations can be expressed

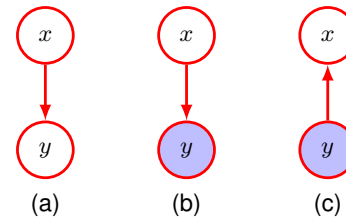


Figure 11.13 A graphical representation of Bayes' theorem showing (a) a joint distribution over two variables x and y expressed in factorized form, (b) the case with y set to an observed value, and (c) the resulting posterior distribution over x, given by Bayes' theorem.

in terms of elegant calculations on the graph that involve sending local messages between nodes. Such methods give exact answers for tree-structured graphs and give approximate iterative algorithms for graphs with loops. Since we will not discuss these further here, see Bishop (2006) for a more comprehensive discussion in the context of machine learning.

11.2. Conditional Independence

An important concept for probability distributions over multiple variables is that of *conditional independence* (Dawid, 1980). Consider three variables a, b, and c, and suppose that the conditional distribution of a given b and c is such that it does not depend on the value of b, so that

$$p(a|b,c) = p(a|c). \tag{11.22}$$

We say that a is conditionally independent of b given c. This can be expressed in a slightly different way if we consider the joint distribution of a and b conditioned on c, which we can write in the form

$$\begin{aligned} p(a,b|c) &= p(a|b,c)p(b|c) \\ &= p(a|c)p(b|c) \end{aligned} \tag{11.23}$$

where we have used the product rule of probability together with (11.22). We see that, conditioned on c, the joint distribution of a and b factorizes into the product of the marginal distribution of a and the marginal distribution of b (again both conditioned on c). This says that the variables a and b are statistically independent, given c. Note that our definition of conditional independence will require that (11.22), or equivalently (11.23), must hold for every possible value of c, and not just for some values. We will sometimes use a shorthand notation for conditional independence (Dawid, 1979) in which

$$a \perp\!\!\!\perp b \mid c \tag{11.24}$$

denotes that a is conditionally independent of b given c. Conditional independence properties play an important role in probabilistic models for machine learning because they simplify both the structure of a model and the computations needed to perform inference and learning under that model.

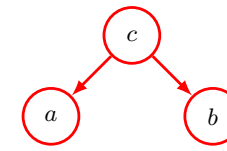


Figure 11.14 The first of three examples of graphs over three variables a, b, and c used to discuss conditional independence properties of directed graphical models.

If we are given an expression for the joint distribution over a set of variables in terms of a product of conditional distributions (i.e., the mathematical representation underlying a directed graph), then we could in principle test whether any potential conditional independence property holds by repeated application of the sum and product rules of probability. In practice, such an approach would be very time-consuming. An important and elegant feature of graphical models is that conditional independence properties of the joint distribution can be read directly from the graph without having to perform any analytical manipulations. The general framework for achieving this is called *d-separation*, where the 'd' stands for 'directed' (Pearl, 1988). Here we will motivate the concept of d-separation and give a general statement of the d-separation criterion. A formal proof can be found in Lauritzen (1996).

11.2.1 Three example graphs

We begin our discussion of the conditional independence properties of directed graphs by considering three simple examples each involving graphs having just three nodes. Together, these will motivate and illustrate the key concepts of d-separation. The first of the three examples is shown in Figure 11.14, and the joint distribution corresponding to this graph is easily written down using the general result (11.6) to give

$$p(a, b, c) = p(a|c)p(b|c)p(c). \tag{11.25}$$

If none of the variables are observed, then we can investigate whether a and b are independent by marginalizing both sides of (11.25) with respect to c to give

$$p(a, b) = \sum_c p(a|c)p(b|c)p(c). \tag{11.26}$$

In general, this does not factorize into the product $p(a)p(b)$, and so

$$a \not\perp\!\!\!\perp b \mid \emptyset \tag{11.27}$$

where $\emptyset$ denotes the empty set, and the symbol $\not\perp\!\!\!\perp$ means that the conditional independence property does not hold in general. Of course, it may hold for a particular distribution by virtue of the specific numerical values associated with the various conditional probabilities, but it does not follow in general from the structure of the graph.

Now suppose we condition on the variable c, as represented by the graph of Figure 11.15. From (11.25), we can easily write down the conditional distribution of

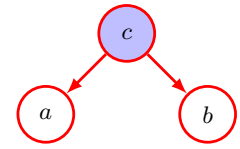


Figure 11.15 As in Figure 11.14 but where we have conditioned on the value of variable c.

a and b, given c, in the form

$$\begin{aligned} p(a,b|c) &= \frac{p(a,b,c)}{p(c)} \\ &= p(a|c)p(b|c) \end{aligned}$$

and so we obtain the conditional independence property

$$a \perp\!\!\!\perp b \mid c.$$

We can provide a simple graphical interpretation of this result by considering the path from node a to node b via c. The node c is said to be *tail-to-tail* with respect to this path because the node is connected to the tails of the two arrows, and the presence of such a path connecting nodes a and b causes these nodes to be dependent. However, when we condition on node c, as in Figure 11.15, the conditioned node 'blocks' the path from a to b and causes a and b to become (conditionally) independent.

We can similarly consider the graph shown in Figure 11.16. The joint distribution corresponding to this graph is again obtained from our general formula (11.6) to give

$$p(a,b,c) = p(a)p(c|a)p(b|c). \tag{11.28}$$

First, suppose that none of the variables are observed. Again, we can test to see if a and b are independent by marginalizing over c to give

$$p(a,b) = p(a)\sum_c p(c|a)p(b|c) = p(a)p(b|a)$$

which in general does not factorize into $p(a)p(b)$, and so

$$a \not\perp\!\!\!\perp b \mid \emptyset \tag{11.29}$$

as before.

Now suppose we condition on node c, as shown in Figure 11.17. Using Bayes'

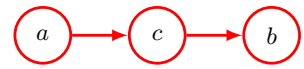
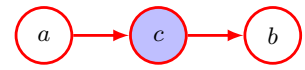


Figure 11.16 The second of our three examples of three-node graphs used to motivate the conditional independence framework for directed graphical models.

Figure 11.17 As in Figure 11.16 but now conditioning on node c.



theorem together with (11.28), we obtain

$$\begin{aligned} p(a,b|c) &= \frac{p(a,b,c)}{p(c)} \\ &= \frac{p(a)p(c|a)p(b|c)}{p(c)} \\ &= p(a|c)p(b|c) \end{aligned}$$

and so again we obtain the conditional independence property

$$a \perp\!\!\!\perp b \mid c.$$

As before, we can interpret these results graphically. The node c is said to be *head-to-tail* with respect to the path from node a to node b. Such a path connects nodes a and b and renders them dependent. If we now observe c, as in Figure 11.17, then this observation 'blocks' the path from a to b and so we obtain the conditional independence property $a \perp\!\!\!\perp b \mid c$.

Finally, we consider the third of our three-node examples, shown by the graph in Figure 11.18. As we will see, this has a more subtle behaviour than the two previous graphs. The joint distribution can again be written down using our general result (11.6) to give

$$p(a,b,c) = p(a)p(b)p(c|a,b). \tag{11.30}$$

Consider first the case where none of the variables are observed. Marginalizing both sides of (11.30) over c we obtain

$$p(a,b) = p(a)p(b)$$

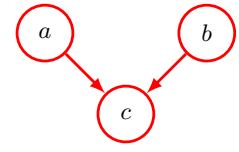
and so a and b are independent with no variables observed, in contrast to the two previous examples. We can write this result as

$$a \perp\!\!\!\perp b \mid \emptyset. \tag{11.31}$$

Now suppose we condition on c, as indicated in Figure 11.19. The conditional distribution of a and b is then given by

$$\begin{aligned} p(a,b|c) &= \frac{p(a,b,c)}{p(c)} \\ &= \frac{p(a)p(b)p(c|a,b)}{p(c)}, \end{aligned}$$

Figure 11.18 The last of our three examples of three-node graphs used to explore conditional independence properties in graphical models. This graph has rather different properties from the two previous examples.



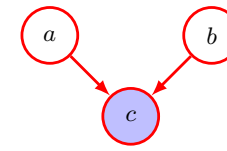


Figure 11.19 As in Figure 11.18 but conditioning on the value of node c. In this graph, the act of conditioning induces a dependence between a and b.

which in general does not factorize into the product $p(a|c)p(b|c)$, and so

$$a \not\perp\!\!\!\perp b \mid c.$$

Thus, our third example has the opposite behaviour from the first two. Graphically, we say that node c is *head-to-head* with respect to the path from a to b because it connects to the heads of the two arrows. The node c is sometimes called a *collider node*. When node c is unobserved, it 'blocks' the path, and the variables a and b are independent. However, conditioning on c 'unblocks' the path and renders a and b dependent.

There is one more subtlety associated with this third example that we need to consider. First we introduce some more terminology. We say that node y is a *descendant* of node x if there is a path from x to y in which each step of the path follows the directions of the arrows. Then it can be shown that a head-to-head path will become unblocked if either the node, *or any of its descendants*, is observed.

Exercise 11.13

In summary, a tail-to-tail node or a head-to-tail node leaves a path unblocked unless it is observed, in which case it blocks the path. By contrast, a head-to-head node blocks a path if it is unobserved, but once the node and/or at least one of its descendants is observed the path becomes unblocked.

11.2.2 Explaining away

It is worth spending a moment to understand further the unusual behaviour of the graph in Figure 11.19. Consider a particular instance of such a graph corresponding to a problem with three binary random variables relating to the fuel system on a car, as shown in Figure 11.20. The variables are B, which represents the state of a battery that is either charged ($B = 1$) or flat ($B = 0$), F which represents the state of the fuel tank that is either full of fuel ($F = 1$) or empty ($F = 0$), and G, which is the state of an electric fuel gauge and which indicates that the fuel tank is either full ($G = 1$) or empty ($G = 0$). The battery is either charged or flat, and independently,

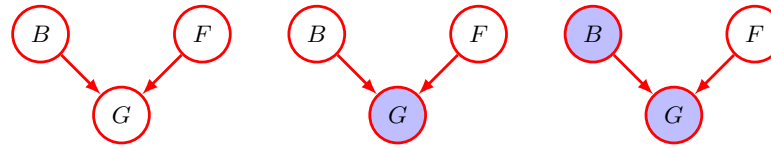


Figure 11.20 An example of a three-node graph used to illustrate 'explaining away'. The three nodes represent the state of the battery (B), the state of the fuel tank (F), and the reading on the electric fuel gauge (G). See the text for details.

the fuel tank is either full or empty, with prior probabilities

$$\begin{aligned} p(B=1) &= 0.9 \\ p(F=1) &= 0.9. \end{aligned}$$

Given the state of the fuel tank and the battery, the fuel gauge reads full with probabilities given by

$$\begin{aligned} p(G=1|B=1,F=1) &= 0.8 \\ p(G=1|B=1,F=0) &= 0.2 \\ p(G=1|B=0,F=1) &= 0.2 \\ p(G=1|B=0,F=0) &= 0.1 \end{aligned}$$

so this is a rather unreliable fuel gauge! All remaining probabilities are determined by the requirement that probabilities sum to one, and so we have a complete specification of the probabilistic model.

Before we observe any data, the prior probability of the fuel tank being empty is $p(F=0)=0.1$. Now suppose that we observe the fuel gauge and discover that it reads empty, i.e., $G=0$, corresponding to the middle graph in Figure 11.20. We can use Bayes' theorem to evaluate the posterior probability of the fuel tank being empty. First we evaluate the denominator for Bayes' theorem:

$$p(G=0) = \sum_{B\in\{0,1\}} \sum_{F\in\{0,1\}} p(G=0|B,F)p(B)p(F) = 0.315 \tag{11.32}$$

and similarly we evaluate

$$p(G=0|F=0) = \sum_{B\in\{0,1\}} p(G=0|B,F=0)p(B) = 0.81 \tag{11.33}$$

and using these results, we have

$$p(F=0|G=0) = \frac{p(G=0|F=0)p(F=0)}{p(G=0)} \simeq 0.257 \tag{11.34}$$

and so $p(F=0|G=0) > p(F=0)$. Thus, observing that the gauge reads empty makes it more likely that the tank is indeed empty, as we would intuitively expect. Next suppose that we also check the state of the battery and find that it is flat, i.e., $B=0$. We have now observed the states of both the fuel gauge and the battery, as shown by the right-hand graph in Figure 11.20. The posterior probability that the fuel tank is empty given the observations of both the fuel gauge and the battery state is then given by

$$p(F=0|G=0,B=0) = \frac{p(G=0|B=0,F=0)p(F=0)}{\sum_{F\in\{0,1\}} p(G=0|B=0,F)p(F)} \simeq 0.111 \tag{11.35}$$

where the prior probability $p(B=0)$ has cancelled between the numerator and denominator. Thus, the probability that the tank is empty has *decreased* (from 0.257

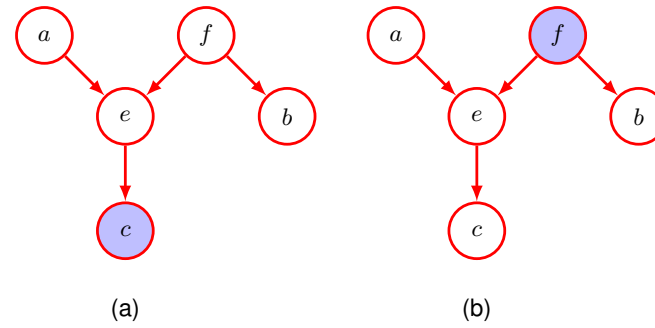


Figure 11.21 Illustration of d-separation. See the text for details.

to 0.111) as a result of the observation of the state of the battery. This accords with our intuition that finding that the battery is flat *explains away* the observation that the fuel gauge reads empty. We see that the state of the fuel tank and that of the battery have indeed become dependent on each other as a result of observing the reading on the fuel gauge. In fact, this would also be the case if, instead of observing the fuel gauge directly, we observed the state of some descendant of G, for example a rather unreliable witness who reports seeing that the gauge was reading empty. Note that the probability $p(F = 0|G = 0, B = 0) \simeq 0.111$ is greater than the prior probability $p(F = 0) = 0.1$ because the observation that the fuel gauge reads zero still provides some evidence in favour of an empty fuel tank.

Exercise 11.14

11.2.3 D-separation

We now give a general statement of the d-separation property (Pearl, 1988) for directed graphs. Consider a general directed graph in which A, B, and C are arbitrary non-intersecting sets of nodes (whose union may be smaller than the complete set of nodes in the graph). We wish to ascertain whether a particular conditional independence statement $A \perp\!\!\!\perp B \mid C$ is implied by a given directed acyclic graph. To do so, we consider all possible paths from any node in A to any node in B. Any such path is said to be *blocked* if it includes a node such that either

(a) the arrows on the path meet either head-to-tail or tail-to-tail at the node, and the node is in the set C, or

(b) the arrows meet head-to-head at the node and neither the node, nor any of its descendants is in the set C.

If all paths are blocked, then A is said to be d-separated from B by C, and the joint distribution over all the variables in the graph will satisfy $A \perp\!\!\!\perp B \mid C$.

D-separation is illustrated in Figure 11.21. In graph (a), the path from a to b is not blocked by node f because it is a tail-to-tail node for this path and is not observed, nor is it blocked by node e because, although the latter is a head-to-head node, it has a descendant c in the conditioning set. Thus, the conditional independence statement $a \perp\!\!\!\perp b \mid c$ does *not* follow from this graph. In graph (b), the path from a to b is blocked by node f because this is a tail-to-tail node that is observed, and so the conditional independence property $a \perp\!\!\!\perp b \mid f$ will be satisfied by any distribution that factorizes

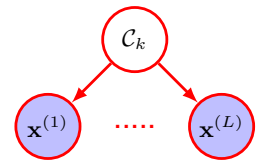


Figure 11.22 A graphical representation of the naive Bayes model for classification. Conditioned on the class label $\mathcal{C}_k$, the elements of the observed vector $\mathbf{x} = (\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(L)})$ are assumed to be independent.

according to this graph. Note that this path is also blocked by node e because e is a head-to-head node and neither it nor its descendant are in the conditioning set. In d-separation, parameters such as λ in Figure 11.12, which are indicated by floating variables, behave in the same way as observed nodes. However, there are no marginal distributions associated with such nodes, and consequently parameter nodes never themselves have parents and so all paths through these nodes will always be tail-to-tail and hence blocked. Consequently they play no role in d-separation.

Section 2.3.2

Another example of conditional independence and d-separation is provided by i.i.d. (independent and identically distributed) data. Consider the binary classification model shown in Figure 11.12. Here the stochastic nodes correspond to $\{t_n\}$, $\mathbf{w}$, and $\widehat{t}$. We see that the node for $\mathbf{w}$ is tail-to-tail with respect to the path from $\widehat{t}$ to any one of the nodes t_n, and so we have the following conditional independence property:

$$\widehat{t} \perp\!\!\!\perp t_n \mid \mathbf{w}. \tag{11.36}$$

Thus, conditioned on the network parameters $\mathbf{w}$, the predictive distribution for $\widehat{t}$ is independent of the training data $\{t_1, \ldots, t_N\}$. We can therefore first use the training data to determine the posterior distribution (or some approximation to the posterior distribution) over the coefficients $\mathbf{w}$ and then we can discard the training data and use the posterior distribution for $\mathbf{w}$ to make predictions of $\widehat{t}$ for new input observations $\widehat{x}$.

11.2.4 Naive Bayes

A related graphical structure arises in an approach to classification called the *naive Bayes* model, in which we use conditional independence assumptions to simplify the model structure. Suppose our data consists of observations of a vector $\mathbf{x}$, and we wish to assign values of $\mathbf{x}$ to one of K classes. We can define a class-conditional density $p(\mathbf{x}|\mathcal{C}_k)$ for each of the classes, along with prior class probabilities $p(\mathcal{C}_k)$. The key assumption of the naive Bayes model is that, conditioned on the class $\mathcal{C}_k$, the distribution of the input variable factorizes into the product of two or more densities. Suppose we partition $\mathbf{x}$ into L elements $\mathbf{x} = (\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(L)})$. Naive Bayes then takes the form

$$p(\mathbf{x}|\mathcal{C}_k) = \prod_{l=1}^{L} p(\mathbf{x}^{(l)}|\mathcal{C}_k) \tag{11.37}$$

where it is assumed that (11.37) holds for each of the classes $\mathcal{C}_k$ separately. The graphical representation of this model is shown in Figure 11.22. We see that an observation of $\mathcal{C}_k$ would block the path between $\mathbf{x}^{(i)}$ and $\mathbf{x}^{(j)}$ for $j \neq i$ because such

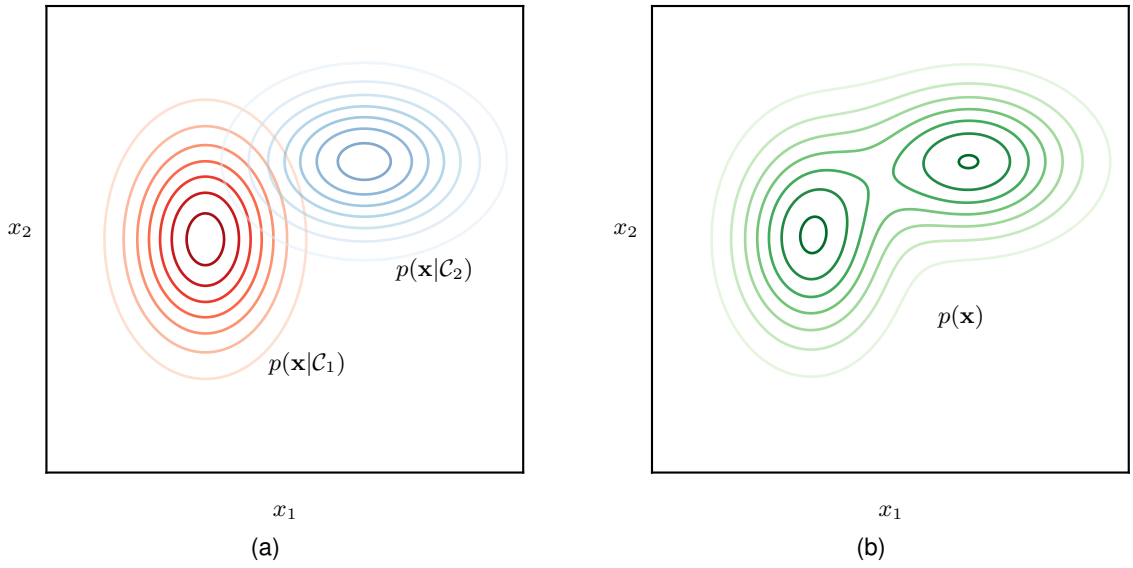


Figure 11.23 Illustration of a naive Bayes classifier for a two-dimensional data space, showing (a) the conditional distributions $p(\mathbf{x}|\mathcal{C}_k)$ for each of the two classes and (b) the marginal distribution $p(\mathbf{x})$ in which we have assumed equal class priors $p(\mathcal{C}_1) = p(\mathcal{C}_2) = 0.5$. Note that the conditional distributions factorize with respect to x_1 and x_2, whereas the marginal distribution does not.

paths are tail-to-tail at the node $\mathcal{C}_k$, and so $\mathbf{x}^{(i)}$ and $\mathbf{x}^{(j)}$ are conditionally independent given $\mathcal{C}_k$. If, however, we marginalize out $\mathcal{C}_k$, the tail-to-tail path from $\mathbf{x}^{(i)}$ to $\mathbf{x}^{(j)}$ is no longer blocked, which tells us that in general the marginal density $p(\mathbf{x})$ will not factorize with respect to the elements $\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(L)}$.

If we are given a labelled training set, comprising observations $\{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$ together with their class labels, then we can fit the naive Bayes model to the training data using maximum likelihood by assuming that the data are drawn independently from the model. The solution is obtained by fitting the model for each class separately using the corresponding labelled data and then setting the class priors $p(\mathcal{C}_k)$ equal to the fraction of training data points in each class. The probability that a vector $\mathbf{x}$ belongs to class $\mathcal{C}_k$ is then given by Bayes' theorem in the form

Exercise 11.15

$$p(\mathcal{C}_k|\mathbf{x}) = \frac{p(\mathbf{x}|\mathcal{C}_k)p(\mathcal{C}_k)}{p(\mathbf{x})} \tag{11.38}$$

where $p(\mathbf{x}|\mathcal{C}_k)$ is given by (11.37), and $p(\mathbf{x})$ can be evaluated using

$$p(\mathbf{x}) = \sum_{k=1}^{K} p(\mathbf{x}|\mathcal{C}_k)p(\mathcal{C}_k). \tag{11.39}$$

The naive Bayes model is illustrated for a two-dimensional data space in Figure 11.23 in which $\mathbf{x} = (x_1, x_2)$. Here we assume that the conditional densities

$p(\mathbf{x}|\mathcal{C}_k)$ for each of the two classes are axis-aligned Gaussians, and hence that they each factorize with respect to x_1 and x_2 so that

$$p(\mathbf{x}|\mathcal{C}_k) = p(x_1|\mathcal{C}_k)p(x_2|\mathcal{C}_k). \tag{11.40}$$

However, the marginal density $p(\mathbf{x})$ given by

$$p(\mathbf{x}) = \sum_{k=1}^{K} p(\mathbf{x}|\mathcal{C}_k)p(\mathcal{C}_k) \tag{11.41}$$

is now a mixture of Gaussians and does not factorize with respect to x_1 and x_2. We have already encountered a simple application of the naive Bayes model in the context of fusing data from different sources, such as blood tests and skin images for medical diagnosis.

Section 5.2.4

The naive Bayes assumption is helpful when the dimensionality D of the input space is high, making density estimation in the full D-dimensional space more challenging. It is also useful if the input vector contains both discrete and continuous variables, since each can be represented separately using appropriate models (e.g., Bernoulli distributions for binary observations or Gaussians for real-valued variables). The conditional independence assumption of this model is clearly a strong one that may lead to rather poor representations of the class-conditional densities. Nevertheless, even if this assumption is not precisely satisfied, the model may still give good classification performance in practice because the decision boundaries can be insensitive to some of the details in the class-conditional densities, as illustrated in Figure 5.8.

11.2.5 Generative models

Many applications of machine learning can be viewed as examples of *inverse problems* in which there is an underlying, often physical, process that generates data, and the goal is to learn now to invert this process. For example, an image of an object can be viewed as the output of a generative process in which the type of object is selected from some distribution of possible object classes. The position and orientation of the object are also chosen from some prior distributions, and then the resulting image is created. Given a large data set of images labelled with the type, position, and scale of the objects they contain, the goal is to train a machine learning model that can take new, unlabelled images and detect the presence of an object including its location within the image and its size. The machine learning solution therefore represents the inverse of the process that generated the data.

One approach would be to train a deep neural network, such as a convolutional network, to take an image as input and to generate outputs that describe the object's type, position, and scale. This approach therefore tries to solve the inverse problem directly and is an example of a *discriminative model*. It can achieve high accuracy provided ample examples of labelled images are available. In practice, unlabelled images are often plentiful, and much of the effort in obtaining a training set goes into proving the labels, which may be done by hand. Our simple discriminative model cannot directly make use of unlabelled images during training.

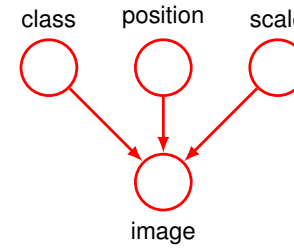


Figure 11.24 A graphical model representing the process by which images of objects are created. The identity of an object (a discrete variable) and the position and orientation of that object (continuous variables) have independent prior probabilities. The image (an array of pixel intensities) has a probability distribution that is dependent on the identity of the object as well as on its position and orientation.

An alternative approach is to model the generative process and then subsequently to invert it computationally. In our image example, if we assume that the object's class, position, and scale are all chosen independently, then we can represent the generative process using a directed graphical model as shown in Figure 11.24. Note that the directions of the arrows correspond to the sequence of generative steps, and so the model represents the *causal* process (Pearl, 1988) by which the observed data is generated. This is an example of a *generative model* because once it is trained, it can be used to generate synthetic images by first selecting values for object's class, position, and scale from the learned prior distributions and then subsequently sampling an image from the learned conditional distribution. We will later see how diffusion models and other generative models can synthesize impressive high-resolution images based on a textual description of the desired content and style of the image.

Chapter 20

The graph in Figure 11.24 assumes that, when no image is observed, the class, position, and scale variables are independent. This follows because every path between any two of these variables is head-to-head with respect to the image variable, which is unobserved. However, when we observe an image, those paths become unblocked, and the class, position, and scale variables are no longer independent. Intuitively this is reasonable because being told the identity of the object within the image provides us with very relevant information to assist us with determining its location.

The hidden variables in a probabilistic model need not, however, have any explicit physical interpretation but may be introduced simply to allow a more complex joint distribution to be constructed from simpler components. For example, models such as normalizing flows, variational autoencoders, and diffusion models all use deep neural networks to create complex distributions in the data space by transforming hidden variables having a simple Gaussian distribution.

11.2.6 Markov blanket

A conditional independence property that is helpful when discussing more complex directed graphs is called the *Markov blanket* or *Markov boundary*. Consider a joint distribution $p(\mathbf{x}_1, \ldots, \mathbf{x}_D)$ represented by a directed graph having D nodes, and consider the conditional distribution of a particular node with variables $\mathbf{x}_i$ conditioned on all the remaining variables $\mathbf{x}_{j \neq i}$. Using the factorization property (11.6),

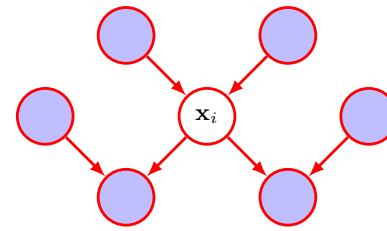


Figure 11.25 The Markov blanket of a node $\mathbf{x}_i$ comprises the set of parents, children, and co-parents of the node. It has the property that the conditional distribution of $\mathbf{x}_i$, conditioned on all the remaining variables in the graph, is dependent only on the variables in the Markov blanket.

we can express this conditional distribution in the form

$$p(\mathbf{x}_i|\mathbf{x}_{\{j\neq i\}}) = \frac{p(\mathbf{x}_1,\ldots,\mathbf{x}_D)}{\displaystyle\int p(\mathbf{x}_1,\ldots,\mathbf{x}_D)\,\mathrm{d}\mathbf{x}_i} = \frac{\displaystyle\prod_k p(\mathbf{x}_k|\mathrm{pa}(k))}{\displaystyle\int \prod_k p(\mathbf{x}_k|\mathrm{pa}(k))\,\mathrm{d}\mathbf{x}_i}$$

in which the integral is replaced by a summation for discrete variables. We now observe that any factor $p(\mathbf{x}_k|\mathrm{pa}(k))$ that does not have any functional dependence on $\mathbf{x}_i$ can be taken outside the integral over $\mathbf{x}_i$ and will therefore cancel between numerator and denominator. The only factors that remain will be the conditional distribution $p(\mathbf{x}_i|\mathrm{pa}(i))$ for node $\mathbf{x}_i$ itself, together with the conditional distributions for any nodes $\mathbf{x}_k$ such that node $\mathbf{x}_i$ is in the conditioning set of $p(\mathbf{x}_k|\mathrm{pa}(k))$, in other words for which $\mathbf{x}_i$ is a parent of $\mathbf{x}_k$. The conditional $p(\mathbf{x}_i|\mathrm{pa}(i))$ will depend on the parents of node $\mathbf{x}_i$, whereas the conditionals $p(\mathbf{x}_k|\mathrm{pa}(k))$ will depend on the children of $\mathbf{x}_i$ as well as on the *co-parents*, in other words variables corresponding to parents of node $\mathbf{x}_k$ other than node $\mathbf{x}_i$. The set of nodes comprising the parents, the children, and the co-parents is called the *Markov blanket* and is illustrated in Figure 11.25.

We can think of the Markov blanket of a node $\mathbf{x}_i$ as being the minimal set of nodes that isolates $\mathbf{x}_i$ from the rest of the graph. Note that it is not sufficient to include only the parents and children of node $\mathbf{x}_i$ because explaining away means that observations of the child nodes will not block paths to the co-parents. We must therefore observe the co-parent nodes as well.

11.2.7 Graphs as filters

We have seen that a particular directed graph represents a specific decomposition of a joint probability distribution into a product of conditional probabilities, and it also expresses a set of conditional independence statements obtained through the d-separation criterion. The d-separation theorem is really an expression of the equivalence of these two properties. To make this clear, it is helpful to think of a directed graph as a filter. Suppose we consider a particular joint probability distribution $p(\mathbf{x})$ over the variables $\mathbf{x}$ corresponding to the (unobserved) nodes of the graph. The filter will allow this distribution to pass through if, and only if, it can be expressed in

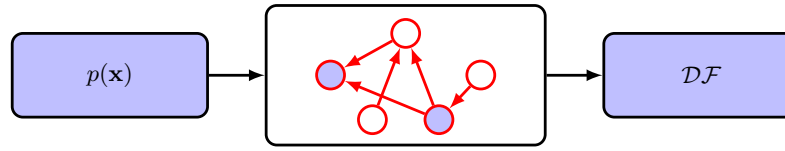


Figure 11.26 We can view a graphical model (in this case a directed graph) as a filter in which a probability distribution $p(\mathbf{x})$ is allowed through the filter if, and only if, it satisfies the directed factorization property (11.6). The set of all possible probability distributions $p(\mathbf{x})$ that pass through the filter is denoted $\mathcal{DF}$. We can alternatively use the graph to filter distributions according to whether they respect all the conditional independence properties implied by the d-separation properties of the graph. The d-separation theorem says the same set of distributions $\mathcal{DF}$ will be allowed through this second kind of filter.

terms of the factorization (11.6) implied by the graph. If we present to the filter the set of all possible distributions $p(\mathbf{x})$ over the set of variables $\mathbf{x}$, then the subset of distributions that are passed by the filter is denoted $\mathcal{DF}$, for *directed factorization*. This is illustrated in Figure 11.26.

Alternatively, we can use the graph as a different kind of filter by first listing all the conditional independence properties obtained by applying the d-separation criterion to the graph and then allowing a distribution to pass only if it satisfies all of these properties. If we present all possible distributions $p(\mathbf{x})$ to this second kind of filter, then the d-separation theorem tells us that the set of distributions that will be allowed through is precisely the set $\mathcal{DF}$.

It should be emphasized that the conditional independence properties obtained from d-separation apply to any probabilistic model described by that particular directed graph. This will be true, for instance, whether the variables are discrete or continuous or a combination of these. Again, we see that a particular graph describes a whole family of probability distributions.

At one extreme, we have a fully connected graph that exhibits no conditional independence properties at all and which can represent any possible joint probability distribution over the given variables. The set $\mathcal{DF}$ will contain all possible distributions $p(\mathbf{x})$. At the other extreme, we have a fully disconnected graph, i.e., one having no links at all. This corresponds to joint distributions that factorize into the product of the marginal distributions over the variables comprising the nodes of the graph. Note that for any given graph, the set of distributions $\mathcal{DF}$ will include any distributions that have additional independence properties beyond those described by the graph. For instance, a fully factorized distribution will always be passed through the filter implied by any graph over the corresponding set of variables.

11.3. Sequence Models

There are many important applications of machine learning in which the data consists of a *sequence* of values. For example, text comprises a sequence of words, whereas a protein comprises a sequence of amino acids. Many sequences are ordered by

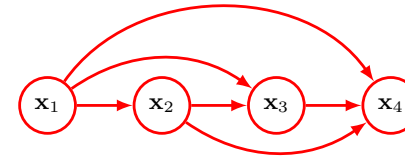


Figure 11.27 An illustration of a general autoregressive model of the form (11.42) with four nodes.

time, such as the audio signals from a microphone or daily rainfall measurements at a particular location. Sometimes the terminology of 'time' as well as 'past' and 'future' are used when referring to other types of sequential data, not just temporal sequences. Applications involving sequences include speech recognition, automatic translation between languages, detecting genes in DNA, synthesizing music, writing computer code, holding a conversation with a modern search engine, and many others.

We will denote a data sequence by $\mathbf{x}_1, \ldots, \mathbf{x}_N$ where each element $\mathbf{x}_n$ of the sequence comprises a vector of values. Note that we might have several such sequences drawn independently from the same distribution, in which case the joint distribution over all the sequences factorizes into the product of the distributions over each sequence individually. From now on, we focus on modelling just one of those sequences.

We have already seen in (11.4) that by repeated application of the product rule of probability, a general distribution over N variables can be written as the product of conditional distributions, and that the form of this decomposition depends on a specific of ordering for the variables. For vector-valued variables, and if we chose an ordering that corresponds to the order of the variables in the sequence, then we can write

$$p(\mathbf{x}_1, \ldots, \mathbf{x}_N) = \prod_{n=1}^{N} p(\mathbf{x}_n | \mathbf{x}_1, \ldots, \mathbf{x}_{n-1}). \tag{11.42}$$

This corresponds to a directed graph in which each node receives a link from every previous node in the sequence, as illustrated using four variables in Figure 11.27. This is known as an *autoregressive* model.

This representation has complete generality and therefore from a modelling perspective adds no value since it encodes no assumptions. We can constrain the space of models by introducing conditional independence properties by removing links from the graph, or equivalently by removing variables from the conditioning set of the factors on the right-hand-side of (11.42).

The strongest assumption would be to remove all conditioning variables, giving a joint distribution of the form

$$p(\mathbf{x}_1, \ldots, \mathbf{x}_N) = \prod_{n=1}^{N} p(\mathbf{x}_n), \tag{11.43}$$

which treats the variables as independent and therefore completely ignores the ordering information. This corresponds to a probabilistic graphical model without links, as shown in Figure 11.28.

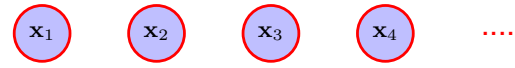


Figure 11.28 The simplest approach to modelling a sequence of observations is to treat them as independent, corresponding to a probabilistic graphical model without links.

Interesting models that capture sequential properties while introducing modelling assumptions lie between these two extremes. One strong assumption would be to assume that each conditional distribution depends only on the immediately preceding variable in the sequence, giving a joint distribution of the form

$$p(\mathbf{x}_1, \ldots, \mathbf{x}_N) = p(\mathbf{x}_1) \prod_{n=2}^{N} p(\mathbf{x}_n|\mathbf{x}_{n-1}). \tag{11.44}$$

Note that the first variable in the sequence is treated slightly differently since it has no conditioning variable. The functional form (11.44) is known as a *Markov model*, or *Markov chain*, and is represented by a graph consisting of a simple chain of nodes, as seen in Figure 11.29. Using d-separation, we see that the conditional distribution for observation $\mathbf{x}_n$, given all of the observations up to time n, is given by

Section 11.2.3

$$p(\mathbf{x}_n|\mathbf{x}_1, \ldots, \mathbf{x}_{n-1}) = p(\mathbf{x}_n|\mathbf{x}_{n-1}), \tag{11.45}$$

which is easily verified by direct evaluation starting from (11.44) and using the product rule of probability. Thus, if we use such a model to predict the next observation in a sequence, the distribution of predictions will depend only on the value of the immediately preceding observation and will be independent of all earlier observations.

Exercise 11.16

More specifically, (11.44) is known as a first-order Markov model because only one conditioning variable appears in each conditional distribution. We can extend the model by allowing each conditional distribution to depend on the two preceding variables, giving a second-order Markov model of the form

$$p(\mathbf{x}_1, \ldots, \mathbf{x}_N) = p(\mathbf{x}_1)p(\mathbf{x}_2|\mathbf{x}_1) \prod_{n=3}^{N} p(\mathbf{x}_n|\mathbf{x}_{n-1}, \mathbf{x}_{n-2}). \tag{11.46}$$

Note that the first two variables are treated differently as they have fewer than two conditioning variables. This model is shown as a directed graph in Figure 11.30.

Exercise 11.17

By using d-separation (or by direct evaluation using the rules of probability), we see that in the second-order Markov model, the conditional distribution of $\mathbf{x}_n$ given all previous observations $\mathbf{x}_1, \ldots, \mathbf{x}_{n-1}$ is independent of the observations $\mathbf{x}_1, \ldots \mathbf{x}_{n-3}$. We can similarly consider extensions to an Mth order Markov chain in

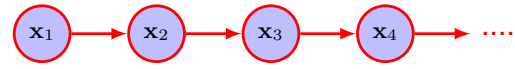


Figure 11.29 A first-order Markov chain of observations in which the distribution of a particular observation $\mathbf{x}_n$ is conditioned on the value of the previous observation $\mathbf{x}_{n-1}$.

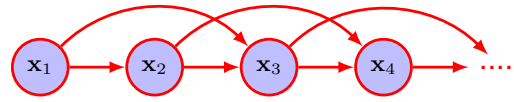


Figure 11.30 A second-order Markov chain in which the conditional distribution of a particular observation $\mathbf{x}_n$ depends on the values of the two previous observations $\mathbf{x}_{n-1}$ and $\mathbf{x}_{n-2}$.

which the conditional distribution for a particular variable depends on the previous M variables. However, we have paid a price for this increased flexibility because the number of parameters in the model is now much larger. Suppose the observations are discrete variables having K states. Then the conditional distribution $p(\mathbf{x}_n|\mathbf{x}_{n-1})$ in a first-order Markov chain will be specified by a set of $K-1$ parameters for each of the K states of $\mathbf{x}_{n-1}$ giving a total of $K(K-1)$ parameters. Now suppose we extend the model to an Mth order Markov chain, so that the joint distribution is built up from conditionals $p(\mathbf{x}_n|\mathbf{x}_{n-M},\ldots,\mathbf{x}_{n-1})$. If the variables are discrete and if the conditional distributions are represented by general conditional probability tables, then such a model will have $K^{M-1}(K-1)$ parameters. Thus, the number of parameters grows exponentially with M, which will generally render this approach impractical for larger values of M.

11.3.1 Hidden variables

Suppose we wish to build a model for sequences that is not limited by the Markov assumption to any order and yet can be specified using a limited number of free parameters. We can achieve this by introducing additional latent variables, thus permitting a rich class of models to be constructed out of simple components. For each observation $\mathbf{x}_n$, we introduce a corresponding latent variable $\mathbf{z}_n$ (which may be of different type or dimensionality to the observed variable). We now assume that it is the latent variables that form a Markov chain, giving rise to the graphical structure known as a *state-space model*, which is shown in Figure 11.31. It satisfies the key conditional independence property that $\mathbf{z}_{n-1}$ and $\mathbf{z}_{n+1}$ are independent given $\mathbf{z}_n$, so that

$$\mathbf{z}_{n+1} \perp\!\!\!\perp \mathbf{z}_{n-1} \mid \mathbf{z}_n. \tag{11.47}$$

The joint distribution for this model is given by

$$p(\mathbf{x}_1,\ldots,\mathbf{x}_N,\mathbf{z}_1,\ldots,\mathbf{z}_N) = p(\mathbf{z}_1)\left[\prod_{n=2}^{N} p(\mathbf{z}_n|\mathbf{z}_{n-1})\right]\prod_{n=1}^{N} p(\mathbf{x}_n|\mathbf{z}_n). \tag{11.48}$$

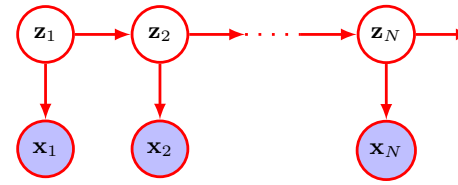


Figure 11.31 A state-space model expresses the joint probability distribution over a sequence of observed states $\mathbf{x}_1,\ldots,\mathbf{x}_N$ in terms of a Markov chain of hidden states $\mathbf{z}_1,\ldots,\mathbf{z}_N$ in the form (11.48).

Using the d-separation criterion, we see that in the state-space model there is always a path connecting any two observed variables $\mathbf{x}_n$ and $\mathbf{x}_m$ via the latent variables and that this path is never blocked. Thus, the predictive distribution $p(\mathbf{x}_{n+1}|\mathbf{x}_1, \ldots, \mathbf{x}_n)$ for observation $\mathbf{x}_{n+1}$ given all previous observations does not exhibit any conditional independence properties, and so our predictions for $\mathbf{x}_{n+1}$ depend on all previous observations. The observed variables, therefore, do not satisfy the Markov property at any order.

There are two important models for sequential data that are described by this graph. If the latent variables are discrete, then we obtain a *hidden Markov model* (Elliott, Aggoun, and Moore, 1995). Note that the observed variables in a hidden Markov model may be discrete or continuous, and a variety of different conditional distributions can be used to model them. If both the latent and the observed variables are Gaussian (with a linear-Gaussian dependence of the conditional distributions on their parents), then we obtain a *linear dynamical system*, also known as a *Kalman filter* (Zarchan and Musoff, 2005). Both hidden Markov models and Kalman filters are discussed at length, along with algorithms for training them, in Bishop (2006). Such models can be made considerably more flexible by replacing the simple discrete probability tables, or linear-Gaussian distributions, used to define $p(\mathbf{x}_n|\mathbf{z}_n)$ with deep neural networks.

Exercises

11.1 ($\star$) By marginalizing out the variables in order, show that the representation (11.6) for the joint distribution of a directed graph is correctly normalized, provided each of the conditional distributions is normalized.

11.2 ($\star$) Show that the property of there being no directed cycles in a directed graph follows from the statement that there exists an ordered numbering of the nodes such that for each node there are no links going to a lower-numbered node.

11.3 ($\star\star$) Consider three binary variables $a, b, c \in \{0, 1\}$ having the joint distribution given in Table 11.1. Show by direct evaluation that this distribution has the property that a and b are marginally dependent, so that $p(a, b) \neq p(a)p(b)$, but that they become independent when conditioned on c, so that $p(a, b|c) = p(a|c)p(b|c)$ for both $c = 0$ and $c = 1$.

Table 11.1 The joint distribution over three binary variables.

a	b	c	$p(a, b, c)$
0	0	0	0.192
0	0	1	0.144
0	1	0	0.048
0	1	1	0.216
1	0	0	0.192
1	0	1	0.064
1	1	0	0.048
1	1	1	0.096

11.4 (⋆⋆) Evaluate the distributions $p(a)$, $p(b|c)$, and $p(c|a)$ corresponding to the joint distribution given in Table 11.1. Hence, show by direct evaluation that $p(a, b, c) = p(a)p(c|a)p(b|c)$. Draw the corresponding directed graph.

11.5 (⋆) For the model shown in Figure 11.6, we have seen that the number of parameters required to specify the conditional distribution $p(y|x_1, \ldots, x_M)$, where $x_i \in \{0, 1\}$, could be reduced from 2^M to $M + 1$ by making use of the logistic sigmoid representation (11.8). An alternative representation (Pearl, 1988) is given by

$$p(y = 1|x_1, \ldots, x_M) = 1 - (1 - \mu_0) \prod_{i=1}^{M} (1 - \mu_i)^{x_i} \tag{11.49}$$

where the parameters μ_i represent the probabilities $p(x_i = 1)$ and μ_0 is an additional parameter satisfying $0 \leqslant \mu_0 \leqslant 1$. The conditional distribution (11.49) is known as the *noisy-OR*. Show that this can be interpreted as a 'soft' (probabilistic) form of the logical OR function (i.e., the function that gives $y = 1$ whenever at least one of the $x_i = 1$). Discuss the interpretation of μ_0.

11.6 (⋆⋆) Starting from the definition (11.9) for the conditional distributions, derive the recursion relation (11.12) for the mean of the joint distribution for a linear-Gaussian model.

11.7 (⋆⋆) Starting from the definition (11.9) for the conditional distributions, derive the recursion relation (11.13) for the covariance matrix of the joint distribution for a linear-Gaussian model.

11.8 (⋆⋆) Show that the number of parameters in the covariance matrix of a fully connected linear-Gaussian graphical model over D variables defined by (11.9) is $D(D + 1)/2$.

11.9 (⋆⋆) Using the recursion relations (11.12) and (11.13), show that the mean and covariance of the joint distribution for the graph shown in Figure 11.7 are given by (11.14) and (11.15), respectively.

11.10 (⋆) Verify that the joint distribution over a set of vector-valued variables defined by a linear-Gaussian model in which each node corresponds to a distribution of the form (11.16) is itself a Gaussian.

11.11 (⋆) Show that $a \perp\!\!\!\perp b, c \mid d$ implies $a \perp\!\!\!\perp b \mid d$.

11.12 (⋆) Using the d-separation criterion, show that the conditional distribution for a node $\mathbf{x}$ in a directed graph, conditioned on all the nodes in the Markov blanket, is independent of the remaining variables in the graph.

11.13 (⋆) Consider the directed graph shown in Figure 11.32 in which none of the variables is observed. Show that $a \perp\!\!\!\perp b \mid \emptyset$. Suppose we now observe the variable d. Show that in general $a \not\perp\!\!\!\perp b \mid d$.

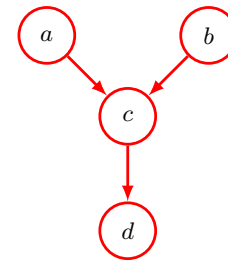


Figure 11.32 Example of a graphical model used to explore the conditional independence properties of the head-to-head path a–c–b when a descendant of c, namely the node d, is observed.

11.14 ($\star\star$) Consider the example of the car fuel system shown in Figure 11.20, and suppose that instead of observing the state of the fuel gauge G directly, the gauge is seen by the driver D, who reports to us the reading on the gauge. This report says that the gauge shows either that the tank is full $D = 1$ or that it is empty $D = 0$. Our driver is a bit unreliable, as expressed through the following probabilities:

$$p(D = 1|G = 1) = 0.9 \tag{11.50}$$

$$p(D = 0|G = 0) = 0.9. \tag{11.51}$$

Suppose that the driver tells us that the fuel gauge shows empty, in other words that we observe $D = 0$. Evaluate the probability that the tank is empty given only this observation. Similarly, evaluate the corresponding probability given also the observation that the battery is flat, and note that this second probability is lower. Discuss the intuition behind this result, and relate the result to Figure 11.32.

11.15 ($\star\star$) Suppose we train a naive Bayes model, with the assumption (11.37), using maximum likelihood. Assume that each of the class-conditional densities $p(\mathbf{x}^{(l)}|\mathcal{C}_k)$ is governed by its own independent parameters $\mathbf{w}^{(l)}$. Show that the maximum likelihood solution involves fitting each of the class-conditional densities using the corresponding observed data vectors $\mathbf{x}_1^{(l)}, \ldots, \mathbf{x}_N^{(l)}$ by maximizing the likelihood with respect to the corresponding class label data, and then setting the class priors $p(\mathcal{C}_k)$ to the fraction of training data points in each class.

11.16 ($\star\star$) Consider the joint probability distribution (11.44) corresponding to the directed graph of Figure 11.29. Using the sum and product rules of probability, verify that this joint distribution satisfies the conditional independence property (11.45) for $n = 2, \ldots, N$. Similarly, show that the second-order Markov model described by the joint distribution (11.46) satisfies the conditional independence property

$$p(\mathbf{x}_n|\mathbf{x}_1, \ldots, \mathbf{x}_{n-1}) = p(\mathbf{x}_n|\mathbf{x}_{n-1}, \mathbf{x}_{n-2}) \tag{11.52}$$

for $n = 3, \ldots, N$.

11.17 ($\star$) Use d-separation, as discussed in Section 11.2, to verify that the Markov model shown in Figure 11.29 having N nodes in total satisfies the conditional independence properties (11.45) for $n = 2, \ldots, N$. Similarly, show that a model described by the graph in Figure 11.30 in which there are N nodes in total satisfies the conditional independence properties (11.52) for $n = 3, \ldots, N$.

11.18 ($\star$) Consider a second-order Markov process described by the graph in Figure 11.30. By combining adjacent pairs of variables, show that this can be expressed as a first-order Markov process over the new variables.

11.19 ($\star$) By using d-separation, show that the distribution $p(\mathbf{x}_1, \ldots, \mathbf{x}_N)$ of the observed data for the state-space model represented by the directed graph in Figure 11.31 does not satisfy any conditional independence properties and hence does not exhibit the Markov property at any finite order.



12 Transformers

Transformers represent one of the most important developments in deep learning. They are based on a processing concept called *attention*, which allows a network to give different weights to different inputs, with weighting coefficients that themselves depend on the input values, thereby capturing powerful inductive biases related to sequential and other forms of data.

These models are known as transformers because they transform a set of vectors in some representation space into a corresponding set of vectors, having the same dimensionality, in some new space. The goal of the transformation is that the new space will have a richer internal representation that is better suited to solving downstream tasks. Inputs to a transformer can take the form of unstructured sets of vectors, ordered sequences, or more general representations, giving transformers broad applicability.

Transformers were originally introduced in the context of natural language pro-

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024
C. M. Bishop, H. Bishop, *Deep Learning*, https://doi.org/10.1007/978-3-031-45468-4_12

cessing, or NLP (where a 'natural' language is one such as English or Mandarin) and have greatly surpassed the previous state-of-the-art approaches based on recurrent neural networks (RNNs). Transformers have subsequently been found to achieve excellent results in many other domains. For example, vision transformers often outperform CNNs in image processing tasks, whereas multimodal transformers that combine multiple types of data, such as text, images, audio, and video, are amongst the most powerful deep learning models.

One major advantage of transformers is that transfer learning is very effective, so that a transformer model can be trained on a large body of data and then the trained model can be applied to many downstream tasks using some form of fine-tuning. A large-scale model that can subsequently be adapted to solve multiple different tasks is known as a *foundation model*. Furthermore, transformers can be trained in a self-supervised way using unlabelled data, which is especially effective with language models since transformers can exploit vast quantities of text available from the internet and other sources. The *scaling hypothesis* asserts that simply by increasing the scale of the model, as measured by the number of learnable parameters, and training on a commensurately large data set, significant improvements in performance can be achieved, even with no architectural changes. Moreover, the transformer is especially well suited to massively parallel processing hardware such as *graphical processing units*, or GPUs, allowing exceptionally large neural network language models having of the order of a trillion (10^{12}) parameters to be trained in reasonable time. Such models have extraordinary capabilities and show clear indications of emergent properties that have been described as the early signs of artificial general intelligence (Bubeck *et al.*, 2023).

The architecture of a transformer can seem complex, or even daunting, to a newcomer as it involves multiple different components working together, in which the various design choices can seem arbitrary. In this chapter we therefore aim to give a comprehensive step-by-step introduction to all the key ideas behind transformers and to provide clear intuition to motivate the design of the various elements. We first describe the transformer architecture and then focus on natural language processing, before exploring other application domains.

12.1. Attention

Section 12.2.5

The fundamental concept that underpins a transformer is *attention*. This was originally developed as an enhancement to RNNs for machine translation (Bahdanau, Cho, and Bengio, 2014). However, Vaswani *et al.* (2017) later showed that significantly improved performance could be obtained by eliminating the recurrence structure and instead focusing exclusively on the attention mechanism. Today, transformers based on attention have completely superseded RNNs in almost all applications.

We will motivate the use of attention using natural language as an example,

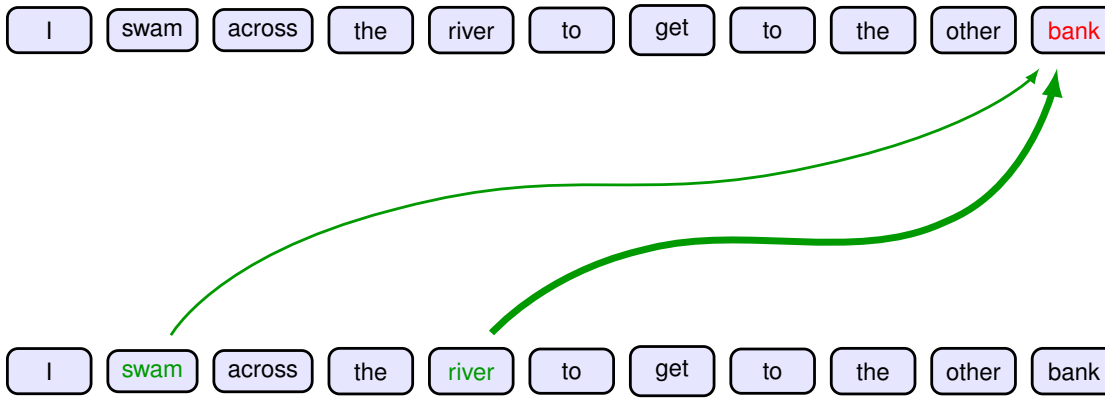


Figure 12.1 Schematic illustration of attention in which the interpretation of the word 'bank' is influenced by the words 'river' and 'swam', with the thickness of each line being indicative of the strength of its influence.

although it has much broader applicability. Consider the following two sentences:

I swam across the river to get to the other bank.

I walked across the road to get cash from the bank.

Here the word 'bank' has different meanings in the two sentences. However, this can be detected only by looking at the context provided by other words in the sequence. We also see that some words are more important than others in determining the interpretation of 'bank'. In the first sentence, the words 'swam' and 'river' most strongly indicate that 'bank' refers to the side of a river, whereas in the second sentence, the word 'cash' is a strong indicator that 'bank' refers to a financial institution. We see that to determine the appropriate interpretation of 'bank', a neural network processing such a sentence should *attend* to, in other words rely more heavily on, specific words from the rest of the sequence. This concept of attention is illustrated in Figure 12.1.

Moreover, we also see that the particular locations that should receive more attention depend on the input sequence itself: in the first sentence it is the second and fifth words that are important whereas in the second sentence it is the eighth word. In a standard neural network, different inputs will influence the output to different extents according to the values of the weights that multiply those inputs. Once the network is trained, however, those weights, and their associated inputs, are fixed. By contrast, attention uses weighting factors whose values depend on the specific input data. Figure 12.2 shows the attention weights from a section of a transformer network trained on natural language.

When we discuss natural language processing, we will see how word embedding can be used to map words into vectors in an embedding space. These vectors can then be used as inputs for subsequent neural network processing. These embeddings capture elementary semantic properties, for example by mapping words with similar meanings to nearby locations in the embedding space. One characteristic of such embeddings is that a given word always maps to the same embedding vector.

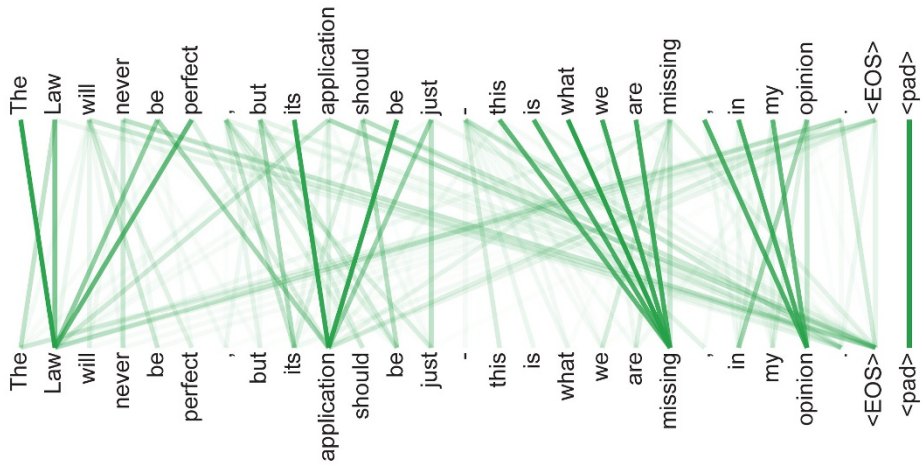


Figure 12.2 An example of learned attention weights. [From Vaswani *et al.* (2017) with permission.]

A transformer can be viewed as a richer form of embedding in which a given vector is mapped to a location that depends on the other vectors in the sequence. Thus, the vector representing 'bank' in our example above could map to different places in a new embedding space for the two different sentences. For example, in the first sentence the transformed representation might put 'bank' close to 'water' in the embedding space, whereas in the second sentence the transformed representation might put it close to 'money'.

As an example of attention, consider the modelling of proteins. We can view a protein as a one-dimensional sequence of molecular units called amino acids. A protein can comprise potentially hundreds or thousands of such units, each of which is given by one of 22 possibilities. In a living cell, a protein folds up into a three-dimensional structure in which amino acids that are widely separated in the one-dimensional sequence can become physically close in three-dimensional space and thereby interact. Transformer models allows these distant amino acids to 'attend' to each other thereby greatly improving the accuracy with which their 3-dimensional structure can be modelled (Vig *et al.*, 2020).

Figure 1.2

12.1.1 Transformer processing

The input data to a transformer is a set of vectors $\{\mathbf{x}_n\}$ of dimensionality D, where $n = 1, \ldots, N$. We refer to these data vectors as *tokens*, where a token might, for example, correspond to a word within a sentence, a patch within an image, or an amino acid within a protein. The elements x_{ni} of the tokens are called *features*. Later we will see how to construct these token vectors for natural language data and for images. A powerful property of transformers is that we do not have to design a new neural network architecture to handle a mix of different data types but instead can simply combine the data variables into a joint set of tokens.

Before we can gain a clear understanding of the operation of a transformer, it

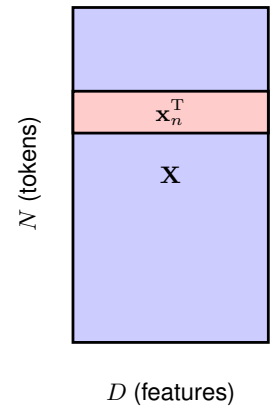


Figure 12.3 The structure of the data matrix $\mathbf{X}$, of dimension $N \times D$, in which row n represents the transposed data vector $\mathbf{x}_n^{\mathrm{T}}$.

is important to be precise about notation. We will follow the standard convention and combine the data vectors into a matrix $\mathbf{X}$ of dimensions $N \times D$ in which the nth row comprises the token vector $\mathbf{x}_n^{\mathrm{T}}$, and where $n = 1, \ldots, N$ labels the rows, as illustrated in Figure 12.3. Note that this matrix represents one set of input tokens, and that for most applications, we will require a data set containing many sets of tokens, such as independent passages of text where each word is represented as one token. The fundamental building block of a transformer is a function that takes a data matrix as input and creates a transformed matrix $\widetilde{\mathbf{X}}$ of the same dimensionality as the output. We can write this function in the form

$$\widetilde{\mathbf{X}} = \mathrm{TransformerLayer}\left[\mathbf{X}\right]. \tag{12.1}$$

We can then apply multiple transformer layers in succession to construct deep networks capable of learning rich internal representations. Each transformer layer contains its own weights and biases, which can be learned using gradient descent using an appropriate cost function, as we will discuss in detail later in the chapter.

Section 12.3

A single transformer layer itself comprises two stages. The first stage, which implements the attention mechanism, mixes together the corresponding features from different token vectors across the columns of the data matrix, whereas the second stage then acts on each row independently and transforms the features within each token vector. We start by looking at the attention mechanism.

12.1.2 Attention coefficients

Suppose that we have a set of input tokens $\mathbf{x}_1, \ldots, \mathbf{x}_N$ in an embedding space and we want to map this to another set $\mathbf{y}_1, \ldots, \mathbf{y}_N$ having the same number of tokens but in a new embedding space that captures a richer semantic structure. Consider a particular output vector $\mathbf{y}_n$. The value of $\mathbf{y}_n$ should depend not just on the corresponding input vector $\mathbf{x}_n$ but on all the vectors $\mathbf{x}_1, \ldots, \mathbf{x}_N$ in the set. With attention, this dependence should be stronger for those inputs $\mathbf{x}_m$ that are particularly important for determining the modified representation of $\mathbf{y}_n$. A simple way to achieve this is to define each output vector $\mathbf{y}_n$ to be a linear combination of the input vectors

$\mathbf{x}_1, \ldots, \mathbf{x}_N$ with weighting coefficients a_{nm}:

$$\mathbf{y}_n = \sum_{m=1}^{N} a_{nm}\mathbf{x}_m \tag{12.2}$$

where a_{nm} are called *attention weights*. The coefficients should be close to zero for input tokens that have little influence on the output $\mathbf{y}_n$ and largest for inputs that have most influence. We therefore constrain the coefficients to be non-negative to avoid situations in which one coefficient can become large and positive while another coefficient compensates by becoming large and negative. We also want to ensure that if an output pays more attention to a particular input, this will be at the expense of paying less attention to the other inputs, and so we constrain the coefficients to sum to unity. Thus, the weighting coefficients must satisfy the following two constraints:

$$a_{nm} \geqslant 0 \tag{12.3}$$

$$\sum_{m=1}^{N} a_{nm} = 1. \tag{12.4}$$

Exercise 12.1

Together these imply that each coefficient lies in the range $0 \leqslant a_{nm} \leqslant 1$ and so the coefficients define a 'partition of unity'. For the special case $a_{mm} = 1$, it follows that $a_{nm} = 0$ for $n \neq m$, and therefore $\mathbf{y}_m = \mathbf{x}_m$ so that the input vector is unchanged by the transformation. More generally, the output $\mathbf{y}_m$ is a blend of the input vectors with some inputs given more weight than others.

Note that we have a different set of coefficients for each output vector $\mathbf{y}_n$, and the constraints (12.3) and (12.4) apply separately for each value of n. These coefficients a_{nm} depend on the input data, and we will shortly see how to calculate them.

12.1.3 Self-attention

The next question is how to determine the coefficients a_{nm}. Before we discuss this in detail, it is useful to first introduce some terminology taken from the field of information retrieval. Consider the problem of choosing which movie to watch in an online movie streaming service. One approach would be to associate each movie with a list of attributes describing things such as the genre (comedy, action, etc.), the names of the leading actors, the length of the movie, and so on. The user could then search through a catalogue to find a movie that matches their preferences. We could automate this by encoding the attributes of each movie in a vector called the *key*. The corresponding movie file itself is called a *value*. Similarly, the user could then provide their own personal vector of values for the desired attributes, which we call the *query*. The movie service could then compare the query vector with all the key vectors to find the best match and send the corresponding movie to the user in the form of the value file. We can think of the user 'attending' to the particular movie whose key most closely matches their query. This would be considered a form of *hard attention* in which a single value vector is returned. For the transformer, we generalize this to *soft attention* in which we use continuous variables to measure

the degree of match between queries and keys and we then use these variables to weight the influence of the value vectors on the outputs. This will also ensure that the transformer function is differentiable and can therefore be trained by gradient descent.

Following the analogy with information retrieval, we can view each of the input vectors $\mathbf{x}_n$ as a value vector that will be used to create the output tokens. We also use the vector $\mathbf{x}_n$ directly as the key vector for input token n. That would be analogous to using the movie itself to summarize the characteristics of the movie. Finally, we can use $\mathbf{x}_m$ as the query vector for output $\mathbf{y}_m$, which can then be compared to each of the key vectors. To see how much the token represented by $\mathbf{x}_n$ should attend to the token represented by $\mathbf{x}_m$, we need to work out how similar these vectors are. One simple measure of similarity is to take their dot product $\mathbf{x}_n^{\mathrm{T}}\mathbf{x}_m$. To impose the constraints (12.3) and (12.4), we can define the weighting coefficients a_{nm} by using the *softmax* function to transform the dot products:

Section 5.3

$$a_{nm} = \frac{\exp(\mathbf{x}_n^{\mathrm{T}}\mathbf{x}_m)}{\sum_{m'=1}^{N}\exp(\mathbf{x}_n^{\mathrm{T}}\mathbf{x}_{m'})}. \tag{12.5}$$

Note that in this case there is no probabilistic interpretation of the softmax function and it is simply being used to normalize the attention weights appropriately.

So in summary, each input vector $\mathbf{x}_n$ is transformed to a corresponding output vector $\mathbf{y}_n$ by taking a linear combination of input vectors of the form (12.2) in which the weight a_{nm} applied to input vector $\mathbf{x}_m$ is given by the softmax function (12.5) defined in terms of the dot product $\mathbf{x}_n^{\mathrm{T}}\mathbf{x}_m$ between the query $\mathbf{x}_n$ for input n and the key $\mathbf{x}_m$ associated with input m. Note that, if all the input vectors are orthogonal, then each output vector is simply equal to the corresponding input vector so that $\mathbf{y}_m = \mathbf{x}_m$ for $m = 1, \ldots, N$.

Exercise 12.3

We can write (12.2) in matrix notation by using the data matrix $\mathbf{X}$, along with the analogous $N \times D$ output matrix $\mathbf{Y}$, whose rows are given by $\mathbf{y}_m$, so that

$$\mathbf{Y} = \mathrm{Softmax}\left[\mathbf{X}\mathbf{X}^{\mathrm{T}}\right]\mathbf{X} \tag{12.6}$$

where $\mathrm{Softmax}[\mathbf{L}]$ is an operator that takes the exponential of every element of a matrix $\mathbf{L}$ and then normalizes each row independently to sum to one. From now on, we will focus on matrix notation for clarity.

This process is called *self-attention* because we are using the same sequence to determine the queries, keys, and values. We will encounter variants of this attention mechanism later in this chapter. Also, because the measure of similarity between query and key vectors is given by a dot product, this is known as *dot-product self-attention*.

12.1.4 Network parameters

As it stands, the transformation from input vectors $\{\mathbf{x}_n\}$ to output vectors $\{\mathbf{y}_n\}$ is fixed and has no capacity to learn from data because it has no adjustable parameters. Furthermore, each of the feature values within a token vector $\mathbf{x}_n$ plays an equal role in determining the attention coefficients, whereas we would like the network to

have the flexibility to focus more on some features than others when determining token similarity. We can address both issues if we define modified feature vectors given by a linear transformation of the original vectors in the form

$$\widetilde{\mathbf{X}} = \mathbf{X}\mathbf{U} \tag{12.7}$$

where $\mathbf{U}$ is a $D \times D$ matrix of learnable weight parameters, analogous to a 'layer' in a standard neural network. This gives a modified transformation of the form

$$\mathbf{Y} = \text{Softmax}\left[\mathbf{X}\mathbf{U}\mathbf{U}^{\mathrm{T}}\mathbf{X}^{\mathrm{T}}\right]\mathbf{X}\mathbf{U}. \tag{12.8}$$

Although this has much more flexibility, it has the property that the matrix

$$\mathbf{X}\mathbf{U}\mathbf{U}^{\mathrm{T}}\mathbf{X}^{\mathrm{T}} \tag{12.9}$$

is symmetric, whereas we would like the attention mechanism to support significant asymmetry. For example, we might expect that 'chisel' should be strongly associated with 'tool' since every chisel is a tool, whereas 'tool' should only be weakly associated with 'chisel' because there are many other kinds of tools besides chisels. Although the softmax function means the resulting matrix of attention weights is not itself symmetric, we can create a much more flexible model by allowing the queries and the keys to have independent parameters. Furthermore, the form (12.8) uses the same parameter matrix $\mathbf{U}$ to define both the value vectors and the attention coefficients, which again seems like an undesirable restriction.

We can overcome these limitations by defining separate query, key, and value matrices each having their own independent linear transformations:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^{(\mathrm{q})} \tag{12.10}$$

$$\mathbf{K} = \mathbf{X}\mathbf{W}^{(\mathrm{k})} \tag{12.11}$$

$$\mathbf{V} = \mathbf{X}\mathbf{W}^{(\mathrm{v})} \tag{12.12}$$

where the weight matrices $\mathbf{W}^{(\mathrm{q})}$, $\mathbf{W}^{(\mathrm{k})}$, and $\mathbf{W}^{(\mathrm{v})}$ represent parameters that will be learned during the training of the final transformer architecture. Here the matrix $\mathbf{W}^{(\mathrm{k})}$ has dimensionality $D \times D_{\mathrm{k}}$ where D_{k} is the length of the key vector. The matrix $\mathbf{W}^{(\mathrm{q})}$ must have the same dimensionality $D \times D_{\mathrm{k}}$ as $\mathbf{W}^{(\mathrm{k})}$ so that we can form dot products between the query and key vectors. A typical choice is $D_{\mathrm{k}} = D$. Similarly, $\mathbf{W}^{(\mathrm{v})}$ is a matrix of size $D \times D_{\mathrm{v}}$, where D_{v} governs the dimensionality of the output vectors. If we set $D_{\mathrm{v}} = D$, so that the output representation has the same dimensionality as the input, this will facilitate the inclusion of residual connections, which we discuss later. Also, multiple transformer layers can be stacked on top of each other if each layer has the same dimensionality. We can then generalize (12.6) to give

Section 12.1.7

$$\mathbf{Y} = \text{Softmax}\left[\mathbf{Q}\mathbf{K}^{\mathrm{T}}\right]\mathbf{V} \tag{12.13}$$

where $\mathbf{Q}\mathbf{K}^{\mathrm{T}}$ has dimension $N \times N$, and the matrix $\mathbf{Y}$ has dimension $N \times D_{\mathrm{v}}$. The calculation of the matrix $\mathbf{Q}\mathbf{K}^{\mathrm{T}}$ is illustrated in Figure 12.4, whereas the evaluation of the matrix $\mathbf{Y}$ is illustrated in Figure 12.5.

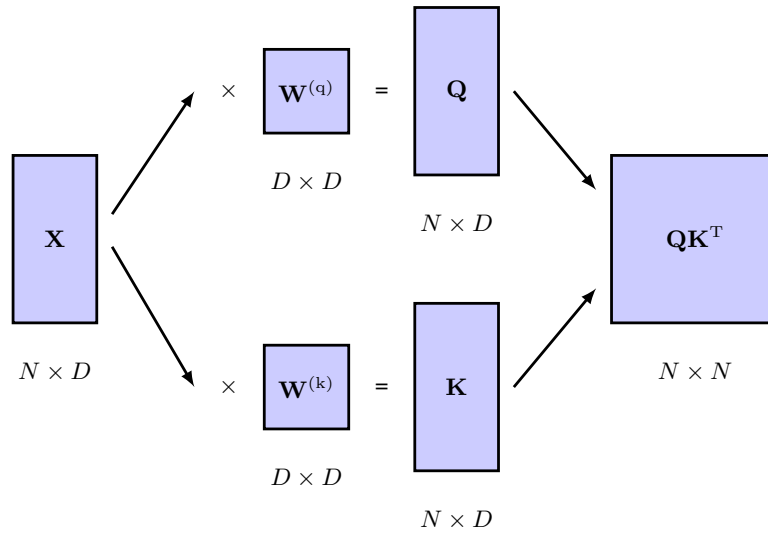


Figure 12.4 Illustration of the evaluation of the matrix $\mathbf{Q}\mathbf{K}^{\mathrm{T}}$, which determines the attention coefficients in a transformer. The input $\mathbf{X}$ is separately transformed using (12.10) and (12.11) to give the query matrix $\mathbf{Q}$ and key matrix $\mathbf{K}$, respectively, which are then multiplied together.

Section 6.2.1

In practice we can also include bias parameters in these linear transformations. However, the bias parameters can be absorbed into the weight matrices, as we did with standard neural networks, by augmenting the data matrix $\mathbf{X}$ with an additional column of 1's and by augmenting the weight matrices with an additional row of parameters to represent the biases. From now on we will treat the bias parameters as implicit to avoid cluttering the notation.

Compared to a conventional neural network, the signal paths have multiplicative relations between activation values. Whereas standard networks multiply activations by fixed weights, here the activations are multiplied by the data-dependent attention coefficients. This means, for example, that if one of the attention coefficients is close to zero for a particular choice of input vector, the resulting signal path will ignore the corresponding incoming signal, which will therefore have no influence

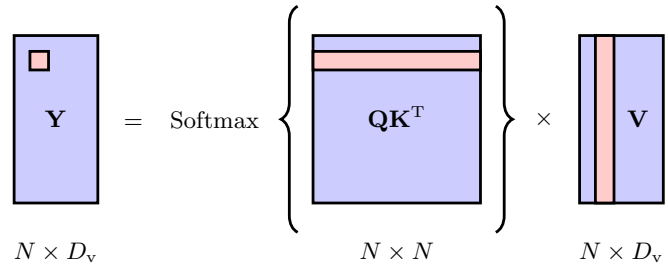


Figure 12.5 Illustration of the evaluation of the output from an attention layer given the query, key, and value matrices $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$, respectively. The entry at the position highlighted in the output matrix $\mathbf{Y}$ is obtained from the dot product of the highlighted row and column of the $\mathrm{Softmax}\left[\mathbf{Q}\mathbf{K}^{\mathrm{T}}\right]$ and $\mathbf{V}$ matrices, respectively.

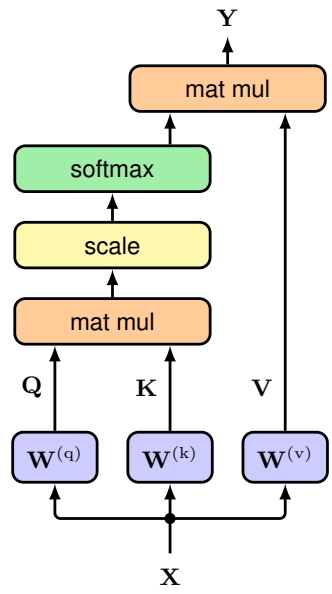


Figure 12.6 Information flow in a scaled dot-product self-attention neural network layer. Here 'mat mul' denotes matrix multiplication, and 'scale' refers to the normalization of the argument to the softmax using $\sqrt{D_{\mathrm{k}}}$. This structure constitutes a single attention 'head'.

on the network outputs. By contrast, if a standard neural network learns to ignore a particular input or hidden-unit variable, it does so for all input vectors.

12.1.5 Scaled self-attention

There is one final refinement we can make to the self-attention layer. Recall that the gradients of the softmax function become exponentially small for inputs of high magnitude, just as happens with $\tanh$ or logistic-sigmoid activation functions. To help prevent this from happening, we can re-scale the product of the query and key vectors before applying the softmax function. To derive a suitable scaling, note that if the elements of the query and key vectors were all independent random numbers with zero mean and unit variance, then the variance of the dot product would be D_{k}. We therefore normalize the argument to the softmax using the standard deviation given by the square root of D_{k}, so that the output of the attention layer takes the form

Exercise 12.4

$$\mathbf{Y} = \mathrm{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \equiv \mathrm{Softmax}\left[\frac{\mathbf{Q}\mathbf{K}^{\mathrm{T}}}{\sqrt{D_{\mathrm{k}}}}\right]\mathbf{V}. \tag{12.14}$$

This is called *scaled dot-product self-attention*, and is the final form of our self-attention neural network layer. The structure of this layer is summarized in Figure 12.6 and in Algorithm 12.1.

12.1.6 Multi-head attention

The attention layer described so far allows the output vectors to attend to data-dependent patterns of input vectors and is called an *attention head*. However, there

Algorithm 12.1: Scaled dot-product self-attention

Input: Set of tokens $\mathbf{X} \in \mathbb{R}^{N \times D} : \{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$
Weight matrices $\{\mathbf{W}^{(\mathrm{q})}, \mathbf{W}^{(\mathrm{k})}\} \in \mathbb{R}^{D \times D_\mathrm{k}}$ and $\mathbf{W}^{(\mathrm{v})} \in \mathbb{R}^{D \times D_\mathrm{v}}$
Output: $\mathrm{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \in \mathbb{R}^{N \times D_\mathrm{v}} : \{\mathbf{y}_1, \ldots, \mathbf{y}_N\}$

$\mathbf{Q} = \mathbf{X}\mathbf{W}^{(\mathrm{q})}$ // compute queries $\mathbf{Q} \in \mathbb{R}^{N \times D_\mathrm{k}}$
$\mathbf{K} = \mathbf{X}\mathbf{W}^{(\mathrm{k})}$ // compute keys $\mathbf{K} \in \mathbb{R}^{N \times D_\mathrm{k}}$
$\mathbf{V} = \mathbf{X}\mathbf{W}^{(\mathrm{v})}$ // compute values $\mathbf{V} \in \mathbb{R}^{N \times D}$
return $\mathrm{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathrm{Softmax}\left[\frac{\mathbf{Q}\mathbf{K}^{\mathrm{T}}}{\sqrt{D_\mathrm{k}}}\right]\mathbf{V}$

might be multiple patterns of attention that are relevant at the same time. In natural language, for example, some patterns might be relevant to tense whereas others might be associated with vocabulary. Using a single attention head can lead to averaging over these effects. Instead we can use multiple attention heads in parallel. These consist of identically structured copies of the single head, with independent learnable parameters that govern the calculation of the query, key, and value matrices. This is analogous to using multiple different filters in each layer of a convolutional network.

Suppose we have H heads indexed by $h = 1, \ldots, H$ of the form

$$\mathbf{H}_h = \mathrm{Attention}(\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h) \tag{12.15}$$

where $\mathrm{Attention}(\cdot, \cdot, \cdot)$ is given by (12.14), and we have defined separate query, key, and value matrices for each head using

$$\mathbf{Q}_h = \mathbf{X}\mathbf{W}_h^{(\mathrm{q})} \tag{12.16}$$

$$\mathbf{K}_h = \mathbf{X}\mathbf{W}_h^{(\mathrm{k})} \tag{12.17}$$

$$\mathbf{V}_h = \mathbf{X}\mathbf{W}_h^{(\mathrm{v})}. \tag{12.18}$$

The heads are first concatenated into a single matrix, and the result is then linearly transformed using a matrix $\mathbf{W}^{(\mathrm{o})}$ to give a combined output in the form

$$\mathbf{Y}(\mathbf{X}) = \mathrm{Concat}\left[\mathbf{H}_1, \ldots, \mathbf{H}_H\right]\mathbf{W}^{(\mathrm{o})}. \tag{12.19}$$

This is illustrated in Figure 12.7.

Each matrix $\mathbf{H}_h$ has dimension $N \times D_\mathrm{v}$, and so the concatenated matrix has dimension $N \times HD_\mathrm{v}$. This is transformed by the linear matrix $\mathbf{W}^{(\mathrm{o})}$ of dimension $HD_\mathrm{v} \times D$ to give the final output matrix $\mathbf{Y}$ of dimension $N \times D$, which is the same as the original input matrix $\mathbf{X}$. The elements of the matrix $\mathbf{W}^{(\mathrm{o})}$ are learned during the training phase along with the query, key, and value matrices. Typically D_v is

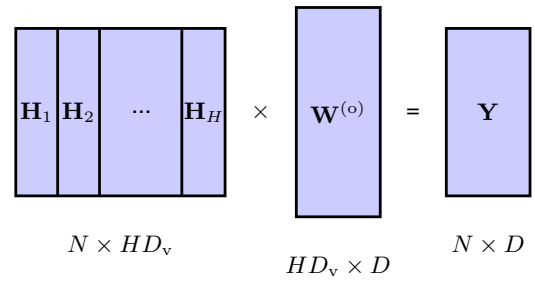


Figure 12.7 Network architecture for multi-head attention. Each head comprises the structure shown in Figure 12.6, and has its own key, query, and value parameters. The outputs of the heads are concatenated and then linearly projected back to the input data dimensionality.

chosen to be equal to D/H so that the resulting concatenated matrix has dimension $N \times D$. Multi-head attention is summarized in Algorithm 12.2, and the information flow in a multi-head attention layer is illustrated in Figure 12.8.

Note that the formulation of multi-head attention given above, which follows that used in the research literature, includes some redundancy in the successive multiplication of the $\mathbf{W}^{(\mathrm{v})}$ matrix for each head and the output matrix $\mathbf{W}^{(\mathrm{o})}$. Removing this redundancy allows a multi-head self-attention layer to be written as a sum over contributions from each of the heads separately.

Exercise 12.5

12.1.7 Transformer layers

Multi-head self-attention forms the core architectural element in a transformer network. We know that neural networks benefit greatly from depth, and so we would like to stack multiple self-attention layers on top of each other. To improve training

Algorithm 12.2: Multi-head attention

Input: Set of tokens $\mathbf{X} \in \mathbb{R}^{N\times D} : \{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$
Query weight matrices $\{\mathbf{W}_1^{(\mathrm{q})}, \ldots, \mathbf{W}_H^{(\mathrm{q})}\} \in \mathbb{R}^{D\times D}$
Key weight matrices $\{\mathbf{W}_1^{(\mathrm{k})}, \ldots, \mathbf{W}_H^{(\mathrm{k})}\} \in \mathbb{R}^{D\times D}$
Value weight matrices $\{\mathbf{W}_1^{(\mathrm{v})}, \ldots, \mathbf{W}_H^{(\mathrm{v})}\} \in \mathbb{R}^{D\times D_\mathrm{v}}$
Output weight matrix $\mathbf{W}^{(\mathrm{o})} \in \mathbb{R}^{HD_\mathrm{v}\times D}$
Output: $\mathbf{Y} \in \mathbb{R}^{N\times D} : \{\mathbf{y}_1, \ldots, \mathbf{x}_N\}$

```
// compute self-attention for each head (Algorithm 12.1)
```
for $h = 1, \ldots, H$ **do**
  $\mathbf{Q}_h = \mathbf{X}\mathbf{W}_h^{(\mathrm{q})}, \quad \mathbf{K}_h = \mathbf{X}\mathbf{W}_h^{(\mathrm{k})}, \quad \mathbf{V}_h = \mathbf{X}\mathbf{W}_h^{(\mathrm{v})}$
  $\mathbf{H}_h = \mathrm{Attention}\left(\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h\right)$ // $\mathbf{H}_h \in \mathbb{R}^{N\times D_\mathrm{v}}$
end for
$\mathbf{H} = \mathrm{Concat}\left[\mathbf{H}_1, \ldots, \mathbf{H}_N\right]$ // concatenate heads
return $\mathbf{Y}(\mathbf{X}) = \mathbf{H}\mathbf{W}^{(\mathrm{o})}$

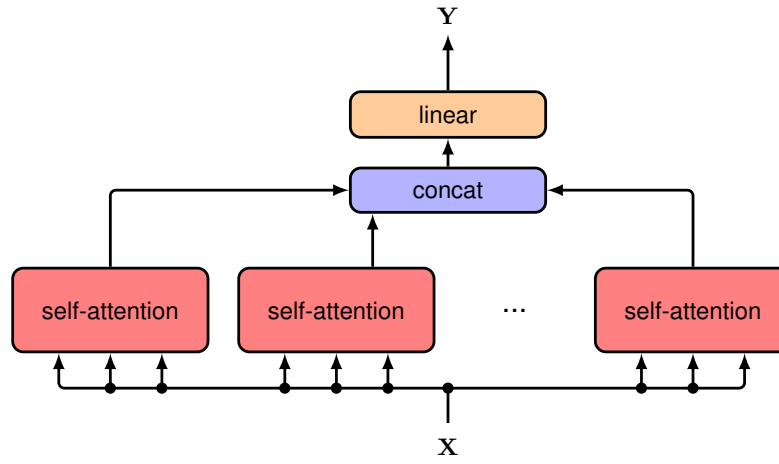


Figure 12.8 Information flow in a multi-head attention layer. The associated computation, given by Algorithm 12.2, is illustrated in Figure 12.7.

Section 9.5

Section 7.4.3

efficiency, we can introduce residual connections that bypass the multi-head structure. To do this we require that the output dimensionality is the same as the input dimensionality, namely $N \times D$. This is then followed by *layer normalization* (Ba, Kiros, and Hinton, 2016), which improves training efficiency. The resulting transformation can be written as

$$\mathbf{Z} = \text{LayerNorm}\left[\mathbf{Y}(\mathbf{X}) + \mathbf{X}\right] \tag{12.20}$$

where $\mathbf{Y}$ is defined by (12.19). Sometimes the layer normalization is replaced by *pre-norm* in which the normalization layer is applied before the multi-head self-attention instead of after, as this can result in more effective optimization, in which case we have

$$\mathbf{Z} = \mathbf{Y}(\mathbf{X}') + \mathbf{X}, \quad \text{where} \quad \mathbf{X}' = \text{LayerNorm}\left[\mathbf{X}\right]. \tag{12.21}$$

In each case, $\mathbf{Z}$ again has the same dimensionality $N \times D$ as the input matrix $\mathbf{X}$.

We have seen that the attention mechanism creates linear combinations of the value vectors, which are then linearly combined to produce the output vectors. Also, the values are linear functions of the input vectors, and so we see that the outputs of an attention layer are constrained to be linear combinations of the inputs. Nonlinearity does enter through the attention weights, and so the outputs will depend nonlinearly on the inputs via the softmax function, but the output vectors are still constrained to lie in the subspace spanned by the input vectors and this limits the expressive capabilities of the attention layer. We can enhance the flexibility of the transformer by post-processing the output of each layer using a standard nonlinear neural network with D inputs and D outputs, denoted $\text{MLP}[\cdot]$ for 'multilayer perceptron'. For example, this might consist of a two-layer fully connected network with ReLU hidden units. This needs to be done in a way that preserves the ability

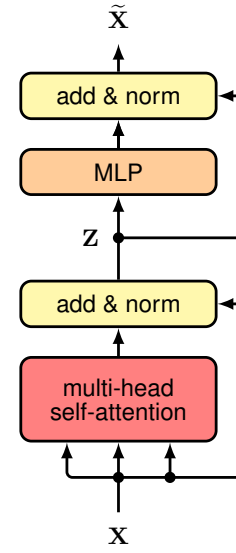


Figure 12.9 One layer of the transformer architecture that implements the transformation (12.1). Here 'MLP' stands for multilayer perceptron, while 'add and norm' denotes a residual connection followed by layer normalization.

of the transformer to process sequences of variable length. To achieve this, the same shared network is applied to each of the output vectors, corresponding to the rows of $\mathbf{Z}$. Again, this neural network layer can be improved by using a residual connection. It also includes layer normalization so that the final output from the transformer layer has the form

$$\widetilde{\mathbf{X}} = \text{LayerNorm}\left[\text{MLP}\left[\mathbf{Z}\right] + \mathbf{Z}\right]. \tag{12.22}$$

This leads to an overall architecture for a transformer layer shown in Figure 12.9 and summarized in Algorithm 12.3. Again, we can use a pre-norm instead, in which case the final output is given by

$$\widetilde{\mathbf{X}} = \text{MLP}(\mathbf{Z}') + \mathbf{Z}, \quad \text{where} \quad \mathbf{Z}' = \text{LayerNorm}\left[\mathbf{Z}\right]. \tag{12.23}$$

In a typical transformer there are multiple such layers stacked on top of each other. The layers generally have identical structures, although there is no sharing of weights and biases between different layers.

12.1.8 Computational complexity

The attention layer discussed so far takes a set of N vectors each of length D and maps them into another set of N vectors having the same dimensionality. Thus, the inputs and outputs each have overall dimensionality ND. If we had used a standard fully connected neural network to map the input values to the output values, it would have $\mathcal{O}(N^2D^2)$ independent parameters. Likewise the computational cost of evaluating one forward pass through such a network would also be $\mathcal{O}(N^2D^2)$.

In the attention layer, the matrices $\mathbf{W}^{(\text{q})}$, $\mathbf{W}^{(\text{k})}$, and $\mathbf{W}^{(\text{v})}$ are shared across input tokens, and therefore the number of independent parameters is $\mathcal{O}(D^2)$, assuming $D_\text{k} \simeq D_\text{v} \simeq D$. Since there are N input tokens, the number of computational steps

Algorithm 12.3: Transformer layer

Input: Set of tokens $\mathbf{X} \in \mathbb{R}^{N \times D} : \{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$
Multi-head self-attention layer parameters
Feed-forward network parameters
Output: $\widetilde{\mathbf{X}} \in \mathbb{R}^{N \times D} : \{\widetilde{\mathbf{x}}_1, \ldots, \widetilde{\mathbf{x}}_N\}$

$\mathbf{Z} = \text{LayerNorm}\left[\mathbf{Y}(\mathbf{X}) + \mathbf{X}\right]$ `// Y(X) from Algorithm 12.2`
$\widetilde{\mathbf{X}} = \text{LayerNorm}\left[\text{MLP}\left[\mathbf{Z}\right] + \mathbf{Z}\right]$ `// shared neural network`
return $\widetilde{\mathbf{X}}$

in evaluating the dot products in a self-attention layer is $\mathcal{O}(N^2D)$. We can think of a self-attention layer as a sparse matrix in which parameters are shared between specific blocks of the matrix. The subsequent neural network layer, which has D inputs and D outputs, has a cost that is $\mathcal{O}(D^2)$. Since it is shared across tokens, it has a complexity that is linear in N, and therefore overall this layer has a cost that is $\mathcal{O}(ND^2)$. Depending on the relative sizes of N and D, either the transformer layer or the MLP layer may dominate the computational cost. Compared to a fully connected network, a transformer layer is computationally more efficient. Many variants of the transformer architecture have been proposed (Lin *et al.*, 2021; Phuong and Hutter, 2022) including modifications aimed at improving efficiency (Tay *et al.*, 2020).

Exercise 12.6

12.1.9 Positional encoding

In the transformer architecture, the matrices $\mathbf{W}_h^{(\text{q})}$, $\mathbf{W}_h^{(\text{k})}$, and $\mathbf{W}_h^{(\text{v})}$ are shared across the input tokens, as is the subsequent neural network. As a consequence, the transformer has the property that permuting the order of the input tokens, i.e., the rows of $\mathbf{X}$, results in the same permutation of the rows of the output matrix $\widetilde{\mathbf{X}}$. In other words a transformer is *equivariant* with respect to input permutations. The sharing of parameters in the network architecture facilitates the massively parallel processing of the transformer, and also allows the network to learn long-range dependencies just as effectively as short-range dependencies. However, the lack of dependence on token order becomes a major limitation when we consider sequential data, such as the words in a natural language, because the representation learned by a transformer will be independent of the input token ordering. The two sentences '*The food was bad, not good at all.*' and '*The food was good, not bad at all.*' contain the same tokens but they have very different meanings because of the different token ordering. Clearly token order is crucial for most sequential processing tasks including natural language processing, and so we need to find a way to inject token order information into the network.

Exercise 12.7
Section 10.2

Since we wish to retain the powerful properties of the attention layers that we have carefully constructed, we aim to encode the token order in the data itself in-

stead of having to be represented in the network architecture. We will therefore construct a position encoding vector $\mathbf{r}_n$ associated with each input position n and then combine this with the associated input token embedding $\mathbf{x}_n$. One obvious way to combine these vectors would be to concatenate them, but this would increase the dimensionality of the input space and hence of all subsequent attention spaces, creating a significant increase in computational cost. Instead, we can simply add the position vectors onto the token vectors to give

$$\widetilde{\mathbf{x}}_n = \mathbf{x}_n + \mathbf{r}_n. \tag{12.24}$$

This requires that the positional encoding vectors have the same dimensionality as the token-embedding vectors.

At first it might seem that adding position information onto the token vector would corrupt the input vectors and make the task of the network much more difficult. However, some intuition as to why this can work well comes from noting that two randomly chosen uncorrelated vectors tend to be nearly orthogonal in spaces of high dimensionality, indicating that the network is able to process the token identity information and the position information relatively separately. Note also that, because of the residual connections across every layer, the position information does not get lost in going from one transformer layer to the next. Moreover, due to the linear processing layers in the transformer, a concatenated representation has similar properties to an additive one.

Exercise 12.8

Exercise 12.9

The next task is to construct the embedding vectors $\{\mathbf{r}_n\}$. A simple approach would be to associate an integer $1, 2, 3, \ldots$ with each position. However, this has the problem that the magnitude of the value increases without bound and therefore may start to corrupt the embedding vector significantly. Also it may not generalize well to new input sequences that are longer than those used in training, since these will involve coding values that lie outside the range of those used in training. Alternatively we could assign a number in the range $(0, 1)$ to each token in the sequence, which keeps the representation bounded. However, this representation is not unique for a given position as it depends on the overall sequence length.

An ideal positional encoding should provide a unique representation for each position, it should be bounded, it should generalize to longer sequences, and it should have a consistent way to express the number of steps between any two input vectors irrespective of their absolute position because the relative position of tokens is often more important than the absolute position.

There are many approaches to positional encoding (Dufter, Schmitt, and Schütze, 2021). Here we describe a technique based on sinusoidal functions introduced by Vaswani *et al.* (2017). For a given position n the associated position-encoding vector has components r_{ni} given by

$$r_{ni} = \begin{cases} \sin\left(\dfrac{n}{L^{i/D}}\right), & \text{if } i \text{ is even,} \\ \cos\left(\dfrac{n}{L^{(i-1)/D}}\right), & \text{if } i \text{ is odd.} \end{cases} \tag{12.25}$$

We see that the elements of the embedding vector $\mathbf{r}_n$ are given by a series of sine and cosine functions of steadily increasing wavelength, as illustrated in Figure 12.10(a).

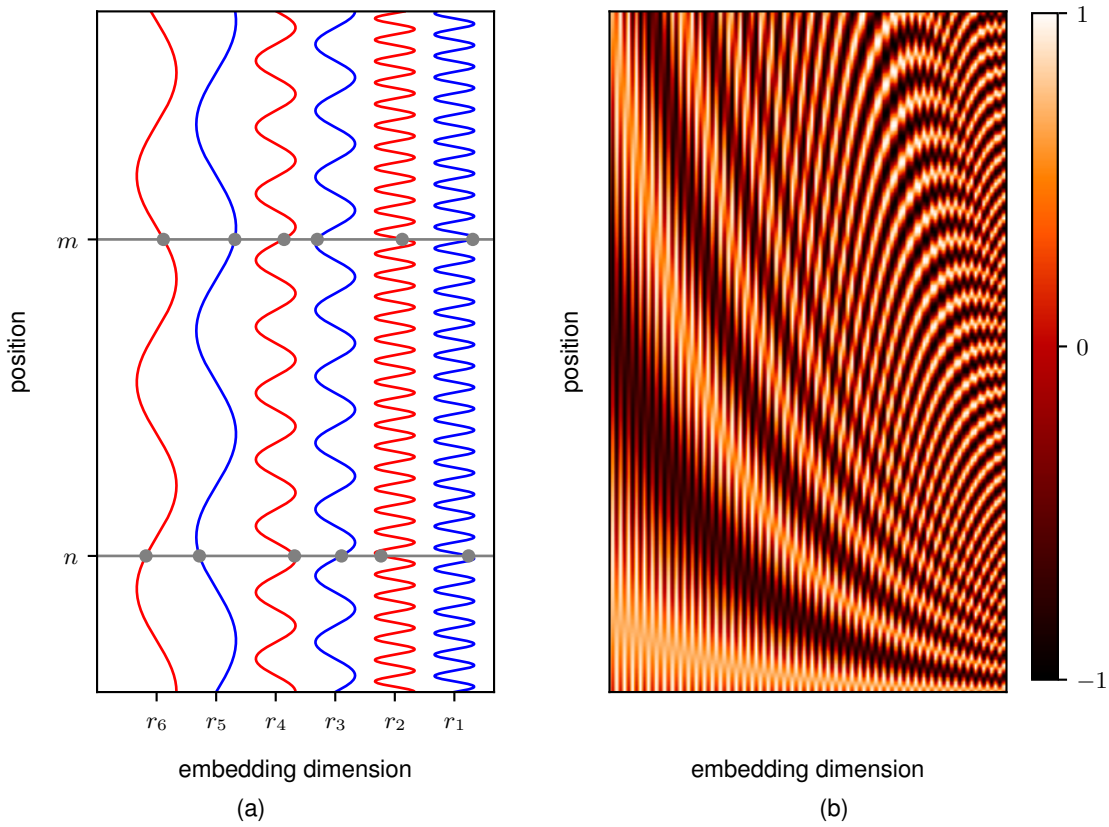


Figure 12.10 Illustrations of the functions defined by (12.25) and used to construct position-encoding vectors. (a) A plot in which the horizontal axis shows the different components of the embedding vector $\mathbf{r}$ whereas the vertical axis shows the position in the sequence. The values of the vector elements for two positions n and m are shown by the intersections of the sine and cosine curves with the horizontal grey lines. (b) A heat map illustration of the position-encoding vectors defined by (12.25) for dimension $D = 100$ with $L = 30$ for the first $N = 200$ positions.

This encoding has the property that the elements of the vector $\mathbf{r}_n$ all lie in the range $(-1, 1)$. It is reminiscent of the way binary numbers are represented, with the lowest order bit alternating with high frequency, and subsequent bits alternating with steadily decreasing frequencies:

$$\begin{array}{ll} \mathbf{1}: & 0\ 0\ 0\ 1 \\ \mathbf{2}: & 0\ 0\ 1\ 0 \\ \mathbf{3}: & 0\ 0\ 1\ 1 \\ \mathbf{4}: & 0\ 1\ 0\ 0 \\ \mathbf{5}: & 0\ 1\ 0\ 1 \\ \mathbf{6}: & 0\ 1\ 1\ 0 \\ \mathbf{7}: & 0\ 1\ 1\ 1 \\ \mathbf{8}: & 1\ 0\ 0\ 0 \\ \mathbf{9}: & 1\ 0\ 0\ 1 \end{array}$$

For the encoding given by (12.25), however, the vector elements are continuous variables rather than binary. A plot of the position-encoding vectors is shown in Figure 12.10(b).

One nice property of the sinusoidal representation given by (12.25) is that, for any fixed offset k, the encoding at position $n + k$ can be represented as a linear combination of the encoding at position n, in which the coefficients do not depend on the absolute position but only on the value of k. The network should therefore be able to learn to attend to relative positions. Note that this property requires that the encoding makes use of both sine and cosine functions.

Exercise 12.10

Another popular approach to positional representation is to use learned position encodings. This is done by having a vector of weights at each token position that can be learned jointly with the rest of the model parameters during training, and avoids using hand-crafted representations. Because the parameters are not shared between the token positions, the tokens are no longer invariant under a permutation, which is the purpose of a positional encoding. However, this approach does not meet the criteria we mentioned earlier of generalizing to longer input sequences, as the encoding will be untrained for positional encodings not seen during training. Therefore, this approach is generally most suitable when the input length is relatively constant during both training and inference.

12.2. Natural Language

Now that we have studied the architecture of the transformer, we will explore how this can be used to process language data consisting of words, sentences, and paragraphs. Although this is the modality that transformers were originally developed to operate on, they have proved to be a very general class of models and have become the state-of-the-art for most input data types. Later in this chapter we will look at their use in other domains.

Section 12.4

Many languages, including English, comprise a series of words separated by white space, along with punctuation symbols, and therefore represent an example of

Section 11.3
Section 12.2.2

sequential data. For the moment we will focus on the words, and we will return to punctuation later.

The first challenge is to convert the words into a numerical representation that is suitable for use as the input to a deep neural network. One simple approach is to define a fixed dictionary of words and then introduce vectors of length equal to the size of the dictionary along with a 'one hot' representation for each word, in which the kth word in the dictionary is encoded with a vector having a 1 in position k and 0 in all other positions. For example if 'aardwolf' is the third word in our dictionary then its vector representation would be $(0, 0, 1, 0, \ldots, 0)$.

An obvious problem with a one-hot representation is that a realistic dictionary might have several hundred thousand entries leading to vectors of very high dimensionality. Also, it does not capture any similarities or relationships that might exist between words. Both issues can be addressed by mapping the words into a lower-dimensional space through a process called *word embedding* in which each word is represented as a dense vector in a space of typically a few hundred dimensions.

12.2.1 Word embedding

The embedding process can be defined by a matrix $\mathbf{E}$ of size $D \times K$ where D is the dimensionality of the embedding space and K is the dimensionality of the dictionary. For each one-hot encoded input vector $\mathbf{x}_n$ we can then calculate the corresponding embedding vector using

$$\mathbf{v}_n = \mathbf{E}\mathbf{x}_n. \tag{12.26}$$

Because $\mathbf{x}_n$ has a one-hot encoding, the vector $\mathbf{v}_n$ is simply given by the corresponding column of the matrix $\mathbf{E}$.

We can learn the matrix $\mathbf{E}$ from a corpus (i.e., a large data set) of text, and there are many approaches to doing this. Here we look at a popular technique called *word2vec* (Mikolov *et al.*, 2013), which can be viewed as a simple two-layer neural network. A training set is constructed in which each sample is obtained by considering a 'window' of M adjacent words in the text, where a typical value might be $M = 5$. The samples are considered to be independent, and the error function is defined as the sum of the error functions for each sample. There are two variants of this approach. In *continuous bag of words*, the target variable for network training is the middle word, and the remaining *context* words form the inputs, so that the network is being trained to 'fill in the blank'. A closely related approach, called *skip-grams*, reverses the inputs and outputs, so that the centre word is presented as the input and the target values are the context words. These models are illustrated in Figure 12.11.

This training procedure can be viewed as a form of *self-supervised* learning since the data consists simply of a large corpus of unlabelled text from which many small windows of word sequences are drawn at random. Labels are obtained from the text itself by 'masking' out those words whose values the network is trying to predict.

Once the model is trained, the embedding matrix $\mathbf{E}$ is given by the transpose of the second-layer weight matrix for the continuous bag-of-words approach and by the first-layer weight matrix for skip-grams. Words that are semantically related are mapped to nearby positions in the embedding space. This is to be expected

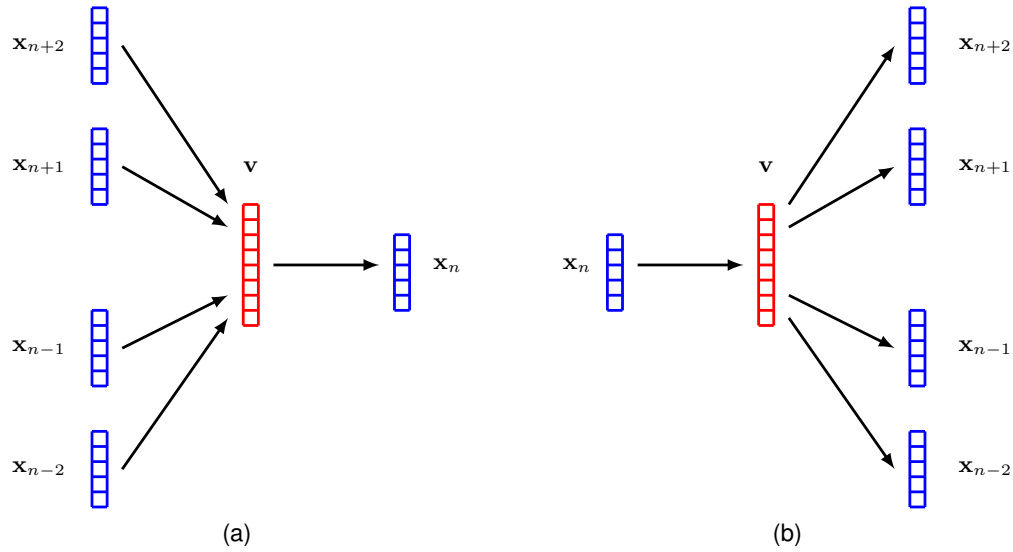


Figure 12.11 Two-layer neural networks used to learn word embeddings, where (a) shows the continuous bag-of-words approach, and (b) shows the skip-grams approach.

since related words are more likely to occur with similar context words compared to unrelated words. For example, the words 'city' and 'capital' might occur with higher frequency as context for target words such as 'Paris' or 'London' and less frequently as context for 'orange' or 'polynomial'. The network can more easily predict the probability of the missing words if 'Paris' and 'London' are mapped to nearby embedding vectors.

It turns out that the learned embedding space often has an even richer semantic structure than just the proximity of related words, and that this allows for simple vector arithmetic. For example, the concept that 'Paris is to France as Rome is to Italy' can be expressed through operations on the embedding vectors. If we use $\mathbf{v}(\text{word})$ to denote the embedding vector for 'word', then we find

$$\mathbf{v}(\text{Paris}) - \mathbf{v}(\text{France}) + \mathbf{v}(\text{Italy}) \simeq \mathbf{v}(\text{Rome}). \tag{12.27}$$

Word embeddings were originally developed as natural language processing tools in their own right. Today, they are more likely to be used as pre-processing steps for deep neural networks. In this regard they can be viewed as the first layer in a deep neural network. They can be fixed using some standard pre-trained embedding matrix, or they can be treated as an adaptive layer that is learned as part of the overall end-to-end training of the system. In the latter case the embedding layer can be initialized either using random weight values or using a standard embedding matrix.

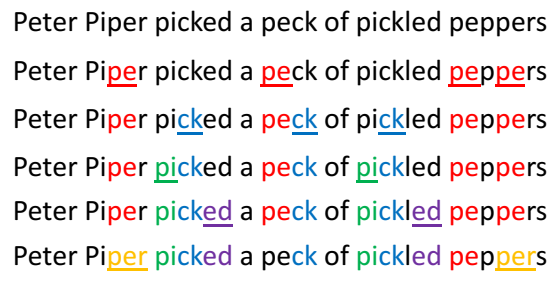


Figure 12.12 An illustration of the process of tokenizing natural language by analogy with byte pair encoding. In this example, the most frequently occurring pair of characters is 'pe', which occurs four times, and so these form a new token that replaces all the occurrences of 'pe'. Note that 'Pe' is not included in this since upper-case 'P' and lower-case 'p' are distinct characters. Next the pair 'ck' is added since this occurs three times. This is followed by tokens such as 'pi', 'ed', and 'per', all of which occur twice, and so on.

12.2.2 Tokenization

One problem with using a fixed dictionary of words is that it cannot cope with words not in the dictionary or which are misspelled. It also does not take account of punctuation symbols or other character sequences such as computer code. An alternative approach that addresses these problems would be to work at the level of characters instead of using words, so that our dictionary comprises upper-case and lower-case letters, numbers, punctuation, and white-space symbols such as spaces and tabs. A disadvantage of this approach, however, is that it discards the semantically important word structure of language, and the subsequent neural network would have to learn to reassemble words from elementary characters. It would also require a much larger number of sequential steps for a given body of text, thereby increasing the computational cost of processing the sequence.

We can combine the benefits of character-level and word-level representations by using a pre-processing step that converts a string of words and punctuation symbols into a string of *tokens*, which are generally small groups of characters and might include common words in their entirety, along with fragments of longer words as well as individual characters that can be assembled into less common words (Schuster and Nakajima, 2012). This tokenization also allows the system to process other kinds of sequences such as computer code or even other modalities such as images. It also means that variations of the same word can have related representations. For example, 'cook', 'cooks', 'cooked', 'cooking', and 'cooker' are all related and share the common element 'cook', which itself could be represented as one of the tokens.

Section 12.4.1

There are many approaches to tokenization. As an example, a technique called *byte pair encoding* that is used for data compression, can be adapted to text tokenization by merging characters instead of bytes (Sennrich, Haddow, and Birch, 2015). The process starts with the individual characters and iteratively merges them into longer strings. The list of tokens is first initialized with the list of individual characters. Then a body of text is searched for the most frequently occurring adjacent pairs of tokens and these are replaced with a new token. To ensure that words are not merged, a new token is not formed from two tokens if the second token starts with a white space. The process is repeated iteratively as illustrated in Figure 12.12.

Initially the number of tokens is equal to the number of characters, which is relatively small. As tokens are formed, the total number of tokens increases, and

if this is continued long enough, the tokens will eventually correspond to the set of words in the text. The total number of tokens is generally fixed in advance, as a compromise between character-level and word-level representations. The algorithm is stopped when this number of tokens is reached.

In practical applications of deep learning to natural language, the input text is typically first mapped into a tokenized representation. However, for the remainder of this chapter, we will use word-level representations as this makes it easier to illustrate and motivate key concepts.

12.2.3 Bag of words

We now turn to the task of modelling the joint distribution $p(\mathbf{x}_1, \ldots, \mathbf{x}_N)$ of an ordered sequence of vectors, such as words (or tokens) in a natural language. The simplest approach is to assume that the words are drawn independently from the same distribution and hence that the joint distribution is fully factorized in the form

$$p(\mathbf{x}_1, \ldots, \mathbf{x}_N) = \prod_{n=1}^{N} p(\mathbf{x}_n). \tag{12.28}$$

This can be expressed as a probabilistic graphical model in which the nodes are isolated with no interconnecting links.

Figure 11.28

The distribution $p(\mathbf{x})$ is shared across the variables and can be represented, without loss of generality, as a simple table listing the probabilities of each of the possible states of $\mathbf{x}$ (corresponding to the dictionary of words or tokens). The maximum likelihood solution for this model is obtained simply by setting each of these probabilities to the fraction of times that the word occurs in the training set. This is known as a *bag-of-words* model because it completely ignores the ordering of the words.

Exercise 12.11

We can use the bag-of-words approach to construct a simple text classifier. This could be used for example in sentiment analysis in which a passage of text representing a restaurant review is to be classified as positive or negative. The *naive Bayes* classifier assumes that the words are independent within each class $\mathcal{C}_k$, but with a different distribution for each class, so that

$$p(\mathbf{x}_1, \ldots, \mathbf{x}_N | \mathcal{C}_k) = \prod_{n=1}^{N} p(\mathbf{x}_n | \mathcal{C}_k). \tag{12.29}$$

Given prior class probabilities $p(\mathcal{C}_k)$, the posterior class probabilities for a new sequence are given by:

$$p(\mathcal{C}_k | \mathbf{x}_1, \ldots, \mathbf{x}_N) \propto p(\mathcal{C}_k) \prod_{n=1}^{N} p(\mathbf{x}_n | \mathcal{C}_k). \tag{12.30}$$

Both the class-conditional densities $p(\mathbf{x}|\mathcal{C}_k)$ and the prior probabilities $p(\mathcal{C}_k)$ can be estimated using frequencies from the training data set. For a new sequence, the table entries are multiplied together to get the desired posterior probabilities. Note that if a word occurs in the test set that was not present in the training set then the

corresponding probability estimate will be zero, and so these estimates are typically 'smoothed' after training by reassigning a small level of probability uniformly across all entries to avoid zero values.

12.2.4 Autoregressive models

One obvious major limitation of the bag-of-words model is that it completely ignores word order. To address this we can take an autoregressive approach. Without loss of generality we can decompose the distribution over the sequence of words into a product of conditional distributions in the form

$$p(\mathbf{x}_1, \ldots, \mathbf{x}_N) = \prod_{n=1}^{N} p(\mathbf{x}_n | \mathbf{x}_1, \ldots, \mathbf{x}_{n-1}). \tag{12.31}$$

Figure 11.27

This can be represented as a probabilistic graphical model in which each node in the sequence receives a link from every previous node. We could represent each term on the right-hand side of (12.31) by a table whose entries are once again estimated using simple frequency counts from the training set. However, the size of these tables grows exponentially with the length of the sequence, and so this approach would become prohibitively expensive.

Exercise 12.12

We can simplify the model dramatically by assuming that each of the conditional distributions on the right-hand side of (12.31) is independent of all previous observations except the L most recent words. For example, if $L = 2$ then the joint distribution for a sequence of N observations under this model is given by

$$p(\mathbf{x}_1, \ldots, \mathbf{x}_N) = p(\mathbf{x}_1)p(\mathbf{x}_2|\mathbf{x}_1) \prod_{n=3}^{N} p(\mathbf{x}_n | \mathbf{x}_{n-1}, \mathbf{x}_{n-2}). \tag{12.32}$$

Figure 11.30

In the corresponding graphical model each node has links from the two previous nodes. Here we assume that the conditional distributions $p(\mathbf{x}_n|\mathbf{x}_{n-1})$ are shared across all variables. Again each of the distributions on the right-hand side of (12.32) can be represented as tables whose values are estimated from the statistics of triplets of successive words drawn from a training corpus.

The case with $L = 1$ is known as a *bi-gram* model because it depends on pairs of adjacent words. Similarly $L = 2$, which involves triplets of adjacent words, is called a *tri-gram* model, and in general these are called *n-gram* models.

All the models discussed so far in this section can be run *generatively* to synthesize novel text. For example, if we provide the first and second words in a sequence, then we can sample from the tri-gram statistics $p(\mathbf{x}_n|\mathbf{x}_{n-1}, \mathbf{x}_{n-2})$ to generate the third word, and then we can use the second and third words to sample the fourth word, and so on. The resulting text, however, will be incoherent because each word is predicted only on the basis of the two previous words. High-quality text models must take account of the long-range dependencies in language. On the other hand, we cannot simply increase the value of L because the size of the probability tables grows exponentially in L so that it is prohibitively expensive to go much beyond tri-gram models. However, the autoregressive representation will play a central role

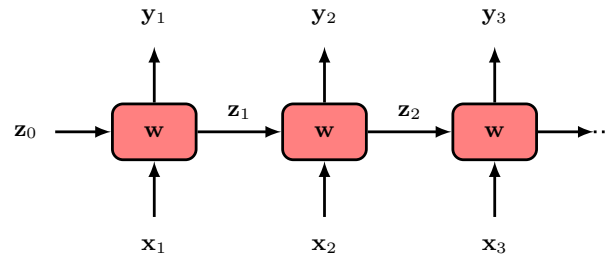


Figure 12.13 A general RNN with parameters $\mathbf{w}$. It takes a sequence $\mathbf{x}_1, \ldots, \mathbf{x}_N$ as input and generates a sequence $\mathbf{y}_1, \ldots, \mathbf{y}_N$ as output. Each of the boxes corresponds to a multilayer network with nonlinear hidden units.

when we consider modern language models based not on probability tables but on deep neural networks configured as transformers.

One way to allow longer-range dependencies, while avoiding the exponential growth in the number of parameters of an n-gram model, is to use a *hidden Markov model* whose graphical structure is shown in Figure 11.31. The number of learnable parameters is governed by the dimensionality of the latent variables whereas the distribution over a given observation $\mathbf{x}_n$ depends, in principle, on all previous observations. However, the influence of more distant observations is still very limited since their effect must be carried through the chain of latent states which are themselves being updated by more recent observations.

Section 11.3.1

12.2.5 Recurrent neural networks

Techniques such as n-grams have very poor scaling with sequence length because they store completely general tables of conditional distributions. We can achieve much better scaling by using parameterized models based on neural networks. Suppose we simply apply a standard feed-forward neural network to sequences of words in natural language. One problem that arises is that the network has a fixed number of inputs and outputs, whereas we need to be able to handle sequences in the training and test sets that have variable length. Furthermore, if a word, or group of words, at a particular location in a sequence represents some concept then the same word, or group of words, at a different location is likely to represent the same concept at that new location. This is reminiscent of the equivariance property we encountered in processing image data. If we can construct a network architecture that is able to share parameters across the sequence then not only can we capture this equivariance property but we can greatly reduce the number of free parameters in the model as well as handle sequences having different lengths.

Chapter 10

To address this we can borrow inspiration from the hidden Markov model and introduce an explicit hidden variable $\mathbf{z}_n$ associated with each step n in the sequence. The neural network takes as input both the current word $\mathbf{x}_n$ and the current hidden state $\mathbf{z}_{n-1}$ and produces an output word $\mathbf{y}_n$ as well as the next state $\mathbf{z}_n$ of the hidden variable. We can then chain together copies of this network, in which the weight values are shared across the copies. The resulting architecture is called a *recurrent neural network* (RNN) and is illustrated in Figure 12.13. Here the initial value of the hidden state may be initialized for example to some default value such as $\mathbf{z}_0 =$

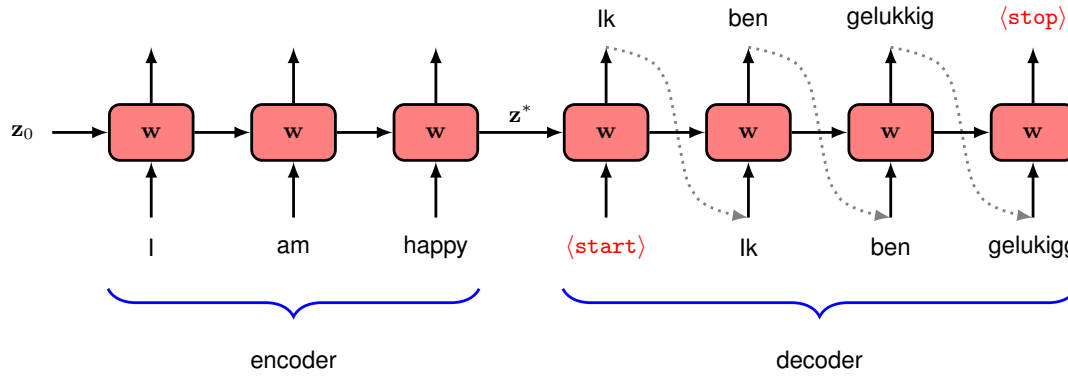


Figure 12.14 An example of a recurrent neural network used for language translation. See the text for details.

$(0, 0, \ldots, 0)^{\mathrm{T}}$.

As an example of how an RNN might be used in practice, consider the specific task of translating sentences from English into Dutch. The sentences can have variable length, and each output sentence might have a different length from the corresponding input sentence. Furthermore, the network may need to see the whole of the input sentence before it can even start to generate the output sentence. We can address this using an RNN by feeding in the complete English sentence followed by a special input token, which we denote by ⟨start⟩, to trigger the start of translation. During training the network learns to associate ⟨start⟩ with the beginning of the output sentence. We also take each successively generated word and feed it into the input at the next time step, as shown in Figure 12.14. The network can be trained to generate a specific ⟨stop⟩ token to signify the completion of the translation. The first few stages of the network are used to absorb the input sequence, and the associated output vectors are simply ignored. This part of the network can be viewed as an 'encoder' in which the entire input sentence has been compressed into the state $\mathbf{z}^\star$ of the hidden variable. The remaining network stages function as the 'decoder', which generates the translated sentence as output one word at a time. Notice that each output word is fed as input to the next stage of the network, and so this approach has an autoregressive structure analogous to (12.31).

12.2.6 Backpropagation through time

RNNs can be trained by stochastic gradient descent using gradients calculated by backpropagation and evaluated through automatic differentiation, just as with regular neural networks. The error function consists of a sum over all output units of the error for each unit, in which each output unit has a softmax activation function along with an associated cross-entropy error function. During forward propagation, the activation values are propagated all the way from the first input in the sequence through to all the output nodes in the sequence, and error signals are then backpropagated along the same paths. This process is called *backpropagation through time*

Section 5.4.4

and in principle is straightforward. However, in practice, for very long sequences, training can be difficult due to the problems of *vanishing gradients* or *exploding gradients* that arise with very deep network architectures.

Section 7.4.2

Another problem with standard RNNs is that they deal poorly with long-range dependencies. This is especially problematic for natural language where such dependencies are widespread. In a long passage of text, a concept might be introduced that plays an important role in predicting words occurring much later in the text. In the architecture shown in Figure 12.14, the entire concept of the English sentence must be captured in the single hidden vector $\mathbf{z}^\star$ of fixed length, and this becomes increasingly problematic with longer sequences. This is known as the *bottleneck problem* because a sequence of arbitrary length has to be summarized in a single hidden vector of activations and the network can start to generate the output translation only once the full input sequence has been processed.

One approach for addressing both the vanishing and exploding gradients problems and the limited long-range dependencies is to modify the architecture of the neural network to allow additional signal paths that bypass many of the processing steps within each stage of the network and hence allow information to be remembered over a larger number of time steps. *Long short-term memory* (LSTM) models (Hochreiter and Schmidhuber, 1997) and *gated recurrent unit* (GRU) models (Cho *et al.*, 2014) are the most widely known examples. Although they improve performance compared to standard RNNs, they still have a limited ability to model long-range dependencies. Also, the additional complexity of each cell means that LSTMs are even slower to train than standard RNNs. Furthermore, all recurrent networks have signal paths that grow linearly with the number of steps in the sequence. Moreover, they do not support parallel computation within a single training example due to the sequential nature of the processing. In particular, this means that RNNs struggle to make efficient use of modern highly parallel hardware based on GPUs. These problems are addressed by replacing RNNs with transformers.

12.3. Transformer Language Models

The transformer processing layer is a highly flexible component for building powerful neural network models with broad applicability. In this section we explore the application of transformers to natural language. This has given rise to the development of massive neural networks known as *large language models* (LLMs), which have proven to be exceptionally capable (Zhao *et al.*, 2023).

Transformers can be applied to many different kinds of language processing task, and can be grouped into three categories according to the form of the input and output data. In a problem such as sentiment analysis, we take a sequence of words as input and provide a single variable representing the sentiment of the text, for example happy or sad, as output. Here a transformer is acting as an 'encoder' of the sequence. Other problems might take a single vector as input and generate a word sequence as output, for example if we wish to generate a text caption given an input image. In such cases the transformer functions as a 'decoder', generating

a sequence as output. Finally, in sequence-to-sequence processing tasks, both the input and the output comprise a sequence of words, for example if our goal is to translate from one language to another. In this case, transformers are used in both encoder and decoder roles. We discuss each of these classes of language model in turn, using illustrative examples of model architectures.

12.3.1 Decoder transformers

We start by considering decoder-only transformer models. These can be used as *generative models* that create output sequences of tokens. As an illustrative example, we will focus on a class of models called *GPT* which stands for *generative pre-trained transformer* (Radford *et al.*, 2019; Brown *et al.*, 2020; OpenAI, 2023). The goal is to use the transformer architecture to construct an autoregressive model of the form defined by (12.31) in which the conditional distributions $p(\mathbf{x}_n|\mathbf{x}_1, \ldots, \mathbf{x}_{n-1})$ are expressed using a transformer neural network that is learned from data.

The model takes as input a sequence consisting of the first $n-1$ tokens, and its corresponding output represents the conditional distribution for token n. If we draw a sample from this distribution then we have extended the sequence to n tokens and this new sequence can be fed back through the model to give a distribution over token $n+1$, and so on. The process can be repeated to generate sequences up to a maximum length determined by the number of inputs to the transformer. We will shortly discuss strategies for sampling from the conditional distributions, but for the moment we focus on how to construct and train the network.

Section 12.3.2

The architecture of a GPT model consists of a stack of transformer layers that take a sequence $\mathbf{x}_1, \ldots, \mathbf{x}_N$ of tokens, each of dimensionality D, as input and produce a sequence $\widetilde{\mathbf{x}}_1, \ldots, \widetilde{\mathbf{x}}_N$ of tokens, again of dimensionality D, as output. Each output needs to represent a probability distribution over the dictionary of tokens at that time step, and this dictionary has dimensionality K whereas the tokens have a dimensionality of D. We therefore make a linear transformation of each output token using a matrix $\mathbf{W}^{(\mathrm{p})}$ of dimensionality $D \times K$ followed by a softmax activation function in the form

$$\mathbf{Y} = \mathrm{Softmax}\left(\widetilde{\mathbf{X}}\mathbf{W}^{(\mathrm{p})}\right) \tag{12.33}$$

where $\mathbf{Y}$ is a matrix whose nth row is $\mathbf{y}_n^{\mathrm{T}}$, and $\widetilde{\mathbf{X}}$ is a matrix whose nth row is $\widetilde{\mathbf{x}}_n^{\mathrm{T}}$. Each softmax output unit has an associated cross-entropy error function. The architecture of the model is shown in Figure 12.15.

Section 5.4.4

The model can be trained using a large corpus of unlabelled natural language by taking a self-supervised approach. Each training sample consists of a sequence of tokens $\mathbf{x}_1, \ldots, \mathbf{x}_n$, which form the input to the network, along with an associated target value $\mathbf{x}_{n+1}$ consisting of the next token in the sequence. The sequences are considered to be independent and identically distributed so that the error function used for training is the sum of the cross-entropy error values summed over the training set, grouped into appropriate mini-batches. Naively we could process each such training sample independently using a forward pass through the model. However, we can achieve much greater efficiency by processing an entire sequence at once so that each token acts both as a target value for the sequence of previous tokens and as

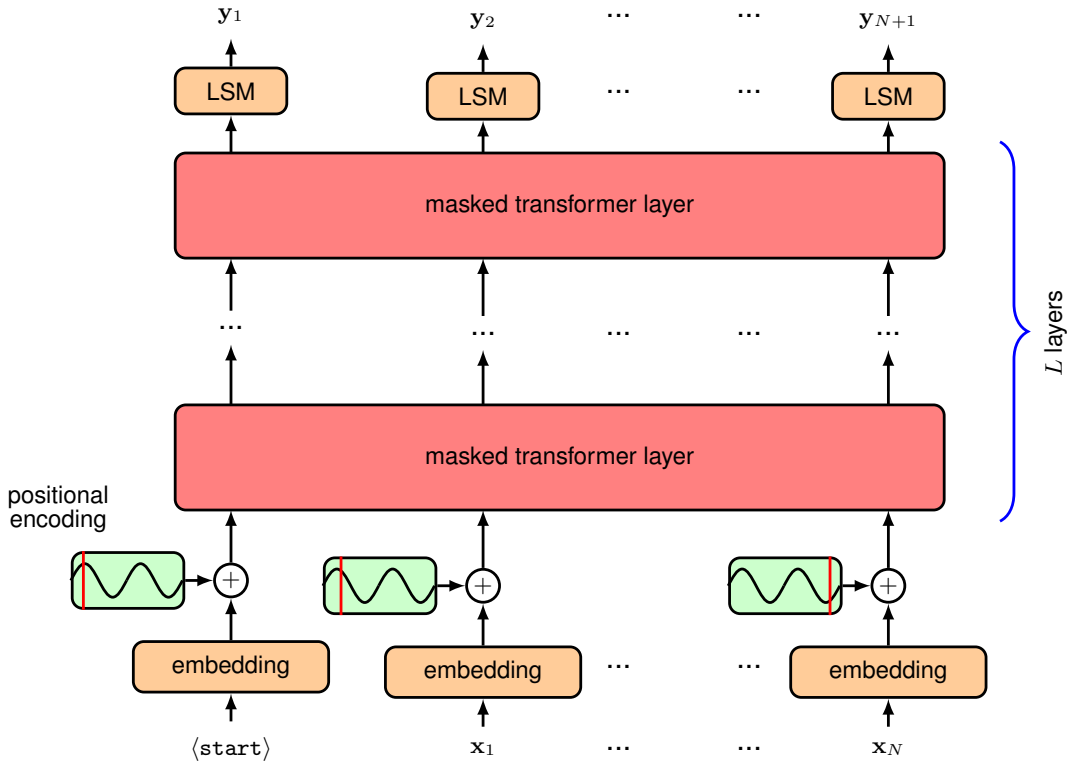


Figure 12.15 Architecture of a GPT decoder transformer network. Here 'LSM' stands for linear-softmax and denotes a linear transformation whose learnable parameters are shared across the token positions, followed by a softmax activation function. Masking is explained in the text.

an input value for subsequent tokens. For example, consider the word sequence

I swam across the river to get to the other bank.

We can use 'I swam across' as an input sequence with an associated target of 'the', and also use 'I swam across the' as an input sequence with an associated target of 'river', and so on. However, to process these in parallel we have to ensure that the network is not able to 'cheat' by looking ahead in the sequence, otherwise it will simply learn to copy the next input directly to the output. If it did this, it would then be unable to generate new sequences since the subsequent token by definition is not available at test time. To address this problem we do two things. First, we shift the input sequence to the right by one step, so that input $\mathbf{x}_n$ corresponds to output $\mathbf{y}_{n+1}$, with target $\mathbf{x}_{n+1}$, and an additional special token denoted $\langle\texttt{start}\rangle$ is prepended in the first position of the input sequence. Second, note that the tokens in a transformer are processed independently, except when they are used to compute the attention weights, when they interact in pairs through the dot product. We therefore introduce *masked attention*, sometimes called *causal attention*, into each of the at-

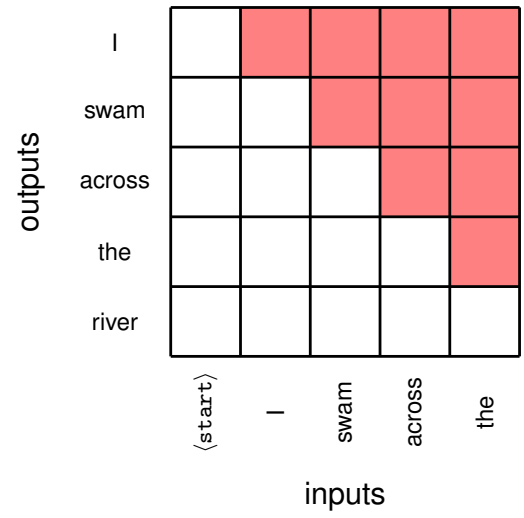


Figure 12.16 An illustration of the mask matrix for masked self-attention. Attention weights corresponding to the red elements are set to zero. Thus, in predicting the token 'across', the output can depend only on the input tokens '⟨start⟩' 'I' and 'swam'.

tention layers, in which we set to zero all of the attention weights that correspond to a token attending to any later token in the sequence. This simply involves setting to zero all the corresponding elements of the attention matrix $\mathrm{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V})$ defined by (12.14) and then normalizing the remaining elements so that each row once again sums to one. In practice, this can be achieved by setting the corresponding pre-activation values to $-\infty$ so that the softmax evaluates to zero for the associated outputs and also takes care of the normalization across the non-zero outputs. The structure of the masked attention matrix is illustrated in Figure 12.16.

In practice, we wish to make efficient use of the massive parallelism of GPUs, and hence multiple sequences may be stacked together into an input tensor for parallel processing in a single batch. However, this requires the sequences to be of the same length, whereas text sequences naturally have variable length. This can be addressed by introducing a specific token, which we denote by ⟨pad⟩, that is used to fill unused positions to bring all sequences up to the same length so that they can be combined into a single tensor. An additional mask is then used in the attention weights to ensure that the output vectors do not pay attention to any inputs occupied by the ⟨pad⟩ token. Note that the form of this mask depends on the particular input sequence.

The output of the trained model is a probability distribution over the space of tokens, given by the softmax output activation function, which represents the probability of the next token given the current token sequence. Once this next word is chosen, the token sequence with the new token included can then be fed through the model again to generate the subsequent token in the sequence, and this process can be repeated indefinitely or until an end-of-sequence token is generated. This may appear to be quite inefficient since data must be fed through the whole model for each new generated token. However, note that due to the masked attention, the embedding learned for a particular token depends only on that token itself and on earlier tokens

and hence does not change when a new, later token is generated. Consequently, much of the computation can be recycled when processing a new token.

12.3.2 Sampling strategies

We have seen that the output of a decoder transformer is a probability distribution over values for the next token in the sequence, from which a particular value for that token must be chosen to extend the sequence. There are several options for selecting the value of the token based on the computed probabilities (Holtzman *et al.*, 2019). One obvious approach, called greedy search, is simply to select the token with the highest probability. This has the effect of making the model deterministic, in that a given input sequence always generates the same output sequence. Note that simply choosing the highest probability token at each stage is not the same as selecting the highest probability sequence of tokens. To find the most probable sequence, we would need to maximize the joint distribution over all tokens, which is given by

Exercise 12.15

$$p(\mathbf{y}_1, \ldots, \mathbf{y}_N) = \prod_{n=1}^{N} p(\mathbf{y}_n | \mathbf{y}_1, \ldots, \mathbf{y}_{n-1}). \tag{12.34}$$

If there are N steps in the sequence and the number of token values in the dictionary is K then the total number of sequences is $\mathcal{O}(K^N)$, which grows exponentially with the length of the sequence, and hence finding the single most probable sequence is infeasible. By comparison, greedy search has cost $\mathcal{O}(KN)$, which is linear in the sequence length.

One technique that has the potential to generate higher probability sequences than greedy search is called *beam search*. Instead of choosing the single most probable token value at each step, we maintain a set of B hypotheses, where B is called the *beam width*, each consisting of a sequence of token values up to step n. We then feed all these sequences through the network, and for each sequence we find the B most probable token values, thereby creating B^2 possible hypotheses for the extended sequence. This list is then pruned by selecting the most probable B hypotheses according to the total probability of the extended sequence. Thus, the beam search algorithm maintains B alternative sequences and keeps track of their probabilities, finally selecting the most probable sequence amongst those considered. Because the probability of a sequence is obtained by multiplying the probabilities at each step of the sequence and since these probability are always less than or equal to one, a long sequence will generally have a lower probability than a short one, biasing the results towards short sequences. For this reason the sequence probabilities are generally normalized by the corresponding lengths of the sequence before making comparisons. Beam search has cost $\mathcal{O}(BKN)$, which is again linear in the sequence length. However, the cost of generating a sequence is increased by a factor of B, and so for very large language models, where the cost of inference can become significant, this makes beam search much less attractive.

One problem with approaches such as greedy search and beam search is that they limit the diversity of potential outputs and can even cause the generation process to become stuck in a loop, where the same sub-sequence of words is repeated over and

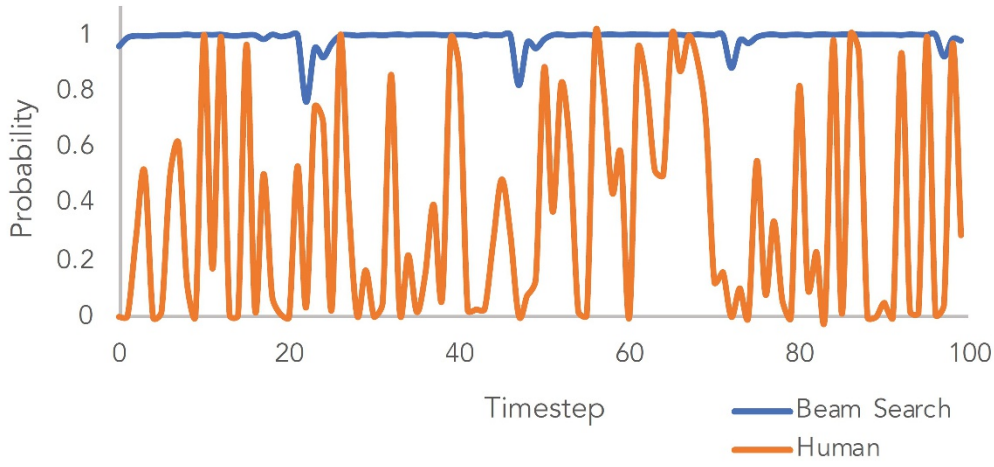


Figure 12.17 A comparison of the token probabilities from beam search and human text for a given trained transformer language model and a given initial input sequence, showing how the human sequence has much lower token probabilities. [From Holtzman *et al.* (2019) with permission.]

over. As can be seen in Figure 12.17, human-generated text may have lower probability and hence be more surprising with respect to a given model than automatically generated text.

Instead of trying to find a sequence with the highest probability, we can instead generate successive tokens simply by sampling from the softmax distribution at each step. However, this can lead to sequences that are nonsensical. This arises from the typically very large size of the token dictionary, in which there is a long tail of many token states each of which has a very small probability but which in aggregate account for a significant fraction of the total probability mass. This leads to the problem in which there is a significant chance that the system will make a bad choice for the next token.

As a balance between these extremes, we can consider only the states having the top K probabilities, for some choice of K, and then sample from these according to their renormalized probabilities. A variant of this approach, called top-p sampling or *nucleus sampling*, calculates the cumulative probability of the top outputs until a threshold is reached and then samples from this restricted set of token states.

A 'softer' version of top-K sampling is to introduce a parameter T called *temperature* into the definition of the softmax function (Hinton, Vinyals, and Dean, 2015) so that

$$y_i = \frac{\exp(a_i/T)}{\sum_j \exp(a_j/T)} \tag{12.35}$$

and then sample the next token from this modified distribution. When $T = 0$, the probability mass is concentrated on the most probable state, with all other states having zero probability, and hence this becomes greedy selection. For $T = 1$, we

recover the unmodified softmax distribution, and as $T \to \infty$, the distribution becomes uniform across all states. By choosing a value in the range $0 < T < 1$, the probability is concentrated towards the higher values.

One challenge with sequence generation is that during the learning phase, the model is trained on a human-generated input sequence, whereas when it is running generatively, the input sequence is itself generated from the model. This means that the model can drift away from the distribution of sequences seen during training.

12.3.3 Encoder transformers

We next consider transformer language models based on encoders, which are models that take sequences as input and produce fixed-length vectors, such as class labels, as output. An example of such a model is *BERT*, which stands for *bidirectional encoder representations from transformers* (Devlin *et al.*, 2018). The goal is to pre-train a language model using a large corpus of text and then to fine-tune the model using *transfer learning* for a broad range of downstream tasks each of which requires a smaller application-specific training data set. The architecture of an encoder transformer is illustrated in Figure 12.18. This approach is a straightforward application of the transformer layers discussed previously.

Section 12.1.7

The first token of every input string is given by a special token ⟨`class`⟩, and the corresponding output of the model is ignored during pre-training. Its role will become apparent when we discuss fine-tuning. The model is pre-trained by presenting token sequences at the input. A randomly chosen subset of the tokens, say 15%, are replaced with a special token denoted ⟨`mask`⟩. The model is trained to predict the missing tokens at the corresponding output nodes. This is analogous to the masking used in word2vec to learn word embeddings. For example, an input sequence might be

Section 12.2.1

I ⟨`mask`⟩ across the river to get to the ⟨`mask`⟩ bank.

and the network should predict ‘swam’ at output node 2 and ‘other’ at output node 10. In this case only two of the outputs contribute to the error function and the other outputs are ignored.

The term ‘bidirectional’ refers to the fact that the network sees words both before and after the masked word and can use both sources of information to make a prediction. As a consequence, unlike decoder models, there is no need to shift the inputs to the right by one place, and there is no need to mask the outputs of each layer from seeing input tokens occurring later in the sequence. Compared to the decoder model, an encoder is less efficient since only a fraction of the sequence tokens are used as training labels. Moreover, an encoder model is unable to generate sequences.

The procedure of replacing randomly selected tokens with ⟨`mask`⟩ means the training set has a mismatch compared to subsequent fine-tuning sets in that the latter will not contain any ⟨`mask`⟩ tokens. To mitigate any problems this might cause, Devlin *et al.* (2018) modified the procedure slightly, so that of the 15% of randomly selected tokens, 80% are replaced with ⟨`mask`⟩, 10% are replaced with a word selected at random from the vocabulary, and in 10% of the cases, the original words are retained at the input, but they still have to be correctly predicted at the output.

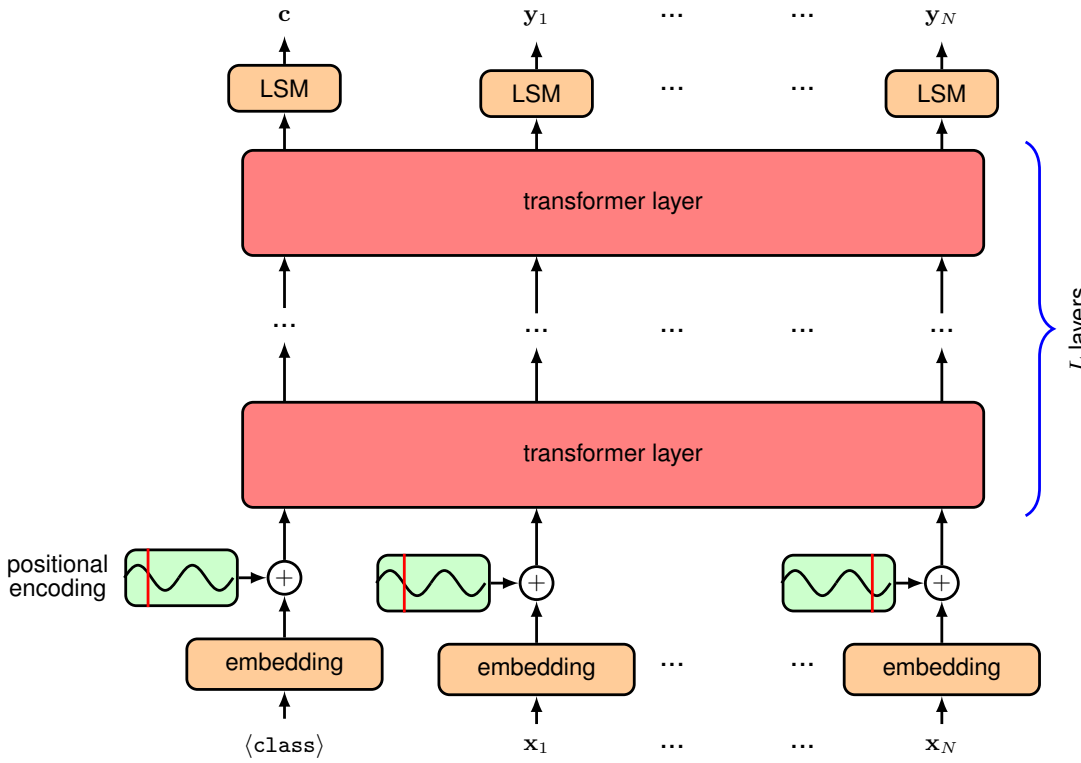


Figure 12.18 Architecture of an encoder transformer model. The boxes labelled 'LSM' denote a linear transformation whose learnable parameters are shared across the token positions, followed by a softmax activation function. The main differences compared to the decoder model are that the input sequence is not shifted to the right, and the 'look ahead' masking matrix is omitted and therefore, within each self-attention layer, every output token can attend to any of the input tokens.

Once the encoder model is trained it can then be fine-tuned for a variety of different tasks. To do this a new output layer is constructed whose form is specific to the task being solved. For a text classification task, only the first output position is used, which corresponds to the ⟨class⟩ token that always appears in the first position of the input sequence. If this output has dimension D then a matrix of parameters of dimension $D \times K$, where K is the number of classes, is appended to the first output node and this in turn feeds into a K-dimensional softmax function or a vector of dimension $D \times 1$ followed by a logistic sigmoid for $K = 2$. The linear output transformation could alternatively be replaced with a more complex differentiable model such as an MLP. If the goal is to classify each token of the input string, for example to assign each token to a category (such as person, place, colour, etc) then the first output is ignored and the subsequent outputs have a shared linear-plus-softmax layer. During fine-tuning all model parameters including the new output matrix are learned by stochastic gradient descent using the log probability

of the correct label. Alternatively the output of a pre-trained model might feed into a sophisticated generative deep learning model for applications such as text-to-image synthesis.

Chapter 20

12.3.4 Sequence-to-sequence transformers

For completeness, we discuss briefly the third category of transformer model, which combines an encoder with a decoder, as discussed in the original transformer paper of Vaswani *et al.* (2017). Consider the task of translating an English sentence into a Dutch sentence. We can use a decoder model to generate the token sequence corresponding to the Dutch output, token by token, as discussed previously. The main difference is that this output needs to be conditioned on the entire input sequence corresponding to the English sentence. An encoder transformer can be used to map the input token sequence into a suitable internal representation, which we denote by $\mathbf{Z}$. To incorporate $\mathbf{Z}$ into the generative process for the output sequence, we use a modified form of the attention mechanism called *cross attention*. This is the same as self-attention except that although the query vectors come from the sequence being generated, in this case the Dutch output sequence, the key and value vectors come from the sequence represented by $\mathbf{Z}$, as illustrated in Figure 12.19. Returning to our analogy with a video streaming service, this would be like the user sending their query vector to a different streaming company who then compares it with their own set of key vectors to find the best match and then returns the associated value vector in the form of a movie.

Section 12.3.1

When we combine the encoder and decoder modules, we obtain the architecture of the model shown in Figure 12.20. The model can be trained using paired input and output sentences.

12.3.5 Large language models

The most important recent development in the field of machine learning has been the creation of very large transformer-based neural networks for natural language processing, known as *large language models* or LLMs. Here 'large' refers to the number of weight and bias parameters in the network, which can number up to around one trillion (10^{12}) at the time of writing. Such models are expensive to train, and the motivation for building them comes from their extraordinary capabilities.

In addition to the availability of large data sets, the training of ever larger models has been facilitated by the advent of massively parallel training hardware based on GPUs (graphics processing units) and similar processors tightly coupled in large clusters equipped fast interconnect and lots of onboard memory. The transformer architecture has played a key role in the development of these models because it is able to make very efficient use of such hardware. Very often, increasing the size of the training data set, along with a commensurate increase in the number of model parameters, leads to improvements in performance that outpace architectural improvements or other ways to incorporate more domain knowledge (Sutton, 2019; Kaplan *et al.*, 2020). For example, the impressive increase in performance of the GPT series of models (Radford *et al.*, 2019; Brown *et al.*, 2020; OpenAI, 2023) through successive generations has come primarily from an increase in scale. These kinds

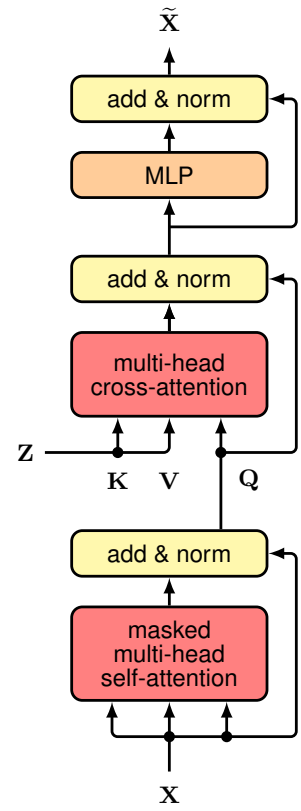


Figure 12.19 Schematic illustration of one cross-attention layer as used in the decoder section of a sequence-to-sequence transformer. Here $\mathbf{Z}$ denotes the output from the encoder section. $\mathbf{Z}$ determines the key and value vectors for the cross-attention layer, whereas the query vectors are determined within the decoder section.

of performance improvements have driven a new kind of Moore's law in which the number of compute operations required to train a state-of-the-art machine learning model has grown exponentially since about 2012 with a doubling time of around 3.4 months.

Figure 1.16

Early language models were trained using supervised learning. For example, to build a translation system, the training set would consist of matched pairs of sentences in two languages. A major limitation of supervised learning, however, is that the data typically has to be human-curated to provide labelled examples, and this severely limits the quantity of data available, thereby requiring heavy use of inductive biases such as feature engineering and architecture constraints to achieve reasonable performance.

Large language models are trained instead by self-supervised learning on very large data sets of text, along with potentially other token sequences such as computer code. We have seen how a decoder transformer can be trained on token sequences in which each token acts as a labelled target example, with the preceding sequence as input, to learn a conditional probability distribution. This 'self-labelling' hugely expands the quantity of training data available and therefore allows exploitation of deep neural networks having large numbers of parameters.

Section 12.3.1

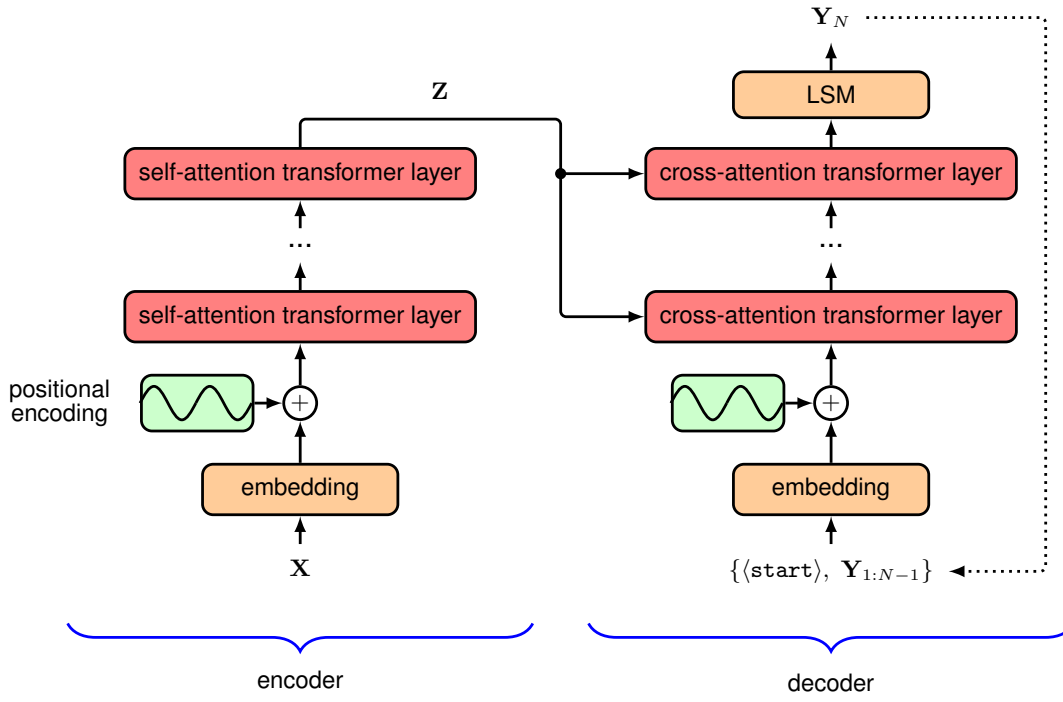


Figure 12.20 Schematic illustration of a sequence-to-sequence transformer. To keep the diagram uncluttered the input tokens are collectively shown as a single box, and likewise for the output tokens. Positional-encoding vectors are added to the input tokens for both the encoder and decoder sections. Each layer in the encoder corresponds to the structure shown in Figure 12.9, and each cross-attention layer is of the form shown in Figure 12.19.

This use of self-supervised learning led to a paradigm shift in which a large model is first *pre-trained* using unlabelled data and then subsequently *fine-tuned* using supervised learning based on a much smaller set of labelled data. This is effectively a form of transfer learning, and the same pre-trained model can be used for multiple 'downstream' applications. A model with broad capabilities that can be subsequently fine-tuned for specific tasks is called a *foundation model* (Bommasani *et al.*, 2021).

The fine-tuning can be done by adding extra layers to the outputs of the network or by replacing the last few layers with fresh parameters and then using the labelled data to train these final layers. During the fine-tuning stage, the weights and biases in the main model can either be left unchanged or be allowed to undergo small levels of adaptation. Typically the cost of the fine-tuning is small compared to that of pre-training.

One very efficient approach to fine-tuning is called *low-rank adaptation* or LoRA (Hu *et al.*, 2021). This approach is inspired by results which show that a trained over-parameterized model has a low intrinsic dimensionality with respect to fine-tuning, meaning that changes in the model parameters during fine-tuning lie on a manifold

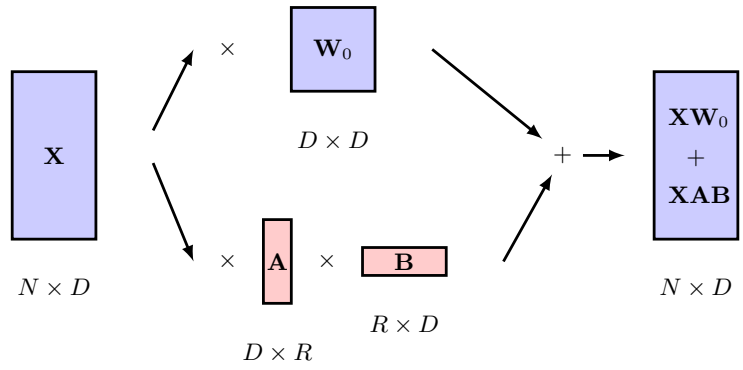


Figure 12.21 Schematic illustration low-rank adaptation showing a weight matrix $\mathbf{W}_0$ from one of the attention layers in a pre-trained transformer. Additional weights given by matrices $\mathbf{A}$ and $\mathbf{B}$ are adapted during fine-tuning and their product $\mathbf{AB}$ is then added to the original matrix for subsequent inference.

whose dimensionality is much smaller than the total number of learnable parameters in the model (Aghajanyan, Zettlemoyer, and Gupta, 2020). LoRa exploits this by freezing the weights of the original model and adding additional learnable weight matrices into each layer of the transformer in the form of low-rank products. Typically only attention-layer weights are modified, whereas MLP-layer weights are kept fixed. Consider a weight matrix $\mathbf{W}_0$ having dimension $D \times D$, which might represent a query, key, or value matrix in which the matrices from multiple attention heads are treated together as a single matrix. We introduce a parallel set of weights defined by the product of two matrices $\mathbf{A}$ and $\mathbf{B}$ with dimensions $D \times R$ and $R \times D$, respectively, as shown schematically in Figure 12.21. This layer then generates an output given by $\mathbf{XW}_0 + \mathbf{XAB}$. The number of parameters in the additional weight matrix $\mathbf{AB}$ is $2RD$ compared to the D^2 parameters in the original weight matrix $\mathbf{W}_0$, and so if $R \ll D$ then the number of parameters that need to be adapted during fine-tuning is much smaller than the number in the original transformer. In practice, this can reduce the number of parameters that need to be trained by a factor of 10,000. Once the fine-tuning is complete, the additional weights can be added to the original weight matrices to give a new weight matrix

$$\widehat{\mathbf{W}} = \mathbf{W}_0 + \mathbf{AB} \tag{12.36}$$

so that during inference there is no additional computational overhead compared to running the original model since the updated model has the same size as the original.

As language models have become larger and more powerful, the need for fine-tuning has diminished, with generative language models now able to solve a broad range of tasks simply through text-based interaction. For example, if a text string

English: the cat sat on the mat. French:

is given as the input sequence, an autoregressive language model can continue to generate subsequent tokens until a ⟨stop⟩ token is generated, in which the newly gen-

erated tokens represent the French translation. Note that the model was not trained specifically to do translation but has learned to do so as a result of being trained on a vast corpus of data that includes multiple languages.

A user can interact with such models using a natural language dialogue, making them very accessible to broad audiences. To improve the user experience and the quality of the generated outputs, techniques have been developed for fine-tuning large language models through human evaluation of generated output, using methods such as *reinforcement learning through human feedback* or RLHF (Christiano *et al.*, 2017). Such techniques have helped to create large language models with impressively easy-to-use conversational interfaces, most notably the system from OpenAI called *ChatGPT*.

The sequence of input tokens given by the user is called a *prompt*. For example, it might consist of the opening words of a story, which the model is required to complete. Or it might comprise a question, and the model should provide the answer. By using different prompts, the same trained neural network may be capable of solving a broad range of tasks such as generating computer code from a simple text request or writing rhyming poetry on demand. The performance of the model now depends on the form of the prompt, leading to a new field called *prompt engineering* (Liu *et al.*, 2021), which aims to design a good form for a prompt that results in high-quality output for the downstream task. The behaviour of the model can also be modified by adapting the user's prompt before feeding it into the language model by pre-pending an additional token sequence called a *prefix prompt* to the user prompt to modify the form of the output. For example, the pre-prompt might consist of instructions, expressed in standard English, to tell the network not to include offensive language in its output.

This allows the model to solve new tasks simply by providing some examples within the prompt, without needing to adapt the parameters of the model. This is an example of *few-shot learning*.

Current state-of-the-art models such as GPT-4 have become so powerful that they are exhibiting remarkable properties which have been described as the first indications of artificial general intelligence (Bubeck *et al.*, 2023) and are driving a new wave of technological innovation. Moreover, the capabilities of these models continue to improve at an impressive pace.

12.4. Multimodal Transformers

Although transformers were initially developed as an alternative to recurrent networks for processing sequential language data, they have become prevalent in nearly all areas of deep learning. They have proved to be general-purpose models, as they make very few assumptions about the input data, in contrast, for example, to convolutional networks, which make strong assumptions about equivariances and locality. Due to their generality, transformers have become the state-of-the-art for many different modalities, including text, image, video, point cloud, and audio data, and have been used for both discriminative and generative applications within each of these

Chapter 10

domains. The core architecture of the transformer layer has remained relatively constant, both over time and across applications. Therefore, the key innovations that enabled the use of transformers in areas other than natural language have largely focused on the representation and encoding of the inputs and outputs.

One big advantage of a single architecture that is capable of processing many different kinds of data is that it makes *multimodal* computation relatively straightforward. In this context, multimodal refers to applications that combine two or more different types of of data, either in the inputs or outputs or both. For example, we may wish to generate an image from a text prompt or design a robot that can combine information from multiple sensors such as cameras, radar, and microphones. The important thing to note is that if we can tokenize the inputs and decode the output tokens, then it is likely that we can use a transformer.

12.4.1 Vision transformers

Transformers have been applied with great success to computer vision and have achieved state-of-the-art performance on many tasks. The most common choice for discriminative tasks is a standard transformer encoder, and this approach in the vision domain is known as a *vision transformer*, or ViT (Dosovitskiy *et al.*, 2020). When using a transformer, we need to decide how to convert an input image into tokens, and the simplest choice is to use each pixel as a token, following a linear projection. However, the memory required by a standard transformer implementation grows quadratically with the number of input tokens, and so this approach is generally infeasible. Instead, the most common approach to tokenization is to split the image into a set of patches of the same size. Suppose the images have dimension $\mathbf{x} \in \mathbb{R}^{H \times W \times C}$ where H and W are the height and width of the image in pixels and C is the number of channels (where typically $C = 3$ for R, G, and B colours). Each image is split into non-overlapping patches of size $P \times P$ (where $P = 16$ is a common choice) and then 'flattened' into a one-dimensional vector, which gives a representation $\mathbf{x}_p \in \mathbb{R}^{N \times (P^2 C)}$ where $N = HW/P^2$ is the total number of patches for one image. The ViT architecture is shown in Figure 12.22.

Another approach to tokenization is to feed the image through a small convolutional neural network (CNN). This can down-sample the image to give a manageable number of tokens each represented by one of the network outputs. For example a typical ResNet18 encoder architecture down-samples an image by a factor of 8 in both the height and width dimensions, giving 64 times fewer tokens than pixels.

Chapter 10

We also need a way to encode positional information in the tokens. It is possible to construct explicit positional embeddings that encode the two-dimensional positional information of the image patches, but in practice this does not generally improve performance, and so it is most common to just use learned positional embeddings. In contrast to the transformers used for natural language, vision transformers generally take a fixed number of tokens as input, which avoids the problem of learned positional encodings not generalizing to inputs of a different size.

A vision transformer has a very different architectural design compared to a CNN. Although strong inductive biases are baked into a CNN model, the only two-dimensional inductive bias in a vision transformer is due to the patches used to tok-

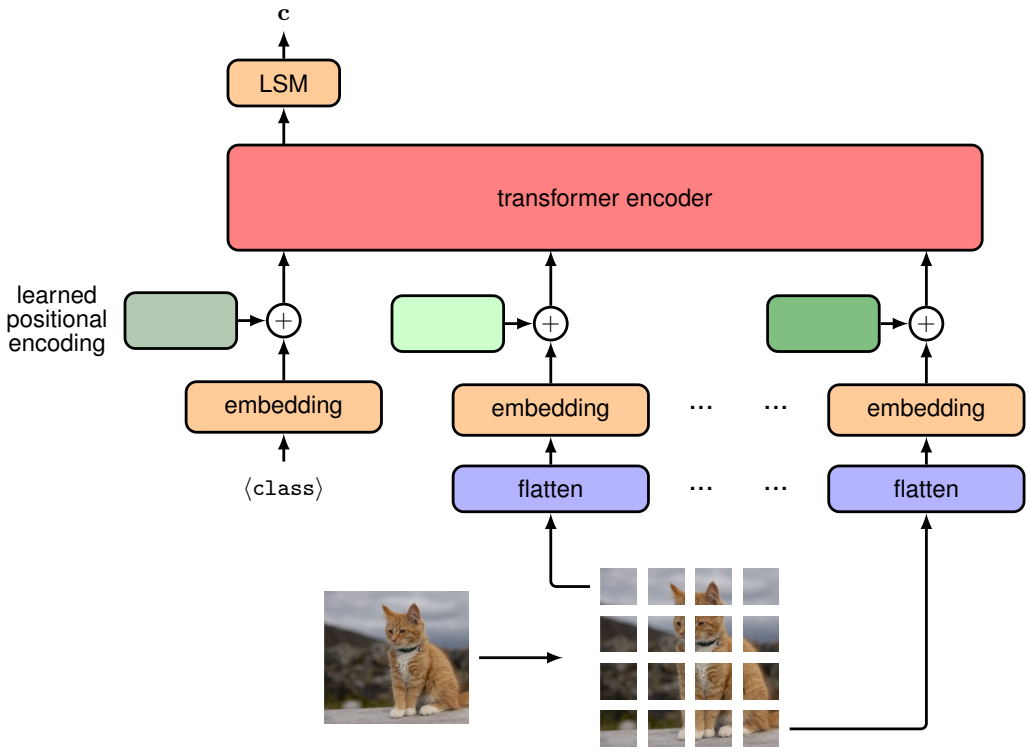


Figure 12.22 Illustration of the vision transformer architecture for a classification task. Here a learnable ⟨class⟩ token is included as an additional input, and the associated output is transformed by a linear layer with a softmax activation, denoted by LSM, to give the final class-vector output $\mathbf{c}$.

enize the input. A transformer therefore generally requires more training data than a comparable CNN as it has to learn the geometrical properties of images from scratch. However, because there are no strong assumptions about the structure of the inputs, transformers are often able to converge to a higher accuracy. This provides another illustration of the trade-off between inductive bias and the scale of the training data (Sutton, 2019).

12.4.2 Generative image transformers

In the language domain, the most impressive results have come when transformers are used as an autoregressive generative model for synthesizing text. It is therefore natural to ask whether we can also use transformers to synthesize realistic images. Since natural language is inherently sequential, it fits neatly into the autoregressive framework, whereas images have no natural ordering of their pixels so that it is not as intuitive that decoding them autoregressively would be useful. However, any distribution can be decomposed into a product of conditionals, provided we first define some ordering of the variables. Thus, the joint distribution over ordered

Section 11.1.2

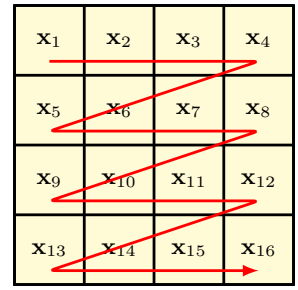


Figure 12.23 Illustration of a *raster scan* that defines a specific linear ordering of the pixels in a two-dimensional image.

variables $\mathbf{x}_1, \ldots, \mathbf{x}_N$ can be written

$$p(\mathbf{x}_1, \ldots, \mathbf{x}_N) = \prod_{n=1}^{N} p(\mathbf{x}_n | \mathbf{x}_1, \ldots, \mathbf{x}_{n-1}). \tag{12.37}$$

This factorization is completely general and makes no restrictions on the form of the individual conditional distributions $p(\mathbf{x}_n|\mathbf{x}_1, \ldots, \mathbf{x}_{n-1})$.

For an image we can choose $\mathbf{x}_n$ to represent the nth pixel as a three-dimensional vector of the RGB values. We now need to decide on an ordering for the pixels, and one widely used choice is called a *raster scan* as illustrated in Figure 12.23. A schematic illustration of an image being generated using an autoregressive model, based on a raster-scan ordering, is shown in Figure 12.24.

Note that the use of autoregressive generative models of images predates the introduction of transformers. For example, PixelCNN (Oord *et al.*, 2016) and PixelRNN (Oord, Kalchbrenner, and Kavukcuoglu, 2016) used bespoke masked convolution layers that preserve the conditional independence defined for each pixel by the corresponding term on the right-hand side of 12.37.

Representations of an image using continuous values can work well in discriminative tasks. However, much better results are obtained for image generation by using discrete representations. Continuous conditional distributions learned by maximum likelihood, such as Gaussians for which the negative log likelihood function is a sum-of-squares error function, tend to learn averages of the training data, leading to blurry images. Conversely, discrete distributions can handle multimodality with ease. For example, one of the conditional distributions $p(\mathbf{x}_n|\mathbf{x}_1, \ldots, \mathbf{x}_{n-1})$ in

Section 4.2

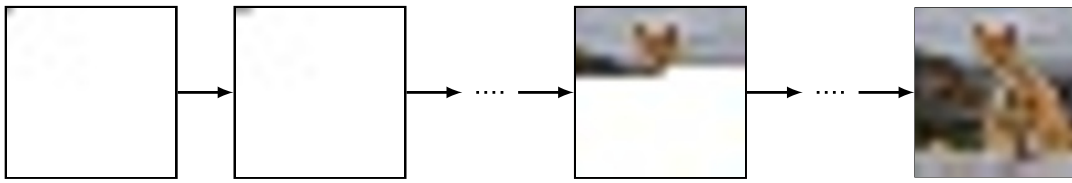


Figure 12.24 An illustration of how an image can be sampled from an autoregressive model. The first pixel is sampled from the marginal distribution $p(\mathbf{x}_{11})$, the second pixel from the conditional distribution $p(\mathbf{x}_{12}|\mathbf{x}_{11})$, and so on in raster scan order until we have a complete image.

(12.37) might learn that a pixel could be either black or white, whereas a regression model might learn that the pixel should be grey.

However, working with discrete spaces also brings its challenges. The R, G, and B values of image pixels are typically represented with at least 8 bits of precision, so that each pixel has $2^{24} \simeq 16\text{M}$ possible values. Learning a conditional softmax distribution over a such a high-dimensional space is infeasible.

One way to address the problem of the high dimensionality is to use the technique of *vector quantization*, which can be viewed as a form of data compression. Suppose we have a set of data vectors $\mathbf{x}_1, \ldots, \mathbf{x}_N$ each of dimensionality D, which might, for example, represent image pixels, and we then introduce a set of K *codebook vectors* $\mathcal{C} = \mathbf{c}_1, \ldots, \mathbf{c}_K$ also of dimensionality D, where typically $K \ll D$. We now approximate each data vector by its nearest codebook vector according to some similarity metric, usually Euclidean distance, so that

Section 15.1.1

$$\mathbf{x}_n \rightarrow \arg\min_{\mathbf{c}_k \in \mathcal{C}} ||\mathbf{x}_n - \mathbf{c}_k||^2. \tag{12.38}$$

Since there are K codebook vectors, we can represent each $\mathbf{x}_n$ by a one-hot encoded K-dimensional vector, and since we can choose the value of K, we can control the trade-off between more accurate representation of the data, by using a larger value of K, or greater compression, by using a smaller value of K.

We can therefore take the original image pixels and map them into the lower-dimensional codebook space. An autoregressive transformer can then be trained to generate a sequence of codebook vectors, and this sequence can be mapped back into the original image space by replacing each codebook index k with the corresponding D-dimensional codebook vector $\mathbf{c}_k$.

Autoregressive transformers were first applied to images in ImageGPT (Chen, Radford, *et al.*, 2020). Here each pixel is treated as one of a discrete set of three-dimensional colour codebook vectors, each corresponding to a cluster in a K-means clustering of the colour space. A one-hot encoding therefore gives discrete tokens, analogous to language tokens, and allows the transformer to be trained in the same way as language models, with a next-token classification objective. This is a powerful objective for representation learning for subsequent fine-tuning, again in a similar way to language modelling.

Section 15.1

Section 6.3.3

Using the individual pixels as tokens directly, however, can lead to high computational cost since a forward pass is required per pixel, which means that both training and inference scale poorly with image resolution. Also, using individual pixels as inputs means that low-resolution images have to be used to give a reasonable context length when decoding the pixels later in the raster scan. As we saw with the ViT model, it is preferable to use patches of the image as tokens instead of pixels, as this can result in dramatically fewer tokens and therefore facilitates working with higher-resolution images. As before, we need to work with a discrete space of token values due to the potential multimodality of the conditional distributions. Again, this raises the challenge of dimensionality, which is now much more severe with patches than with individual pixels since the dimensionality is exponential with respect to the number of pixels in the patch. For example, even with just two possible pixel

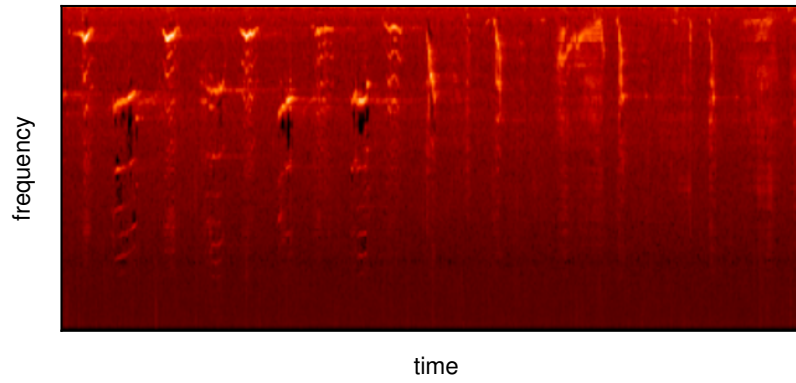


Figure 12.25 An example mel spectrogram of a humpback whale song. [Source data copyright ©2013–2023, librosa development team.]

tokens, representing black and white, and patches of size 16×16, we would have a dictionary of patch tokens of size $2^{256} \simeq 10^{77}$.

Once again we turn to vector quantization to address the challenge of dimensionality. The codebook vectors can be learned from a data set of image patches using simple clustering algorithms such as K-means or with more sophisticated methods such as fully convolutional networks (Oord, Vinyals, and Kavukcuoglu, 2017; Esser, Rombach, and Ommer, 2020) or even vision transformers (Yu *et al.*, 2021). One problem with learning to map each patch to a discrete set of codes and back again, is that vector quantization is a non-differentiable operation. Fortunately we can use a technique called straight-through gradient estimation (Bengio, Léonard, and Courville, 2013), which is a simple approximation that just copies the gradients through the non-differentiable function during backpropagation.

The use of autoregressive transformers to generate images can be extended to videos by treating a video as one long sequence of these vector-quantized tokens (Rakhimov *et al.*, 2020; Yan *et al.*, 2021; Hu *et al.*, 2023).

12.4.3 Audio data

We next look at the application of transformers to audio data. Sound is generally stored as a waveform obtained by measuring the amplitude of the air pressure at regular time intervals. Although this waveform could be used directly as input to a deep learning model, in practice it is more effective to pre-process it into a *mel spectrogram*. This is a matrix whose columns represent time steps and whose rows correspond to frequencies. The frequency bands follow a standard convention that was chosen through subjective assessment to give equal perceptual differences between successive frequencies (the word 'mel' comes from melody). An example of a mel spectrogram is shown in Figure 12.25.

One application for transformers in the audio domain is classification in which segments of audio are assigned to one of a number of predefined categories. For example, the *AudioSet* data set (Gemmeke *et al.*, 2017) is a widely used benchmark.

It contains classes such as 'car', 'animal', and 'laughter'. Until the development of the transformer, the state-of-the-art approach for audio classification was based on mel spectrograms treated as images and used as the input to a convolutional neural network (CNN). However, although a CNN is good at understanding local relationships, one drawback is that it struggles with longer-range dependencies, which can be important in processing audio.

Chapter 10

Just as transformers replaced RNNs as the state-of-the-art in natural language processing, they have also come to replace CNNs for tasks such as audio classification. For example, a transformer encoder model of identical structure to that used for both language and vision, as shown in Figure 12.18, can be used to predict the class of audio inputs (Gong, Chung, and Glass, 2021). Here the mel spectrogram is viewed as an image which is then tokenized. This is done by splitting the image into patches in a similar way to vision transformers, possibly with some overlap so as not to lose any important neighbourhood relations. Each patch is then flattened, meaning it is converted to a one-dimensional array, in this case of length 256. A unique positional encoding is then added to each token, a specific ⟨`class`⟩ token is appended, and the tokens are then fed through the transformer encoder. The output token corresponding to the ⟨`class`⟩ input token from the last transformer layer can then be decoded using a linear layer followed by a softmax activation function, and the whole model can be trained end-to-end using a cross-entropy loss.

12.4.4 Text-to-speech

Classification is not the only task that deep learning, and more specifically the transformer architecture, has revolutionized in the audio domain. The success of transformers at synthesizing speech that imitates the voice of a given speaker is another demonstration of their versatility, and their application to this task is an informative case study in how to apply transformers in a new context.

Generating speech corresponding to a given passage of text is known as *text-to-speech synthesis*. A more traditional approach would be to collect recordings of speech from a given speaker and train a supervised regression model to predict the speech output, possibly in the form of a mel spectrogram, from corresponding transcribed text. During inference, the text for which we would like to synthesize speech is presented as input and the resulting mel spectrogram output can then be decoded back to an audio waveform since this is a fixed mapping.

This approach has a few major drawbacks, however. First, if we predict speech at a low level, for example using sub-word components known as *phonemes*, a larger context is needed to make the resulting sentences sound fluid. However, if we predict longer segments, then the space of possible inputs grows significantly, and an infeasible amount of training data might be required to achieve good generalization. Second, this approach does not transfer knowledge across speakers, and so a lot of data will be required for each new speaker. Finally, the problem is really a generative modelling task, as there are multiple correct speech outputs for a given speaker and text pair, so regression may not be suitable since it tends to average over target values.

Section 4.2

If instead we treat audio data in the same way as natural language and frame text-to-speech as a conditional language modelling task, then we should be able to

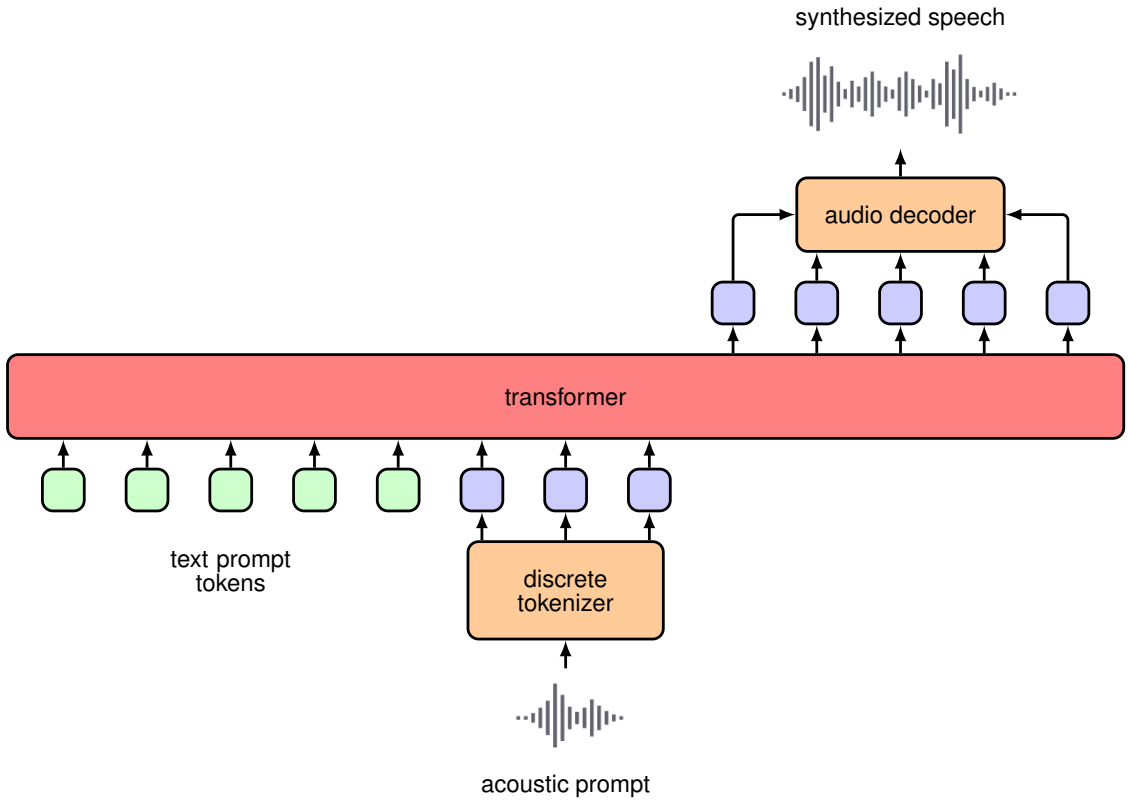


Figure 12.26 A diagram showing the high-level architecture of Vall-E. The input to the transformer model consists of standard text tokens, which prompt the model as to what words the synthesized speech should contain, together with acoustic prompt tokens that determine the speaker style and tone information. The sampled model output tokens are decoded back to speech with the learned decoder. For simplicity, the positional encodings and linear projections are not shown.

train the model in much the same way as with text-based large language models. There are two main implementation details that need to be addressed. The first is how to tokenize the training data and decode the predictions, and the second is how to condition the model on the speaker's voice.

One approach to text-to-speech synthesis that makes use of transformers and language modelling techniques is *Vall-E* (Wang *et al.*, 2023). New text can be mapped into speech in the voice of a new speaker using only a few seconds of sample speech from that person. Speech data is converted into a sequence of discrete tokens from a learned dictionary or *codebook* obtained using vector quantization, and we can think of these tokens as analogous to the one-hot encoded tokens from the natural language domain. The input consists of text tokens from a passage of text whereas the target outputs for training consist of the corresponding speech tokens. Additional speech tokens from a short segment of unrelated speech from the same speaker are

Section 12.4.2

appended to the input text tokens, as illustrated in Figure 12.26. By including examples from many different speakers, the system can learn to read out a passage of text while imitating the voice represented by the additional speech input tokens. Once trained the system can be presented with new text, along with audio tokens from a brief segment of speech captured from a new speaker, and the resulting output tokens can be decoded, using the same codebook used during training, to create a speech waveform. This allows the system to synthesize speech corresponding to the input text in the voice of the new speaker.

12.4.5 Vision and language transformers

We have seen how to generate discrete tokens for text, audio, and images, and so it is a natural next step to ask if we can train a model with input tokens of one modality and output tokens of another, or whether we can have a combination of different modalities for either inputs or outputs or both. We will focus on the combination of text and vision data as this is the most widely studied example, but in principle the approaches discussed here could be applied to other combinations of input and output modalities.

The first requirement is that we have a large data set for training. The LAION-400M data set (Schuhmann *et al.*, 2021) has greatly accelerated research in text-to-image generation and image-to-text captioning in much the same way that ImageNet was critical in the development of deep image classification models. Text-to-image generation is actually much like the unconditional image generation we have looked at so far, except that we also allow the model to take as input the text information to condition the generation process. This is straightforward when using transformers as we can simply provide the text tokens as additional input when decoding each image token.

This approach can also be viewed as treating the text-to-image problem as a sequence-to-sequence language modelling problem, such as machine translation, except that the target tokens are discrete image tokens rather than language tokens. It therefore makes sense to choose a full encoder-decoder transformer model, as shown in Figure 12.20, in which $\mathbf{X}$ corresponds to the input text tokens and $\mathbf{Y}$ corresponds to the output image tokens. This is the approach taken in a model called *Parti* (Yu *et al.*, 2022) in which the transformer is scaled to 20 billion parameters while showing consistent performance improvements with increasing model size.

A lot of research has also been done on using pre-trained language models, and modifying or fine-tuning them so that they can also accept visual data as input (Alayrac *et al.*, 2022; Li *et al.*, 2022). These approaches largely use bespoke architectures, along with continuous-valued image tokens, and therefore are not natural fits for also generating visual data. Moreover, they cannot be used directly if we wish to include new modalities such as audio tokens. Although this is a step towards multimodality, we would ideally like to use both text and image tokens as both input and output. The simplest approach is to treat everything as a sequence of tokens as if this were natural language but with a dictionary that is the concatenation of a language token dictionary and the image token codebook. We can then treat any stream of audio and visual data as simply a sequence of tokens.

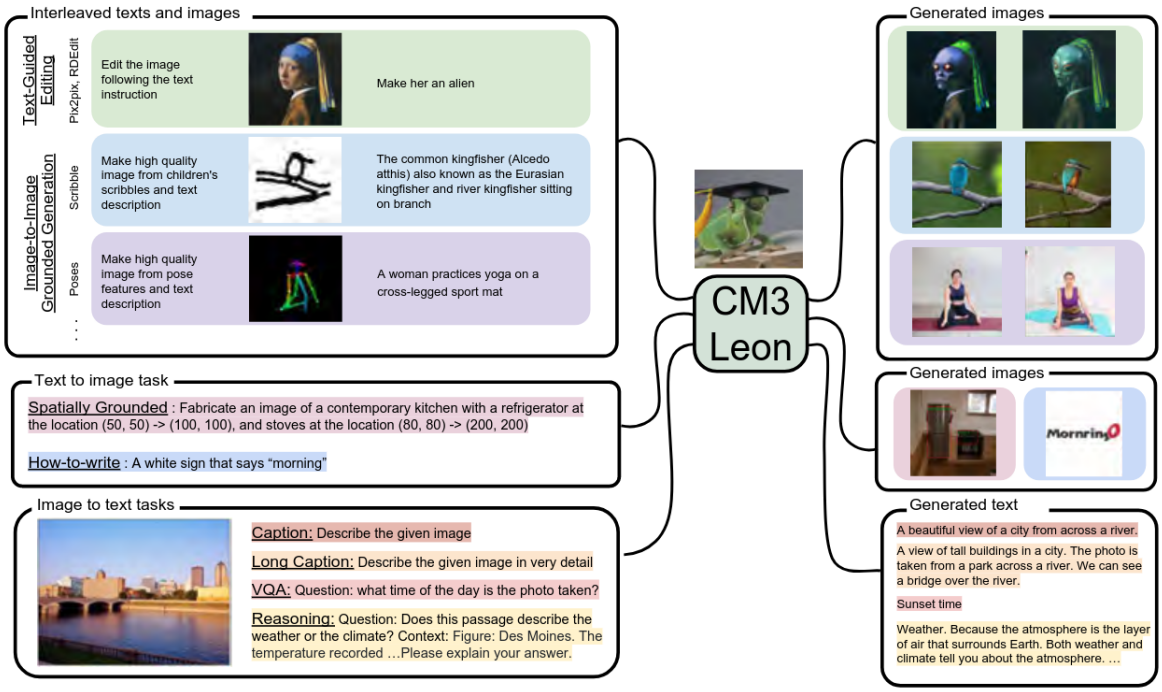


Figure 12.27 Examples of the CM3Leon model performing a variety of different tasks in the joint space of text and images. [From (Yu *et al.*, 2023) with permission.]

In CM3 (Aghajanyan *et al.*, 2022) and CM3Leon (Yu *et al.*, 2023), a variation of language modelling is used to train on HTML documents containing both image and text data taken from online sources. When this large quantity of training data was combined with a scalable architecture, the models became very powerful. Moreover, the multimodal nature of the training means that the models are very flexible. Such models are capable of completing many tasks that otherwise might require task-specific model architectures and training regimes, such as text-to-image generation, image-to-text captioning, image editing, text completion, and many more, including anything a regular language model is capable of. Examples of the CM3Leon model completing instances of a few different tasks are shown in Figure 12.27.

Exercises

12.1 ($\star\star$) Consider a set of coefficients a_{nm}, for $m = 1, \ldots, N$, with the properties that

$$a_{nm} \geqslant 0 \tag{12.39}$$

$$\sum_m a_{nm} = 1. \tag{12.40}$$

Appendix C By using a Lagrange multiplier show that the coefficients must also satisfy

$$a_{nm} \leqslant 1 \quad \text{for} \quad n = 1, \ldots, N. \tag{12.41}$$

12.2 (⋆) Verify that the softmax function (12.5) satisfies the constraints (12.3) and (12.4) for any values of the vectors $\mathbf{x}_1, \ldots, \mathbf{x}_N$.

12.3 (⋆) Consider the input vectors $\mathbf{x}_n$ in the simple transformation defined by (12.2), in which the weighting coefficients a_{nm} are defined by (12.5). Show that if all the input vectors are orthogonal, so that $\mathbf{x}_n^{\mathrm{T}}\mathbf{x}_m = 0$ for $n \neq m$, then the output vectors will simply be equal to the input vectors so that $\mathbf{y}_n = \mathbf{x}_n$ for $n = 1, \ldots, N$.

12.4 (⋆) Consider two independent random vectors $\mathbf{a}$ and $\mathbf{b}$ each of dimension D and each being drawn from a Gaussian distribution with zero mean and unit variance $\mathcal{N}(\cdot|\mathbf{0}, \mathbf{I})$. Show that the expected value of $(\mathbf{a}^{\mathrm{T}}\mathbf{b})^2$ is given by D.

12.5 (⋆⋆⋆) Show that multi-head attention defined by (12.19) can be rewritten in the form

$$\mathbf{Y} = \sum_{h=1}^{H} \mathbf{H}_h \mathbf{X} \mathbf{W}^{(h)} \tag{12.42}$$

where $\mathbf{H}_h$ is given by (12.15) and we have defined

$$\mathbf{W}^{(h)} = \mathbf{W}_h^{(v)} \mathbf{W}_h^{(\mathrm{o})}. \tag{12.43}$$

Figure 12.7 Here we have partitioned the matrix $\mathbf{W}^{(\mathrm{o})}$ horizontally into sub-matrices denoted $\mathbf{W}_h^{(\mathrm{o})}$ each of dimension $D_{\mathrm{v}} \times D$, corresponding to the vertical segments of the concatenated attention matrix. Since D_{v} is typically smaller than D, for example $D_{\mathrm{v}} = D/H$ is a common choice, this combined matrix is rank deficient. Therefore, using a fully flexible matrix to replace $\mathbf{W}_h^{(\mathrm{v})} \mathbf{W}_h^{(\mathrm{o})}$ would not be equivalent to the original formulation given in the text.

12.6 (⋆⋆) Express the self-attention function (12.14) as a fully connected network in the form of a matrix that maps the full input sequence of concatenated word vectors into an output vector of the same dimension. Note that such a matrix would have $\mathcal{O}(N^2 D^2)$ parameters. Show that the self-attention network corresponds to a sparse version of this matrix with parameter sharing. Draw a sketch showing the structure of this matrix, indicating which blocks of parameters are shared and which blocks have all elements equal to zero.

12.7 (⋆) Show that if we omit the positional encoding of input vectors then the outputs of a multi-head attention layer defined by (12.19) are equivariant with respect to a reordering of the input sequence.

12.8 (⋆⋆⋆) Consider two D-dimensional unit vectors $\mathbf{a}$ and $\mathbf{b}$, satisfying $\|\mathbf{a}\| = 1$ and $\|\mathbf{b}\| = 1$, drawn from a random distribution. Assume that the distribution is symmetrical around the origin, i.e., it depends only on the distance from the origin and

not the direction. Show that for large values of D the magnitude of the cosine of the angle between these vectors is close to zero and hence that these random vectors are nearly orthogonal in a high-dimensional space. To do this, consider an orthonormal basis set $\{\mathbf{u}_i\}$ where $\mathbf{u}_i^{\mathrm{T}}\mathbf{u}_j = \delta_{ij}$ and express $\mathbf{a}$ and $\mathbf{b}$ as expansions in this basis.

12.9 ($\star\star$) Consider a position encoding in which the input token vector $\mathbf{x}$ is concatenated with a position-encoding vector $\mathbf{e}$. Show that when this concatenated vector undergoes a general linear transformation by multiplication using a matrix, the result can be expressed as the sum of a linearly transformed input and a linearly transformed position vector.

12.10 ($\star\star$) Show that the positional encoding defined by (12.25) has the property that, for a fixed offset k, the encoding at position $n + k$ can be represented as a linear combination of the encoding at position n with coefficients that depend only on k and not on n. To do this make use of the following trigonometric identities:

$$\cos(A + B) = \cos A \cos B - \sin A \sin B \tag{12.44}$$

$$\sin(A + B) = \cos A \sin B + \sin A \cos B. \tag{12.45}$$

Show that if the encoding is based purely on sine functions, without cosine functions, then this property no longer holds.

12.11 ($\star$) Consider the bag-of-words model (12.28) in which each of the component distributions $p(\mathbf{x}_n)$ is given by a general probability table that is shared across all words. Show that the maximum likelihood solution, given a training set of vectors, is given by a table whose entries are the fractions of times each word occurs in the training set.

12.12 ($\star$) Consider the autoregressive language model given by (12.31) and suppose that the terms $p(\mathbf{x}_n|\mathbf{x}_1, \ldots, \mathbf{x}_{n-1})$ on the right-hand side are represented by general probability tables. Show that the number of entries in these tables grows exponentially with the value of n.

12.13 ($\star$) When using n-grams it is usual to train the n-gram and $(n-1)$-gram models at the same time and then compute the conditional probability using the product rule of probability in the form

$$p(\mathbf{x}_n|\mathbf{x}_{n-L+1}, \ldots, \mathbf{x}_{n-1}) = \frac{p_L(\mathbf{x}_{n-L+1}, \ldots, \mathbf{x}_n)}{p_{L-1}(\mathbf{x}_{n-L+1}, \ldots, \mathbf{x}_{n-1})}. \tag{12.46}$$

Explain why this is more convenient than storing the left-hand-side directly, and show that to obtain the correct probabilities the final token from each sequence must be omitted when evaluating $p_{L-1}(\cdots)$.

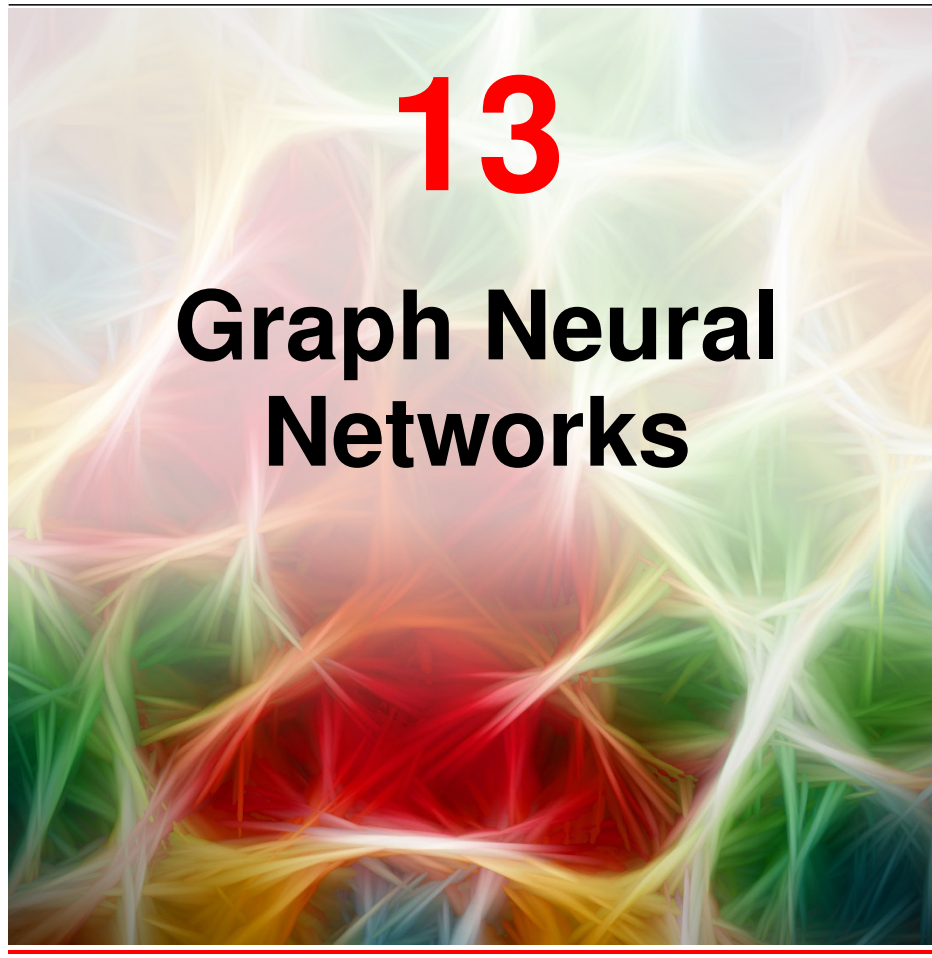
12.14 ($\star\star$) Write down pseudo-code for the inference process in a trained RNN with an architecture of the form depicted in Figure 12.13.

12.15 ($\star\star$) Consider a sequence of two tokens y_1 and y_2 each of which can take the states A or B. The table below shows the joint probability distribution $p(y_1, y_2)$:

	$y_1 = A$	$y_1 = B$
$y_2 = A$	0.0	0.4
$y_2 = B$	0.1	0.25

We see that the most probable sequence is $y_1 = B$, $y_2 = B$ and that this has probability 0.4. Using the sum and product rules of probability, write down the values of the marginal distribution $p(y_1)$ and the conditional distribution $p(y_2|y_1)$. Show that if we first maximize $p(y_1)$ to give a value $y_1^\star$ and then subsequently maximize $p(y_2|y_1^\star)$ then we obtain a sequence that is different from the overall most probable sequence. Find the probability of the sequence.

12.16 ($\star$) The BERT-Large model (Devlin *et al.*, 2018) has a maximum input length of 512 tokens, each of dimensionality $D = 1{,}024$ and taken from a vocabulary of 30,000. It has 24 transformer layers each with 16 self-attention heads with $D_{\mathrm{q}} = D_{\mathrm{k}} = D_{\mathrm{v}} = 64$, and the MLP position-wise networks have two layers with 4,096 hidden nodes. Show that the total number of parameters in the BERT encoder transformer language model is approximately 340 million.



13 Graph Neural Networks

In previous chapters we have encountered *structured data* in the form of sequences and images, corresponding to one-dimensional and two-dimensional arrays of variables respectively. More generally, there are many types of structured data that are best described by a *graph* as illustrated in Figure 13.1. In general a graph consists of a set of objects, known as *nodes*, connected by *edges*. Both the nodes and the edges can have data associated with them. For example, in a molecule the nodes and edges are associated with discrete variables corresponding to the types of atom (carbon, nitrogen, hydrogen, etc.) and the types of bonds (single bond, double bond, etc.). For a rail network, each railway line might be associated with a continuous variable given by the average journey time between two cities. Here we are assuming that the edges are symmetrical, for example that the journey time from London to Cambridge is the same as the journey time from Cambridge to London. Such edges are depicted by undirected links between the nodes. For the worldwide web the edges are directed

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024
C. M. Bishop, H. Bishop, *Deep Learning*, https://doi.org/10.1007/978-3-031-45468-4_13

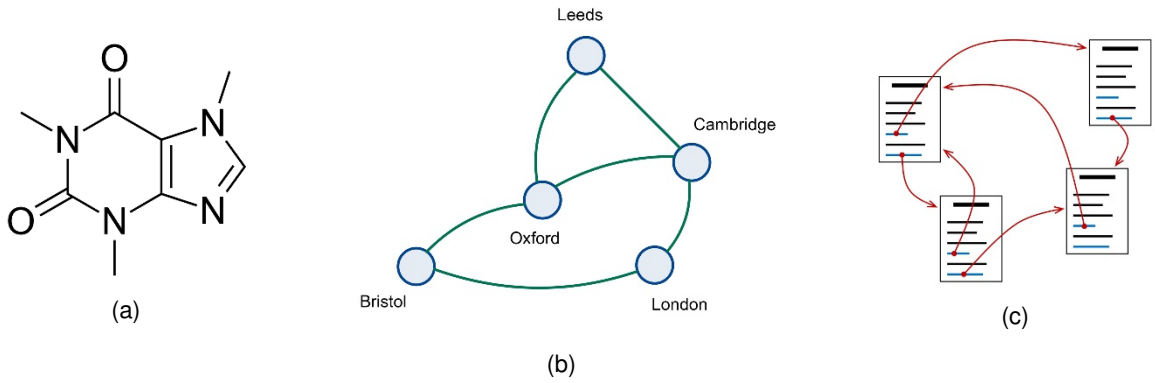


Figure 13.1 Three examples of graph-structured data: (a) the caffeine molecule consisting of atoms connected by chemical bonds, (b) a rail network consisting of cities connected by railway lines, and (c) the worldwide web consisting of pages connected by hyperlinks.

since if there is a hyperlink on page A that points to page B there is not necessarily a hyperlink on page B pointing back to page A.

Other examples of graph-structured data include a protein interaction network, in which the nodes are proteins and the edges express how strongly pairs of proteins interact, an electrical circuit where the nodes are components and the edges are conductors, or a social network where the nodes are people and the edges are 'friendships'. More complex graphical structures are also possible, for example the knowledge graph inside a company comprises multiple different kinds of nodes such as people, documents, and meetings, along with multiple kinds of edges capturing different properties such as a person being present at a meeting or a document referencing another document.

In this chapter we explore how to apply deep learning to graph-structured data. We have already encountered an example of structured data when we discussed images, in which the individual elements of an image data vector $\mathbf{x}$ correspond to pixels on a regular grid. An image is therefore a special instance of graph-structured data in which the nodes are the pixels and the edges describe which pixels are adjacent. *Chapter 10* Convolutional neural networks (CNNs) take this structure into account, incorporating prior knowledge of the relative positions of the pixels, together with the equivariance of properties such as segmentation and the invariance of properties such as classification. We will use CNNs for images as a source of inspiration to construct more general approaches to deep learning for graphical data known as *graph neural networks* (Zhou *et al.*, 2018; Wu *et al.*, 2019; Hamilton, 2020; Veličković, 2023). We will see that a key consideration when applying deep learning to graph-structured data is to ensure either equivariance or invariance with respect to a reordering of the nodes in the graph.

13.1. Machine Learning on Graphs

There are many kinds of applications that we might wish to address using graph-structured data, and we can group these broadly according to whether the goal is to predict properties of nodes, of edges, or of the whole graph. An example of node prediction would be to classify documents according to their topic based on the hyperlinks and citations between the documents.

Regarding edges we might, for example, know some of the interactions in a protein network and would like to predict the presence of any additional ones. Such tasks are called *edge prediction* or *graph completion* tasks. There are also tasks where the edges are known in advance and the goal is to discover clusters or 'communities' within the graph.

Finally, we may wish to predict properties that relate to the graph as a whole. For example, we might wish to predict whether a particular molecule is soluble in water. Here instead of being given a single graph we will have a data set of different graphs, which we can view as being drawn from some common distribution, in other words we assume that the graphs themselves are *independent and identically distributed*. Such tasks can be considered as graph regression or graph classification tasks.

For the molecule solubility classification example, we might be given a labelled training set of molecules, along with a test set of new molecules whose solubility needs to be predicted. This is a standard example of an *inductive* task of the kind we have seen many times in previous chapters. However, some graph prediction examples are *transductive* in which we are given the structure of the entire graph along with labels for some of the nodes and the goal is to predict the labels of the remaining nodes. An example would be a large social network in which our goal is to classify each node as either a real person or an automated bot. Here a small number of nodes might be manually labelled, but it would be prohibitive to investigate every node individually in a large and ever-changing social network. During training, we therefore have access to the whole graph along with labels for a subset of the nodes, and we wish to predict the labels for the remaining nodes. This can be viewed as a form of semi-supervised learning.

As well as solving prediction tasks directly, we can also use deep learning on graphs to discover useful internal representations that can subsequently facilitate a range of downstream tasks. This is known as *graph representation learning*. For example we could seek to build a *foundation model* for molecules by training a deep learning system on a large corpus of molecular structures. The goal is that once trained, such a foundation model can be fine-tuned to specific tasks by using a small, labelled data set.

Graph neural networks define an *embedding vector* for each of the nodes, usually initialized with the observed node properties, which are then transformed through a series of learnable layers to create a learned representation. This is analogous to the way word embeddings, or tokens, are processed through a series of layers in the transformer to give a representation that better captures the meaning of the words in the context of the rest of the text. Graph neural networks can also use learned embeddings associated with the edges and with the graph as a whole.

Chapter 12

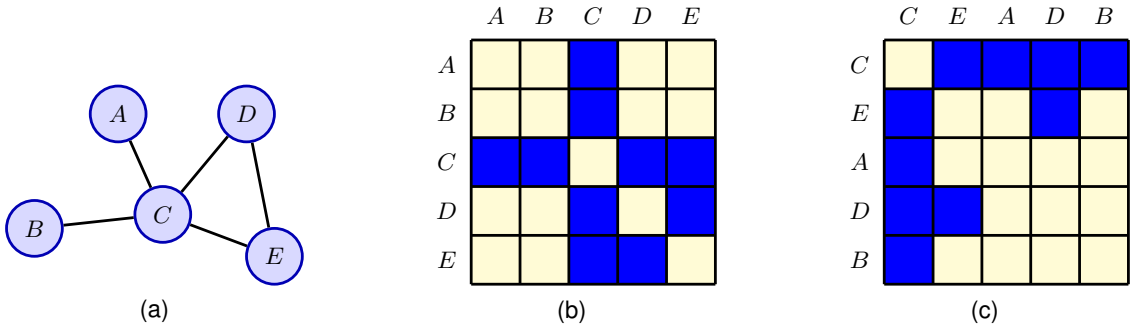


Figure 13.2 An example of an adjacency matrix showing (a) an example of a graph with five nodes, (b) the associated adjacency matrix for a particular choice of node order, and (c) the adjacency matrix corresponding to a different choice for the node order.

13.1.1 Graph properties

In this chapter we will focus on *simple graphs* where there is at most one edge between any pair of nodes, where the edges are undirected, and where there are no self-edges that connect a node to itself. This suffices to introduce the key concepts of graph neural networks, and it also encompasses a wide range of practical applications. These concepts can then be applied to more complex graphical structures.

We begin by introducing some notation associated with graphs and by defining some important properties. A graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ consists of a set of *nodes* or *vertices*, denoted by $\mathcal{V}$, along with a set of *edges* or *links*, denoted by $\mathcal{E}$. We index the nodes by $n = 1, \ldots N$, and we write the edge from node n to node m as (n, m). If two nodes are linked by an edge they are called *neighbours*, and the set of all neighbours of node n is denoted by $\mathcal{N}(n)$.

In addition to the graph structure, we usually also have observed data associated with the nodes. For each node n we can represent the corresponding node variables as a D-dimensional column vector $\mathbf{x}_n$ and we can group these into a data matrix $\mathbf{X}$ of dimensionality $N \times D$ in which row n is given by $\mathbf{x}_n^{\mathrm{T}}$. There may also be data variables associated with the edges in the graph, although to start with we will focus just on node variables.

Section 13.3.2

13.1.2 Adjacency matrix

A convenient way to specify the edges in a graph is to use an *adjacency matrix* denoted by $\mathbf{A}$. To define the adjacency matrix we first have to choose an ordering for the nodes. If there are N nodes in the graph, we can index them using $n = 1, \ldots, N$. The adjacency matrix has dimensions $N \times N$ and contains a 1 in every location n, m for which there is an edge going from node n to node m, with all other entries being 0. For graphs with undirected edges, the adjacency matrix will be symmetric since the presence of an edge from node n to node m implies that there is also an edge from node m to node n, and therefore $A_{mn} = A_{nm}$ for all n and m. An example of an adjacency matrix is shown in Figure 13.2.

Since the adjacency matrix defines the structure of a graph, we could consider

using it directly as the input to a neural network. To do this we could 'flatten' the matrix, for example by concatenating the columns into one long column vector. However, a major problem with this approach is that the adjacency matrix depends on the arbitrary choice of node ordering, as seen in Figure 13.2. Suppose for instance that we want to predict the solubility of a molecule. This clearly should not depend on the ordering assigned to the nodes when writing down an adjacency matrix. Because the number of permutations increases factorially with the number of nodes, it is impractical to try to learn permutation invariance by using large data sets or by data augmentation. Instead, we should treat this invariance property as an inductive bias when constructing a network architecture.

13.1.3 Permutation equivariance

We can express node label permutation mathematically by introducing the concept of a *permutation matrix* $\mathbf{P}$, which has the same size as the adjacency matrix and which specifies a particular permutation of a node ordering. It contains a single 1 in each row and a single 1 in each column, with 0 in all the other elements, such that a 1 in position n, m indicates that node n will be relabelled as node m after the permutation. Consider, for example, the permutation from $(A, B, C, D, E) \rightarrow (C, E, A, D, B)$ corresponding to the two choices of node ordering in Figure 13.2.

Exercise 13.1

The corresponding permutation matrix takes the form

$$\mathbf{P} = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{pmatrix}. \tag{13.1}$$

We can define the permutation matrix more formally as follows. First we introduce the *standard unit vector* $\mathbf{u}_n$, for $n = 1, \ldots, N$. This is a column vector in which all elements are 0 except element n, which equals 1. In this notation the identity matrix is given by

$$\mathbf{I} = \begin{pmatrix} \mathbf{u}_1^{\mathrm{T}} \\ \mathbf{u}_2^{\mathrm{T}} \\ \cdots \\ \mathbf{u}_N^{\mathrm{T}} \end{pmatrix}. \tag{13.2}$$

We can now introduce a permutation function $\pi(\cdot)$ that maps n to $m = \pi(n)$. The associated permutation matrix is given by

$$\mathbf{P} = \begin{pmatrix} \mathbf{u}_{\pi(1)}^{\mathrm{T}} \\ \mathbf{u}_{\pi(2)}^{\mathrm{T}} \\ \cdots \\ \mathbf{u}_{\pi(N)}^{\mathrm{T}} \end{pmatrix}. \tag{13.3}$$

When we reorder the labelling on the nodes of a graph, the effect on the corresponding node data matrix $\mathbf{X}$ is to permute the rows according to $\pi(\cdot)$, which can be achieved by pre-multiplication by $\mathbf{P}$ to give

Exercise 13.4

$$\widetilde{\mathbf{X}} = \mathbf{P}\mathbf{X}. \tag{13.4}$$

For the adjacency matrix, both the rows and the columns become permuted. Again the rows can be permuted using pre-multiplication by $\mathbf{P}$ whereas the columns are permuted using post-multiplication by $\mathbf{P}^{\mathrm{T}}$, giving a new adjacency matrix:

Exercise 13.5

$$\widetilde{\mathbf{A}} = \mathbf{P}\mathbf{A}\mathbf{P}^{\mathrm{T}}. \tag{13.5}$$

When applying deep learning to graph-structured data, we will need to represent the graph structure in numerical form so that it can be fed into a neural network, which requires that we assign an ordering to the nodes. However, the specific ordering we choose is arbitrary and so it will be important to ensure that any global property of the graph does not depend on this ordering. In other words, the network predictions must be *invariant* to node label reordering, so that

$$y(\widetilde{\mathbf{X}}, \widetilde{\mathbf{A}}) = y(\mathbf{X}, \mathbf{A}) \qquad \text{Invariance} \tag{13.6}$$

where $y(\cdot, \cdot)$ is the output of the network.

We may also want to make predictions that relate to individual nodes. In this case, if we reorder the node labelling then the corresponding predictions should show the same reordering so that a given prediction is always associated with the same node irrespective of the choice of order. In other words, node predictions should be *equivariant* with respect to node label reordering. This can be expressed as

$$\mathbf{y}(\widetilde{\mathbf{X}}, \widetilde{\mathbf{A}}) = \mathbf{P}\mathbf{y}(\mathbf{X}, \mathbf{A}) \qquad \text{Equivariance} \tag{13.7}$$

where $\mathbf{y}(\cdot, \cdot)$ is a vector of network outputs, with one element per node.

13.2. Neural Message-Passing

Ensuring invariance or equivariance under node label permutations is a key design consideration when we apply deep neural networks to graph-structured data. Another consideration is that we want to exploit the representational capabilities of deep neural networks and so we retain the concept of a 'layer' as a computational transformation that can be applied repeatedly. If each layer of the network is equivariant under node reordering then multiple layers applied in succession will also exhibit equivariance, while allowing each layer of the network to be informed by the graph structure.

For networks whose outputs represent node-level predictions, the whole network will be equivariant as required. If the network is being used to predict a graph-level property then a final layer can be included that is invariant to permutations of its inputs. We also want to ensure that each layer is a highly flexible nonlinear function and is differentiable with respect to its parameters so that it can be trained by stochastic gradient descent using gradients obtained by automatic differentiation.

Graphs come in various sizes. For example different molecules can have different numbers of atoms, so a fixed-length representation as used for standard neural

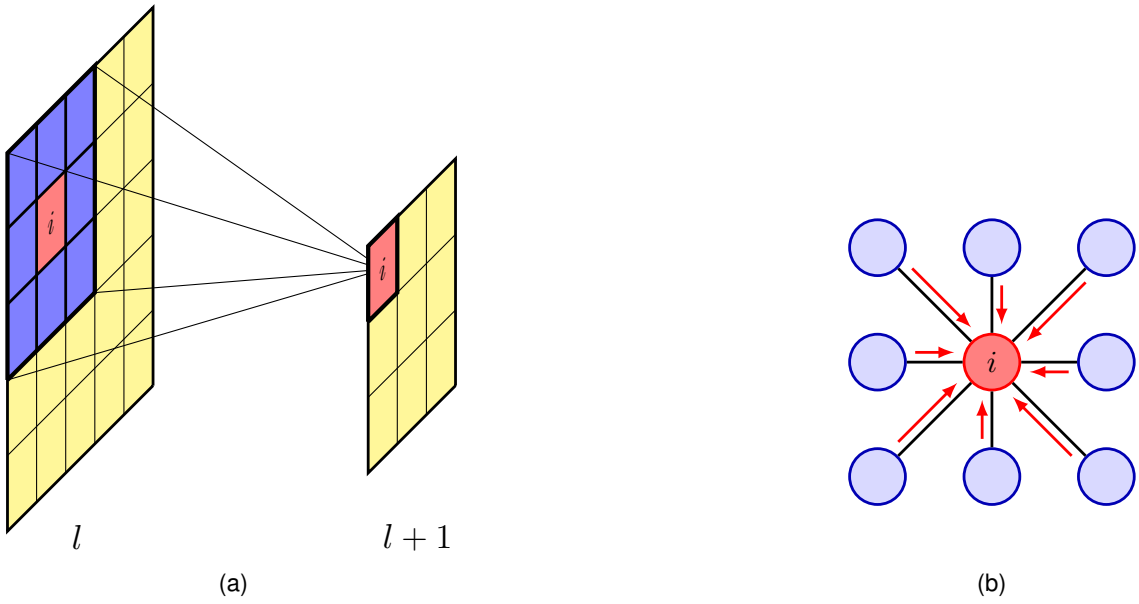


Figure 13.3 A convolutional filter for images can be represented as a graph-structured computation. (a) A filter computed by node i in layer $l+1$ of a deep convolutional network is a function of the activation values in layer l over a local patch of pixels. (b) The same computation structure expressed as a graph showing 'messages' flowing into node i from its neighbours.

Chapter 12

networks is unsuitable. A further requirement is therefore that the network should be able to handle variable-length inputs, as we saw with transformer networks. Some graphs can be very large, for example a social network with many millions of participants, and so we also want to construct models that scale well. Not surprisingly, parameter sharing will play an important role, both to allow the invariance and equivariance properties to be built into the network architecture but also to facilitate scaling to large graphs.

13.2.1 Convolutional filters

Chapter 10

To develop a framework that meets all of these requirements, we can seek inspiration from image processing using convolutional neural networks. First note that an image can be viewed as a specific instance of graph-structured data, in which the nodes are the pixels and the edges represent pairs of pixels that are adjacent in the image, where adjacency includes nodes that are diagonally adjacent as well as those that are horizontally or vertically adjacent.

Section 10.2

In a convolutional network, we make successive transformations of the image domain such that a pixel at a particular layer computes a function of states of pixels in the previous layer through a local function called a *filter*. Consider a convolutional layer using 3×3 filters, as illustrated in Figure 13.3(a). The computation performed

by a single filter at a single pixel in layer $l+1$ can be expressed as

$$z_i^{(l+1)} = f\left(\sum_j w_j z_j^{(l)} + b\right) \tag{13.8}$$

where $f(\cdot)$ is a differentiable nonlinear activation function such as ReLU, and the sum over j is taken over all nine pixels in a small patch in layer l. The same function is applied across multiple patches in the image, so that the weights w_j and bias b are shared across the patches (and therefore do not carry the index i).

As it stands, (13.8) is not equivariant under reordering of the nodes in layer l because the weight vector, with elements w_j, is not invariant under permutation of its elements. However, we can achieve equivariance with some simple modifications as follows. We first view the filter as a graph, as shown in Figure 13.3(b), and separate out the contribution from node i. The other eight 8 nodes are its neighbours $\mathcal{N}(i)$. We then assume that a single weight parameter w_{neigh} is shared across the neighbours so that

$$z_i^{(l+1)} = f\left(w_{\text{neigh}} \sum_{j\in\mathcal{N}(i)} z_j^{(l)} + w_{\text{self}} z_i^{(l)} + b\right) \tag{13.9}$$

where node i has its own weight parameter w_{self}.

We can interpret (13.9) as updating a local representation z_i at node i by gathering information from the neighbouring nodes by passing *messages* from the neighbouring nodes into node i. In this case the messages are simply the activations of the other nodes. These messages are then combined with information from node i, and the result is transformed using a nonlinear function. The information from the neighbouring nodes is *aggregated* through a simple summation in (13.9), and this is clearly invariant to any permutation of the labels associated with those nodes. Furthermore, the operation (13.9) is applied synchronously to every node in a graph, and so if the nodes are permuted then the resulting computations will be unchanged but their ordering will be likewise permuted, and hence, this calculation is equivariant under node reordering. Note that this depends on the parameters w_{neigh}, w_{self}, and b being shared across all nodes.

13.2.2 Graph convolutional networks

We now use the convolution example as a template to construct deep neural networks for graph-structured data. Our goal is to define a flexible, nonlinear transformation of the node embeddings that is differentiable with respect to a set of weight and bias parameters and which maps the variables in layer l into corresponding variables in layer $l+1$. For each node n in the graph and for each layer l in the network, we introduce a D-dimensional column vector $\mathbf{h}_n^{(l)}$ of node-embedding variables, where $n = 1, \ldots, N$ and $l = 1, \ldots, L$.

We see that the transformation given by (13.9) first gathers and combines information from neighbouring nodes and then updates the node as a function of the

Algorithm 13.1: Simple message-passing neural network

Input: Undirected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$
Initial node embeddings $\{\mathbf{h}_n^{(0)} = \mathbf{x}_n\}$
Aggregate$(\cdot)$ function
Update$(\cdot, \cdot)$ function
Output: Final node embeddings $\{\mathbf{h}_n^{(L)}\}$

```
// Iterative message-passing
```
for $l \in \{0, \ldots, L-1\}$ **do**
  $\mathbf{z}_n^{(l)} \leftarrow \text{Aggregate}\left(\left\{\mathbf{h}_m^{(l)} : m \in \mathcal{N}(n)\right\}\right)$
  $\mathbf{h}_n^{(l+1)} \leftarrow \text{Update}\left(\mathbf{h}_n^{(l)}, \mathbf{z}_n^{(l)}\right)$
end for
return $\{\mathbf{h}_n^{(L)}\}$

current embedding of the node and the incoming messages. We can therefore view each layer of processing as having two successive stages. The first is the *aggregation* stage in which, for each node n, messages are passed to that node from its neighbours and combined to form a new vector $\mathbf{z}_n^{(l)}$ in a way that is permutation invariant. This is followed by an *update* step in which the aggregated information from neighbouring nodes is combined with local information from the node itself and used to calculate a revised embedding vector for that node.

Consider a specific node n in the graph. We first aggregate the node vectors from all the neighbours of node n:

$$\mathbf{z}_n^{(l)} = \text{Aggregate}\left(\left\{\mathbf{h}_m^{(l)} : m \in \mathcal{N}(n)\right\}\right). \tag{13.10}$$

The form of this aggregation function is very flexible if it is well defined for a variable number of neighbouring nodes and does not depend on the ordering of those nodes. It can potentially contain learnable parameters as long as it is a differentiable function with respect to those parameters to facilitate gradient descent training.

We then use another operation to update the embedding vector at node n:

$$\mathbf{h}_n^{(l+1)} = \text{Update}\left(\mathbf{h}_n^{(l)}, \mathbf{z}_n^{(l)}\right). \tag{13.11}$$

Again, this can be a differentiable function of a set of learnable parameters. Application of the Aggregate operation followed by the Update operation in parallel for every node in the graph represents one layer of the network. The node embeddings are typically initialized using observed node data so that $\mathbf{h}_n^{(0)} = \mathbf{x}_n$. Note that each layer generally has its own independent parameters, although the parameters can also be shared across layers. This framework is called a *message-passing neural network* (Gilmer *et al.*, 2017) and is summarized in Algorithm 13.1.

13.2.3 Aggregation operators

There are many possible forms for the Aggregate function, but it must depend only on the set of inputs and not on their ordering. It must also be a differentiable function of any learnable parameters. The simplest such aggregation function, following from (13.9), is summation:

$$\text{Aggregate}\left(\{\mathbf{h}_m^{(l)} : m \in \mathcal{N}(n)\}\right) = \sum_{m \in \mathcal{N}(n)} \mathbf{h}_m^{(l)}. \tag{13.12}$$

A simple summation is clearly independent of the ordering of the neighbouring nodes and is also well defined no matter how many nodes are in the neighbourhood set. Note that this has no learnable parameters.

A summation gives a stronger influence over nodes that have many neighbours compared to those with few neighbours, and this can lead to numerical issues, particularly in applications such as social networks where the size of the neighbourhood set can vary by several orders of magnitude. A variation of this approach is to define the Aggregation operation to be the average of the neighbouring embedding vectors so that

$$\text{Aggregate}\left(\{\mathbf{h}_m^{(l)} : m \in \mathcal{N}(n)\}\right) = \frac{1}{|\mathcal{N}(n)|} \sum_{m \in \mathcal{N}(n)} \mathbf{h}_m^{(l)} \tag{13.13}$$

where $|\mathcal{N}(n)|$ denotes the number of nodes in the neighbourhood set $\mathcal{N}(n)$. However, this normalization also discards information about the network structure and is provably less powerful than a simple summation (Hamilton, 2020), and so the choice of whether to use it depends on the relative importance of node features compared to graph structure.

Another variation of this approach (Kipf and Welling, 2016) takes account of the number of neighbours for each of the neighbouring nodes:

$$\text{Aggregate}\left(\{\mathbf{h}_m^{(l)} : m \in \mathcal{N}(n)\}\right) = \sum_{m \in \mathcal{N}(n)} \frac{\mathbf{h}_m^{(l)}}{\sqrt{|\mathcal{N}(n)|\,|\mathcal{N}(m)|}}. \tag{13.14}$$

Yet another possibility is to take the element-wise maximum (or minimum) of the neighbouring embedding vectors, which also satisfies the desired properties of being well defined for a variable number of neighbours and of being independent of their order.

Since each node in a given layer of the network is updated by aggregating information from its neighbours in the previous layer, this defines a *receptive field* analogous to the receptive fields of filters used in CNNs. As information is processed through successive layers, the updates to a given node depend on a steadily increasing fraction of other nodes in earlier layers until the effective receptive field potentially spans the whole graph as illustrated in Figure 13.4. However, large, sparse graphs may require an excessive number of layers before each output is influenced by every input. Some architectures therefore introduce an additional 'super-node'

Chapter 10

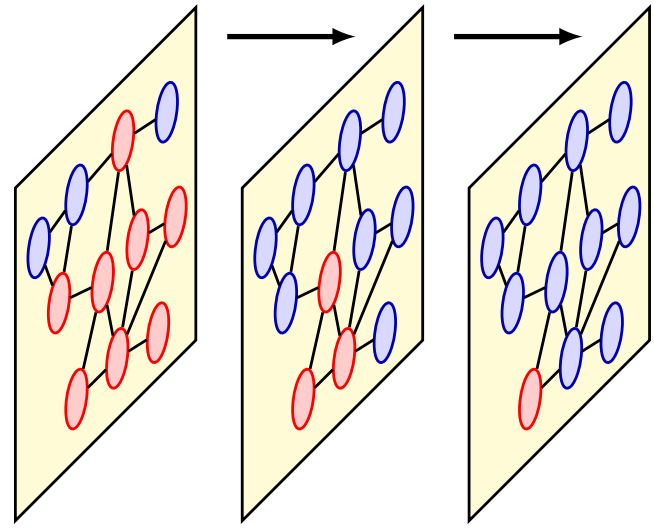


Figure 13.4 Schematic illustration of information flow through successive layers of a graph neural network. In the third layer a single node is highlighted in red. It receives information from its two neighbours in the previous layer and those in turn receive information from their neighbours in the first layer. As with convolutional neural networks for images, we see that the effective receptive field, corresponding to the number of nodes shown in red, grows with the number of processing layers.

that connects directly to every node in the original graph to ensure fast propagation of information.

The aggregation operators discussed so far have no learnable parameters. We can introduce such parameters if we first transform each of the embedding vectors from neighbouring nodes using a multilayer neural network, denoted by $\mathrm{MLP}_{\boldsymbol{\phi}}$, before combining their outputs, where MLP denotes 'multilayer perceptron' and $\boldsymbol{\phi}$ represents the parameters of the network. So long as the network has a structure and parameter values that are shared across nodes then this aggregation operator again be permutation invariant. We can also transform the combined vector with another neural network $\mathrm{MLP}_{\boldsymbol{\theta}}$, with parameters $\boldsymbol{\theta}$, to give an overall aggregation operator:

$$\text{Aggregate}\left(\{\mathbf{h}_m^{(l)} : m \in \mathcal{N}(n)\}\right) = \mathrm{MLP}_{\boldsymbol{\theta}}\left(\sum_{m\in\mathcal{N}(n)} \mathrm{MLP}_{\boldsymbol{\phi}}(\mathbf{h}_m^{(l)})\right) \tag{13.15}$$

in which $\mathrm{MLP}_{\boldsymbol{\phi}}$ and $\mathrm{MLP}_{\boldsymbol{\theta}}$ are shared across layer l. Due to the flexibility of MLPs, the transformation defined by (13.15) represents a universal approximator for any permutation-invariant function that maps a set of embeddings to a single embedding (Zaheer *et al.*, 2017). Note that the summation can be replaced by other invariant functions such as averages or an element-wise maximum or minimum.

A special case of graph neural networks arises if we consider a graph having no edges, which corresponds simply to an unstructured set of nodes. In this case if we use (13.15) for each vector $\mathbf{h}_n^{(l)}$ in the set, in which the summation is taken over all other vectors except $\mathbf{h}_n^{(l)}$, then we have a general framework for learning functions over unstructured sets of variables known as *deep sets*.

13.2.4 Update operators

Having chosen a suitable Aggregate operator, we similarly need to decide on the form of the Update operator. By analogy with (13.9) for the CNN, a simple form for this operator would be

$$\text{Update}\left(\mathbf{h}_n^{(l)}, \mathbf{z}_n^{(l)}\right) = f\left(\mathbf{W}_{\text{self}}\mathbf{h}_n^{(l)} + \mathbf{W}_{\text{neigh}}\mathbf{z}_n^{(l)} + \mathbf{b}\right) \tag{13.16}$$

where $f(\cdot)$ is a nonlinear activation function such as ReLU applied element-wise to its vector argument, and where $\mathbf{W}_{\text{self}}$, $\mathbf{W}_{\text{neigh}}$, and $\mathbf{b}$ are the learnable weights and biases and $\mathbf{z}_n^{(l)}$ is defined by the Aggregate operator (13.10).

If we choose a simple summation (13.12) as the aggregation function and if we also share the same weight matrix between nodes and their neighbours so that $\mathbf{W}_{\text{self}} = \mathbf{W}_{\text{neigh}}$, we obtain a particularly simple form of Update operator given by

$$\mathbf{h}_n^{(l+1)} = \text{Update}\left(\mathbf{h}_n^{(l)}, \mathbf{z}_n^{(l)}\right) = f\left(\mathbf{W}_{\text{neigh}} \sum_{m \in \mathcal{N}(n), n} \mathbf{h}_m^{(l)} + \mathbf{b}\right). \tag{13.17}$$

The message-passing algorithm is typically initialized by setting $\mathbf{h}_n^{(0)} = \mathbf{x}_n$. Sometimes, however, we may want to have an internal representation vector for each node that has a higher, or lower, dimensionality than that of $\mathbf{x}_n$. Such a representation can be initialized by padding the node vectors $\mathbf{x}_n$ with additional zeros (to achieve a higher dimensionality) or simply by transforming the node vectors using a learnable linear transformation to a space of the desired number of dimensions. An alternative form of initialization, particularly when there are no data variables associated with the nodes, is to use a one-hot vector that labels the degree of each node (i.e., the number of neighbours).

Overall, we can represent a graph neural network as a sequence of layers that successively transform the node embeddings. If we group these embeddings into a matrix $\mathbf{H}$ whose nth row is the vector $\mathbf{h}_n^{\mathrm{T}}$, which is initialized to the data matrix $\mathbf{X}$, then we can write the successive transformations in the form

$$\begin{aligned}
\mathbf{H}^{(1)} &= \mathbf{F}\left(\mathbf{X}, \mathbf{A}, \mathbf{W}^{(1)}\right) \\
\mathbf{H}^{(2)} &= \mathbf{F}\left(\mathbf{H}^{(1)}, \mathbf{A}, \mathbf{W}^{(2)}\right) \\
\vdots \quad &= \quad \vdots \\
\mathbf{H}^{(L)} &= \mathbf{F}\left(\mathbf{H}^{(L-1)}, \mathbf{A}, \mathbf{W}^{(L)}\right)
\end{aligned} \tag{13.18}$$

where $\mathbf{A}$ is the adjacency matrix, and $\mathbf{W}^{(l)}$ represents the complete set of weight and biases in layer l of the network. Under a node reordering defined by a permutation matrix $\mathbf{P}$, the transformation of the node embeddings computed by layer l is equivariant:

$$\mathbf{P}\mathbf{H}^{(l)} = \mathbf{F}\left(\mathbf{P}\mathbf{H}^{(l-1)}, \mathbf{P}\mathbf{A}\mathbf{P}^{\mathrm{T}}, \mathbf{W}^{(l)}\right). \tag{13.19}$$

Exercise 13.7 As a consequence, the complete network computes an equivariant transformation.

13.2.5 Node classification

A graph neural network can be viewed as a series of layers each of which transforms a set of node-embedding vectors $\{\mathbf{h}_n^{(l)}\}$ into a new set $\{\mathbf{h}_n^{(l+1)}\}$ of the same size and dimensionality. After the final convolutional layer of the network, we need to obtain predictions so that we can define a cost function for training and also for making predictions on new data using the trained network.

Consider first the task of classifying the nodes in a graph, which is one of the most common uses for graph neural networks. We can define an output layer, sometimes called a *readout layer*, which calculates a softmax function for each node corresponding to a classification over C classes, of the form

$$y_{ni} = \frac{\exp(\mathbf{w}_i^{\mathrm{T}}\mathbf{h}_n^{(L)})}{\sum_j \exp(\mathbf{w}_j^{\mathrm{T}}\mathbf{h}_n^{(L)})} \tag{13.20}$$

where $\{\mathbf{w}_i\}$ is a set of learnable weight vectors and $i = 1, \ldots, C$. We can then define a loss function as the sum of the cross-entropy loss across all nodes and all classes:

$$\mathcal{L} = -\sum_{n \in \mathcal{V}_{\text{train}}} \sum_{i=1}^{C} y_{ni}^{t_{ni}} \tag{13.21}$$

where $\{t_{ni}\}$ are target values with a one-hot encoding for each value of n. Because the weight vectors $\{\mathbf{w}_i\}$ are shared across the output nodes, the outputs y_{ni} are equivariant to permutation of the node ordering, and hence the loss function (13.21) is invariant. If the goal is to predict continuous values at the outputs then a simple linear transformation can be combined with a sum-of-squares error to define a suitable loss function.

The sum over n in (13.21) is taken over the subset of the nodes denoted by $\mathcal{V}_{\text{train}}$ and used for training. We can distinguish between three types of nodes as follows:

1. The nodes $\mathcal{V}_{\text{train}}$ are labelled and included in the message-passing operations of the graph neural network and are also used to compute the loss function used for training.

2. There is potentially also a *transductive* subset of nodes denoted by $\mathcal{V}_{\text{trans}}$, which are unlabelled and which do not contribute to the evaluation of the loss function used for training. However, they still participate in the message-passing operations during both training and inference, and their labels may be predicted as part of the inference process.

3. The remaining nodes, denoted $\mathcal{V}_{\text{induct}}$, are a set of *inductive* nodes that are not used to compute the loss function, and neither these nodes nor their associated edges participate in message-passing during the training phase. However, they do participate in message-passing during the inference phase and their labels are predicted as the outcome of inference.

If there are no transductive nodes, and hence the test nodes (and their associated edges) are not available during the training phase, then the training is generally referred to as *inductive learning*, which can be considered to be a form of *supervised learning*. However, if there are transductive nodes then it is called *transductive learning*, which may be viewed as a form of *semi-supervised* learning.

13.2.6 Edge classification

In some applications we wish to make predictions about the edges of the graph rather than the nodes. A common form of edge classification task is edge completion in which the goal is to determine whether an edge should be present between two nodes. Given a set of node embeddings, the dot product between pairs of embeddings can be used to define a probability $p(n, m)$ for the presence of an edge between nodes n and m by using the logistic sigmoid function:

$$p(n, m) = \sigma\left(\mathbf{h}_n^{\mathrm{T}}\mathbf{h}_m\right). \tag{13.22}$$

An example application would be predicting whether two people in a social network have shared interests and therefore might wish to connect.

13.2.7 Graph classification

In some applications of graph neural networks, the goal is to predict the properties of new graphs given a training set of labelled graphs $\mathcal{G}_1, \ldots, \mathcal{G}_N$. This requires that we combine the final-layer embedding vectors in a way that does not depend on the arbitrary node ordering, thereby ensuring that the output predictions will be invariant to that ordering. The goal is somewhat like that of the Aggregate function except that all nodes in the graph are included, not just the neighbourhood sets of the individual nodes. The simplest approach is to take the sum of the node-embedding vectors:

$$\mathbf{y} = \mathbf{f}\left(\sum_{n\in\mathcal{V}} \mathbf{h}_n^{(L)}\right) \tag{13.23}$$

where the function $\mathbf{f}$ may contain learnable parameters such as a linear transformation or a neural network. Other invariant aggregation functions can be used such as averages or element-wise minimum or maximum.

A cross-entropy loss is typically used for classification problems, such as labelling a candidate drug molecule as toxic or safe, and a squared-error loss for regression problems, such as predicting the solubility of a candidate drug molecule. Graph-level predictions correspond to an inductive task since there must be separate sets of graphs for training and for inference.

13.3. General Graph Networks

There are many variations and extensions of the graph networks considered so far. Here we outline a few of the key concepts along with some practical considerations.

13.3.1 Graph attention networks

Section 12.1

The attention mechanism is very powerful when used as the basis of a transformer architecture. It can be used in the context of graph neural networks to construct an aggregation function that combines messages from neighbouring nodes. The incoming messages are weighted by attention coefficients A_{nm} to give

$$\mathbf{z}_n^{(l)} = \text{Aggregate}\left(\left\{\mathbf{h}_m^{(l)} : m \in \mathcal{N}(n)\right\}\right) = \sum_{m\in\mathcal{N}(n)} A_{nm}\mathbf{h}_m^{(l)} \tag{13.24}$$

where the attention coefficients satisfy

$$A_{nm} \geqslant 0 \tag{13.25}$$

$$\sum_{m\in\mathcal{N}(n)} A_{nm} = 1. \tag{13.26}$$

This is known as a *graph attention network* (Veličković *et al.*, 2017) and can capture an inductive bias that says some neighbouring nodes will be more important than others in determining the best update in a way that depends on the data itself.

There are multiple ways to construct the attention coefficients, and these generally employ a softmax function. For example, we can use a bilinear form:

$$A_{nm} = \frac{\exp\left(\mathbf{h}_n^{\mathrm{T}}\mathbf{W}\mathbf{h}_m\right)}{\sum_{m'\in\mathcal{N}(n)}\exp\left(\mathbf{h}_n^{\mathrm{T}}\mathbf{W}\mathbf{h}_{m'}\right)} \tag{13.27}$$

where $\mathbf{W}$ is a $D \times D$ matrix of learnable parameters. A more general option is to use a neural network to combine the embedding vectors from the nodes at each end of the edge:

$$A_{nm} = \frac{\exp\left\{\text{MLP}\left(\mathbf{h}_n, \mathbf{h}_m\right)\right\}}{\sum_{m'\in\mathcal{N}(n)}\exp\left\{\text{MLP}\left(\mathbf{h}_n, \mathbf{h}_{m'}\right)\right\}} \tag{13.28}$$

where the MLP has a single continuous output variable whose value is invariant if the input vectors are exchanged. Provided the MLP is shared across all the nodes in the network, this aggregation function will be equivariant under node reordering.

Exercise 13.8

A graph attention network can be extended by introducing multiple attention heads in which H distinct sets of attention weights $A_{nm}^{(h)}$ are defined, for $h = 1, \ldots, H$, in which each head is evaluated using one of the mechanisms described above and with its own independent parameters. These are then combined in the aggregation step using concatenation and linear projection. Note that, for a fully-connected network, a multi-head graph attention network becomes a standard transformer encoder.

Section 12.1.6

Exercise 13.9

13.3.2 Edge embeddings

The graph neural networks discussed above use embedding vectors that are associated with the nodes. We have seen that some networks also have data associated with the edges. Even when there are no observable values associated with the edges,

we can still maintain and update edge-based hidden variables and these can contribute to the internal representations learned by the graph neural network.

In addition to the node embeddings given by $\mathbf{h}_n^{(l)}$, we therefore introduce edge embeddings $\mathbf{e}_{nm}^{(l)}$. We can then define general message-passing equations in the form

$$\mathbf{e}_{nm}^{(l+1)} = \text{Update}_{\text{edge}}\left(\mathbf{e}_{nm}^{(l)}, \mathbf{h}_n^{(l)}, \mathbf{h}_m^{(l)}\right) \tag{13.29}$$

$$\mathbf{z}_n^{(l+1)} = \text{Aggregate}_{\text{node}}\left(\{\mathbf{e}_{nm}^{(l+1)} : m \in \mathcal{N}(n)\}\right) \tag{13.30}$$

$$\mathbf{h}_n^{(l+1)} = \text{Update}_{\text{node}}\left(\mathbf{h}_n^{(l)}, \mathbf{z}_n^{(l+1)}\right). \tag{13.31}$$

The learned edge embeddings $\mathbf{e}_{nm}^{(L)}$ from the final layer can be used directly to make predictions associated with the edges.

13.3.3 Graph embeddings

In addition to node and edge embeddings we can also maintain and update an embedding vector $\mathbf{g}^{(l)}$ that relates to the graph as a whole. Bringing all these aspects together allows us to define a more general set of message-passing functions, and a richer set of learned representations, for graph-structured applications. Specifically, we can define general message-passing equations (Battaglia *et al.*, 2018):

$$\mathbf{e}_{nm}^{(l+1)} = \text{Update}_{\text{edge}}\left(\mathbf{e}_{nm}^{(l)}, \mathbf{h}_n^{(l)}, \mathbf{h}_m^{(l)}, \mathbf{g}^{(l)}\right) \tag{13.32}$$

$$\mathbf{z}_n^{(l+1)} = \text{Aggregate}_{\text{node}}\left(\{\mathbf{e}_{nm}^{(l+1)} : m \in \mathcal{N}(n)\}\right) \tag{13.33}$$

$$\mathbf{h}_n^{(l+1)} = \text{Update}_{\text{node}}\left(\mathbf{h}_n^{(l)}, \mathbf{z}_n^{(l+1)}, \mathbf{g}^{(l)}\right) \tag{13.34}$$

$$\mathbf{g}^{(l+1)} = \text{Update}_{\text{graph}}\left(\mathbf{g}^{(l)}, \{\mathbf{h}_n^{(l+1)} : n \in \mathcal{V}\}, \{\mathbf{e}_{nm}^{(l+1)} : (n,m) \in \mathcal{E}\}\right). \tag{13.35}$$

These update equations start in (13.32) by updating the edge embedding vectors $\mathbf{e}_{nm}^{(l+1)}$ based on the previous states of those vectors, on the node embeddings for the nodes connected by each edge, and on a graph-level embedding vector $\mathbf{g}^{(l)}$. These updated edge embeddings are then aggregated across every edge connected to each node using (13.33) to give a set of aggregated vectors. These in turn then contribute to the update of the node-embedding vector $\{\mathbf{h}_n^{(l+1)}\}$ based on the current node-embedding vectors and on the graph-level embedding vector using (13.34). Finally, the graph-level embedding vector is updated using (13.35) based on information from all the nodes and all the edges in the graph along with the graph-level embedding from the previous layer. These message-passing updates are illustrated in Figure 13.5 and are summarized in Algorithm 13.2.

13.3.4 Over-smoothing

One significant problem that can arise with some graph neural networks is called *over-smoothing* in which the node-embedding vectors tend to become very similar to each other after a number of iterations of message-passing, which effectively limits the depth of the network. One way to help alleviate this issue is to introduce residual connections. For example, we can modify the update operator (13.34):

Section 9.5

$$\mathbf{h}_n^{(l+1)} = \text{Update}_{\text{node}}\left(\mathbf{h}_n^{(l)}, \mathbf{z}_n^{(l+1)}, \mathbf{g}^{(l)}\right) + \mathbf{h}_n^{(l)}. \tag{13.36}$$

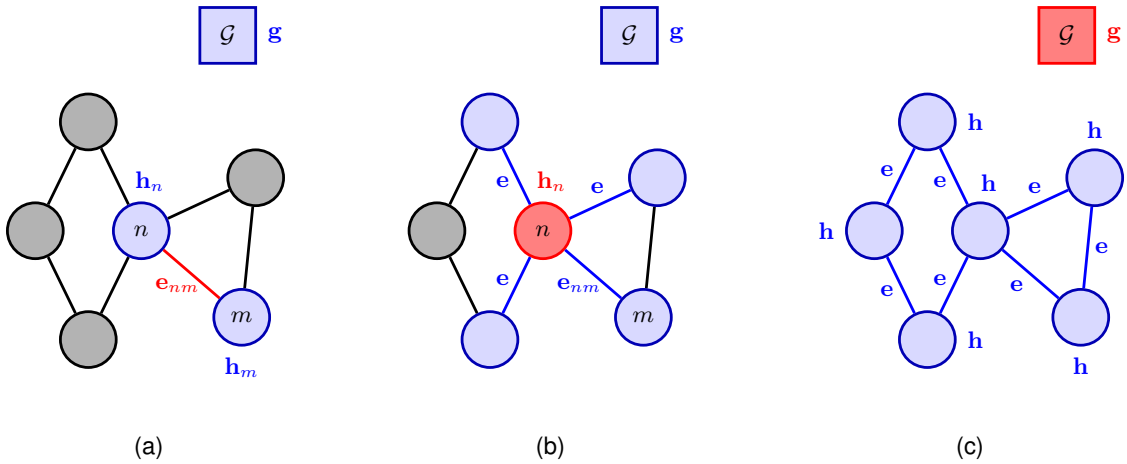


Figure 13.5 Illustration of the general graph message-passing updates defined by (13.32) to (13.35), showing (a) edge updates, (b) node updates, and (c) global graph updates. In each case the variable being updated is shown in red and the variables that contribute to that update are those shown in red and blue.

Another approach for mitigating the effects of over-smoothing is to allow the output layer to take information from all previous layers of the network and not just the final convolutional layer. This can be done for example by concatenating the representations from previous layers:

$$\mathbf{y}_n = \mathbf{f}\left(\mathbf{h}_n^{(1)} \oplus \mathbf{h}_n^{(2)} \oplus \cdots \oplus \mathbf{h}_n^{(L)}\right) \tag{13.37}$$

where $\mathbf{a} \oplus \mathbf{b}$ denotes the concatenation of vectors $\mathbf{a}$ and $\mathbf{b}$. A variant of this would be to combine the vectors using max pooling instead of concatenation. In this case each element of the output vector is given by the max of all the corresponding elements of the embedding vectors from the previous layers.

13.3.5 Regularization

Chapter 9

Standard techniques for regularization can be used with graph neural networks, including the addition of penalty terms, such as the sum-of-squares of the parameter values, to the loss function. In addition, some regularization methods have been developed specifically for graph neural networks.

Graph neural networks already employ weight sharing to achieve permutation equivariance and invariance, but typically they have independent parameters in each layer. However, weights and biases can also be shared across layers to reduce the number of independent parameters.

Dropout in the context of graph neural networks involves omitting random subsets of the graph nodes during training, with a fresh random subset chosen for each forward pass. This can likewise be applied to the edges in the graph in which randomly selected subsets of entries in the adjacency matrix are removed, or masked, during training.

Algorithm 13.2: Graph neural network with node, edge, and graph embeddings

Input: Undirected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$
Initial node embeddings $\{\mathbf{h}_n^{(0)}\}$
Initial edge embeddings $\{\mathbf{e}_{nm}^{(0)}\}$
Initial graph embedding $\mathbf{g}^{(0)}$
Output: Final node embeddings $\{\mathbf{h}_n^{(L)}\}$
Final edge embeddings $\{\mathbf{e}_{nm}^{(L)}\}$
Final graph embedding $\mathbf{g}^{(L)}$

`// Iterative message-passing`

for $l \in \{0, \ldots, L-1\}$ **do**

$$\mathbf{e}_{nm}^{(l+1)} \leftarrow \text{Update}_{\text{edge}}\left(\mathbf{e}_{nm}^{(l)}, \mathbf{h}_n^{(l)}, \mathbf{h}_m^{(l)}, \mathbf{g}^{(l)}\right)$$

$$\mathbf{z}_n^{(l+1)} \leftarrow \text{Aggregate}_{\text{node}}\left(\left\{\mathbf{e}_{nm}^{(l+1)} : m \in \mathcal{N}(n)\right\}\right)$$

$$\mathbf{h}_n^{(l+1)} \leftarrow \text{Update}_{\text{node}}\left(\mathbf{h}_n^{(l)}, \mathbf{z}_n^{(l+1)}, \mathbf{g}^{(l)}\right)$$

$$\mathbf{g}^{(l+1)} \leftarrow \text{Update}_{\text{graph}}\left(\mathbf{g}^{(l)}, \{\mathbf{h}_n^{(l+1)}\}, \{\mathbf{e}_{nm}^{(l+1)}\}\right)$$

end for

return $\{\mathbf{h}_n^{(L)}\}, \{\mathbf{e}_{nm}^{(L)}\}, \mathbf{g}^{(L)}$

13.3.6 Geometric deep learning

We have seen how permutation symmetry is a key consideration when designing deep learning models for graph-structured data. It acts as a form of inductive bias, dramatically reducing the data requirements while improving predictive performance. In applications of graph neural networks associated with spatial properties, such as graphics meshes, fluid flow simulations, or molecular structures, there are additional equivariance and invariance properties that can be built into the network architecture.

Consider the task of predicting the properties of a molecule, for example when exploring the space of candidate drugs. The molecule can be represented as a list of atoms of given types (carbon, hydrogen, nitrogen, etc.) along with the spatial coordinates of each atom expressed as a three-dimensional column vector. We can introduce an associated embedding vector for each atom n at each layer l, denoted by $\mathbf{r}_n^{(l)}$, and these vectors can be initialized with the known atom coordinates. However, the values for the elements of these vectors depends on the arbitrary choice of coordinate system, whereas the properties of the molecule do not. For example, the solubility of the molecule is unchanged if it is rotated in space or translated to a new position relative to the origin of the coordinate system, or if the coordinate system itself is reflected to give the mirror image version of the molecule. The molecular

properties should therefore be invariant under such transformations.

By making careful choices of the functional forms for the update and aggregation operations (Satorras, Hoogeboom, and Welling, 2021), the new embeddings $\mathbf{r}_n^{(l)}$ can be incorporated into the graph neural network update equations (13.29) to (13.31) to achieve the required symmetry properties:

$$\mathbf{e}_{nm}^{(l+1)} = \text{Update}_{\text{edge}}\left(\mathbf{e}_{nm}^{(l)}, \mathbf{h}_n^{(l)}, \mathbf{h}_m^{(l)}, \|\mathbf{r}_n^{(l)} - \mathbf{r}_m^{(l)}\|^2\right) \tag{13.38}$$

$$\mathbf{r}_n^{(l+1)} = \mathbf{r}_n^{(l)} + C \sum_{(n,m)\in\mathcal{E}} \left(\mathbf{r}_n^{(l)} - \mathbf{r}_m^{(l)}\right) \phi\left(\mathbf{e}_{nm}^{(l+1)}\right) \tag{13.39}$$

$$\mathbf{z}_n^{(l+1)} = \text{Aggregate}_{\text{node}}\left(\{\mathbf{e}_{nm}^{(l+1)} : m \in \mathcal{N}(n)\}\right) \tag{13.40}$$

$$\mathbf{h}_n^{(l+1)} = \text{Update}_{\text{node}}\left(\mathbf{h}_n^{(l)}, \mathbf{z}_n^{(l+1)}\right) \tag{13.41}$$

Note that the quantity $\|\mathbf{r}_n^{(l)} - \mathbf{r}_m^{(l)}\|^2$ represents the squared distance between the coordinates $\mathbf{r}_n^{(l)}$ and $\mathbf{r}_m^{(l)}$, and this does not depend on translations, rotations, or reflections. Also, the coordinates $\mathbf{r}_n^{(l)}$ are updated through a linear combination of the relative differences $\left(\mathbf{r}_n^{(l)} - \mathbf{r}_m^{(l)}\right)$. Here $\phi\left(\mathbf{e}_{nm}^{(l+1)}\right)$ is a general scalar function of the edge embeddings and is represented by a neural network, and the coefficient C is typically set equal to the reciprocal of the number of terms in the sum. It follows that under such transformations, the messages in (13.38), (13.40), and (13.41) are invariant and the coordinate embeddings given by (13.39) are equivariant.

Exercise 13.10

We have seen many examples of symmetries in structured data, from translations of objects within images and the permutation of node orderings on graphs, to rotations and translations of molecules in three-dimensional space. Capturing these symmetries in the structure of a deep neural network is a powerful form of inductive bias and forms the basis of a rich field of research known as *geometric deep learning* (Bronstein *et al.*, 2017; Bronstein *et al.*, 2021).

Exercises

13.1 ($\star$) Show that the permutation $(A, B, C, D, E) \to (C, E, A, D, B)$ corresponding to the two choices of node ordering in Figure 13.2 can be expressed in the form (13.5) with a permutation matrix given by (13.1).

13.2 ($\star\star$) Show that the number of edges connected to each node of a graph is given by the corresponding diagonal element of the matrix $\mathbf{A}^2$ where $\mathbf{A}$ is the adjacency matrix.

13.3 ($\star$) Draw the graph whose adjacency matrix is given by

$$\mathbf{A} = \begin{pmatrix} 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \end{pmatrix}. \tag{13.42}$$

13.4 ($\star\star$) Show that the effect of pre-multiplying a data matrix $\mathbf{X}$ using a permutation matrix $\mathbf{P}$ defined by (13.3) is to create a new data matrix $\widetilde{\mathbf{X}}$ given by (13.4) whose rows are permuted according to the permutation function $\pi(\cdot)$.

13.5 ($\star\star$) Show that the transformed adjacency matrix $\widetilde{\mathbf{A}}$ defined by (13.5), where $\mathbf{P}$ is defined by (13.3), is such that both the rows and the columns are permuted according to the permutation function $\pi(\cdot)$ relative to the original adjacency matrix $\mathbf{A}$.

13.6 ($\star\star$) In this exercise we write the update equations (13.16) as graph-level equations using matrices. To keep the notation uncluttered, we omit the layer index l. First, gather the node-embedding vectors $\{\mathbf{h}_n\}$ into an $N \times D$ matrix $\mathbf{H}$ in which row n is given by $\mathbf{h}_n^{\mathrm{T}}$. Then show that the neighbourhood-aggregated vectors $\mathbf{z}_n$ given by

$$\mathbf{z}_n = \sum_{m \in \mathcal{N}(n)} \mathbf{h}_m \tag{13.43}$$

can be written in matrix form as $\mathbf{Z} = \mathbf{A}\mathbf{H}$ where $\mathbf{Z}$ is the $N \times D$ matrix in which row n is given by $\mathbf{z}_n^{\mathrm{T}}$, and $\mathbf{A}$ is the adjacency matrix. Finally, show that the argument to the nonlinear activation function in (13.16) can be written in matrix form as

$$\mathbf{A}\mathbf{H}\mathbf{W}_{\text{neigh}} + \mathbf{H}\mathbf{W}_{\text{self}} + \mathbf{1}_D\mathbf{b}^{\mathrm{T}} \tag{13.44}$$

where $\mathbf{1}_D$ is the D-dimensional column vector in which all elements are 1.

13.7 ($\star\star$) By making use of the equivariance property (13.19) for layer l of a deep graph convolutional network along with the permutation property (13.4) for the node variables, show that a complete deep graph convolutional network defined by (13.18) is also equivariant.

13.8 ($\star\star$) Explain why the aggregation function defined by (13.24), in which the attention weights are given by (13.28), is equivariant under a reordering of the nodes in the graph.

13.9 ($\star$) Show that a graph attention network in which the graph is fully connected, so that there is an edge between every pair of nodes, is equivalent to a standard transformer architecture.

13.10 ($\star\star$) When a coordinate system is translated, the location of an object defined by that coordinate system is transformed using

$$\widetilde{\mathbf{r}} = \mathbf{r} + \mathbf{c} \tag{13.45}$$

where $\mathbf{c}$ is a fixed vector describing the translation. Similarly, if the coordinate system is rotated and/or mirror reflected, the location vector of an object is transformed using

$$\widetilde{\mathbf{r}} = \mathbf{R}\mathbf{r} \tag{13.46}$$

where $\mathbf{R}$ is an *orthogonal matrix* whose inverse is given by its transpose so that

$$\mathbf{R}\mathbf{R}^{\mathrm{T}} = \mathbf{R}^{\mathrm{T}}\mathbf{R} = \mathbf{I}. \tag{13.47}$$

Using these properties, show that under translations, rotations, and reflections, the messages in (13.38), (13.40), and (13.41) are invariant, and that the coordinate embeddings given by (13.39) are equivariant.



14 Sampling

There are many situations in deep learning where we need to create synthetic examples of a variable $\mathbf{z}$ from a probability distribution $p(\mathbf{z})$. Here $\mathbf{z}$ might be a scalar and the distribution might be a univariate Gaussian, or $\mathbf{z}$ might be a high-resolution image and $p(\mathbf{z})$ might be a generative model defined by a deep neural network. The process of creating such examples is known as *sampling*, also known as *Monte Carlo sampling*. For many simple distributions there are numerical techniques that generate suitable samples directly, whereas for more complex distributions, including ones that are defined implicitly, we may need more sophisticated approaches. We adopt the convention of referring to each instantiated value as a sample, in contrast to the convention used in classical statistics whereby 'sample' refers to a set of values.

In this chapter we focus on aspects of sampling that are most relevant to deep learning. Further information on Monte Carlo methods more generally can be found in Gilks, Richardson, and Spiegelhalter (1996) and Robert and Casella (1999).

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024
C. M. Bishop, H. Bishop, *Deep Learning*, https://doi.org/10.1007/978-3-031-45468-4_14

14.1. Basic Sampling Algorithms

In this section, we explore a variety of relatively simple strategies for generating random samples from a given distribution. Because the samples will be generated by a computer algorithm, they will in fact be *pseudo-random*, that is, they will be calculated using a deterministic algorithm but must nevertheless pass appropriate tests for randomness. Here we will assume that an algorithm has been provided that generates pseudo-random numbers distributed uniformly over $(0, 1)$, and indeed most software environments have such a facility built in.

14.1.1 Expectations

Although for some applications the samples themselves may be of direct interest, in other situations the goal is to evaluate *expectations* with respect to the distribution. Suppose we wish to find the expectation of a function $f(\mathbf{z})$ with respect to a probability distribution $p(\mathbf{z})$. Here, the components of $\mathbf{z}$ might comprise discrete or continuous variables or some combination of the two. For continuous variables the expectation is defined by

$$\mathbb{E}[f] = \int f(\mathbf{z})p(\mathbf{z})\,\mathrm{d}\mathbf{z} \tag{14.1}$$

where the integral is replaced by summation for discrete variables. This is illustrated schematically for a single continuous variable in Figure 14.1. We will suppose that such expectations are too complex to be evaluated exactly using analytical techniques.

The general idea behind sampling methods is to obtain a set of samples $\mathbf{z}^{(l)}$ (where $l = 1, \ldots, L$) drawn independently from the distribution $p(\mathbf{z})$. This allows the expectation (14.1) to be approximated by a finite sum:

$$\overline{f} = \frac{1}{L}\sum_{l=1}^{L} f(\mathbf{z}^{(l)}). \tag{14.2}$$

If the samples $\mathbf{z}^{(l)}$ are drawn from the distribution $p(\mathbf{z})$, then $\mathbb{E}[\overline{f}] = \mathbb{E}[f(\mathbf{z})]$ and so the estimator $\overline{f}$ has the correct mean. We can also write this in the form

Exercise 14.1

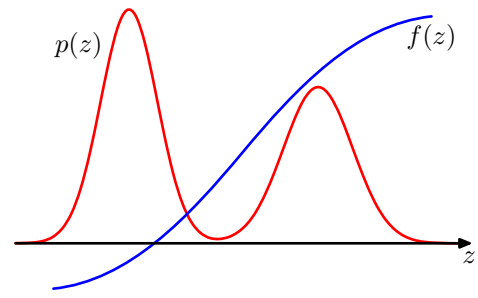


Figure 14.1 Schematic illustration of a function $f(z)$ whose expectation is to be evaluated with respect to a distribution $p(z)$.

$$\mathbb{E}[f(\mathbf{z})] \simeq \frac{1}{L}\sum_{l=1}^{L} f(\mathbf{z}^{(l)}) \tag{14.3}$$

where the symbol $\simeq$ denotes that the right-hand side is an unbiased estimator of the left-hand side, that is the two sides are equal when averaged over the noise distribution.

Exercise 14.2

The variance of the estimator (14.2) is given by

$$\text{var}[\widehat{f}] = \frac{1}{L}\mathbb{E}\left[(f - \mathbb{E}[f])^2\right], \tag{14.4}$$

which is the variance of the function $f(\mathbf{z})$ under the distribution $p(\mathbf{z})$. Note that the linear decrease of this variance with increasing L does not depend on the dimensionality of $\mathbf{z}$, and that, in principle, high accuracy may be achievable with a relatively small number of samples $\{\mathbf{z}^{(l)}\}$. The problem, however, is that the samples $\{\mathbf{z}^{(l)}\}$ might not be independent, and so the effective sample size might be much smaller than the apparent sample size. Also, referring back to Figure 14.1, note that if $f(\mathbf{z})$ is small in regions where $p(\mathbf{z})$ is large and vice versa, then the expectation may be dominated by regions of small probability, implying that relatively large sample sizes will be required to achieve sufficient accuracy.

14.1.2 Standard distributions

We now consider how to generate random numbers from simple nonuniform distributions, assuming that we already have available a source of uniformly distributed random numbers. Suppose that z is uniformly distributed over the interval $(0, 1)$, and that we transform the values of z using some function $g(\cdot)$ so that $y = g(z)$. The distribution of y will be governed by

Section 2.4

$$p(y) = p(z)\left|\frac{\mathrm{d}z}{\mathrm{d}y}\right| \tag{14.5}$$

where, in this case, $p(z) = 1$. Our goal is to choose the function $g(z)$ such that the resulting values of y have some specific desired distribution $p(y)$. Integrating (14.5) we obtain

$$z = \int_{-\infty}^{y} p(\widehat{y})\,\mathrm{d}\widehat{y} \equiv h(y) \tag{14.6}$$

Exercise 14.3

which is the indefinite integral of $p(y)$. Thus, $y = h^{-1}(z)$, and so we have to transform the uniformly distributed random numbers using a function that is the inverse of the indefinite integral of the desired distribution. This is illustrated in Figure 14.2.

Consider for example the *exponential distribution*

$$p(y) = \lambda \exp(-\lambda y) \tag{14.7}$$

where $0 \leqslant y < \infty$. In this case the lower limit of the integral in (14.6) is 0, and so $h(y) = 1 - \exp(-\lambda y)$. Thus, if we transform our uniformly distributed variable z using $y = -\lambda^{-1}\ln(1 - z)$, then y will have an exponential distribution.

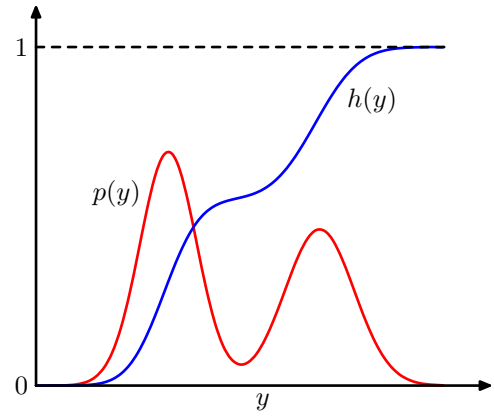


Figure 14.2 Geometrical interpretation of the transformation method for generating non-uniformly distributed random numbers. $h(y)$ is the indefinite integral of the desired distribution $p(y)$. If a uniformly distributed random variable z is transformed using $y = h^{-1}(z)$, then y will be distributed according to $p(y)$.

Another example of a distribution to which the transformation method can be applied is given by the Cauchy distribution

$$p(y) = \frac{1}{\pi}\frac{1}{1+y^2}. \tag{14.8}$$

Exercise 14.4 In this case, the inverse of the indefinite integral can be expressed in terms of the $\tan$ function.

Section 2.4 The generalization to multiple variables involves the Jacobian of the change of variables, so that

$$p(y_1,\ldots,y_M) = p(z_1,\ldots,z_M)\left|\frac{\partial(z_1,\ldots,z_M)}{\partial(y_1,\ldots,y_M)}\right|. \tag{14.9}$$

As a final example of the transformation technique, we consider the Box–Muller method for generating samples from a Gaussian distribution. First, suppose we generate pairs of uniformly distributed random numbers $z_1, z_2 \in (-1, 1)$, which we can do by transforming a variable distributed uniformly over $(0, 1)$ using $z \to 2z - 1$. Next we discard each pair unless it satisfies $z_1^2 + z_2^2 \leqslant 1$. This leads to a uniform distribution of points inside the unit circle with $p(z_1, z_2) = 1/\pi$, as illustrated in Figure 14.3. Then, for each pair z_1, z_2 we evaluate the quantities

$$y_1 = z_1\left(\frac{-2\ln r^2}{r^2}\right)^{1/2} \tag{14.10}$$

$$y_2 = z_2\left(\frac{-2\ln r^2}{r^2}\right)^{1/2} \tag{14.11}$$

Exercise 14.5 where $r^2 = z_1^2 + z_2^2$. Then the joint distribution of y_1 and y_2 is given by

$$\begin{aligned} p(y_1,y_2) &= p(z_1,z_2)\left|\frac{\partial(z_1,z_2)}{\partial(y_1,y_2)}\right| \\ &= \left[\frac{1}{\sqrt{2\pi}}\exp(-y_1^2/2)\right]\left[\frac{1}{\sqrt{2\pi}}\exp(-y_2^2/2)\right] \end{aligned} \tag{14.12}$$

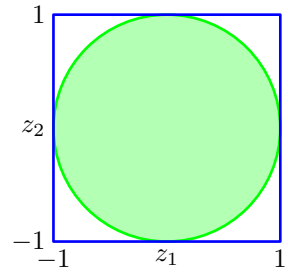


Figure 14.3 The Box–Muller method for generating Gaussian-distributed random numbers starts by generating samples from a uniform distribution inside the unit circle.

and so y_1 and y_2 are independent and each has a Gaussian distribution with zero mean and unit variance.

If y has a Gaussian distribution with zero mean and unit variance, then $\sigma y + \mu$ will have a Gaussian distribution with mean μ and variance σ^2. To generate vector-valued variables having a multivariate Gaussian distribution with mean $\boldsymbol{\mu}$ and covariance $\boldsymbol{\Sigma}$, we can make use of the *Cholesky decomposition*, which takes the form $\boldsymbol{\Sigma} = \mathbf{L}\mathbf{L}^{\mathrm{T}}$ (Deisenroth, Faisal, and Ong, 2020). Then, if $\mathbf{z}$ is a random vector whose components are independent and Gaussian distributed with zero mean and unit variance, then $\mathbf{y} = \boldsymbol{\mu} + \mathbf{L}\mathbf{z}$ will be Gaussian with mean $\boldsymbol{\mu}$ and covariance $\boldsymbol{\Sigma}$.

Exercise 14.6

Clearly, the transformation technique depends for its success on the ability to calculate and then invert the indefinite integral of the required distribution. Such operations are feasible only for a limited number of simple distributions, and so we must turn to alternative approaches in search of a more general strategy. Here we consider two techniques called *rejection sampling* and *importance sampling*. Although mainly limited to univariate distributions and thus not directly applicable to complex problems in many dimensions, they do form important components in more general strategies.

14.1.3 Rejection sampling

The rejection sampling framework allows us to sample from relatively complex distributions, subject to certain constraints. We begin by considering univariate distributions and subsequently discuss the extension to multiple dimensions.

Suppose we wish to sample from a distribution $p(\mathbf{z})$ that is not one of the simple, standard distributions considered so far and that sampling directly from $p(\mathbf{z})$ is difficult. Furthermore suppose, as is often the case, that we are easily able to evaluate $p(\mathbf{z})$ for any given value of $\mathbf{z}$, up to some normalizing constant Z, so that

$$p(z) = \frac{1}{Z_p}\widetilde{p}(z) \tag{14.13}$$

where $\widetilde{p}(z)$ can readily be evaluated, but Z_p is unknown.

To apply rejection sampling, we need some simpler distribution $q(z)$, sometimes called a *proposal distribution*, from which we can readily draw samples. We next introduce a constant k whose value is chosen such that $kq(z) \geqslant \widetilde{p}(z)$ for all values of z. The function $kq(z)$ is called the comparison function and is illustrated

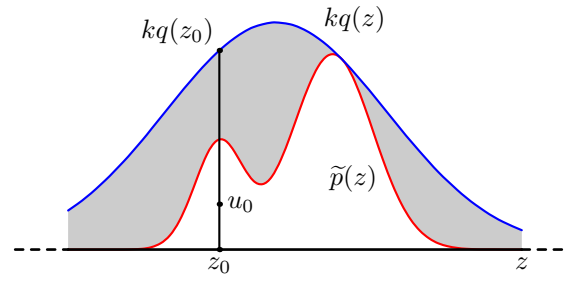


Figure 14.4 In the rejection sampling method, samples are drawn from a simple distribution $q(z)$ and rejected if they fall in the grey area between the unnormalized distribution $\widetilde{p}(z)$ and the scaled distribution $kq(z)$. The resulting samples are distributed according to $p(z)$, which is the normalized version of $\widetilde{p}(z)$.

for a univariate distribution in Figure 14.4. Each step of the rejection sampler involves generating two random numbers. First, we generate a number z_0 from the distribution $q(z)$. Next, we generate a number u_0 from the uniform distribution over $[0, kq(z_0)]$. This pair of random numbers has uniform distribution under the curve of the function $kq(z)$. Finally, if $u_0 > \widetilde{p}(z_0)$ then the sample is rejected, otherwise u_0 is retained. Thus, the pair is rejected if it lies in the grey shaded region in Figure 14.4. The remaining pairs then have uniform distribution under the curve of $\widetilde{p}(z)$, and hence the corresponding z values are distributed according to $p(z)$, as desired.

Exercise 14.7

The original values of z are generated from the distribution $q(z)$, and these samples are then accepted with probability $\widetilde{p}(z)/kq(z)$, and so the probability that a sample will be accepted is given by

$$\begin{aligned} p(\text{accept}) &= \int \{\widetilde{p}(z)/kq(z)\}\, q(z)\, \mathrm{d}z \\ &= \frac{1}{k} \int \widetilde{p}(z)\, \mathrm{d}z. \end{aligned} \tag{14.14}$$

Thus, the fraction of points that are rejected by this method depends on the ratio of the area under the unnormalized distribution $\widetilde{p}(z)$ to the area under the curve $kq(z)$. We therefore see that the constant k should be as small as possible subject to the limitation that $kq(z)$ must be nowhere less than $\widetilde{p}(z)$.

As an illustration of the use of rejection sampling, consider the task of sampling from the gamma distribution

$$\mathrm{Gam}(z|a, b) = \frac{b^a z^{a-1} \exp(-bz)}{\Gamma(a)}, \tag{14.15}$$

which, for $a > 1$, has a bell-shaped form, as shown in Figure 14.5. A suitable proposal distribution is therefore the Cauchy (14.8) because this too is bell-shaped and because we can use the transformation method, discussed earlier, to sample from it. We need to generalize the Cauchy slightly to ensure that it nowhere has a smaller value than the gamma distribution. This can be achieved by transforming a uniform random variable y using $z = b \tan y + c$, which gives random numbers distributed according to

Exercise 14.8

$$q(z) = \frac{k}{1 + (z - c)^2/b^2}. \tag{14.16}$$

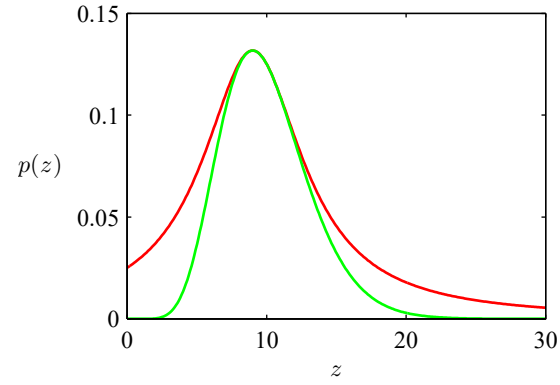


Figure 14.5 Plot showing the gamma distribution given by (14.15) as the green curve, with a scaled Cauchy proposal distribution shown by the red curve. Samples from the gamma distribution can be obtained by sampling from the Cauchy and then applying the rejection sampling criterion.

The minimum reject rate is obtained by setting $c = a-1$, and $b^2 = 2a-1$ and choosing the constant k to be as small as possible while still satisfying the requirement $kq(z) \geqslant \widetilde{p}(z)$. The resulting comparison function is also illustrated in Figure 14.5.

14.1.4 Adaptive rejection sampling

In many instances where we might wish to apply rejection sampling, it can be difficult to determine a suitable analytic form for the envelope distribution $q(z)$. An alternative approach is to construct the envelope function on the fly based on measured values of the distribution $p(z)$ (Gilks and Wild, 1992). Constructing an envelope function is particularly straightforward when $p(z)$ is log concave, in other words when $\ln p(z)$ has derivatives that are non-increasing functions of z. The construction of a suitable envelope function is illustrated graphically in Figure 14.6.

The function $\ln p(z)$ and its gradient are evaluated at some initial set of grid points, and the intersections of the resulting tangent lines are used to construct the envelope function. Next a sample value is drawn from the envelope distribution. This is straightforward because the log of the envelope distribution is a succession of linear functions, and hence the envelope distribution itself comprises a piecewise exponential distribution of the form

Exercise 14.10

$$q(z) = k_i \lambda_i \exp\left\{-\lambda_i (z - z_{i-1})\right\}, \qquad z_{i-1} < z \leqslant z_i. \tag{14.17}$$

Once a sample has been drawn, the usual rejection criterion can be applied. If the sample is accepted, then it will be a draw from the desired distribution. If, however, the sample is rejected, then it is incorporated into the set of grid points, a new tangent line is computed, and the envelope function is thereby refined. As the number of grid points increases, so the envelope function becomes a better approximation of the desired distribution $p(z)$ and the probability of rejection decreases.

There is a variant of the algorithm exists that avoids the evaluation of derivatives (Gilks, 1992). The adaptive rejection sampling framework can also be extended to distributions that are not log concave, simply by following each rejection sampling

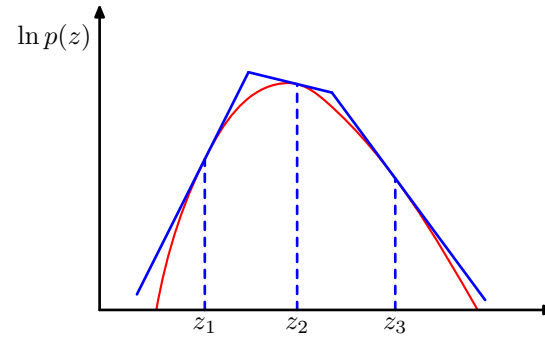


Figure 14.6 In rejection sampling, if a distribution is log concave then an envelope function can be constructed using the tangent lines computed at a set of grid points. If a sample point is rejected, it is added to the set of grid points and used to refine the envelope distribution.

step with a Metropolis–Hastings step (to be discussed in Section 14.2.3), giving rise to *adaptive rejection Metropolis* sampling (Gilks, Best, and Tan, 1995).

For rejection sampling to be of practical value, we require that the comparison function is close to the required distribution so that the rate of rejection is kept to a minimum. Now let us examine what happens when we try to use rejection sampling in spaces of high dimensionality. Consider, for illustration, a somewhat artificial problem in which we wish to sample from a zero-mean multivariate Gaussian distribution with covariance $\sigma_p^2\mathbf{I}$, where $\mathbf{I}$ is the unit matrix, by rejection sampling from a proposal distribution that is itself a zero-mean Gaussian distribution having covariance $\sigma_q^2\mathbf{I}$. Clearly, we must have $\sigma_q^2 \geqslant \sigma_p^2$ to ensure that there exists a k such that $kq(z) \geqslant p(z)$. In D-dimensions, the optimum value of k is given by $k = (\sigma_q/\sigma_p)^D$, as illustrated for $D = 1$ in Figure 14.7. The acceptance rate will be the ratio of volumes under $p(z)$ and $kq(z)$, which, because both distributions are normalized, is just $1/k$. Thus, the acceptance rate diminishes exponentially with dimensionality. Even if σ_q exceeds σ_p by just 1%, for $D = 1{,}000$ the acceptance ratio will be approximately $1/20{,}000$. In this illustrative example, the comparison function is close to the required distribution. For more practical examples, where the desired distribution may be multimodal and sharply peaked, it will be extremely difficult to find

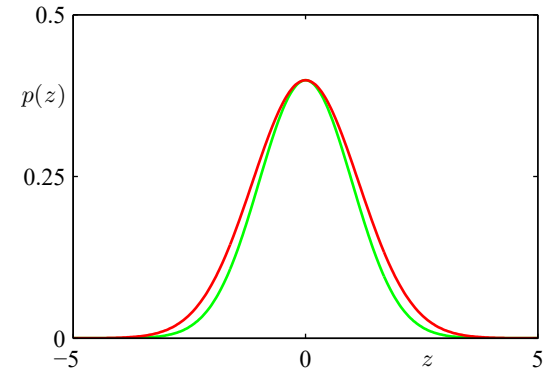


Figure 14.7 Illustrative example used to highlight a limitation of rejection sampling. Samples are drawn from a Gaussian distribution $p(z)$ shown by the green curve, by using rejection sampling from a proposal distribution $q(z)$ that is also Gaussian and whose scaled version $kq(z)$ is shown by the red curve.

a good proposal distribution and comparison function. Furthermore, the exponential decrease of the acceptance rate with dimensionality is a generic feature of rejection sampling. Although rejection can be a useful technique in one or two dimensions, it is unsuited to problems of high dimensionality. It can, however, play a role as a subroutine in more sophisticated algorithms for sampling in high-dimensional spaces.

14.1.5 Importance sampling

One reason for wishing to sample from complicated probability distributions is to evaluate expectations of the form (14.1). The technique of *importance sampling* provides a framework for approximating expectations directly but does not itself provide a mechanism for drawing samples from a distribution $p(\mathbf{z})$.

The finite sum approximation to the expectation, given by (14.2), depends on being able to draw samples from the distribution $p(\mathbf{z})$. Suppose, however, that it is impractical to sample directly from $p(\mathbf{z})$ but that we can evaluate $p(\mathbf{z})$ easily for any given value of $\mathbf{z}$. One simplistic strategy for evaluating expectations would be to discretize $\mathbf{z}$-space into a uniform grid and to evaluate the integrand as a sum of the form

$$\mathbb{E}[f] \simeq \sum_{l=1}^{L} p(\mathbf{z}^{(l)}) f(\mathbf{z}^{(l)}). \tag{14.18}$$

An obvious problem with this approach is that the number of terms in the summation grows exponentially with the dimensionality of $\mathbf{z}$. Furthermore, as we have already noted, the kinds of probability distributions of interest will often have much of their mass confined to relatively small regions of $\mathbf{z}$-space and so uniform sampling will be very inefficient because in high-dimensional problems, only a very small proportion of the samples will make a significant contribution to the sum. We would really like to choose sample points from regions where $p(\mathbf{z})$ is large or ideally where the product $p(\mathbf{z})f(\mathbf{z})$ is large.

As with rejection sampling, importance sampling is based a proposal distribution $q(\mathbf{z})$ from which it is easy to draw samples, as illustrated in Figure 14.8. We can then express the expectation in the form of a finite sum over samples $\{\mathbf{z}^{(l)}\}$ drawn from $q(\mathbf{z})$:

$$\begin{aligned} \mathbb{E}[f] &= \int f(\mathbf{z}) p(\mathbf{z}) \, \mathrm{d}\mathbf{z} \\ &= \int f(\mathbf{z}) \frac{p(\mathbf{z})}{q(\mathbf{z})} q(\mathbf{z}) \, \mathrm{d}\mathbf{z} \\ &\simeq \frac{1}{L} \sum_{l=1}^{L} \frac{p(\mathbf{z}^{(l)})}{q(\mathbf{z}^{(l)})} f(\mathbf{z}^{(l)}). \end{aligned} \tag{14.19}$$

The quantities $r_l = p(\mathbf{z}^{(l)})/q(\mathbf{z}^{(l)})$ are known as *importance weights*, and they correct the bias introduced by sampling from the wrong distribution. Note that, unlike rejection sampling, all the samples generated are retained.

Often the distribution $p(\mathbf{z})$ can be evaluated only up to a normalization constant, so that $p(\mathbf{z}) = \widetilde{p}(\mathbf{z})/Z_p$ where $\widetilde{p}(\mathbf{z})$ can be evaluated easily, whereas Z_p is unknown.

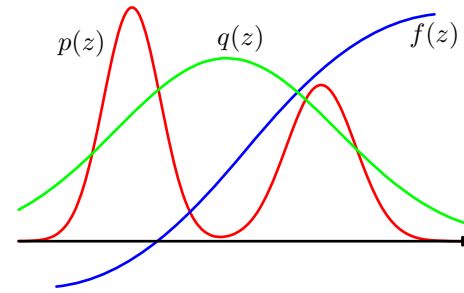


Figure 14.8 Importance sampling addresses the problem of evaluating the expectation of a function $f(z)$ with respect to a distribution $p(z)$ from which it is difficult to draw samples directly. Instead, samples $\{z^{(l)}\}$ are drawn from a simpler distribution $q(z)$, and the corresponding terms in the summation are weighted by the ratios $p(z^{(l)})/q(z^{(l)})$.

Similarly, we may wish to use an importance sampling distribution $q(\mathbf{z}) = \widetilde{q}(\mathbf{z})/Z_q$, which has the same property. We then have

$$\begin{aligned}\mathbb{E}[f] &= \int f(\mathbf{z})p(\mathbf{z})\,\mathrm{d}\mathbf{z} \\ &= \frac{Z_q}{Z_p}\int f(\mathbf{z})\frac{\widetilde{p}(\mathbf{z})}{\widetilde{q}(\mathbf{z})}q(\mathbf{z})\,\mathrm{d}\mathbf{z} \\ &\simeq \frac{Z_q}{Z_p}\frac{1}{L}\sum_{l=1}^{L}\widetilde{r}_l f(\mathbf{z}^{(l)}) \end{aligned} \tag{14.20}$$

where $\widetilde{r}_l = \widetilde{p}(\mathbf{z}^{(l)})/\widetilde{q}(\mathbf{z}^{(l)})$. We can use the same sample set to evaluate the ratio Z_p/Z_q with the result

$$\begin{aligned}\frac{Z_p}{Z_q} &= \frac{1}{Z_q}\int \widetilde{p}(\mathbf{z})\,\mathrm{d}\mathbf{z} = \int \frac{\widetilde{p}(\mathbf{z})}{\widetilde{q}(\mathbf{z})}q(\mathbf{z})\,\mathrm{d}\mathbf{z} \\ &\simeq \frac{1}{L}\sum_{l=1}^{L}\widetilde{r}_l \end{aligned} \tag{14.21}$$

and hence the expectation in (14.20) is given by a weighted sum:

$$\mathbb{E}[f] \simeq \sum_{l=1}^{L} w_l f(\mathbf{z}^{(l)}) \tag{14.22}$$

where we have defined

$$w_l = \frac{\widetilde{r}_l}{\sum_m \widetilde{r}_m} = \frac{\widetilde{p}(\mathbf{z}^{(l)})/q(\mathbf{z}^{(l)})}{\sum_m \widetilde{p}(\mathbf{z}^{(m)})/q(\mathbf{z}^{(m)})}. \tag{14.23}$$

Note that $\{w_l\}$ are non-negative numbers that sum to one.

As with rejection sampling, the success of importance sampling depends crucially on how well the sampling distribution $q(\mathbf{z})$ matches the desired distribution $p(\mathbf{z})$. If, as is often the case, $p(\mathbf{z})f(\mathbf{z})$ is strongly varying and has a significant proportion of its mass concentrated over relatively small regions of $\mathbf{z}$-space, then the

set of importance weights $\{r_l\}$ may be dominated by a few weights having large values, with the remaining weights being relatively insignificant. Thus, the effective sample size can be much smaller than the apparent sample size L. The problem is even more severe if none of the samples falls in the regions where $p(\mathbf{z})f(\mathbf{z})$ is large. In that case, the apparent variances of r_l and $r_l f(\mathbf{z}^{(l)})$ may be small even though the estimate of the expectation may be severely wrong. Hence, a major drawback of importance sampling is its potential to produce results that are arbitrarily in error and with no diagnostic indication. This also highlights a key requirement for the sampling distribution $q(\mathbf{z})$, namely that it should not be small or zero in regions where $p(\mathbf{z})$ may be significant.

14.1.6 Sampling-importance-resampling

The rejection sampling method discussed in Section 14.1.3 depends in part for its success on the determination of a suitable value for the constant k. For many pairs of distributions $p(\mathbf{z})$ and $q(\mathbf{z})$, it will be impractical to determine a suitable value for k as any value that is sufficiently large to guarantee a bound on the desired distribution will lead to impractically small acceptance rates.

As with rejection sampling, the *sampling-importance-resampling* approach also makes use of a sampling distribution $q(\mathbf{z})$ but avoids having to determine the constant k. There are two stages to the scheme. In the first stage, L samples $\mathbf{z}^{(1)}, \ldots, \mathbf{z}^{(L)}$ are drawn from $q(\mathbf{z})$. Then in the second stage, weights $w_1, \ldots, w_L$ are constructed using (14.23). Finally, a second set of L samples is drawn from the discrete distribution $(\mathbf{z}^{(1)}, \ldots, \mathbf{z}^{(L)})$ with probabilities given by the weights $(w_1, \ldots, w_L)$.

The resulting L samples are only approximately distributed according to $p(\mathbf{z})$, but the distribution becomes correct in the limit $L \to \infty$. To see this, consider the univariate case, and note that the cumulative distribution of the resampled values is given by

$$\begin{aligned} p(z \leqslant a) &= \sum_{l: z^{(l)} \leqslant a} w_l \\ &= \frac{\sum_l I(z^{(l)} \leqslant a) \widetilde{p}(z^{(l)})/q(z^{(l)})}{\sum_l \widetilde{p}(z^{(l)})/q(z^{(l)})} \end{aligned} \tag{14.24}$$

where $I(.)$ is the indicator function (which equals 1 if its argument is true and 0 otherwise). Taking the limit $L \to \infty$ and assuming suitable regularity of the distributions, we can replace the sums by integrals weighted according to the original

sampling distribution $q(z)$:

$$\begin{aligned} p(z \leqslant a) &= \frac{\int I(z \leqslant a)\,\{\widetilde{p}(z)/q(z)\}\,q(z)\,\mathrm{d}z}{\int \{\widetilde{p}(z)/q(z)\}\,q(z)\,\mathrm{d}z} \\ &= \frac{\int I(z \leqslant a)\widetilde{p}(z)\,\mathrm{d}z}{\int \widetilde{p}(z)\,\mathrm{d}z} \\ &= \int I(z \leqslant a)p(z)\,\mathrm{d}z, \end{aligned} \tag{14.25}$$

which is the cumulative distribution function of $p(z)$. Again, we see that normalization of $p(z)$ is not required.

For a finite value of L and a given initial sample set, the resampled values will only approximately be drawn from the desired distribution. As with rejection sampling, the approximation improves as the sampling distribution $q(\mathbf{z})$ gets closer to the desired distribution $p(\mathbf{z})$. When $q(\mathbf{z}) = p(\mathbf{z})$, the initial samples $(\mathbf{z}^{(1)}, \ldots, \mathbf{z}^{(L)})$ have the desired distribution and the weights $w_n = 1/L$, so that the resampled values also have the desired distribution.

If moments with respect to the distribution $p(\mathbf{z})$ are required, then they can be evaluated directly using the original samples together with the weights, because

$$\begin{aligned} \mathbb{E}[f(\mathbf{z})] &= \int f(\mathbf{z})p(\mathbf{z})\,\mathrm{d}\mathbf{z} \\ &= \frac{\int f(\mathbf{z})[\widetilde{p}(\mathbf{z})/q(\mathbf{z})]q(\mathbf{z})\,\mathrm{d}\mathbf{z}}{\int [\widetilde{p}(\mathbf{z})/q(\mathbf{z})]q(\mathbf{z})\,\mathrm{d}\mathbf{z}} \\ &\simeq \sum_{l=1}^{L} w_l f(\mathbf{z}_l). \end{aligned} \tag{14.26}$$

14.2. Markov Chain Monte Carlo

In the previous section, we discussed the rejection sampling and importance sampling strategies for evaluating expectations of functions, and we saw that they suffer from severe limitations particularly in spaces of high dimensionality. We therefore turn in this section to a very general and powerful framework called Markov chain Monte Carlo, which allows sampling from a large class of distributions and which scales well with the dimensionality of the sample space. Markov chain Monte Carlo methods have their origins in physics (Metropolis and Ulam, 1949), and it was only

towards the end of the 1980s that they started to have a significant impact in the field of statistics.

As with rejection and importance sampling, we again sample from a proposal distribution. This time, however, we maintain a record of the current state $\mathbf{z}^{(\tau)}$, and the proposal distribution $q(\mathbf{z}|\mathbf{z}^{(\tau)})$ is conditioned on this current state, and so the sequence of samples $\mathbf{z}^{(1)}, \mathbf{z}^{(2)}, \ldots$ forms a Markov chain. Again, if we write $p(\mathbf{z}) = \widetilde{p}(\mathbf{z})/Z_p$, we will assume that $\widetilde{p}(\mathbf{z})$ can readily be evaluated for any given value of $\mathbf{z}$, although the value of Z_p may be unknown. The proposal distribution is chosen to be sufficiently simple that it is straightforward to draw samples from it directly. At each cycle of the algorithm, we generate a candidate sample $\mathbf{z}^\star$ from the proposal distribution and then accept the sample according to an appropriate criterion.

Section 14.2.2

14.2.1 The Metropolis algorithm

In the basic *Metropolis* algorithm (Metropolis *et al.*, 1953), we assume that the proposal distribution is symmetric, that is $q(\mathbf{z}_A|\mathbf{z}_B) = q(\mathbf{z}_B|\mathbf{z}_A)$ for all values of $\mathbf{z}_A$ and $\mathbf{z}_B$. The candidate sample is then accepted with probability

$$A(\mathbf{z}^\star, \mathbf{z}^{(\tau)}) = \min\left(1, \frac{\widetilde{p}(\mathbf{z}^\star)}{\widetilde{p}(\mathbf{z}^{(\tau)})}\right). \tag{14.27}$$

This can be achieved by choosing a random number u with uniform distribution over the unit interval $(0, 1)$ and then accepting the sample if $A(\mathbf{z}^\star, \mathbf{z}^{(\tau)}) > u$. Note that if the step from $\mathbf{z}^{(\tau)}$ to $\mathbf{z}^\star$ causes an increase in the value of $p(\mathbf{z})$, then the candidate point is certain to be kept.

If the candidate sample is accepted, then $\mathbf{z}^{(\tau+1)} = \mathbf{z}^\star$, otherwise the candidate point $\mathbf{z}^\star$ is discarded, $\mathbf{z}^{(\tau+1)}$ is set to $\mathbf{z}^{(\tau)}$, and another candidate sample is drawn from the distribution $q(\mathbf{z}|\mathbf{z}^{(\tau+1)})$. This is in contrast to rejection sampling, where rejected samples are simply discarded. In the Metropolis algorithm, when a candidate point is rejected, the previous sample is included in the final list of samples, leading to multiple copies of samples. Of course, in a practical implementation, only a single copy of each retained sample would be kept, along with an integer weighting factor recording how many times that state appears. As we will see, if $q(\mathbf{z}_A|\mathbf{z}_B)$ is positive for any values of $\mathbf{z}_A$ and $\mathbf{z}_B$ (this is a sufficient but not necessary condition), the distribution of $\mathbf{z}^{(\tau)}$ tends to $p(\mathbf{z})$ as $\tau \to \infty$. It should be emphasized, however, that the sequence $\mathbf{z}^{(1)}, \mathbf{z}^{(2)}, \ldots$ is not a set of independent samples from $p(\mathbf{z})$ because successive samples are highly correlated. If we wish to obtain independent samples, then we can discard most of the sequence and just retain every Mth sample. For M sufficiently large, the retained samples will for all practical purposes be independent. The Metropolis algorithm in summarized in Algorithm 14.1. Figure 14.9 shows a simple illustrative example of sampling from a two-dimensional Gaussian distribution using the Metropolis algorithm in which the proposal distribution is an isotropic Gaussian.

Further insight into the nature of Markov chain Monte Carlo algorithms can be gleaned by looking at the properties of a specific example, namely a simple random

Algorithm 14.1: Metropolis sampling

Input: Unnormalized distribution $\widetilde{p}(\mathbf{z})$
Proposal distribution $q(\mathbf{z}|\widehat{\mathbf{z}})$
Initial state $\mathbf{z}^{(0)}$
Number of iterations T
Output: $\mathbf{z} \sim \widetilde{p}(\mathbf{z})$

$\mathbf{z}_{\text{prev}} \leftarrow \mathbf{z}^{(0)}$
// Iterative message-passing
for $\tau \in \{1, \ldots, T\}$ **do**
  $\mathbf{z}^\star \sim q(\mathbf{z}|\mathbf{z}_{\text{prev}})$ // Sample from proposal distribution
  $u \sim \mathcal{U}(0, 1)$ // Sample from uniform
  if $\widetilde{p}(\mathbf{z}^\star) \,/\, \widetilde{p}(\mathbf{z}_{\text{prev}}) > u$ **then**
    $\mathbf{z}_{\text{prev}} \leftarrow \mathbf{z}^\star$ // $\mathbf{z}^{(\tau)} = \mathbf{z}^\star$
  else
    $\mathbf{z}_{\text{prev}} \leftarrow \mathbf{z}_{\text{prev}}$ // $\mathbf{z}^{(\tau)} = \mathbf{z}^{(\tau-1)}$
  end if
end for
return $\mathbf{z}_{\text{prev}}$ // $\mathbf{z}^{(T)}$

walk. Consider a state space z consisting of the integers, with probabilities

$$p(z^{(\tau+1)} = z^{(\tau)}) = 0.5 \tag{14.28}$$
$$p(z^{(\tau+1)} = z^{(\tau)} + 1) = 0.25 \tag{14.29}$$
$$p(z^{(\tau+1)} = z^{(\tau)} - 1) = 0.25 \tag{14.30}$$

where $z^{(\tau)}$ denotes the state at step τ. If the initial state is $z^{(0)} = 0$, then by symmetry the expected state at time τ will also be zero $\mathbb{E}[z^{(\tau)}] = 0$, and similarly it is easily seen that $\mathbb{E}[(z^{(\tau)})^2] = \tau/2$. Thus, after τ steps, the random walk has travelled only a distance that on average is proportional to the square root of τ. This square root dependence is typical of random walk behaviour and shows that random walks are very inefficient in exploring the state space. As we will see, a central goal in designing Markov chain Monte Carlo methods is to avoid random walk behaviour.

Exercise 14.11

14.2.2 Markov chains

Before discussing Markov chain Monte Carlo methods in more detail, it is useful to study some general properties of Markov chains. In particular, we ask under what circumstances will a Markov chain converge to the desired distribution. A first-order

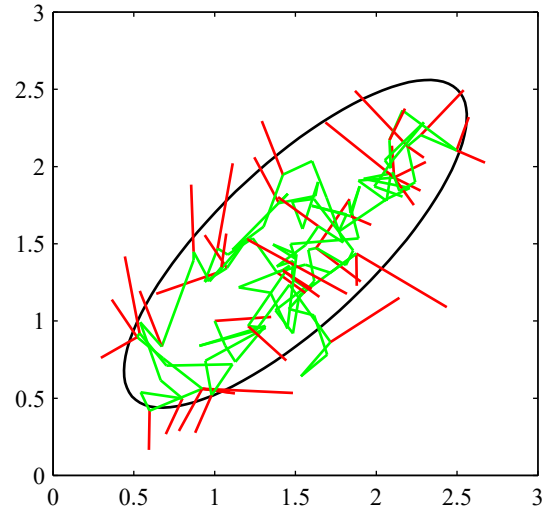


Figure 14.9 A simple illustration using the Metropolis algorithm to sample from a Gaussian distribution whose one standard-deviation contour is shown by the ellipse. The proposal distribution is an isotropic Gaussian distribution whose standard deviation is 0.2. Steps that are accepted are shown as green lines, and rejected steps are shown in red. A total of 150 candidate samples are generated, of which 43 are rejected.

Markov chain is defined to be a series of random variables $\mathbf{z}^{(1)}, \ldots, \mathbf{z}^{(M)}$ such that the following conditional independence property holds for $m \in \{1, \ldots, M-1\}$:

$$p(\mathbf{z}^{(m+1)}|\mathbf{z}^{(1)}, \ldots, \mathbf{z}^{(m)}) = p(\mathbf{z}^{(m+1)}|\mathbf{z}^{(m)}), \tag{14.31}$$

Figure 11.29

which can be represented as a directed graphical model in the form of a chain. We can then specify the Markov chain by giving the probability distribution for the initial variable $p(\mathbf{z}^{(0)})$ together with the conditional distributions for subsequent variables in the form of *transition probabilities* $T_m(\mathbf{z}^{(m)}, \mathbf{z}^{(m+1)}) \equiv p(\mathbf{z}^{(m+1)}|\mathbf{z}^{(m)})$. A Markov chain is called *homogeneous* if the transition probabilities are the same for all m.

The marginal probability for a particular variable can be expressed in terms of the marginal probability for the previous variable in the chain:

$$p(\mathbf{z}^{(m+1)}) = \int p(\mathbf{z}^{(m+1)}|\mathbf{z}^{(m)}) p(\mathbf{z}^{(m)}) \, \mathrm{d}\mathbf{z}^{(m)} \tag{14.32}$$

where the integral is replaced by a summation for discrete variables. A distribution is said to be invariant, or stationary, with respect to a Markov chain if each step in the chain leaves that distribution invariant. Thus, for a homogeneous Markov chain with transition probabilities $T(\mathbf{z}', \mathbf{z})$, the distribution $p^\star(\mathbf{z})$ is invariant if

$$p^\star(\mathbf{z}) = \int T(\mathbf{z}', \mathbf{z}) p^\star(\mathbf{z}') \, \mathrm{d}\mathbf{z}'. \tag{14.33}$$

Note that a given Markov chain may have more than one invariant distribution. For instance, if the transition probabilities are given by the identity transformation, then any distribution will be invariant.

A sufficient (but not necessary) condition for ensuring that the required distribution $p(\mathbf{z})$ is invariant is to choose the transition probabilities to satisfy the property of *detailed balance*, defined by

$$p^\star(\mathbf{z})T(\mathbf{z}, \mathbf{z}') = p^\star(\mathbf{z}')T(\mathbf{z}', \mathbf{z}) \tag{14.34}$$

for the particular distribution $p^\star(\mathbf{z})$. It is easily seen that a transition probability that satisfies detailed balance with respect to a particular distribution will leave that distribution invariant, because

$$\int p^\star(\mathbf{z}')T(\mathbf{z}', \mathbf{z})\,\mathrm{d}\mathbf{z}' = \int p^\star(\mathbf{z})T(\mathbf{z}, \mathbf{z}')\,\mathrm{d}\mathbf{z}' \tag{14.35}$$

$$= p^\star(\mathbf{z}) \int p(\mathbf{z}'|\mathbf{z})\,\mathrm{d}\mathbf{z}' \tag{14.36}$$

$$= p^\star(\mathbf{z}). \tag{14.37}$$

A Markov chain that respects detailed balance is said to be *reversible*.

Our goal is to use Markov chains to sample from a given distribution. We can achieve this if we set up a Markov chain such that the desired distribution is invariant. However, we must also require that for $m \to \infty$, the distribution $p(\mathbf{z}^{(m)})$ converges to the required invariant distribution $p^\star(\mathbf{z})$, irrespective of the choice of initial distribution $p(\mathbf{z}^{(0)})$. This property is called *ergodicity*, and the invariant distribution is then called the *equilibrium* distribution. Clearly, an ergodic Markov chain can have only one equilibrium distribution. It can be shown that a homogeneous Markov chain will be ergodic, subject only to weak restrictions on the invariant distribution and the transition probabilities (Neal, 1993).

In practice we often construct the transition probabilities from a set of 'base' transitions $B_1, \ldots, B_K$. This can be achieved through a mixture distribution of the form

$$T(\mathbf{z}', \mathbf{z}) = \sum_{k=1}^{K} \alpha_k B_k(\mathbf{z}', \mathbf{z}) \tag{14.38}$$

for some set of mixing coefficients $\alpha_1, \ldots, \alpha_K$ satisfying $\alpha_k \geqslant 0$ and $\sum_k \alpha_k = 1$. Alternatively, the base transitions may be combined through successive application, so that

$$T(\mathbf{z}', \mathbf{z}) = \sum_{\mathbf{z}_1} \ldots \sum_{\mathbf{z}_{n-1}} B_1(\mathbf{z}', \mathbf{z}_1) \ldots B_{K-1}(\mathbf{z}_{K-2}, \mathbf{z}_{K-1}) B_K(\mathbf{z}_{K-1}, \mathbf{z}). \tag{14.39}$$

If a distribution is invariant with respect to each of the base transitions, then clearly it will also be invariant with respect to either of the $T(\mathbf{z}', \mathbf{z})$ given by (14.38) or (14.39). For the mixture (14.38), if each of the base transitions satisfies detailed balance, then the mixture transition T will also satisfy detailed balance. This does not hold for the transition probability constructed using (14.39), although by symmetrizing the order of application of the base transitions, namely $B_1, B_2, \ldots, B_K, B_K, \ldots, B_2, B_1$, detailed balance can be restored. A common example of the use of composite transition probabilities is where each base transition changes only a subset of the variables.

14.2.3 The Metropolis–Hastings algorithm

Earlier we introduced the basic Metropolis algorithm without actually demonstrating that it samples from the required distribution. Before giving a proof, we first discuss a generalization, known as the *Metropolis–Hastings* algorithm (Hastings, 1970), which applies when the proposal distribution is no longer a symmetric function of its arguments. In particular at step τ of the algorithm, in which the current state is $\mathbf{z}^{(\tau)}$, we draw a sample $\mathbf{z}^\star$ from the distribution $q_k(\mathbf{z}|\mathbf{z}^{(\tau)})$ and then accept it with probability $A_k(\mathbf{z}^\star, \mathbf{z}^{(\tau)})$ where

$$A_k(\mathbf{z}^\star, \mathbf{z}^{(\tau)}) = \min\left(1, \frac{\widetilde{p}(\mathbf{z}^\star)q_k(\mathbf{z}^{(\tau)}|\mathbf{z}^\star)}{\widetilde{p}(\mathbf{z}^{(\tau)})q_k(\mathbf{z}^\star|\mathbf{z}^{(\tau)})}\right). \tag{14.40}$$

Here k labels the members of the set of possible transitions being considered. Again, evaluating the acceptance criterion does not require knowledge of the normalizing constant Z_p in the probability distribution $p(\mathbf{z}) = \widetilde{p}(\mathbf{z})/Z_p$. For a symmetric proposal distribution, the Metropolis–Hastings criterion (14.40) reduces to the standard Metropolis criterion given by (14.27). Metropolis–Hastings sampling is summarized in Algorithm 14.2.

We can show that $p(\mathbf{z})$ is an invariant distribution of the Markov chain defined by the Metropolis–Hastings algorithm by showing that detailed balance, defined by (14.34), is satisfied. Using (14.40) we have

$$\begin{aligned} p(\mathbf{z})q_k(\mathbf{z}'|\mathbf{z})A_k(\mathbf{z}', \mathbf{z}) &= \min\left(p(\mathbf{z})q_k(\mathbf{z}'|\mathbf{z}), p(\mathbf{z}')q_k(\mathbf{z}|\mathbf{z}')\right) \\ &= \min\left(p(\mathbf{z}')q_k(\mathbf{z}|\mathbf{z}'), p(\mathbf{z})q_k(\mathbf{z}'|\mathbf{z})\right) \\ &= p(\mathbf{z}')q_k(\mathbf{z}|\mathbf{z}')A_k(\mathbf{z}, \mathbf{z}') \end{aligned} \tag{14.41}$$

as required.

The specific choice of proposal distribution can have a marked effect on the performance of the algorithm. For continuous state spaces, a common choice is a Gaussian centred on the current state, leading to an important trade-off in determining the variance parameter of this distribution. If the variance is small, then the proportion of accepted transitions will be high, but progress through the state space takes the form of a slow random walk leading to long correlation times. However, if the variance parameter is large, then the rejection rate will be high because, in the kind of complex problems we are considering, many of the proposed steps will be to states for which the probability $p(\mathbf{z})$ is low. Consider a multivariate distribution $p(\mathbf{z})$ having strong correlations between the components of $\mathbf{z}$, as illustrated in Figure 14.10. The scale ρ of the proposal distribution should be as large as possible without incurring high rejection rates. This suggests that ρ should be of the same order as the smallest length scale $\sigma_{\min}$. The system then explores the distribution along the more extended direction by means of a random walk, and so the number of steps to arrive at a state that is more or less independent of the original state is of order $(\sigma_{\max}/\sigma_{\min})^2$. In fact in two dimensions, the increase in rejection rate as ρ increases is offset by the larger step sizes of those transitions that are accepted, and more generally for a multivariate Gaussian, the number of steps required to obtain independent samples scales like

Algorithm 14.2: Metropolis-Hastings sampling

Input: Unnormalized distribution $\widetilde{p}(\mathbf{z})$
Proposal distributions $\{q_k(\mathbf{z}|\widehat{\mathbf{z}}) : k \in 1, \ldots, K\}$
Mapping from iteration index to distribution index $M(\cdot)$
Initial state $\mathbf{z}^{(0)}$
Number of iterations T

Output: $\mathbf{z} \sim \widetilde{p}(\mathbf{z})$

$\mathbf{z}_{\text{prev}} \leftarrow \mathbf{z}^{(0)}$

// Iterative message-passing

for $\tau \in \{1, \ldots, T\}$ **do**

  $k \leftarrow M(\tau)$ // get distribution index for this iteration

  $\mathbf{z}^\star \sim q_k(\mathbf{z}|\mathbf{z}_{\text{prev}})$ // sample from proposal distribution

  $u \sim \mathcal{U}(0, 1)$ // sample from uniform

  if $\widetilde{p}(\mathbf{z}^\star)q(\mathbf{z}_{\text{prev}}|\mathbf{z}^\star) \;/\; \widetilde{p}(\mathbf{z}_{\text{prev}})q(\mathbf{z}^\star|\mathbf{z}_{\text{prev}}) > u$ **then**

    $\mathbf{z}_{\text{prev}} \leftarrow \mathbf{z}^\star$ // $\mathbf{z}^{(\tau)} = \mathbf{z}^\star$

  else

    $\mathbf{z}_{\text{prev}} \leftarrow \mathbf{z}_{\text{prev}}$ // $\mathbf{z}^{(\tau)} = \mathbf{z}^{(\tau-1)}$

  end if

end for

return $\mathbf{z}_{\text{prev}}$ // $\mathbf{z}^{(T)}$

$(\sigma_{\max}/\sigma_2)^2$ where σ_2 is the second-smallest standard deviation (Neal, 1993). These details aside, if the length scales over which the distributions vary are very different in different directions, then the Metropolis Hastings algorithm can have very slow convergence.

14.2.4 Gibbs sampling

Gibbs sampling (Geman and Geman, 1984) is a simple and widely applicable Markov chain Monte Carlo algorithm and can be seen as a special case of the Metropolis–Hastings algorithm. Consider the distribution $p(\mathbf{z}) = p(z_1, \ldots, z_M)$ from which we wish to sample, and suppose that we have chosen some initial state for the Markov chain. Each step of the Gibbs sampling procedure involves replacing the value of one of the variables by a value drawn from the distribution of that variable conditioned on the values of the remaining variables. Thus, we replace z_i by a value drawn from the distribution $p(z_i|\mathbf{z}_{\setminus i})$, where z_i denotes the ith component of $\mathbf{z}$, and $\mathbf{z}_{\setminus i}$ denotes $\{z_1, \ldots, z_M\}$ but with z_i omitted. This procedure is repeated either by cycling through the variables in some particular order or by choosing the

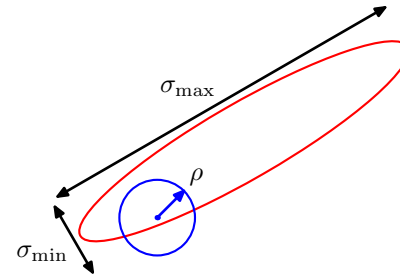


Figure 14.10 Schematic illustration of using an isotropic Gaussian proposal distribution (blue circle) to sample from a correlated multivariate Gaussian distribution (red ellipse) having very different standard deviations in different directions, using the Metropolis–Hastings algorithm. To keep the rejection rate low, the scale ρ of the proposal distribution should be of the order of the smallest standard deviation $\sigma_{\min}$, which leads to random walk behaviour in which the number of steps separating states that are approximately independent is of order $(\sigma_{\max}/\sigma_{\min})^2$ where $\sigma_{\max}$ is the largest standard deviation.

variable to be updated at each step at random from some distribution.

For example, suppose we have a distribution $p(z_1, z_2, z_3)$ over three variables, and at step τ of the algorithm we have selected values $z_1^{(\tau)}$, $z_2^{(\tau)}$, and $z_3^{(\tau)}$. We first replace $z_1^{(\tau)}$ by a new value $z_1^{(\tau+1)}$ obtained by sampling from the conditional distribution

$$p(z_1|z_2^{(\tau)}, z_3^{(\tau)}). \tag{14.42}$$

Next we replace $z_2^{(\tau)}$ by a value $z_2^{(\tau+1)}$ obtained by sampling from the conditional distribution

$$p(z_2|z_1^{(\tau+1)}, z_3^{(\tau)}) \tag{14.43}$$

so that the new value for z_1 is used straight away in subsequent sampling steps. Then we update z_3 with a sample $z_3^{(\tau+1)}$ drawn from

$$p(z_3|z_1^{(\tau+1)}, z_2^{(\tau+1)}) \tag{14.44}$$

and so on, cycling through the three variables in turn. Gibbs sampling is summarized in Algorithm 14.3.

To show that this procedure samples from the required distribution, we first note that the distribution $p(\mathbf{z})$ is an invariant of each of the Gibbs sampling steps individually and hence of the whole Markov chain. This follows since when we sample from $p(z_i|\mathbf{z}_{\setminus i})$, the marginal distribution $p(\mathbf{z}_{\setminus i})$ is clearly invariant because the value of $\mathbf{z}_{\setminus i}$ is unchanged. Also, each step by definition samples from the correct conditional distribution $p(z_i|\mathbf{z}_{\setminus i})$. Because these conditional and marginal distributions together specify the joint distribution, we see that the joint distribution is itself invariant.

The second requirement to be satisfied to ensure that the Gibbs sampling procedure samples from the correct distribution is that it is ergodic. A sufficient condition for ergodicity is that none of the conditional distributions are anywhere zero. If this is the case, then any point in z-space can be reached from any other point in a finite number of steps involving one update of each of the component variables. If this requirement is not satisfied, so that some of the conditional distributions have zeros, then ergodicity, if it applies, must be proven explicitly.

Algorithm 14.3: Gibbs sampling

Input: Initial values $\{z_i : i \in 1, \ldots, M\}$
Conditional distributions $\{p(z_i|\{z_{j\neq i}\}) : i \in 1, \ldots, M\}$
Number of iterations T

Output: Final values $\{z_i : i \in 1, \ldots, M\}$

---

for $\tau \in \{1, \ldots, T\}$ **do**
  for $i \in \{1, \ldots, M\}$ **do**
    $z_i \sim p(z_i|\{z_{j\neq i}\})$
  end for
end for
return $\{z_i : i \in 1, \ldots, M\}$

The distribution of initial states must also be specified to complete the algorithm, although samples drawn after many iterations will effectively become independent of this distribution. Of course, successive samples from the Markov chain will be highly correlated, and so to obtain samples that are nearly independent it will be necessary to sub-sample the sequence.

We can obtain the Gibbs sampling procedure as a particular instance of the Metropolis–Hastings algorithm as follows. Consider a Metropolis–Hastings sampling step involving the variable z_k in which the remaining variables $\mathbf{z}_{\backslash k}$ remain fixed, and for which the transition probability from $\mathbf{z}$ to $\mathbf{z}^\star$ is given by $q_k(\mathbf{z}^\star|\mathbf{z}) = p(z_k^\star|\mathbf{z}_{\backslash k})$. Note that $\mathbf{z}_{\backslash k}^\star = \mathbf{z}_{\backslash k}$ because these components are unchanged by the sampling step. Also, $p(\mathbf{z}) = p(z_k|\mathbf{z}_{\backslash k})p(\mathbf{z}_{\backslash k})$. Thus, the factor that determines the acceptance probability in the Metropolis–Hastings (14.40) is given by

$$A(\mathbf{z}^\star, \mathbf{z}) = \frac{p(\mathbf{z}^\star)q_k(\mathbf{z}|\mathbf{z}^\star)}{p(\mathbf{z})q_k(\mathbf{z}^\star|\mathbf{z})} = \frac{p(z_k^\star|\mathbf{z}_{\backslash k}^\star)p(\mathbf{z}_{\backslash k}^\star)p(z_k|\mathbf{z}_{\backslash k}^\star)}{p(z_k|\mathbf{z}_{\backslash k})p(\mathbf{z}_{\backslash k})p(z_k^\star|\mathbf{z}_{\backslash k})} = 1 \tag{14.45}$$

where we have used $\mathbf{z}_{\backslash k}^\star = \mathbf{z}_{\backslash k}$. Thus, the Metropolis–Hastings steps are always accepted.

As with the Metropolis algorithm, we can gain some insight into the behaviour of Gibbs sampling by investigating its application to a Gaussian distribution. Consider a correlated Gaussian in two variables, as illustrated in Figure 14.11, having conditional distributions of width l and marginal distributions of width L. The typical step size is governed by the conditional distributions and will be of order l. Because the state evolves according to a random walk, the number of steps needed to obtain independent samples from the distribution will be of order $(L/l)^2$. Of course if the Gaussian distribution were uncorrelated, then the Gibbs sampling procedure would be optimally efficient. For this simple problem, we could rotate the coordinate system such that the new variables are uncorrelated. However, in practical applications

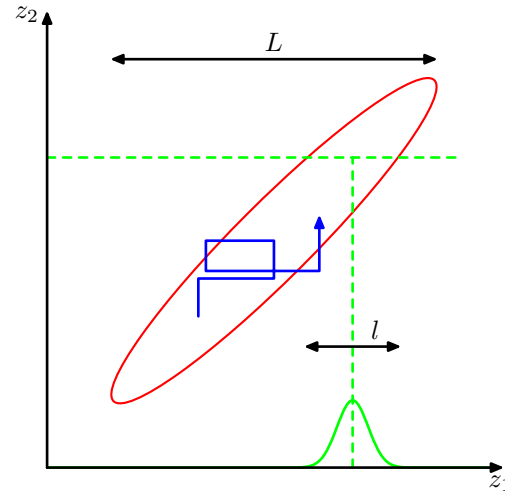


Figure 14.11 Illustration of Gibbs sampling by alternate updates of two variables whose distribution is a correlated Gaussian. The step size is governed by the standard deviation of the conditional distribution (green curve), and is $\mathcal{O}(l)$, leading to slow progress in the direction of elongation of the joint distribution (red ellipse). The number of steps needed to obtain an independent sample from the distribution is $\mathcal{O}((L/l)^2)$.

it will generally be infeasible to find such transformations.

One approach to reducing the random walk behaviour in Gibbs sampling is called *over-relaxation* (Adler, 1981). In its original form, it applies to problems for which the conditional distributions are Gaussian, which represents a more general class of distributions than the multivariate Gaussian because, for example, the non-Gaussian distribution $p(z, y) \propto \exp(-z^2y^2)$ has Gaussian conditional distributions. At each step of the Gibbs sampling algorithm, the conditional distribution for a particular component z_i has some mean μ_i and some variance σ_i^2. In the over-relaxation framework, the value of z_i is replaced with

$$z_i' = \mu_i + \alpha_i(z_i - \mu_i) + \sigma_i(1 - \alpha_i^2)^{1/2}\nu \tag{14.46}$$

where ν is a Gaussian random variable with zero mean and unit variance, and α is a parameter such that $-1 < \alpha < 1$. For $\alpha = 0$, the method is equivalent to standard Gibbs sampling, and for $\alpha < 0$ the step is biased to the opposite side of the mean. This step leaves the desired distribution invariant because if z_i has mean μ_i and variance σ_i^2, then so too does z_i'. The effect of over-relaxation is to encourage directed motion through state space when the variables are highly correlated. The framework of *ordered over-relaxation* (Neal, 1999) generalizes this approach to non-Gaussian distributions.

Exercise 14.14

The practical applicability of Gibbs sampling depends on the ease with which samples can be drawn from the conditional distributions $p(z_k|\mathbf{z}_{\setminus k})$. For probability distributions specified using directed graphical models, the conditional distributions for individual nodes depend only on the variables in the corresponding Markov blanket, as illustrated in Figure 14.12. For directed graphs, a wide choice of conditional distributions for the individual nodes conditioned on their parents will lead to conditional distributions for Gibbs sampling that are log concave. The adaptive rejection sampling methods discussed in Section 14.1.4 therefore provide a framework for Monte Carlo sampling from directed graphs with broad applicability.

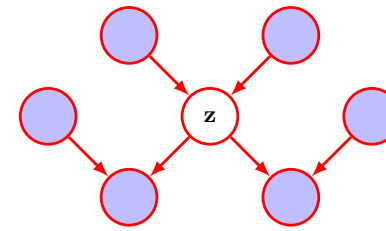


Figure 14.12 The Gibbs sampling method requires samples to be drawn from the conditional distribution of a variable $\mathbf{z}$ conditioned on the remaining variables. For directed graphical models, this conditional distribution is a function of only the states of the nodes in the Markov blanket, shaded in blue, which comprises the parents, the children, and the co-parents.

Because the basic Gibbs sampling technique considers one variable at a time, there are strong dependencies between successive samples. At the opposite extreme, if we could draw samples directly from the joint distribution (an operation that we are supposing is intractable), then successive samples would be independent. We can hope to improve on the simple Gibbs sampler by adopting an intermediate strategy in which we sample successively from groups of variables rather than individual variables. This is achieved in the *blocking Gibbs* sampling algorithm by choosing blocks of variables, not necessarily disjoint, and then sampling jointly from the variables in each block in turn, conditioned on the remaining variables (Jensen, Kong, and Kjaerulff, 1995).

14.2.5 Ancestral sampling

For many models, the joint distribution $p(\mathbf{z})$ is conveniently specified in terms of a graphical model. For a directed graph with no observed variables, it is straightforward to sample from the joint distribution using the following *ancestral sampling* approach. The joint distribution is specified by

$$p(\mathbf{z}) = \prod_{i=1}^{M} p(\mathbf{z}_i|\mathrm{pa}(i)) \tag{14.47}$$

where $\mathbf{z}_i$ are the set of variables associated with node i, and $\mathrm{pa}(i)$ denotes the set of variables associated with the parents of node i. To obtain a sample from the joint distribution, we make one pass through the set of variables in the order $\mathbf{z}_1, \ldots, \mathbf{z}_M$ sampling from the conditional distributions $p(\mathbf{z}_i|\mathrm{pa}(i))$. This is always possible because at each step, all the parent values will have been instantiated. After one pass through the graph, we will have obtained a sample from the joint distribution. This assumes that it is possible to sample from the individual conditional distributions at each node.

Now consider a directed graph in which some of the nodes, which comprise the *evidence set* $\mathcal{E}$, are instantiated with observed values. We can in principle extend the above procedure, at least for nodes representing discrete variables, to give the following *logic sampling* approach (Henrion, 1988), which can be seen as a special case of *importance sampling*. At each step, when a sampled value is obtained for a variable $\mathbf{z}_i$ whose value is observed, the sampled value is compared to the observed value, and if they agree then the sample value is retained and the algorithm proceeds to the next variable in turn. However, if the sampled value and the observed value disagree, then the whole sample so far is discarded and the algorithm starts again

Section 14.1.5

with the first node in the graph. This algorithm samples correctly from the posterior distribution because it corresponds simply to drawing samples from the joint distribution of hidden variables and data variables and then discarding those samples that disagree with the observed data (with the slight saving of not continuing with the sampling from the joint distribution as soon as one contradictory value is observed). However, the overall probability of accepting a sample from the posterior decreases rapidly as the number of observed variables increases and as the number of states that those variables can take increases, and so this approach is rarely used in practice.

An improvement on this approach is called *likelihood weighted sampling* (Fung and Chang, 1990; Shachter and Peot, 1990). It is based on ancestral sampling combined with importance sampling. For each variable in turn, if that variable is in the evidence set, then it is just set to its instantiated value. If it is not in the evidence set, then it is sampled from the conditional distribution $p(\mathbf{z}_i|\mathrm{pa}(i))$ in which the conditioning variables are set to their currently sampled values. The weighting associated with the resulting sample $\mathbf{z}$ is then given by

Exercise 14.15

$$r(\mathbf{z}) = \prod_{\mathbf{z}_i \notin \mathbf{e}} \frac{p(\mathbf{z}_i|\mathrm{pa}(i))}{p(\mathbf{z}_i|\mathrm{pa}(i))} \prod_{\mathbf{z}_i \in \mathbf{e}} \frac{p(\mathbf{z}_i|\mathrm{pa}(i))}{1} = \prod_{\mathbf{z}_i \in \mathbf{e}} p(\mathbf{z}_i|\mathrm{pa}(i)). \tag{14.48}$$

This method can be further extended using *self-importance sampling* (Shachter and Peot, 1990) in which the importance sampling distribution is continually updated to reflect the current estimated posterior distribution.

14.3. Langevin Sampling

The Metropolis–Hastings algorithm draws samples from a probability distribution by creating a Markov chain of candidate samples using a proposal distribution and then accepting or rejecting them using the criterion (14.40). This can be relatively inefficient since the proposal distribution is often a simple, fixed distribution that can generate updates in any direction in the data space, leading to a random walk.

We have seen that when training neural networks, it is hugely advantageous to make use of the gradient of the log likelihood with respect to the learnable parameters of the model in order to maximize the likelihood function. By analogy, we can introduce Markov chain sampling algorithms that make use of the gradient of the probability density with respect to the data vector so as to take steps that preferentially move towards regions of higher probability. One such technique is called *Hamiltonian Monte Carlo*, also known as *hybrid Monte Carlo*. This again makes use of a Metropolis acceptance test (Duane *et al.*, 1987; Bishop, 2006). Here we will focus on a different approach that is widely used in deep learning, called *Langevin sampling*. Although it avoids the use of an acceptance test, the algorithm has to be designed carefully to ensure that the resulting samples are unbiased. An important application of Langevin sampling arises in the context of machine learning models defined in terms of energy functions.

14.3.1 Energy-based models

Many generative models can be expressed as conditional probability distributions $p(\mathbf{x}|\mathbf{w})$ where $\mathbf{x}$ is the data vector and $\mathbf{w}$ represents a vector of learnable parameters. Such models can be trained by maximizing the corresponding likelihood function defined with respect to a training data set. However, to represent a valid probability distribution, the model must satisfy

$$\int p(\mathbf{x}|\mathbf{w})p(\mathbf{x})\,\mathrm{d}\mathbf{x} = 1. \tag{14.49}$$

Ensuring that this requirement is met can significantly limit the allowable forms for the model. If we put aside the normalization constraint then we can consider a much broader class of models called *energy-based models* (LeCun *et al.*, 2006). Suppose we have a function $E(\mathbf{x}, \mathbf{w})$, called the *energy function*, which is a real-valued function of its arguments but which has no other constraints. The exponential $\exp\{-E(\mathbf{x}, \mathbf{w})\}$ is a non-negative quantity and can therefore be viewed as an unnormalized probability distribution over $\mathbf{x}$. Here the introduction of the minus sign in the exponent is simply a convention, and it means that higher values of energy correspond to lower values of probability. We can then define a normalized distribution using

$$p(\mathbf{x}|\mathbf{w}) = \frac{1}{Z(\mathbf{w})} \exp\{-E(\mathbf{x}, \mathbf{w})\} \tag{14.50}$$

Exercise 14.16 where the normalizing constant $Z(\mathbf{w})$, known as the *partition function*, is defined by

$$Z(\mathbf{w}) = \int \exp\{-E(\mathbf{x}, \mathbf{w})\}\,\mathrm{d}\mathbf{x}. \tag{14.51}$$

The energy function is often modelled using a deep neural network with input vector $\mathbf{x}$ and a scalar output $E(\mathbf{x}, \mathbf{w})$, where $\mathbf{w}$ represents the weights and biases in the network.

Note that the partition function depends on $\mathbf{w}$, which creates problems for training. For example, the log likelihood function for a data set $\mathcal{D} = (\mathbf{x}_1, \ldots, \mathbf{x}_N)$ of *Exercise 14.17* i.i.d. data has the form

$$\ln p(\mathcal{D}|\mathbf{w}) = -\sum_{n=1}^{N} E(\mathbf{x}_n, \mathbf{w}) - N \ln Z(\mathbf{w}). \tag{14.52}$$

To compute the gradient of $\ln p(\mathcal{D}|\mathbf{w})$ with respect to $\mathbf{w}$, we need to know the form of $Z(\mathbf{w})$. However, for many choices of the energy function $E(\mathbf{x}, \mathbf{w})$, it will be impractical to evaluate the partition function in (14.51) because this involves integrating (or summing for discrete variables) over all the whole of $\mathbf{x}$-space. The term 'energy-based model' is generally used for models where this integral is intractable. Note, however, that probabilistic models can be seen as special cases of energy-based models, and therefore many of the models discussed in this book can be viewed as energy-based models. The big advantage of energy-based models, therefore, is their flexibility in that they bypass the requirement for normalization. A corresponding disadvantage, however, is that since the normalizing constant is unknown, they can be more difficult to train.

14.3.2 Maximizing the likelihood

Various approximation methods have been developed to train energy-based models without having to evaluate the partition function (Song and Kingma, 2021). Here we look at techniques based on Markov chain Monte Carlo. An alternative approach, called score matching, will be discussed in the context of diffusion models.

Chapter 20

We have seen that for energy-based models, the likelihood function cannot be evaluated explicitly due to the unknown partition function $Z(\mathbf{w})$. However, we can make use of Monte Carlo sampling methods to approximate the gradient of the log likelihood with respect to the model parameters. Once an energy-based model has been trained, by whatever means, we also need a way to draw samples from the model, and again we can make use of Monte Carlo methods.

Using (14.50), the gradient, with respect to the model parameters, of the log likelihood function for an energy-based model can be written in the form

$$\nabla_{\mathbf{w}} \ln p(\mathbf{x}|\mathbf{w}) = -\nabla_{\mathbf{w}} E(\mathbf{x}, \mathbf{w}) - \nabla_{\mathbf{w}} \ln Z(\mathbf{w}). \tag{14.53}$$

This is the likelihood function for a single data point $\mathbf{x}$, but in practice we want to maximize the likelihood defined over a training set of data points drawn from some unknown distribution $p_{\mathcal{D}}(\mathbf{x})$. If we assume the data points are i.i.d., then we can consider the gradient of the expectation of the log likelihood with respect to $p_{\mathcal{D}}(\mathbf{x})$, which is then given by

$$\mathbb{E}_{\mathbf{x}\sim p_{\mathcal{D}}} \left[\nabla_{\mathbf{w}} \ln p(\mathbf{x}|\mathbf{w})\right] = -\mathbb{E}_{\mathbf{x}\sim p_{\mathcal{D}}} \left[\nabla_{\mathbf{w}} E(\mathbf{x}, \mathbf{w})\right] - \nabla_{\mathbf{w}} \ln Z(\mathbf{w}) \tag{14.54}$$

where we have made use of the fact that the final term $-\nabla_{\mathbf{w}} \ln Z(\mathbf{w})$ does not depend on $\mathbf{x}$ and can therefore be taken outside the expectation. The partition function $Z(\mathbf{w})$ is assumed to be unknown, but we can make use of (14.51) and rearrange to obtain

Exercise 14.18

$$-\nabla_{\mathbf{w}} \ln Z(\mathbf{w}) = \int \left\{\nabla_{\mathbf{w}} E(\mathbf{x}, \mathbf{w})\right\} p(\mathbf{x}|\mathbf{w}) \, \mathrm{d}\mathbf{x}. \tag{14.55}$$

The right-hand side of (14.55) corresponds to an expectation over the model distribution $p(\mathbf{x}|\mathbf{w})$ given by

$$\int \left\{\nabla_{\mathbf{w}} E(\mathbf{x}, \mathbf{w})\right\} p(\mathbf{x}|\mathbf{w}) \, \mathrm{d}\mathbf{x} = \mathbb{E}_{\mathbf{x}\sim\mathcal{M}} \left[\nabla_{\mathbf{w}} E(\mathbf{x}, \mathbf{w})\right]. \tag{14.56}$$

Exercise 14.18

Combining (14.54), (14.55), and (14.56) we obtain

$$\nabla_{\mathbf{w}} \mathbb{E}_{\mathbf{x}\sim p_{\mathcal{D}}} \left[\ln p(\mathbf{x}|\mathbf{w})\right] = -\mathbb{E}_{\mathbf{x}\sim p_{\mathcal{D}}} \left[\nabla_{\mathbf{w}} E(\mathbf{x}, \mathbf{w})\right] + \mathbb{E}_{\mathbf{x}\sim p_{\mathcal{M}}(\mathbf{x})} \left[\nabla_{\mathbf{w}} E(\mathbf{x}, \mathbf{w})\right]. \tag{14.57}$$

This result is illustrated in Figure 14.13, and has a nice interpretation, as follows. Our goal is to find values for the parameters $\mathbf{w}$ that maximize the likelihood function, and therefore consider a small change to $\mathbf{w}$ in the direction of the gradient $\nabla_{\mathbf{w}} \ln p(\mathbf{x}|\mathbf{w})$. From (14.57) we see that expected value of this gradient can be expressed as two terms, having opposite signs. The first term on the right-hand side of (14.57) acts

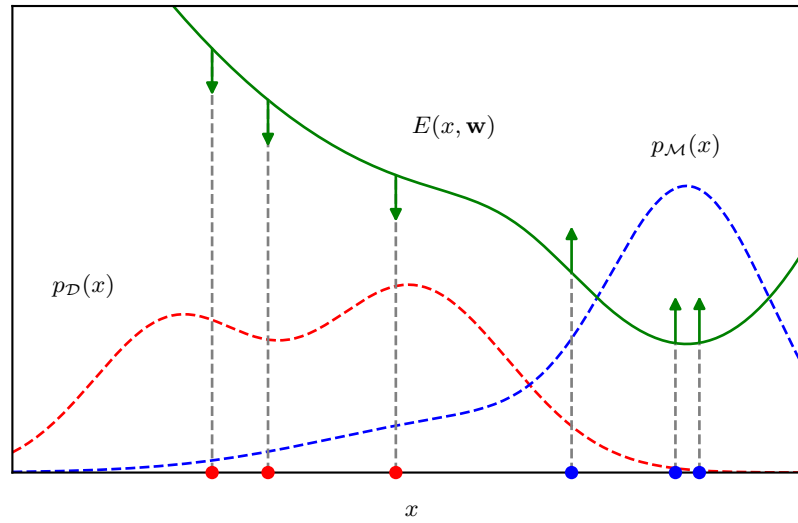


Figure 14.13 Illustration of the training of an energy-based model by maximizing the likelihood, showing the energy function $E(x, \mathbf{w})$ in green along with the associated model distribution $p_{\mathcal{M}}(x)$ and the true data distribution $p_{\mathcal{D}}(x)$. Increasing the expected log likelihood by using (14.57) pushes the energy function up at points corresponding to samples from the model (shown as blue dots) and pushes it down at points corresponding to samples from the data set (shown as red dots).

to decrease $E(\mathbf{x}, \mathbf{w})$, and therefore to increase the probability density defined by the model, for points $\mathbf{x}$ drawn from $p_{\mathcal{D}}(\mathbf{x})$. The second term on the right-hand side of (14.57) acts to increase the value of $E(\mathbf{x}, \mathbf{w})$, and therefore to decrease the probability density defined by the model, for data points drawn from the model itself. In regions where the model density exceeds the training data density, the net effect will be to increase the energy and therefore reduce the probability. Conversely, in regions where training data density exceeds the model density, the net effect will be to reduce the energy and therefore increase the probability density. Together these two terms move probability mass away from regions where there is a low density of training data and towards regions of high data density, as desired. The two terms will be equal in magnitude when the model distribution matches the data distribution, at which point the gradient on the left-hand-side of (14.57) will equal zero.

14.3.3 Langevin dynamics

When applying (14.57) as a practical training method, we need to approximate the two terms on the right-hand side. For any given value of $\mathbf{x}$, we can evaluate $\nabla_{\mathbf{w}} E(\mathbf{x}, \mathbf{w})$ using automatic differentiation. For the first term in (14.57), we can use the training data set to estimate the expectation over $\mathbf{x}$:

$$\mathbb{E}_{\mathbf{x} \sim p_{\mathcal{D}}} \left[\nabla_{\mathbf{w}} E(\mathbf{x}, \mathbf{w}) \right] \simeq \frac{1}{N} \sum_{n=1}^{N} \nabla_{\mathbf{w}} E(\mathbf{x}_n, \mathbf{w}). \tag{14.58}$$

The second term is more challenging because we need to draw samples from the model distribution defined by an energy function whose corresponding partition function is intractable. This can be done using Markov chain Monte Carlo methods. One popular approach is called *stochastic gradient Langevin dynamics* or simply *Langevin sampling* (Parisi, 1981; Welling and Teh, 2011). This term depends on the distribution $p(\mathbf{x}|\mathbf{w})$ only through the *score function*, which is defined to be the gradient of the log likelihood with respect to the data vector $\mathbf{x}$, and is given by

$$\mathbf{s}(\mathbf{x}, \mathbf{w}) = \nabla_{\mathbf{x}} \ln p(\mathbf{x}|\mathbf{w}). \tag{14.59}$$

It is worth emphasising that this gradient is taken with respect to the data point $\mathbf{x}$ and is therefore not the usual gradient with respect to the learnable parameters $\mathbf{w}$. If we substitute (14.50) into (14.59) we obtain

$$\mathbf{s}(\mathbf{x}, \mathbf{w}) = -\nabla_{\mathbf{x}} E(\mathbf{x}, \mathbf{w}) \tag{14.60}$$

where we see that the partition function no longer appears at it is independent of $\mathbf{x}$.

We start by drawing an initial value $\mathbf{x}^{(0)}$ from a prior distribution, and then we iterate the following Markov chain steps:

$$\mathbf{x}^{(\tau+1)} = \mathbf{x}^{(\tau)} + \eta \nabla_{\mathbf{x}} \ln p(\mathbf{x}^{(\tau)}, \mathbf{w}) + \sqrt{2\eta}\boldsymbol{\epsilon}^{(\tau)}, \qquad \tau \in 1, \ldots, \mathcal{T} \tag{14.61}$$

where $\boldsymbol{\epsilon}^{(\tau)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ are independent samples from a zero-mean, unit-covariance Gaussian distribution, and the parameter η controls the step size. Each iteration of the Langevin equation takes a step in the direction of the gradient of the log likelihood, and then adds Gaussian noise. It can be show that, in the limits of $\eta \to 0$ and $\mathcal{T} \to \infty$, the value of $\mathbf{z}^{(\mathcal{T})}$ is an independent sample from the distribution $p(\mathbf{x})$. Langevin sampling is summarized in Algorithm 14.4.

We can repeat the process to generate a set of samples $\{\mathbf{x}_1, \ldots, \mathbf{x}_M\}$ and then approximate the second term in (14.57) using

$$\mathbb{E}_{\mathbf{x} \sim p_{\mathcal{M}}(\mathbf{x})} \left[\nabla_{\mathbf{w}} E(\mathbf{x}, \mathbf{w})\right] \simeq \frac{1}{M} \sum_{m=1}^{M} \nabla_{\mathbf{w}} E(\mathbf{x}_m, \mathbf{w}). \tag{14.62}$$

Running long Markov chains to generate independent samples can be computationally expensive, and so we need to consider practical approximations. One approach is called *contrastive divergence* (Hinton, 2002). Here the samples used to evaluate (14.62) are obtained by running a Monte Carlo chain starting with one of the training data points $\mathbf{x}_n$. If the chain is run for a large number of steps, then the resulting value will be essentially an unbiased sample from the model distribution. Instead Hinton (2002) proposes running for only a few steps of Monte Carlo, perhaps even as few as one step, which is computationally much less costly. The resulting sample will be far from unbiased and will lie close to the data manifold. As a result, the effect of using gradient descent will be to shape the energy surface, and hence the probability density, only in the neighbourhood of the data manifold. This can prove effective for tasks such as discrimination but is expected to be less effective in learning a generative model.

Algorithm 14.4: Langevin sampling

Input: Initial value $\mathbf{x}^{(0)}$
Probability density $p(\mathbf{x}, \mathbf{w})$
Learning rate parameter η
Number of iterations T

Output: Final value $\mathbf{x}^{(T)}$

$\mathbf{x} \leftarrow \mathbf{x}_0$
for $\tau \in \{1, \ldots, T\}$ **do**
  $\boldsymbol{\epsilon} \sim \mathcal{N}(\boldsymbol{\epsilon}|\mathbf{0}, \mathbf{I})$
  $\mathbf{x} \leftarrow \mathbf{x} + \eta \nabla_{\mathbf{x}} \ln p(\mathbf{x}, \mathbf{w}) + \sqrt{2\eta}\boldsymbol{\epsilon}$
end for
return $\mathbf{x}$ `// Final value` $\mathbf{x}^{(T)}$

Exercises

14.1 ($\star$) Show that $\overline{f}$ defined by (14.2) is an unbiased estimator, in other words that the expectation of the right-hand side is equal to $\mathbb{E}[f(\mathbf{z})]$.

14.2 ($\star$) Show that $\overline{f}$ defined by (14.2) has variance given by (14.4).

14.3 ($\star$) Suppose that z is a random variable with uniform distribution over $(0, 1)$ and that we transform z using $y = h^{-1}(z)$ where $h(y)$ is given by (14.6). Show that y has the distribution $p(y)$.

14.4 ($\star\star$) Given a random variable z that is uniformly distributed over $(0, 1)$, find a transformation $y = f(z)$ such that y has a Cauchy distribution given by (14.8).

14.5 ($\star\star$) Suppose that z_1 and z_2 are uniformly distributed over the unit circle, as shown in Figure 14.3, and that we make the change of variables given by (14.10) and (14.11). Show that (y_1, y_2) will be distributed according to (14.12).

14.6 ($\star\star$) Let $\mathbf{z}$ be a D-dimensional random variable having a Gaussian distribution with zero mean and unit covariance matrix, and suppose that the positive definite symmetric matrix $\boldsymbol{\Sigma}$ has the Cholesky decomposition $\boldsymbol{\Sigma} = \mathbf{L}\mathbf{L}^{\mathrm{T}}$, where $\mathbf{L}$ is a lower-triangular matrix (i.e., one with zeros above the leading diagonal). Show that the variable $\mathbf{y} = \boldsymbol{\mu} + \mathbf{L}\mathbf{z}$ has a Gaussian distribution with mean $\boldsymbol{\mu}$ and covariance $\boldsymbol{\Sigma}$. This provides a technique for generating samples from a general multivariate Gaussian using samples from a univariate Gaussian having zero mean and unit variance.

14.7 ($\star\star$) In this exercise, we show more carefully that rejection sampling does indeed draw samples from the desired distribution $p(\mathbf{z})$. Suppose the proposal distribution is $q(\mathbf{z})$. Show that the probability of a sample value $\mathbf{z}$ being accepted is given by

$\widetilde{p}(\mathbf{z})/kq(\mathbf{z})$ where $\widetilde{p}$ is any unnormalized distribution that is proportional to $p(\mathbf{z})$, and the constant k is set to the smallest value that ensures $kq(\mathbf{z}) \geqslant \widetilde{p}(\mathbf{z})$ for all values of $\mathbf{z}$. Note that the probability of drawing a value $\mathbf{z}$ is given by the probability of drawing that value from $q(\mathbf{z})$ times the probability of accepting that value given that it has been drawn. Make use of this, along with the sum and product rules of probability, to write down the normalized form for the distribution over $\mathbf{z}$, and show that it equals $p(\mathbf{z})$.

14.8 ($\star$) Suppose that z has a uniform distribution over the interval $[0, 1]$. Show that the variable $y = b \tan z + c$ has a Cauchy distribution given by (14.16).

14.9 ($\star\star$) Determine expressions for the coefficients k_i in the envelope distribution (14.17) for adaptive rejection sampling using the requirements of continuity and normalization.

14.10 ($\star\star$) By making use of the technique discussed in Section 14.1.2 for sampling from a single exponential distribution, devise an algorithm for sampling from the piecewise exponential distribution defined by (14.17).

14.11 ($\star$) Show that the simple random walk over the integers defined by (14.28), (14.29), and (14.30) has the property that $\mathbb{E}[(z^{(\tau)})^2] = \mathbb{E}[(z^{(\tau-1)})^2] + 1/2$ and hence by induction that $\mathbb{E}[(z^{(\tau)})^2] = \tau/2$.

14.12 ($\star\star$) Show that the Gibbs sampling algorithm, discussed in Section 14.2.4, satisfies detailed balance as defined by (14.34).

14.13 ($\star$) Consider the distribution shown in Figure 14.14. Discuss whether the standard Gibbs sampling procedure for this distribution is ergodic and therefore whether it would sample correctly from this distribution

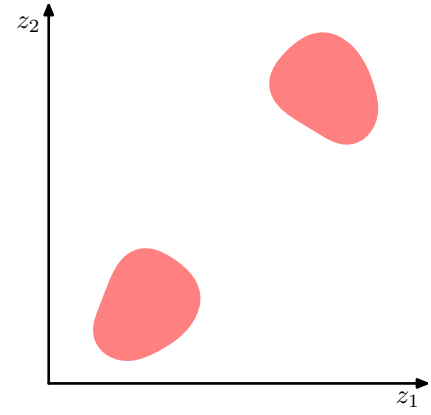


Figure 14.14 A probability distribution over two variables z_1 and z_2 that is uniform over the shaded regions and zero everywhere else.

14.14 ($\star$) Verify that the over-relaxation update (14.46), in which z_i has mean μ_i and variance σ_i and where ν has zero mean and unit variance gives a value z_i' with mean μ_i and variance σ_i^2.

14.15 ($\star$) Show that in likelihood weighted sampling from a directed graph the importance sampling weights are given by (14.48).

14.16 ($\star$) Show that the distribution (14.50) is normalized with respect to $\mathbf{x}$ provided $Z(\mathbf{w})$ satisfies (14.51).

14.17 ($\star\star$) By making use of (14.50) show that the gradient of the log likelihood function for an energy-based model can be written in the form (14.52).

14.18 ($\star\star$) By making use of (14.54), (14.55), and (14.56), show that the gradient of the log likelihood function for an energy-based model can be written in the form (14.57).



15 Discrete Latent Variables

Chapter 11

We have seen how complex distributions can be constructed by combining multiple simple distributions and how the resulting models can be described by directed graphs. In addition to the observed variables, which form part of the data set, such models often introduce additional hidden, or latent, variables. These might correspond to specific quantities involved in the data generation process, such as the unknown orientation of an object in three-dimensional space in the case of images, or they may be introduced simply as modelling constructs to allow much richer models to be created. If we define a joint distribution over observed and latent variables, the corresponding distribution of the observed variables alone is obtained by marginalization. This allows relatively complex marginal distributions over observed variables to be expressed in terms of more tractable joint distributions over the expanded space of observed and latent variables.

In this chapter, we will see that marginalizing over discrete latent variables gives

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024
C. M. Bishop, H. Bishop, *Deep Learning*, https://doi.org/10.1007/978-3-031-45468-4_15

rise to mixture distributions. Our focus will be on mixtures of Gaussians that provide a good illustration of mixture distributions and that are also widely used in machine learning. One simple application for mixture models is to discover clusters in data, and we begin our discussion by considering a technique for clustering called the K-means algorithm, which corresponds to a particular non-probabilistic limit of Gaussian mixtures. Then we introduce the latent-variable view of mixture distributions in which the discrete latent variables can be interpreted as defining assignments of data points to specific components of the mixture.

A general technique for finding maximum likelihood estimators in latent-variable models is the expectation–maximization (EM) algorithm. We first use the Gaussian mixture distribution to motivate the EM algorithm in an informal way, and then we give a more careful treatment based on the latent-variable viewpoint. Finally we provide a general perspective by introducing the evidence lower bound (ELBO), which will play an important role in generative models such as variational autoencoders and diffusion models.

15.1. K-means Clustering

We begin by considering the problem of identifying groups, or clusters, of data points in a multi-dimensional space. Suppose we have a data set $\{\mathbf{x}_1, \ldots, \mathbf{x}_N\}$ consisting of N observations of a D-dimensional Euclidean variable $\mathbf{x}$. Our goal is to partition the data set into some number K of clusters, where we will suppose for the moment that the value of K is given. Intuitively, we might think of a cluster as comprising a group of data points whose inter-point distances are small compared with the distances to points outside the cluster. We can formalize this notion by first introducing a set of D-dimensional vectors $\boldsymbol{\mu}_k$, where $k = 1, \ldots, K$, in which $\boldsymbol{\mu}_k$ is a 'prototype' associated with the kth cluster. As we will see shortly, we can think of the $\boldsymbol{\mu}_k$ as representing the centres of the clusters. Our goal is then to find a set of cluster vectors $\{\boldsymbol{\mu}_k\}$, along with an assignment of data points to clusters, such that the sum of the squares of the distances of each data point to its closest cluster vector $\boldsymbol{\mu}_k$ is a minimum.

It is convenient at this point to define some notation to describe the assignment of data points to clusters. For each data point $\mathbf{x}_n$, we introduce a corresponding set of binary indicator variables $r_{nk} \in \{0, 1\}$, where $k = 1, \ldots, K$. These indicators describe which of the K clusters the data point $\mathbf{x}_n$ is assigned to, so that if data point $\mathbf{x}_n$ is assigned to cluster k then $r_{nk} = 1$, and $r_{nj} = 0$ for $j \neq k$. This is an example of the 1-of-K coding scheme. We can then define an error function:

$$J = \sum_{n=1}^{N} \sum_{k=1}^{K} r_{nk} \|\mathbf{x}_n - \boldsymbol{\mu}_k\|^2, \tag{15.1}$$

which represents the sum of the squares of the distances of each data point to its assigned vector $\boldsymbol{\mu}_k$. Our goal is to find values for the $\{r_{nk}\}$ and the $\{\boldsymbol{\mu}_k\}$ so as to minimize J. We can do this through an iterative procedure in which each iteration